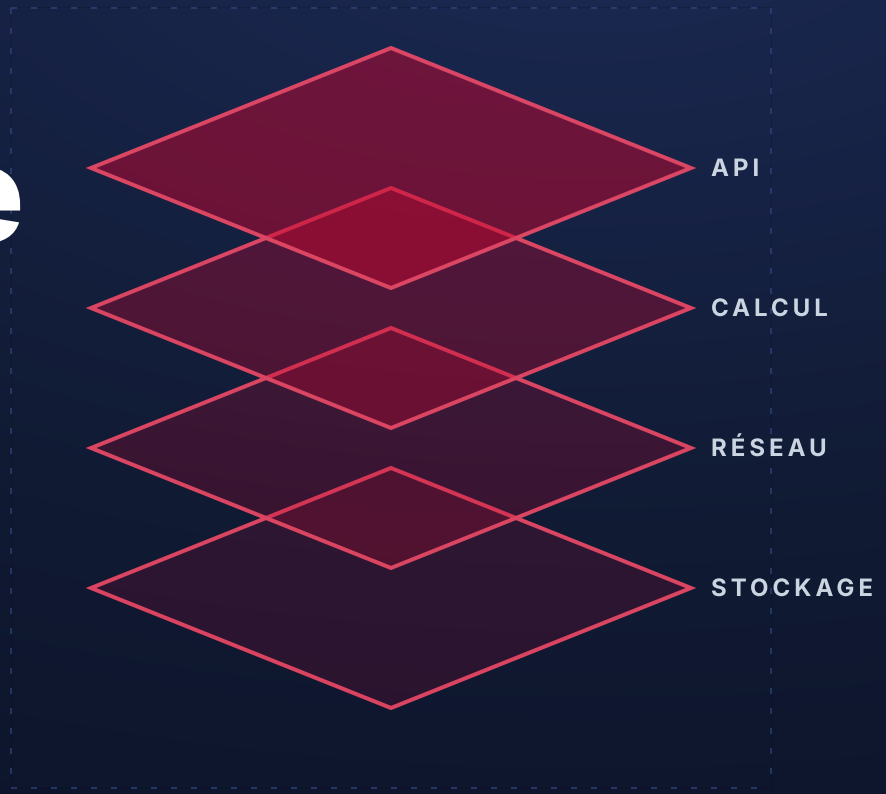
SUPPORT DE FORMATION COMPLET · 13 MODULES

OpenStack, de l'architecture à l'exploitation sécurisée

Concepts, architecture interne, composants, services, déploiement, stack Canonical, MicroCloud, identité, cloisonnement et audit — sur Canonical OpenStack (Sunbeam)

Architecture · Services · Déploiement · Sunbeam · MicroCloud · Keystone · Moindre privilège · Audit



Version de référence : OpenStack 2026.2 « Hibiscus » ; cloud déployé : Canonical OpenStack 2026.1 LTS · documentations consultées en octobre 2026

INTRODUCTION

Plan du support : treize modules

01 Panorama p. 3

02 Architecture interne p. 28

03 Composants système p. 53

04 Principaux services p. 76

05 Méthodes de déploiement p. 101

06 La stack Canonical p. 128

07 MicroCloud p. 153

08 Keystone en profondeur p. 172

09 Les domaines p. 198

10 Projets, hiérarchie et quotas p. 217

11 Authentification externe et mots de passe p. 238

12 Rôles et moindre privilège p. 259

13 Micro-segmentation et audit p. 280

→ Chaque module s'ouvre sur ses objectifs ; les bandeaux bleus renvoient au module qui approfondit un sujet.

01

Panorama d'OpenStack

Le cloud, le projet, les alternatives, l'architecture d'ensemble

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Définir le cloud computing et distinguer **IaaS, PaaS, CaaS et SaaS** par le partage des responsabilités.
- **Comparer** OpenStack à VMware, à OpenShift et aux clouds publics, service par service.
- Présenter le rôle de chaque **service** (Keystone, Nova, Neutron, Cinder, Glance, Swift, Heat…).
- Expliquer ce qu'est OpenStack, d'où il vient, qui le gouverne et comment il est publié.
- Décrire l'**architecture** : plan de contrôle, nœuds, API REST, bus de messages, bases de données.
- Dérouler pas à pas la **création d'une instance** à travers tous les services.

Qu'est-ce qu'un cloud ? Les cinq caractéristiques du NIST

La définition de référence est celle du **NIST (SP 800-145)** : un modèle d'accès par le réseau, à la demande, à un pool partagé de ressources configurables, provisionnées et libérées rapidement avec un minimum d'intervention du fournisseur.

Libre-service à la demande

L'utilisateur provisionne seul, sans ticket ni intervention humaine.

OpenStack : API REST, CLI, Horizon

Accès réseau étendu

Les services sont accessibles par des protocoles standards, depuis n'importe quel client.

OpenStack : endpoints HTTPS publiés dans le catalogue

Mise en commun des ressources

Un même pool sert plusieurs clients isolés logiquement (multi-tenant).

OpenStack : domaines, projets, réseaux isolés

Élasticité rapide

Les ressources s'ajoutent et se retirent en quelques secondes, parfois automatiquement.

OpenStack : création par API, autoscaling Heat

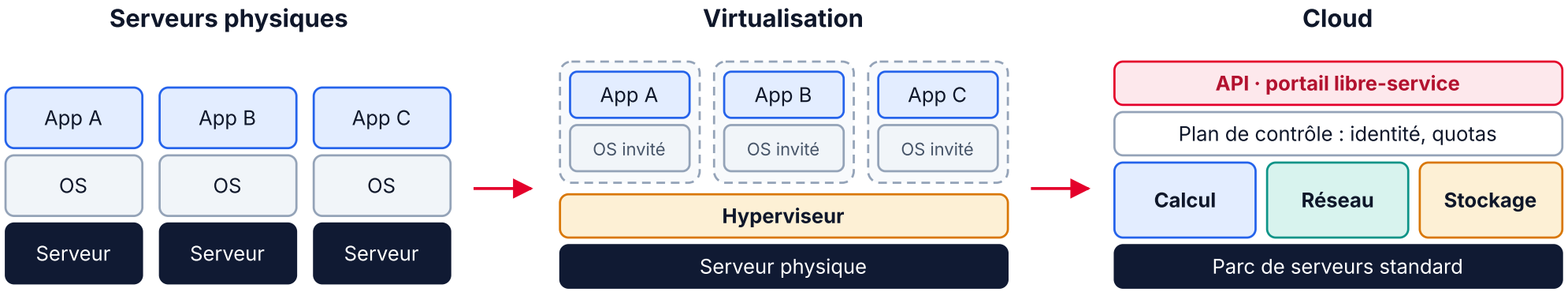
Service mesuré

L'usage est compté, plafonné et restituable (facturation ou refacturation).

OpenStack : quotas, Ceilometer, CloudKitty

> Si l'une de ces cinq propriétés manque — typiquement le libre-service ou la mesure — on parle d'**hébergement** ou de **virtualisation**, pas de cloud. C'est le critère à garder en tête pour toutes les comparaisons qui suivent.

Du serveur physique au cloud : trois générations d'infrastructure



- Une application = un serveur dédié
- Taux d'utilisation faible (souvent < 20 %)
- Achat, câblage, installation à chaque besoin
- Panne matérielle = panne applicative

Délai de mise à disposition
plusieurs semaines

- Consolidation : N machines virtuelles par hôte
- Pilotage par l'équipe infrastructure
- Demande par ticket, création manuelle
- Disponibilité assurée par l'infrastructure

Délai de mise à disposition
quelques heures à quelques jours

- Libre-service par API, multi-tenant
- Pools abstraits, quotas, usage mesuré
- Infrastructure décrite par du code
- Résilience portée par l'application

Délai de mise à disposition
quelques secondes à quelques minutes

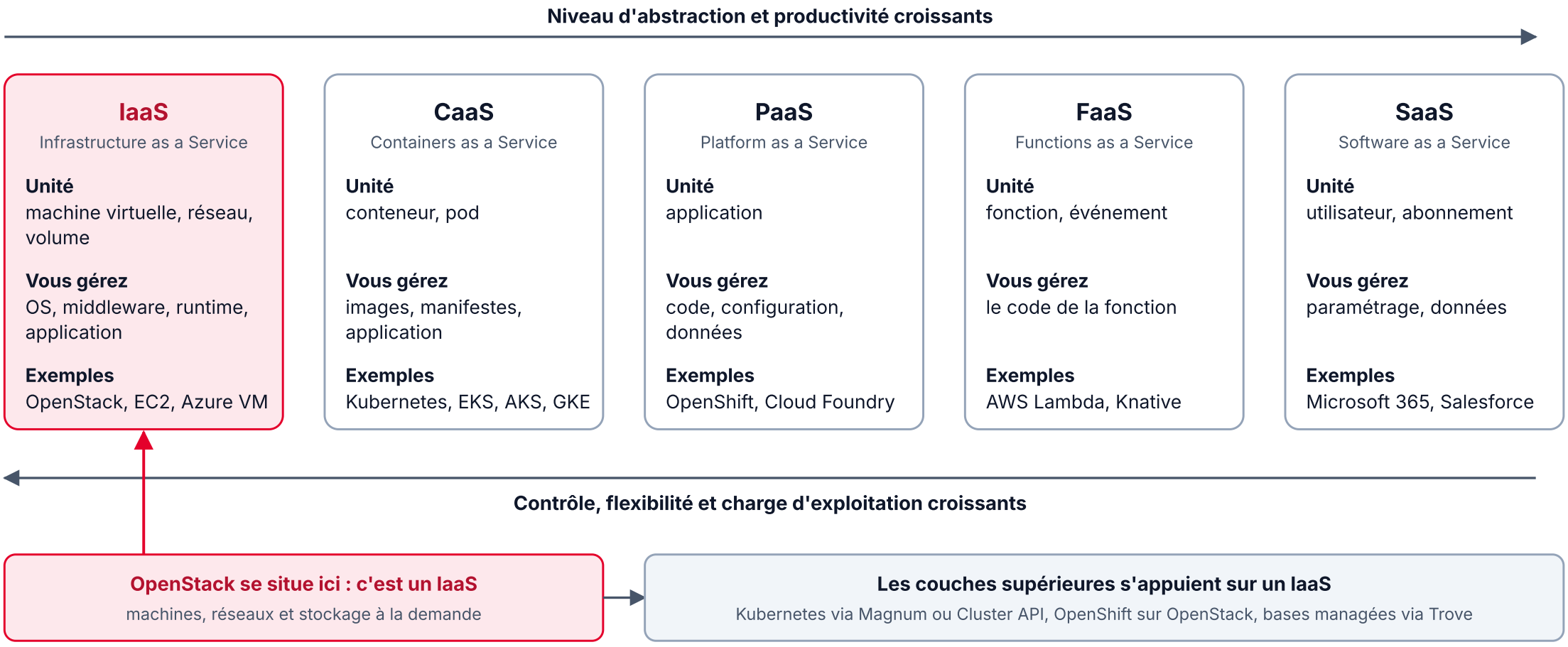
Lecture du schéma : de gauche à droite, l'unité de gestion change : un serveur par application, puis des machines virtuelles consolidées sur un hyperviseur, enfin des ressources tirées par API d'un parc mutualisé. Le bas de chaque colonne donne le délai de mise à disposition, de plusieurs semaines à quelques minutes.

IaaS, PaaS, SaaS : qui gère quoi ?

	Sur site infrastructure traditionnelle	IaaS OpenStack, AWS EC2, Azure VM	PaaS OpenShift, Azure App Service	SaaS Microsoft 365, Salesforce
Applications	vous	vous	vous	fournisseur
Données	vous	vous	vous	fournisseur (hébergement)
Environnement d'exécution	vous	vous	fournisseur	fournisseur
Middleware	vous	vous	fournisseur	fournisseur
Système d'exploitation	vous	vous	fournisseur	fournisseur
Virtualisation	vous	fournisseur	fournisseur	fournisseur
Serveurs	vous	fournisseur	fournisseur	fournisseur
Stockage	vous	fournisseur	fournisseur	fournisseur
Réseau physique	vous	fournisseur	fournisseur	fournisseur

Point clé pour OpenStack : dans un cloud privé, votre organisation tient les deux rôles. L'**équipe plateforme** est le « fournisseur » (elle exploite OpenStack et tout ce qui est en dessous) ; les **équipes projets** sont les « clients » qui consomment de l'IaaS en libre-service.

IaaS, CaaS, PaaS, FaaS, SaaS : un continuum d'abstraction



Lecture du schéma : plus on va vers la droite, plus le fournisseur prend de couches à sa charge et moins vous en gérez — la ligne « Vous gérez » le montre colonne par colonne. OpenStack fournit le premier étage, l'IaaS, sur lequel les autres peuvent s'appuyer.

Modèles de déploiement : public, privé, hybride



Privé

Infrastructure dédiée à une organisation. Maîtrise totale, investissement et exploitation à sa charge. **Terrain naturel d'OpenStack.**

Public

Ressources mutualisées, ouvertes à tous, facturées à l'usage. Certains clouds publics sont eux-mêmes bâtis sur OpenStack (OVHcloud, Infomaniak).

Hybride

Combinaison d'un privé et d'un ou plusieurs publics, reliés et pilotés de façon cohérente : débordement, reprise d'activité, données sensibles en interne.

Communautaire, multi-cloud

Cloud partagé entre organisations d'un même secteur (recherche, État). Multi-cloud : plusieurs fournisseurs pour limiter la dépendance.

« Pets » et « cattle » : l'infrastructure devient du code

	Pets (animaux de compagnie)	Cattle (bétail)
Identité	Serveur nommé, unique	Instance numérotée, interchangeable
En cas de panne	On répare	On détruit et on recrée
Configuration	Manuelle, accumulée au fil du temps	Image + script, reproductible
Évolution	Mise à jour en place	Remplacement par une nouvelle version
État	Sur le disque local	Externalisé : volume, base, objet

- OpenStack sait héberger les deux, mais son modèle natif est le **cattle** : image, gabarit, `cloud-init`, disque éphémère.
- L'état durable vit dans **Cinder** (volumes) et **Swift** (objets), pas dans l'instance.
- Toute l'infrastructure se décrit en fichiers versionnés : Heat, Terraform ou OpenTofu, Ansible.

```
# Une instance complète en une commande, reproductible
openstack server create web-01 \
  --image   ubuntu-24.04 \
  --flavor  m1.small \
  --network net-app \
  --security-group web \
  --key-name cle-deploiement \
  --user-data cloud-init.yaml

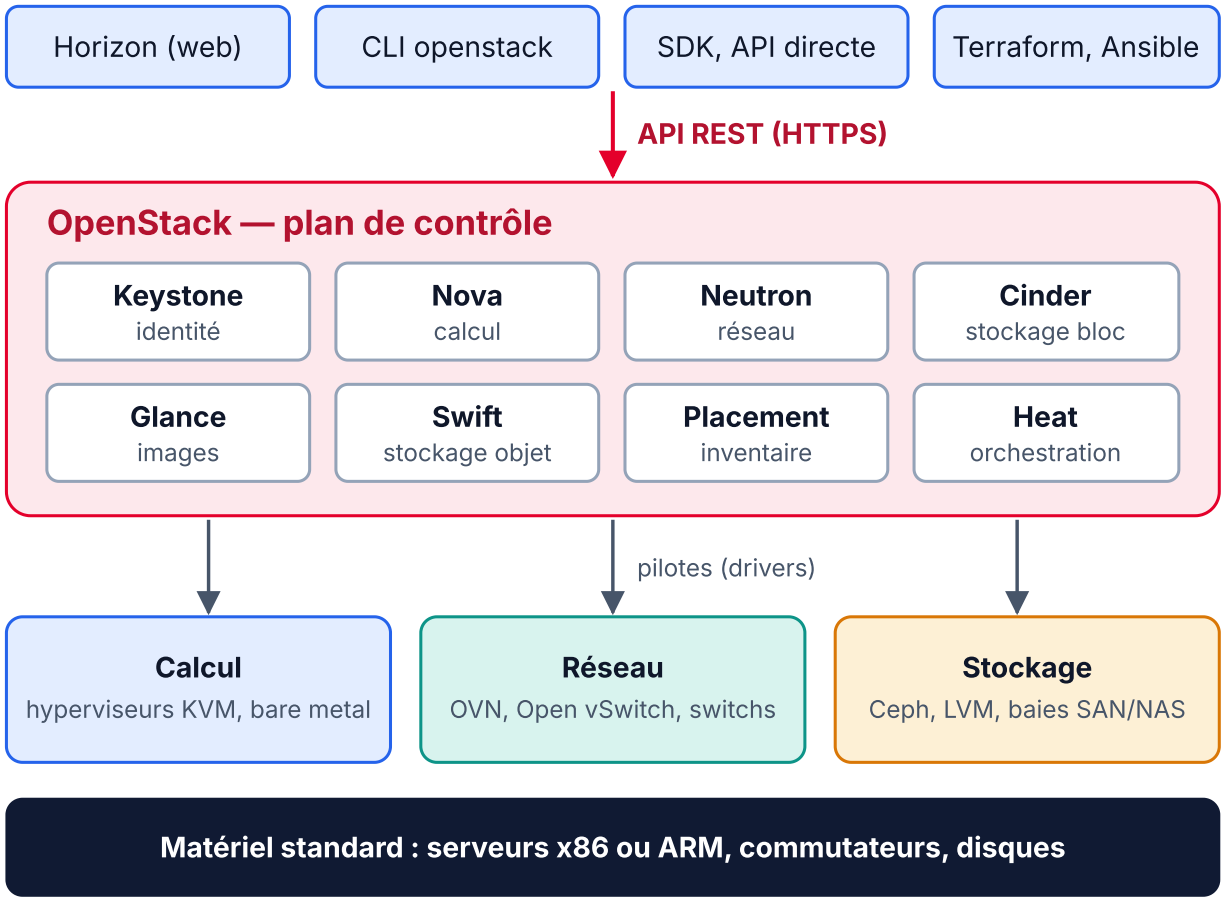
# cloud-init.yaml : configuration au premier démarrage
#cloud-config
packages: [nginx]
runcmd:
  - systemctl enable --now nginx
```

> La même requête peut être émise par la CLI, par Horizon, par Terraform ou par un pipeline CI/CD : tous appellent la **même API REST**.

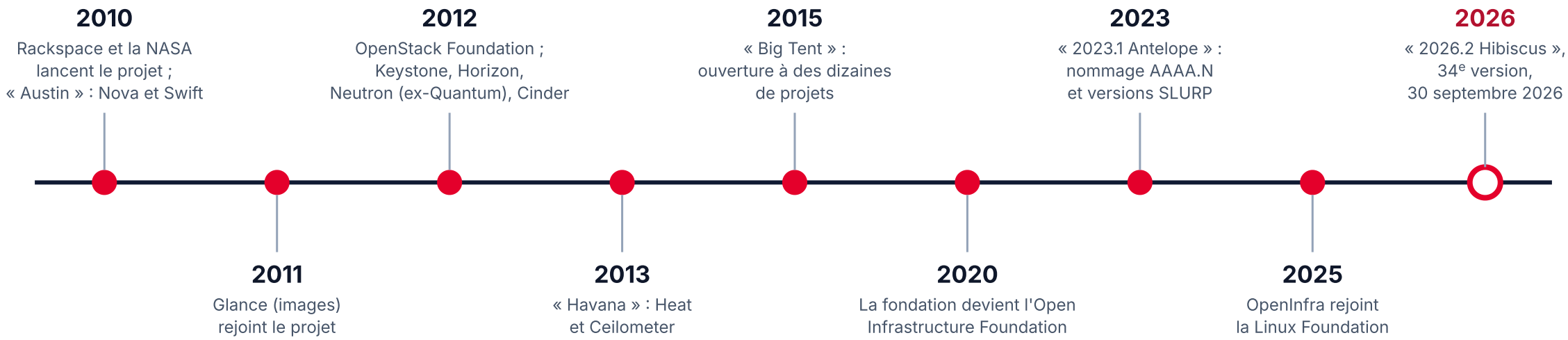
Qu'est-ce qu'OpenStack ?

Un ensemble de **logiciels libres** qui pilote de grands pools de calcul, de réseau et de stockage à l'échelle d'un datacenter, à travers des **API** et un tableau de bord, pour fournir un cloud **IaaS** privé ou public.

- **Licence Apache 2.0**, sans édition « entreprise » fermée : tout le code est public.
- Écrit en **Python**, déployé sur **Linux**, sur du matériel standard.
- **Modulaire :** une trentaine de services indépendants ; on n'installe que ceux dont on a besoin.
- **Agnostique :** il ne virtualise rien lui-même, il **orchestre** des technologies existantes (KVM, Open vSwitch, OVN, Ceph, LVM, baies du marché) grâce à des pilotes.
- Surnommé « **système d'exploitation du cloud** » : comme un OS arbitre CPU, mémoire et disques d'une machine, OpenStack arbitre ceux d'un datacenter entre des projets.



Seize ans d'histoire en neuf jalons



L'origine

La NASA apporte le moteur de calcul (Nova), Rackspace son stockage objet (Swift). L'ambition : un cloud ouvert, sans dépendance à un fournisseur.

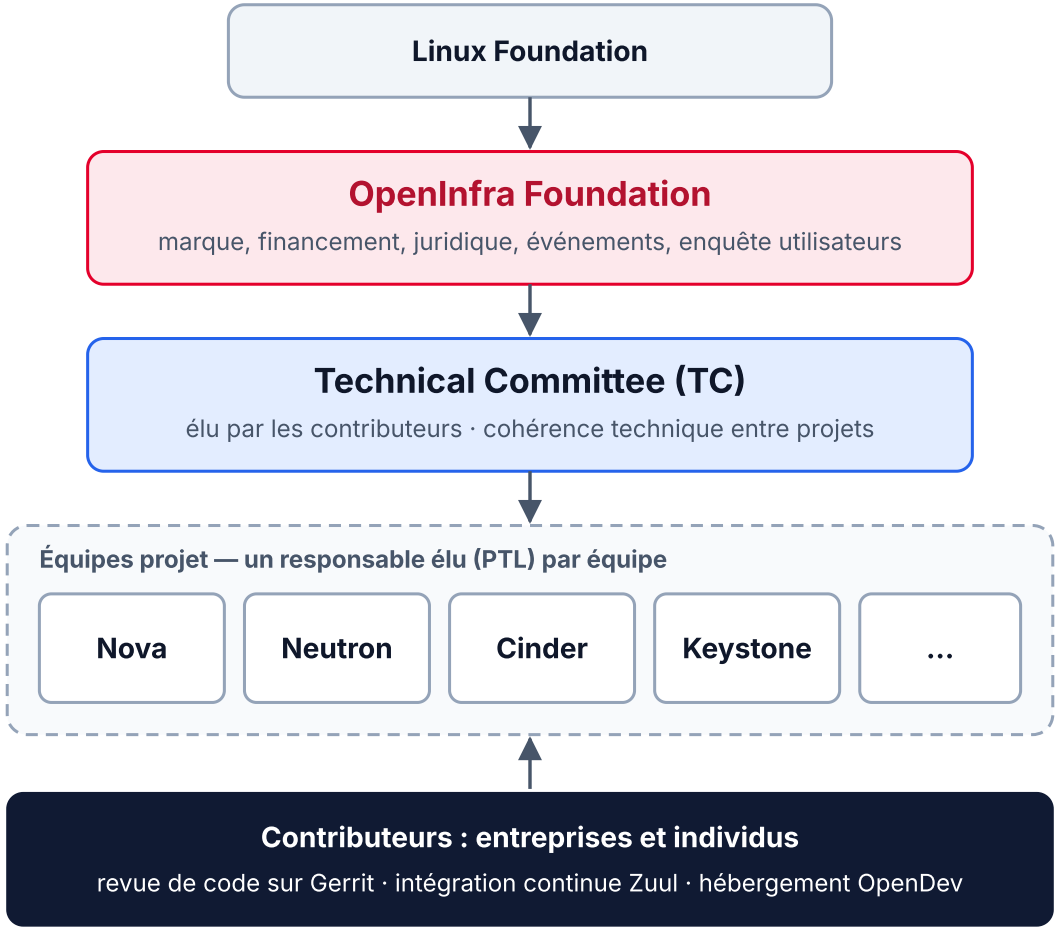
La maturité

Après l'emballement médiatique des années 2015-2017, le projet se recentre sur un cœur stable. Les API principales n'ont plus connu de rupture depuis des années.

Aujourd'hui

Regain d'intérêt porté par la souveraineté numérique, l'évolution des licences VMware et les besoins d'infrastructure pour l'IA (GPU, calcul confidentiel).

Gouvernance : un projet communautaire, sans éditeur unique



Les « Four Opens » — les quatre principes fondateurs :

Open Source
Tout le code sous licence Apache 2.0, sans version réservée aux clients payants.

Open Design
Les orientations se décident en public, lors des PTG (Project Teams Gathering).

Open Development
Chaque modification passe par une revue de code publique et des tests automatiques.

Open Community
Les responsables sont élus ; chacun peut contribuer, quel que soit son employeur.

- Aucune entreprise ne contrôle la feuille de route : les éditeurs (Red Hat, Canonical, Mirantis…) **contribuent** et **distribuent**, ils ne possèdent pas le projet.
- Conséquence pratique : une même API chez tous les fournisseurs, vérifiée par des tests d'interopérabilité.

Cycle de publication : deux versions par an, une SLURP sur deux



Flèches grises : montée de version classique, une version à la fois, tous les six mois.

Cadence

Une version coordonnée tous les **six mois** environ, au printemps et à l'automne. Tous les services sont publiés ensemble et testés ensemble.

Nommage

Depuis 2023 : **année.rang** suivi d'un nom par ordre alphabétique (Antelope, Bobcat, Caracal, Dalmatian, Epoxy, Flamingo, Gazpacho, Hibiscus).

SLURP

Skip Level Upgrade Release Process : les versions **.1** permettent de sauter la version intermédiaire, soit une seule montée de version par an.

Maintenance

Chaque version reçoit des correctifs pendant environ **18 mois** (phase *Maintained*), puis passe en *Unmaintained* avant sa fin de vie.

Distributions et outils de déploiement

OpenStack « amont » est un ensemble de dépôts de code. Pour l'installer, on passe par un **outil de déploiement** communautaire ou par la **distribution** d'un éditeur, qui ajoute intégration, cycle de vie et support.

Solution	Origine	Principe technique	Usage typique
DevStack	Communauté	Scripts shell, installation depuis les sources sur une seule machine	Développement, découverte — jamais en production
Kolla-Ansible	Communauté	Un conteneur par service (Docker ou Podman), orchestré par Ansible	Production, de quelques nœuds à plusieurs centaines
OpenStack-Ansible	Communauté	Playbooks Ansible, services en conteneurs LXC ou directement sur l'hôte	Production, forte personnalisation
OpenStack-Helm	Communauté	Charts Helm : le plan de contrôle tourne dans Kubernetes	Production pour équipes maîtrisant Kubernetes
Canonical OpenStack	Canonical	Sunbeam : snaps, Juju et Kubernetes ; MAAS pour le matériel	Du poste de test au cloud multi-nœuds, support Ubuntu Pro
Red Hat OpenStack Services on OpenShift	Red Hat	Plan de contrôle géré par des opérateurs sur OpenShift, computes sur RHEL	Entreprises et opérateurs déjà clients Red Hat
Mirantis OpenStack for Kubernetes	Mirantis	OpenStack conteneurisé, cycle de vie piloté par Kubernetes	Clouds privés et opérateurs

Tendance de fond : le plan de contrôle est désormais **conteneurisé**, et de plus en plus souvent hébergé sur Kubernetes. Les API exposées aux utilisateurs restent strictement les mêmes quelle que soit la distribution.

OpenStack et VMware : architectures comparées, couche par couche

	VMware Cloud Foundation		OpenStack
Libre-service ce que voit l'utilisateur	**VCF Automation · vSphere Client** catalogue de services, console d'administration	≈	**Horizon · Skyline · CLI · API REST** même API pour l'humain et pour l'automate
Plan de contrôle décisions, état, API	**vCenter Server** inventaire, SSO, DRS, HA, modèles de machines	≈	**Keystone** · **Nova** · **Placement** · **Glance**
Réseau et stockage virtualisés, programmables	**NSX** réseaux logiques, pare-feu **vSAN · VMFS** datastores	≈	**Neutron** OVN, Open vSwitch **Cinder** Ceph, LVM, baies du marché
Hyperviseur exécute les machines	**ESXi** hyperviseur propriétaire	≈	**KVM / QEMU + libvirt** hyperviseur libre, intégré à Linux
Matériel serveurs, disques, switchs	**Serveurs certifiés** liste de compatibilité de l'éditeur	≈	**Matériel standard** multi-constructeurs, x86 ou ARM

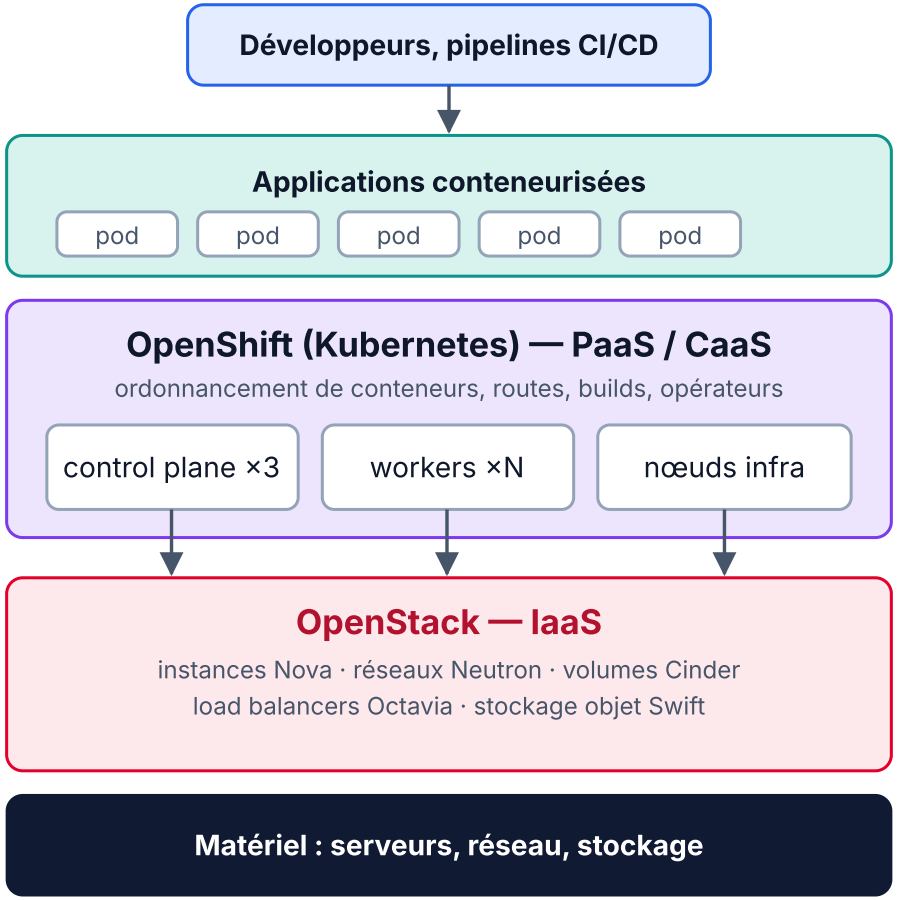
> Les couches se correspondent presque une à une. La différence tient au **découpage** : un produit intégré d'un côté, des services indépendants reliés par des API de l'autre.

OpenStack et VMware : deux philosophies

Dimension	VMware vSphere / Cloud Foundation	OpenStack
Nature	Produit propriétaire intégré, conçu par un éditeur unique	Projet libre modulaire, porté par une communauté et plusieurs distributeurs
Modèle économique	Abonnement par cœur ; depuis le rachat par Broadcom (fin 2023), fin des licences perpétuelles et offres regroupées en bundles	Aucune licence ; le coût porte sur les compétences, l'exploitation et un éventuel contrat de support
Public visé	Administrateurs d'infrastructure qui opèrent des machines pour le compte des métiers	Équipes projets autonomes qui consomment par API ; l'équipe plateforme opère le cloud
Charges de travail	Applications traditionnelles, machines durables, haute disponibilité transparente	Applications conçues pour le cloud, instances jetables, automatisation ; sait aussi héberger du traditionnel
Matériel	Liste de compatibilité matérielle à respecter	Matériel standard, multi-constructeurs ; choix libre du stockage et du réseau
Exploitation	Expérience intégrée, outillage guidé, compétences répandues	Assemblage plus exigeant : Linux, réseau, stockage distribué, automatisation
Multi-tenant	Ajouté par des produits complémentaires	Natif : projets, quotas, réseaux isolés dès l'origine

Migrer de VMware vers OpenStack ne se résume pas à convertir des disques (`virt-v2v` le fait très bien). Il faut aussi revoir les habitudes : réseau à la demande, sauvegardes, haute disponibilité, processus de demande.

OpenStack et OpenShift : deux étages différents de la pile



	OpenStack	OpenShift
Modèle	IaaS	PaaS / CaaS
Unité gérée	Machine virtuelle, réseau, volume	Conteneur, pod, application
Public	Administrateurs système, équipes infrastructure	Développeurs, équipes applicatives
Moteur	Services Python + KVM	Kubernetes + opérateurs
Éditeur	Communauté, multi-distributeurs	Red Hat (amont libre : OKD)
Isolation	Projet, réseau virtuel, hyperviseur	Namespace, NetworkPolicy, noyau partagé

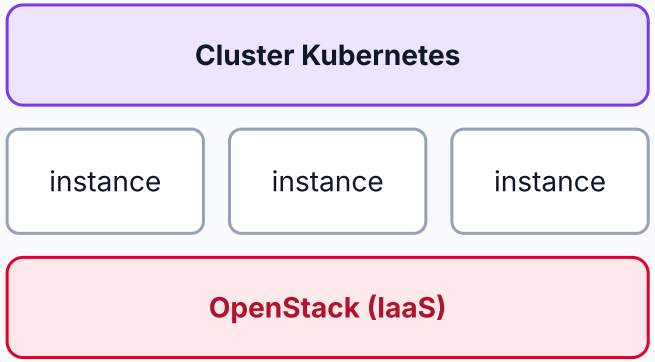
Ce ne sont **pas des concurrents directs** : OpenShift a besoin de machines pour ses nœuds, OpenStack les lui fournit par API.

Zone de recouvrement : OpenShift Virtualization (KubeVirt) exécute des VM dans des pods. Une option pour un parc de VM modeste géré avec les outils Kubernetes ; OpenStack reste plus complet pour un IaaS multi-tenant à grande échelle.

OpenStack et Kubernetes : trois façons de les combiner

A Kubernetes sur OpenStack

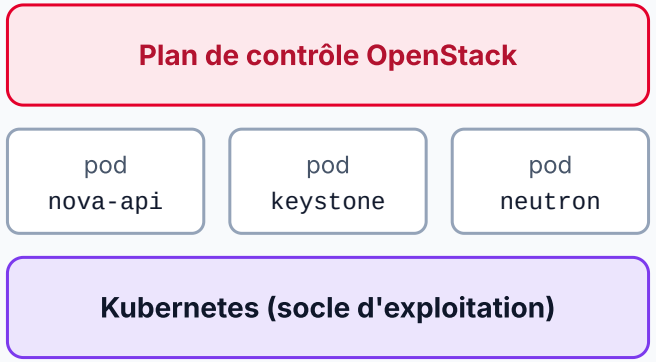
Le cas le plus courant : les clusters tournent dans des instances du cloud.



Création : Magnum ou Cluster API
Volumes : pilote CSI Cinder
Services : LoadBalancer via Octavia
Réseau : ports et sous-réseaux Neutron

B OpenStack sur Kubernetes

Le plan de contrôle d'OpenStack est lui-même déployé comme une application conteneurisée.

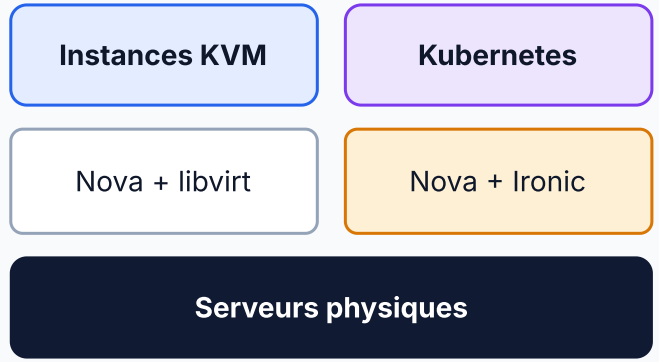


Outils : OpenStack-Helm, Sunbeam, RHOSO, Mirantis MOSK

Intérêt : redémarrage automatique, mises à jour progressives

C Côte à côte sur bare metal

Ironic fournit des serveurs physiques par API ; Kubernetes s'y installe sans hyperviseur.



Usages : GPU, calcul intensif, télécoms, bases de données exigeantes

Intérêt : mêmes API, mêmes réseaux, mêmes quotas pour tout

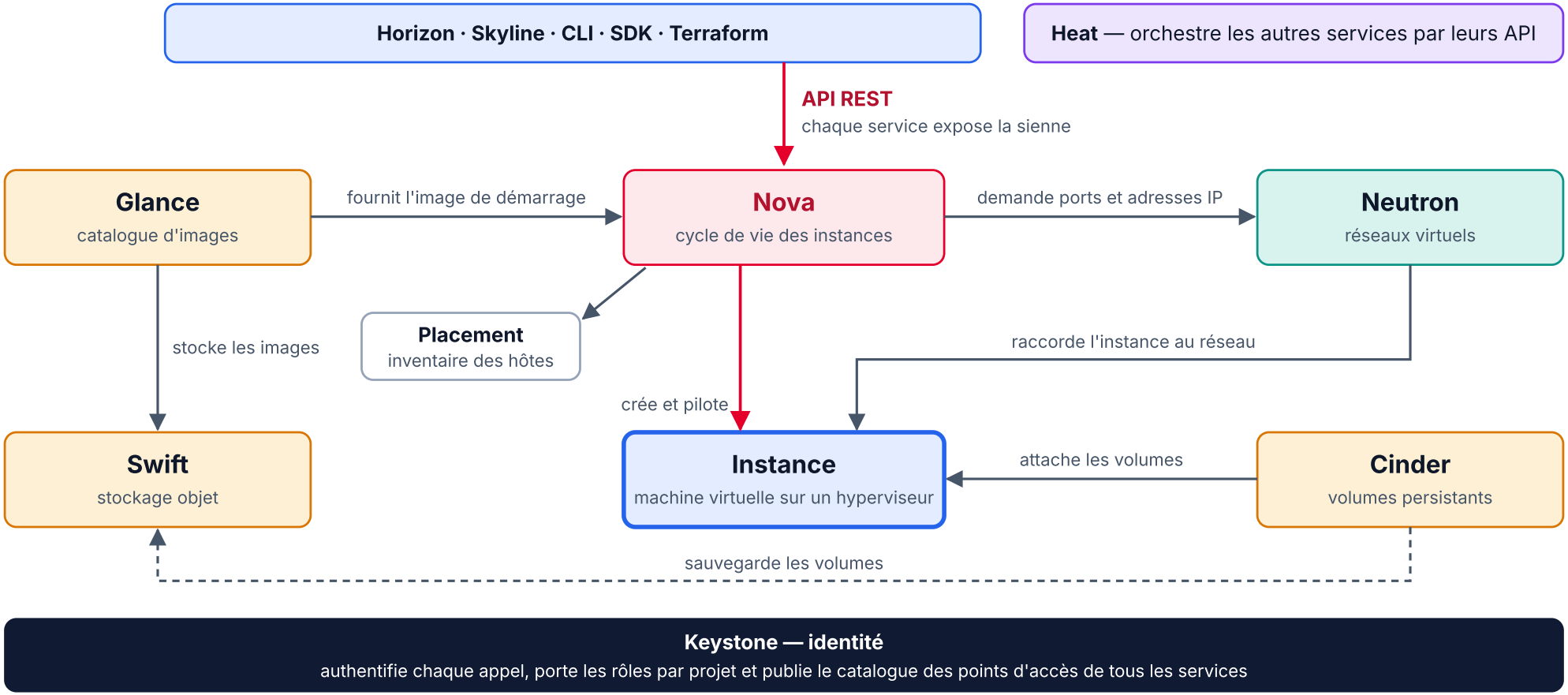
OpenStack gère l'**infrastructure** (machines, réseaux, disques) ; Kubernetes gère les **applications** (conteneurs). Dans la plupart des clouds privés, on trouve les deux, et souvent les schémas A et B simultanément.

Synthèse : quelle solution pour quel besoin ?

	OpenStack	VMware Cloud Foundation	OpenShift	Cloud public
Nature	IaaS libre, privé ou public	Virtualisation et cloud privé propriétaires	Plateforme de conteneurs (PaaS / CaaS)	IaaS, PaaS et SaaS loués
Unité de base	Instance, réseau, volume	Machine virtuelle	Conteneur, application	Tout, de la VM à la fonction
Licence	Apache 2.0, gratuite	Abonnement par cœur	Abonnement (OKD libre)	Paiement à l'usage
Compétences clés	Linux, réseau, stockage, automatisation	Administration vSphere	Kubernetes, conteneurs, CI/CD	Architecture cloud, FinOps
Point fort	Ouverture, multi-tenant natif, coût à grande échelle	Intégration, maturité, confort d'exploitation	Productivité des développeurs	Vitesse, étendue du catalogue
Point de vigilance	Exigeant à exploiter	Coût et dépendance à l'éditeur	Nécessite une infrastructure dessous	Coût à l'échelle, souveraineté
Idéal pour	Cloud privé souverain, opérateurs, fournisseurs de cloud	Parc de VM traditionnelles déjà en place	Applications conteneurisées, microservices	Démarrage rapide, charges variables

À retenir : ces solutions se combinent plus qu'elles ne s'excluent. Un schéma très répandu : **OpenStack** pour l'IaaS privé, **Kubernetes ou OpenShift** par-dessus pour les applications, et le **cloud public** en complément pour le débordement ou les services managés.

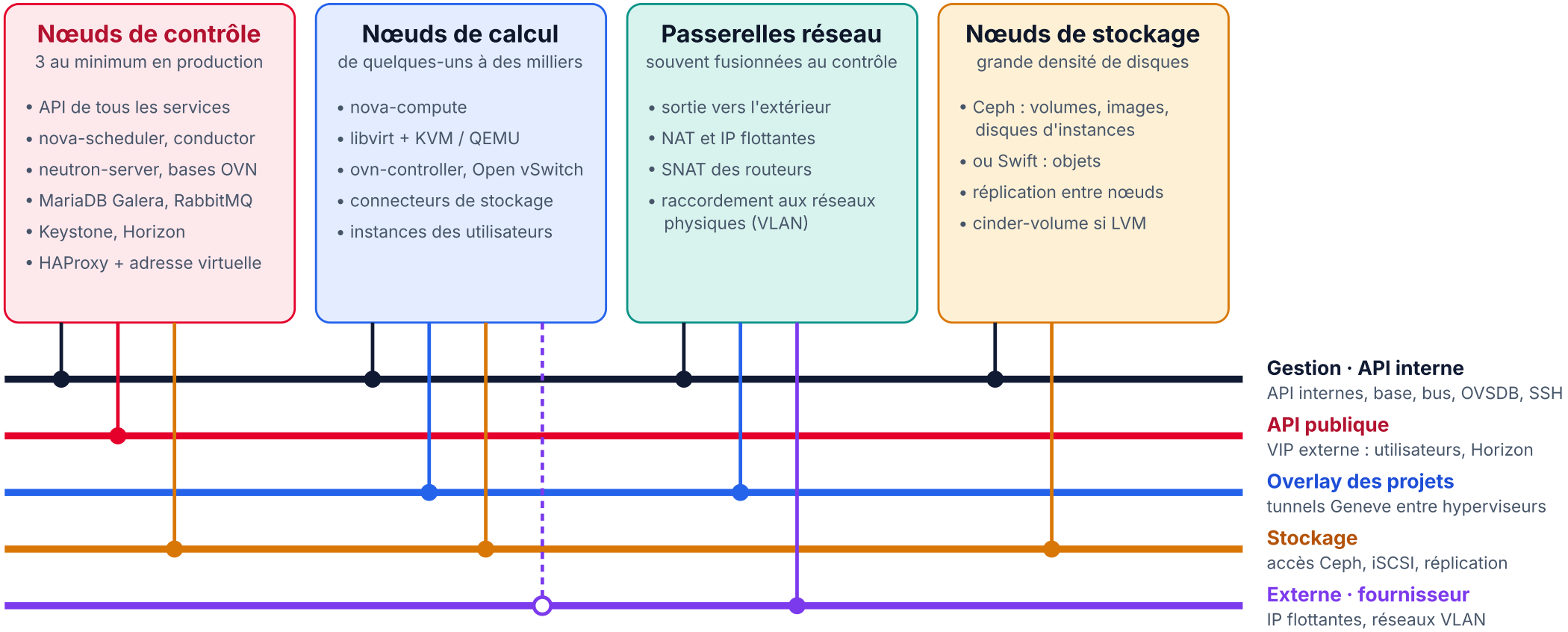
Vue d'ensemble : qui fournit quoi à une instance



Lecture du schéma : au centre, Nova crée l'instance : il obtient l'image de Glance, le réseau de Neutron, les volumes de Cinder et l'hôte par Placement. Keystone, en bas, intervient dans chaque échange — il authentifie et publie l'adresse de tous les services ; Heat et les outils du haut ne font qu'appeler ces API.

 Approfondi au module 2

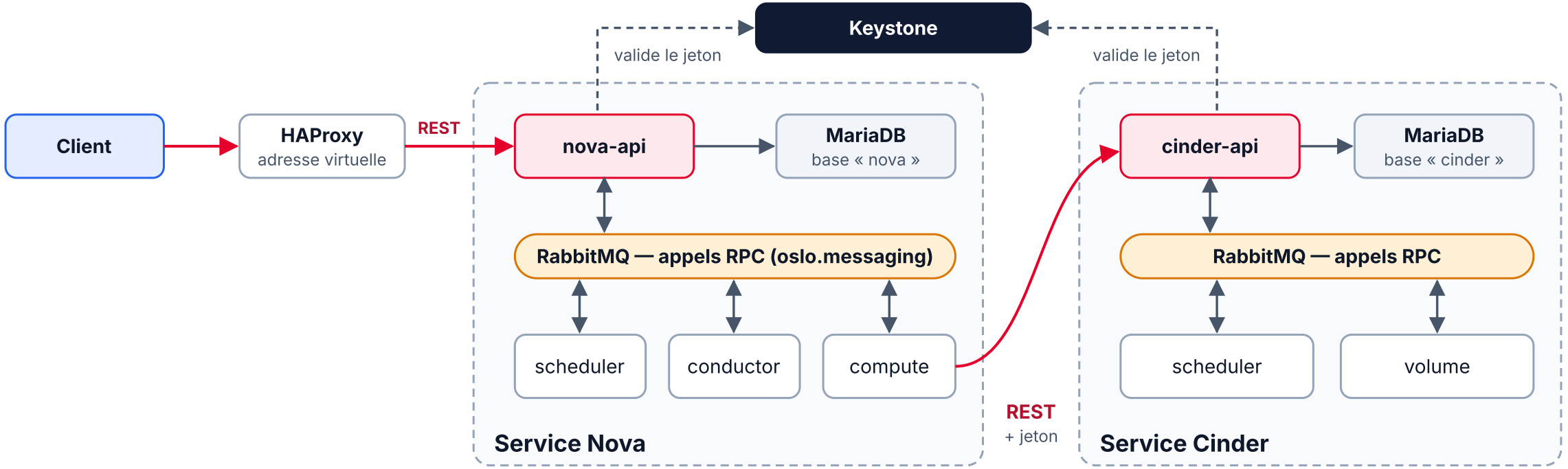
Architecture physique : types de nœuds et réseaux



Séparer les réseaux isole les flux sensibles (gestion, stockage) du trafic des utilisateurs. **Vue simplifiée :** hors bande, provisionnement et réplication du stockage sont détaillés au module 2. Pointillés : le calcul ne rejoint l'externe que si les IP flottantes sont distribuées. Dans un lab, tout peut cohabiter sur une machine.

 Approfondi au module 2

Deux modes de communication : REST entre services, RPC à l'intérieur



Entre services : API REST

Nova ne lit jamais la base de Cinder : il appelle `cinder-api` en HTTP, avec un jeton Keystone, comme le ferait n'importe quel client. Chaque service est donc remplaçable et testable isolément.

Dans un service : RPC asynchrone

Les processus d'un même service échangent par le bus de messages. L'API répond immédiatement (« accepté ») ; le travail se poursuit en arrière-plan, d'où les états transitoires comme `BUILD`.

Un état par service

Chaque service possède sa propre base de données et son propre schéma. Les processus sont sans état : on les multiplie derrière le répartiteur de charge pour la disponibilité et la montée en charge.

M1 · ARCHITECTURE ET COMPOSANTS Approfondi au module 3

Le socle technique partagé par tous les services

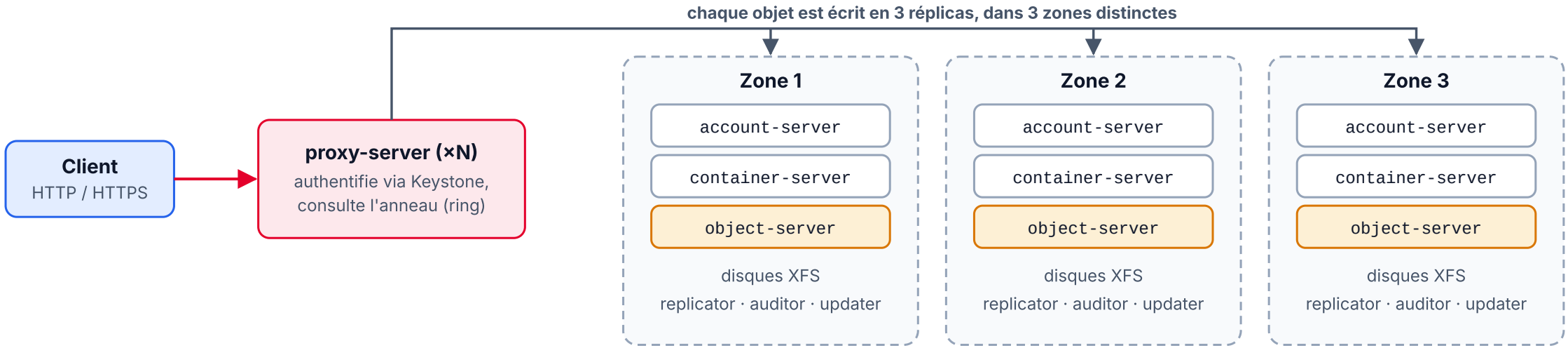
Brique	Technologie de référence	Rôle	Si elle tombe…
Base de données	MariaDB + Galera (3 nœuds)	État de chaque service : instances, réseaux, volumes, utilisateurs	Plus aucune opération d'API possible
Bus de messages	RabbitMQ (files à quorum)	Appels RPC internes et notifications, via la bibliothèque oslo.messaging	Les opérations restent bloquées en cours
Cache	Memcached	Jetons validés, sessions Horizon, résultats fréquents	Ralentissement, charge accrue sur Keystone
Répartiteur de charge	HAProxy + Keepalived ou Pacemaker	Adresse virtuelle unique devant les API, terminaison TLS, bascule	API inaccessibles depuis l'extérieur
Serveur d'applications	Apache mod_wsgi ou uWSGI	Exécute le code Python des API	Le service concerné ne répond plus
Synchronisation horaire	Chrony (NTP)	Horloge commune : validité des jetons, cohérence des clusters	Rejets d'authentification, incohérences

Les bibliothèques Oslo donnent le même comportement à tous les services :

- `oslo.config` : fichiers `/etc/<service>/<service>.conf` au même format partout.
- `oslo.policy` : règles d'autorisation par rôle, surchargeables.
- `oslo.messaging`, `oslo.db`, `oslo.log` : bus, base, journaux.

Point essentiel : une panne du plan de contrôle n'arrête pas les instances. Les machines virtuelles continuent de tourner et de communiquer ; on perd seulement la capacité à créer, modifier ou supprimer des ressources.

Swift : le stockage objet distribué



Concept	Définition
Compte	Espace de stockage d'un projet Keystone
Conteneur	Regroupement d'objets (équivalent d'un bucket S3) ; porte les droits d'accès
Objet	Donnée + métadonnées, lue et écrite en entier par HTTP
Anneau (ring)	Table qui associe chaque objet à ses disques par hachage du nom

- Adresse d'un objet : `/v1/AUTH_<projet>/<conteneur>/<objet>`.
- **Aucun point central :** pas de base de données maîtresse ; la capacité croît en ajoutant des nœuds.
- **Cohérence à terme :** la disponibilité prime ; les réplicas convergent en arrière-plan.
- Réplication ×3 ou **codage à effacement** pour réduire le coût au Go.
- API compatible **S3** via le middleware `s3api`.
- Alternative fréquente : **Ceph RGW**, qui expose les mêmes API Swift et S3.

Quatre types de stockage, quatre usages

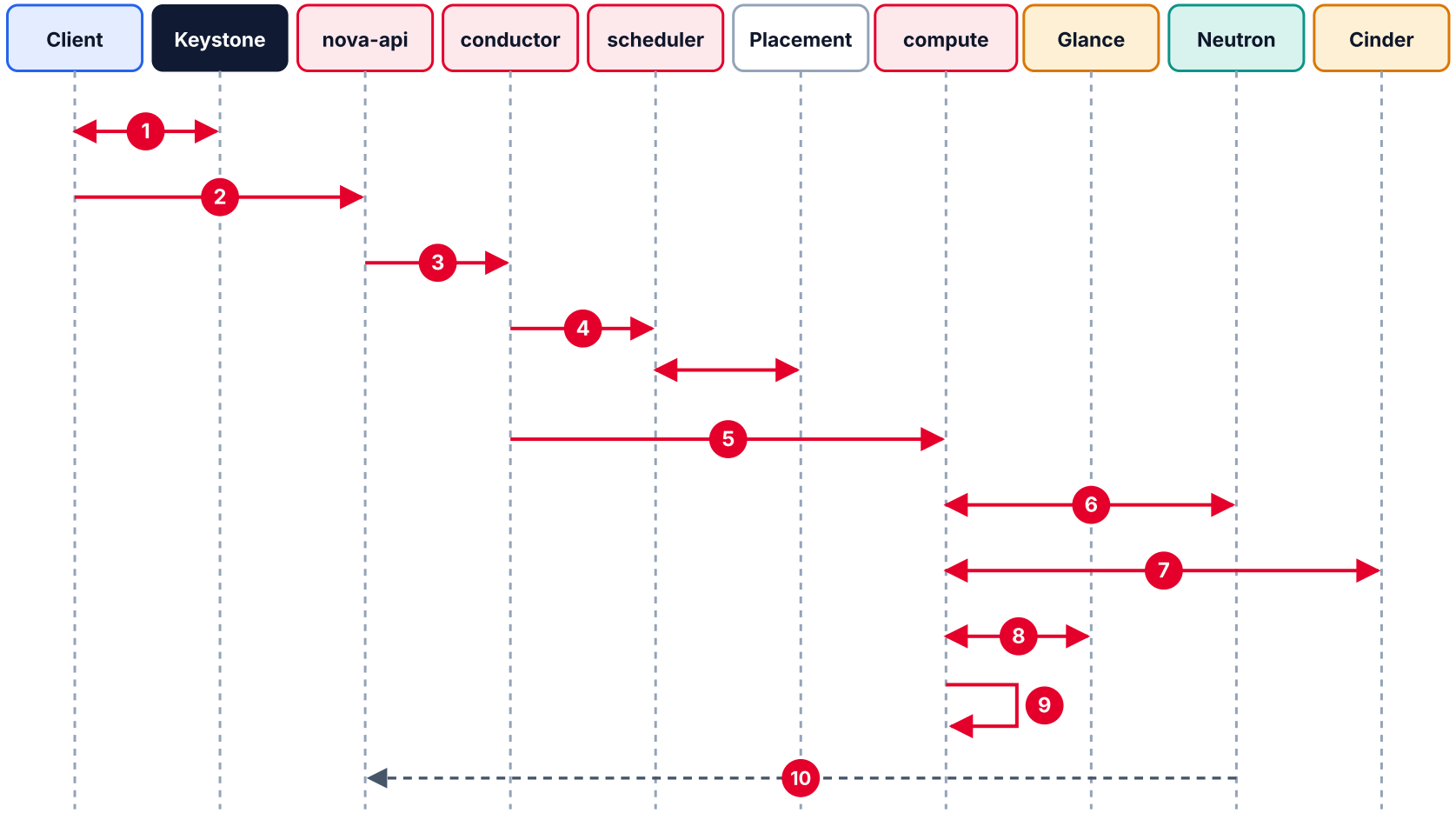
	Éphémère	Bloc	Objet	Fichiers partagés
Service	Nova	Cinder	Swift	Manila
Ce que voit l'utilisateur	Le disque de l'instance	Un disque supplémentaire (`/dev/vdb`)	Une URL HTTP	Un partage réseau à monter
Durée de vie	Celle de l'instance	Indépendante, jusqu'à suppression	Indépendante	Indépendante
Accès	Une seule instance	Une instance (plusieurs en option)	Tout client HTTP, de partout	Plusieurs instances en même temps
Protocole	Disque virtuel local ou Ceph	iSCSI, RBD, Fibre Channel, NVMe-oF	API REST Swift ou S3	NFS, CephFS, SMB
Modification	Lecture et écriture par blocs	Lecture et écriture par blocs	Objet remplacé en entier	Lecture et écriture par fichiers
Usages types	Système, cache, données temporaires	Bases de données, disque système durable	Sauvegardes, médias, journaux, images	Répertoires communs, applications existantes
Équivalent AWS	Instance store	EBS	S3	EFS

Piège classique : supprimer une instance supprime son disque éphémère. Toute donnée à conserver doit être placée sur un volume Cinder, dans Swift ou sur un partage Manila.

Ceph, le backend unificateur : un même cluster Ceph peut servir Glance (images), Cinder (volumes), Nova (disques éphémères), Manila (CephFS) et l'API objet (RGW).

 Approfondi aux modules 2 et 4

Tout assembler : la création d'une instance, étape par étape



1 Authentification : jeton et catalogue.

2 `POST /servers` : image, gabarit, réseau.

3 Jeton, droits et quotas vérifiés ; instance enregistrée à l'état BUILD ; réponse 202.

4 Le scheduler interroge Placement, filtre, pondère et réserve les ressources.

5 Ordre de construction envoyé à l'hôte retenu, par le bus de messages.

6 Création et liaison du port : adresse MAC, IP, groupes de sécurité.

7 Attachement du volume, en cas de démarrage sur volume.

8 Récupération de l'image (ou clone Ceph).

9 libvirt définit et démarre la machine KVM.

10 Neutron signale que le port est actif : l'instance passe à l'état ACTIVE.

Rouge : appels · pointillés : notification

Une simple création d'instance mobilise **six services** et les deux modes de communication : REST entre services (étapes 1, 2, 4, 6, 7, 8, 10) et RPC à l'intérieur de Nova (étapes 3, 4, 5). C'est le meilleur fil conducteur pour diagnostiquer une erreur.

 Approfondi au module 2

Passer à l'échelle : régions, zones, agrégats, cellules



Niveau	Visible par	À quoi il sert
Région	Utilisateur	Séparer des sites : chaque région a ses propres services et points d'accès
Zone de disponibilité	Utilisateur	Isoler les pannes (salle, alimentation) ; choisie à la création de l'instance
Agrégat d'hôtes	Administrateur	Regrouper des hôtes par caractéristique (GPU, SSD, licence) et y diriger certains gabarits
Cellule	Administrateur	Répartir la charge de la base et du bus pour gérer des milliers d'hôtes

Haute disponibilité du plan de contrôle :

- **Trois nœuds de contrôle** au minimum, pour conserver un quorum.
- API sans état, répliquées derrière **HAProxy** et une adresse virtuelle.
- Base **MariaDB Galera** (réplication synchrone) ; **RabbitMQ** en cluster avec files à quorum.
- Côté données : Ceph répliqué ×3, zones de disponibilité, éventuellement Masakari.

02

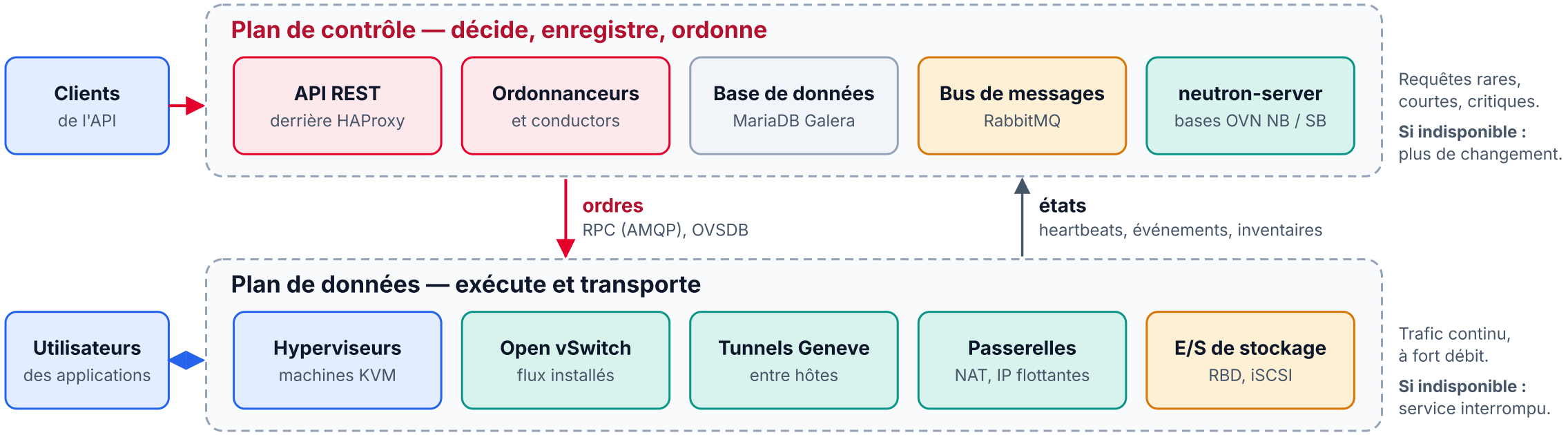
Architecture interne d'OpenStack

Rôle des nœuds, communication entre composants, segmentation réseau du plan de contrôle

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Décrire le contenu d'un nœud de **contrôle**, de **calcul**, de **passerelle** et de **stockage**, processus par processus.
- Suivre une requête API de la VIP jusqu'à la base, puis un appel RPC de file en file.
- Nommer les **réseaux** d'un déploiement, dire quels nœuds y sont raccordés et quels ports y circulent.
- Distinguer les quatre protocoles internes : **REST**, **AMQP (RPC)**, **SQL** et **OVSDB**, et dire qui utilise lequel.
- Expliquer les échanges Nova ↔ Neutron et Nova ↔ Cinder.
- Relier une panne à l'étage qui la provoque.

Plan de contrôle, plan de données : deux circuits distincts



Plan de contrôle

Tout ce qui reçoit une demande, la valide, choisit où la réaliser et en conserve l'état. Il vit sur les **nœuds de contrôle** et ne voit jamais passer le trafic des instances.

Plan de données

Ce qui fait tourner les instances et achemine leurs paquets et leurs E/S. Il vit sur les **nœuds de calcul, de passerelle et de stockage** et continue de fonctionner sans le plan de contrôle.

Plan de gestion

Ce qui sert à exploiter le cloud lui-même : déploiement, accès SSH, supervision, journaux, pilotage matériel (IPMI). Il doit rester joignable quand tout le reste est en panne.

Les familles de nœuds

Contrôle

API, ordonnanceurs, conductors, base de données, bus de messages, bases OVN, HAProxy.

Nombre : 3 (toujours impair)
Perte : plus d'opérations si le quorum disparaît

Calcul

Hyperviseur KVM, nova-compute, ovn-controller, connecteurs de stockage. Héberge les instances.

Nombre : de 1 à plusieurs milliers
Perte : les instances de cet hôte

Passerelle réseau

Sortie vers les réseaux externes, NAT, IP flottantes. Souvent fusionnée avec les nœuds de contrôle.

Nombre : 2 au minimum
Perte : bascule du trafic nord-sud

Stockage

Ceph (MON, MGR, OSD) ou Swift, ou cinder-volume avec LVM. Porte volumes, images et objets.

Nombre : 3 au minimum pour Ceph
Perte : cluster dégradé, sans coupure

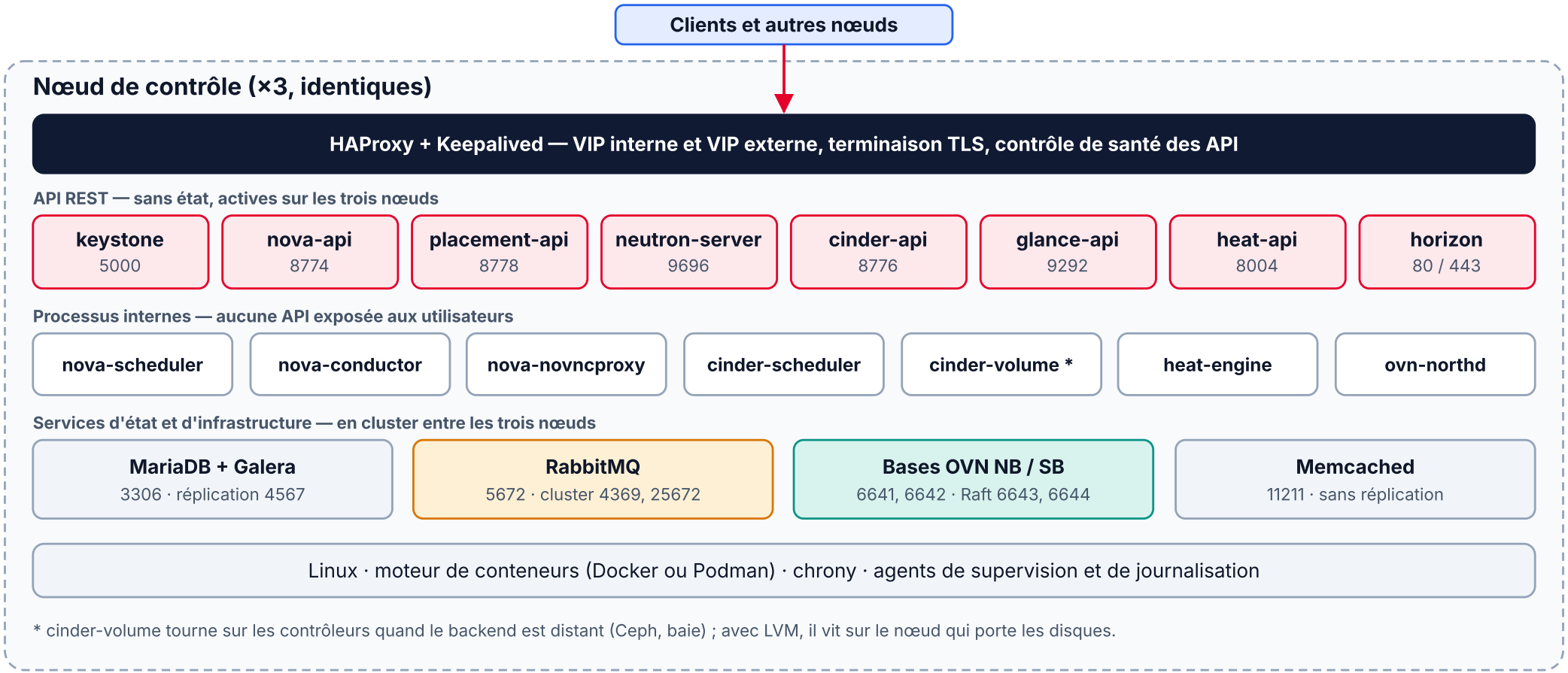
Annexes

Déploiement, bastion, supervision, journaux. Hors du chemin des requêtes des utilisateurs.

Nombre : 1 à 3
Perte : plus de visibilité ni de changement

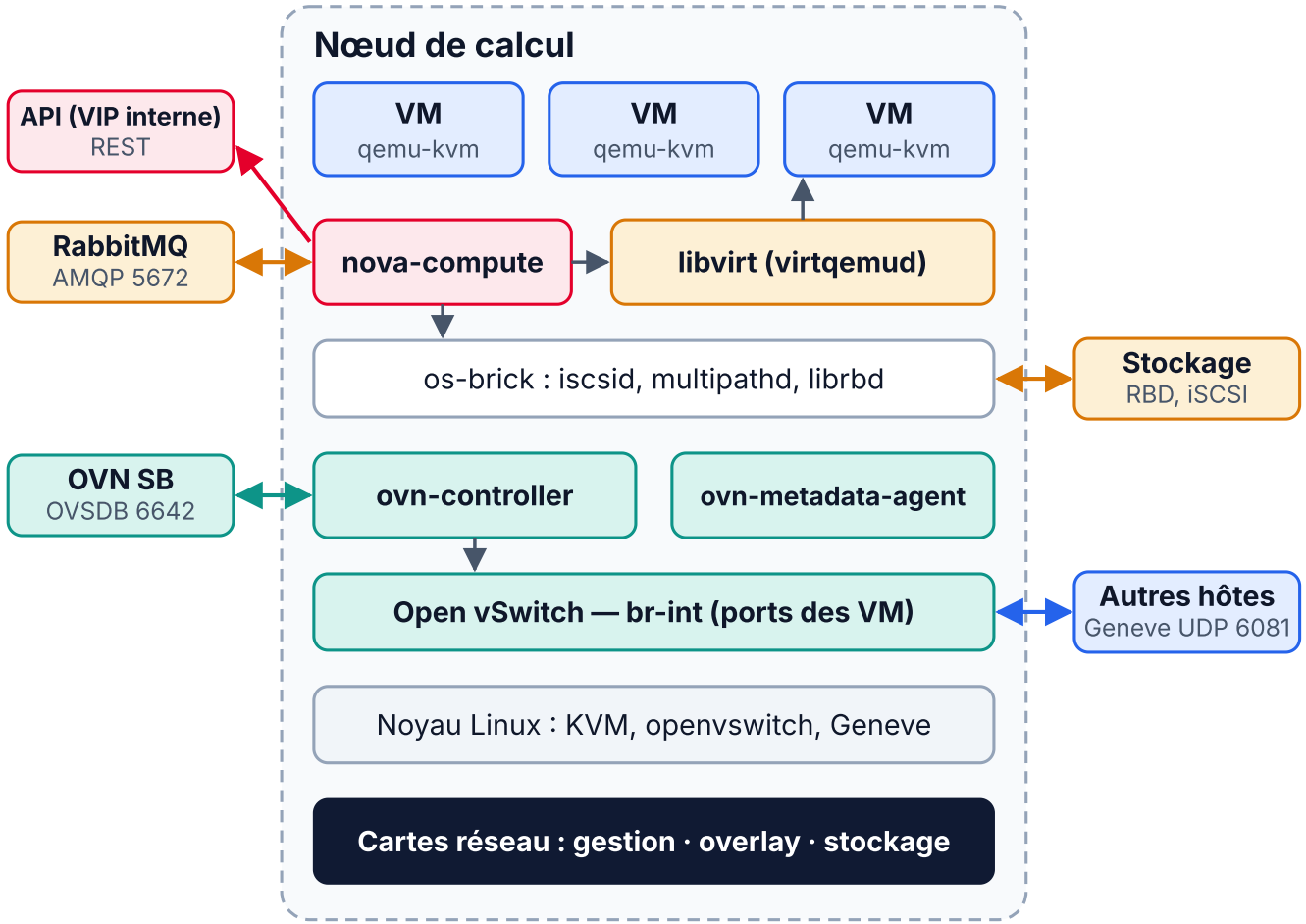
Un rôle est **logique** : on peut en cumuler plusieurs sur une même machine (contrôle + passerelle, calcul + OSD en hyperconvergé) ou au contraire les éclater (base et bus sur des nœuds dédiés). Ce qui compte est de savoir quels processus et quels réseaux chaque machine porte.

Anatomie d'un nœud de contrôle



Lecture du schéma : de haut en bas : HAProxy reçoit tout le trafic et le répartit vers les API, sans état et actives sur les trois nœuds ; les processus internes travaillent derrière elles ; en bas, les services d'état — base, bus, bases OVN — sont en cluster entre les trois nœuds. Ce sont eux qui imposent un nombre impair de contrôleurs.

Anatomie d'un nœud de calcul



Ce qu'il fait

- **nova-compute** reçoit ses ordres par RPC et les traduit en appels libvirt.
- **libvirt** lance un processus `qemu-kvm` par instance.
- **os-brick** connecte les volumes (RBD, iSCSI, FC, NVMe-oF).
- **ovn-controller** programme les flux d'Open vSwitch à partir de la base Southbound.
- **ovn-metadata-agent** relaie les requêtes `169.254.169.254` des instances.

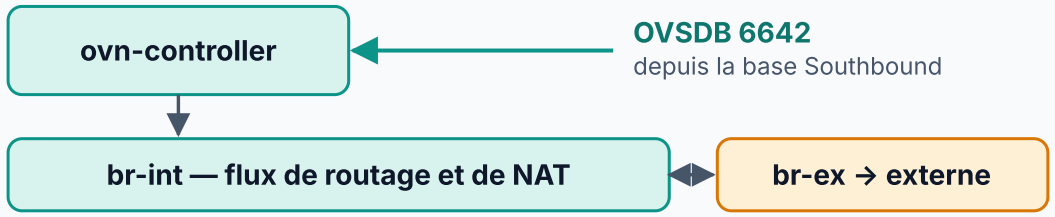
Ce qu'il ne fait pas

- Aucune API exposée aux utilisateurs, aucun accès à la base : voir le motif « conductor », p. 43.

Ports entrants : 22 (SSH), 16509 / 16514 (libvirt), 49152–49215 (migration QEMU), 5900–5999 (consoles VNC), UDP 6081 (Geneve).

Passerelle réseau : chassis OVN ou nœud à agents

ML2/OVN — chassis passerelle



- Aucun agent Neutron ni espace de noms : tout est fait par des flux OVS.
- Rôle activé par l'option ovn-cms-options=enable-chassis-as-gw.
- Chaque routeur est planifié sur plusieurs chassis, par priorité ; BFD détecte la panne et le suivant prend le relais.
- Avec le routage distribué, les IP flottantes sortent directement par l'hyperviseur ; seul le SNAT passe par la passerelle.

ML2/Open vSwitch — nœud réseau à agents



- Agents pilotés par neutron-server à travers RabbitMQ (RPC).
- Agent L3 : un espace de noms qrouter-… par routeur (iptables, keepalived).
- Agent DHCP : un espace de noms qdhcp-… et un dnsmasq par réseau.
- Nœud critique : sans routage distribué (DVR), tout le trafic nord-sud et inter-réseaux y transite.

Dans les deux cas, la passerelle est le point où un réseau de projet (overlay) rejoint un réseau physique. Elle doit donc être raccordée à la fois au réseau **overlay** et au réseau **externe**.

Choix courant : fusionner ce rôle avec les nœuds de contrôle pour les petits clouds, le dédier dès que le débit nord-sud devient important ou que l'on veut isoler la charge.

Trois topologies de référence

A Tout-en-un

Lab, démonstration, développement

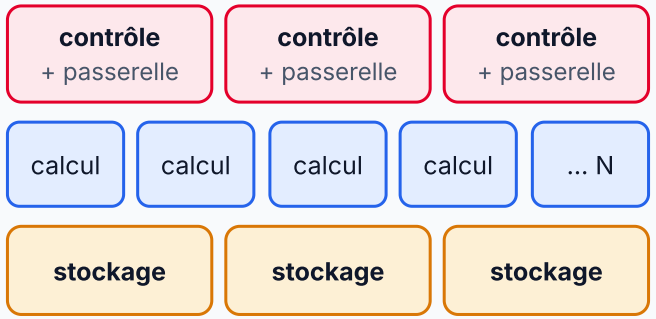
1 machine

contrôle + calcul
+ passerelle + stockage

- DevStack, Kolla all-in-one, Sunbeam mono-nœud
- Aucune haute disponibilité
- Les mêmes processus et les mêmes API qu'en production

B Standard : 3 + N

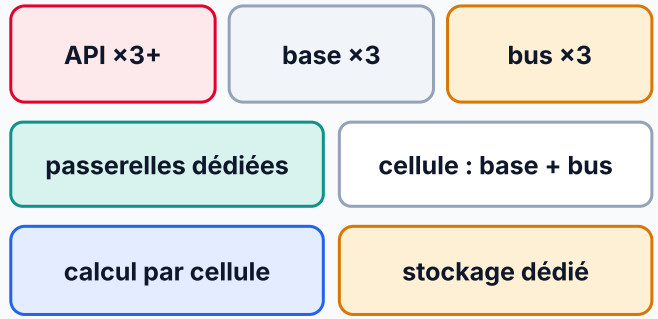
Production courante



- Galera, RabbitMQ, OVN : quorum à trois
- Calcul et stockage croissent séparément
- Convient jusqu'à quelques centaines d'hyperviseurs

C Grande échelle

Plusieurs centaines d'hyperviseurs et au-delà



- Chaque rôle sur ses propres machines
- Cellules Nova : une base et un bus par groupe d'hyperviseurs
- Régions multiples si plusieurs sites

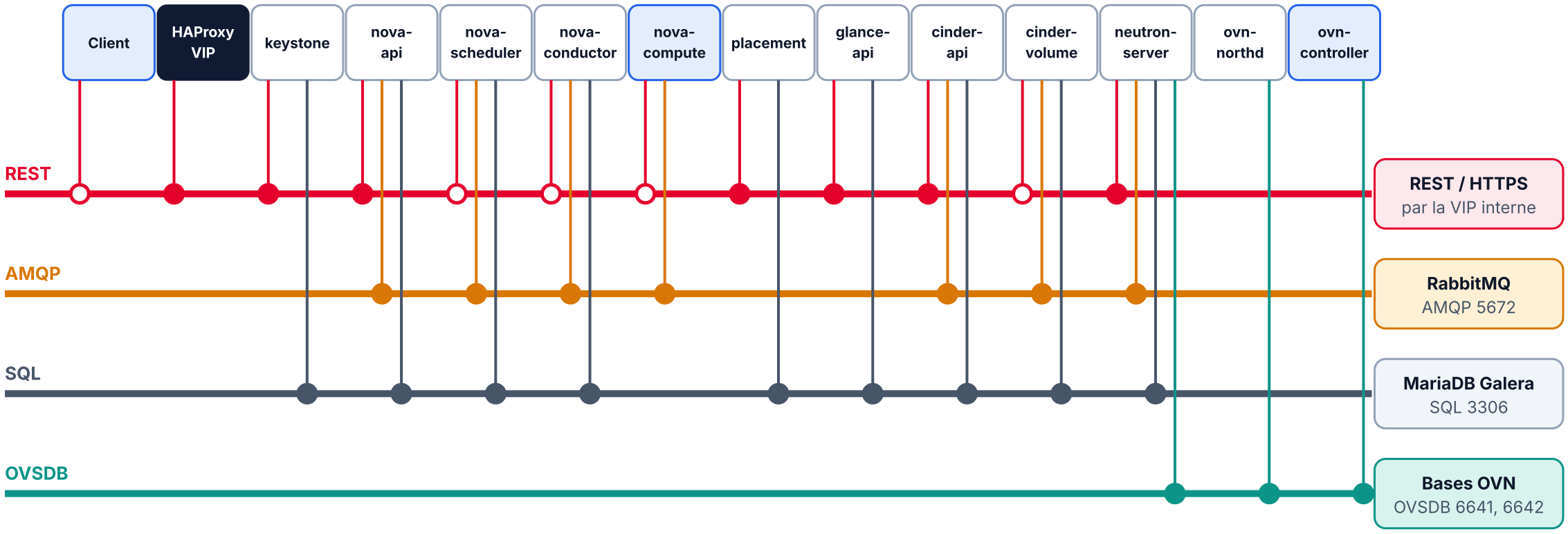
Variante hyperconvergée : chaque nœud de calcul porte aussi des OSD Ceph. Moins de machines, mais calcul et stockage se disputent CPU, mémoire et réseau, et une panne de nœud touche les deux à la fois.

Que se passe-t-il quand un nœud tombe ?

Nœud perdu	Effet immédiat	Ce qui continue de fonctionner	Reprise
1 contrôleur sur 3	Les VIP basculent en quelques secondes ; quelques requêtes en cours échouent	Toutes les API ; Galera, RabbitMQ et OVN gardent leur quorum	Automatique ; réparer puis réintégrer le nœud
2 contrôleurs sur 3	Perte de quorum : base et bus se mettent en retrait, les API ne répondent plus	Les instances, leurs réseaux et leurs volumes	Manuelle : rétablir le quorum (amorçage Galera)
1 nœud de calcul	Ses instances s'arrêtent ; le service passe « down » après 60 s	Tout le reste du cloud	Évacuation vers un autre hôte (manuelle ou Masakari) si le disque est partagé ou sur volume
1 passerelle sur 2	Brève coupure du trafic nord-sud des routeurs qu'elle portait	Trafic entre instances (est-ouest), autres routeurs	Automatique : BFD, bascule vers le chassis suivant
1 nœud OSD Ceph	Cluster « dégradé », puis reconstruction des réplicas	Toutes les E/S, avec une latence accrue	Automatique tant que les réplicas restants suffisent
Nœud de déploiement	Aucun effet sur le cloud en service	Tout	Restaurer inventaire, configuration et secrets

Règle à retenir : la perte du plan de contrôle fige le cloud dans son état ; elle ne l'éteint pas. À l'inverse, la perte d'un nœud du plan de données touche immédiatement des charges utilisateur — d'où l'intérêt de répartir les instances par zones de disponibilité et groupes d'anti-affinité.

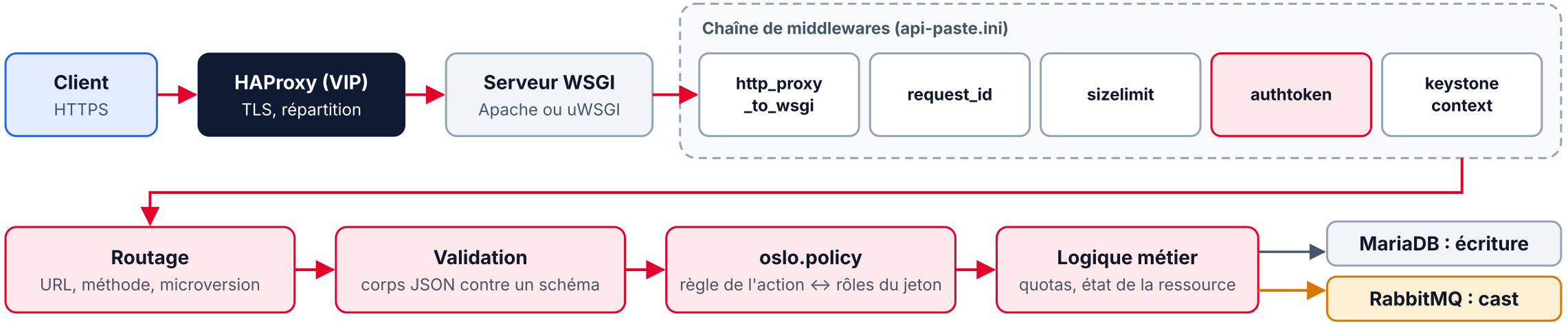
La carte des protocoles : qui parle quoi



nova-compute : REST et AMQP seulement — jamais de SQL, tout passe par nova-conductor.
keystone, placement, glance-api : REST et SQL seulement — aucun bus de messages.
ovn-northd, ovn-controller : OVSDB seulement — neutron-server fait le lien entre les deux mondes.

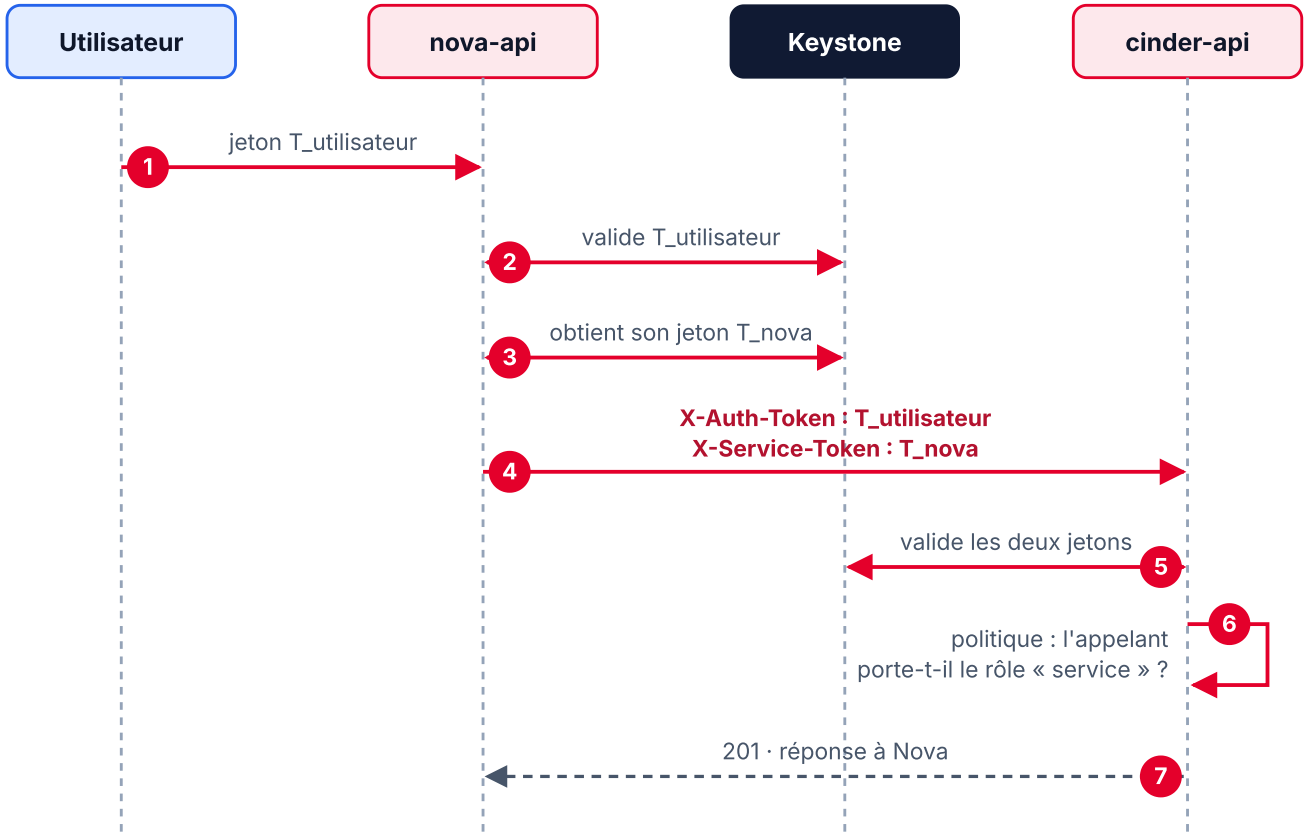
 Codes d'erreur : module 3

Anatomie d'une requête API



Étage	Ce qu'il fait	Ce qu'il transmet à l'étage suivant
HAProxy	Termine TLS, choisit un contrôleur sain	En-têtes `X-Forwarded-For`, `X-Forwarded-Proto`
request_id	Crée l'identifiant de la requête	`req-…`, renvoyé au client dans `X-OpenStack-Request-ID`
sizelimit	Refuse un corps de requête trop volumineux	La requête, inchangée
authtoken	Valide le jeton auprès de Keystone (résultat mis en cache dans Memcached)	Identité : `X-User-Id`, `X-Project-Id`, `X-Roles`
keystonecontext	Construit le contexte de la requête	Identité et identifiant, propagés ensuite dans chaque message RPC
Routage, validation	Associe l'URL à une action, vérifie la microversion et le corps JSON	Action et paramètres validés
oslo.policy	Compare la règle de l'action aux rôles et au projet du jeton	Autorisation accordée
Logique métier	Vérifie quotas et état, écrit en base, émet le RPC	Réponse **202** au client, sans attendre la fin

Authentification entre services : jeton d'utilisateur et jeton de service

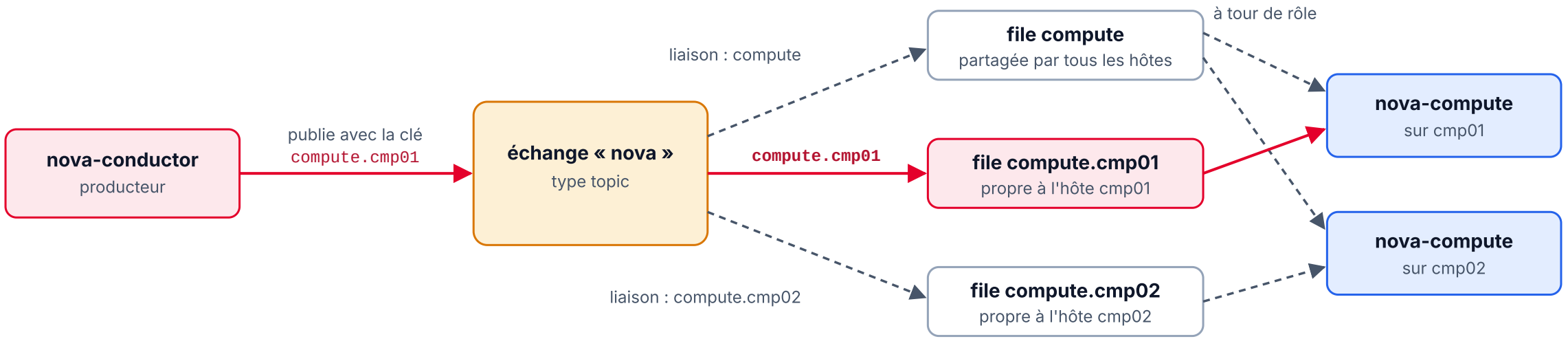


- Le jeton de l'utilisateur dit **pour qui** l'action est faite (utilisateur, projet, rôles).
- Le jeton de service dit **qui relaie** la demande : un compte technique portant le rôle `service`.
- Si le jeton de l'utilisateur expire pendant une opération longue (migration, instantané), l'appel reste accepté grâce au jeton de service.
- Certaines actions de Cinder sont réservées aux appels portant ce rôle : sans jeton de service, l'attachement de volumes échoue.

```
# nova.conf — côté appelant
[service_user]
send_service_user_token = true
auth_type = password
username = nova
project_name = service

# cinder.conf — côté appelé
[keystone_authtoken]
service_token_roles = service
service_token_roles_required = true
```

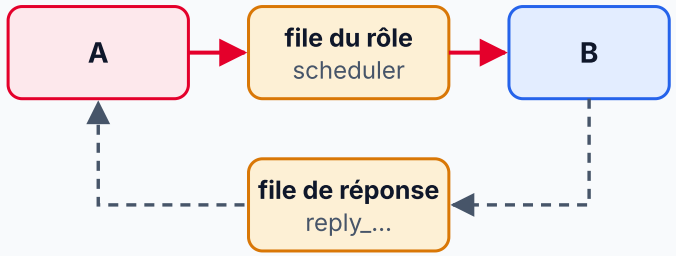
AMQP en cinq notions



Notion	Définition	Dans OpenStack
Échange (exchange)	Point d'entrée du courtier : reçoit les messages et les aiguille, sans les conserver	Un échange de type *topic* par service (`nova` , `cinder` , `neutron` ...), des échanges *fanout* et *direct*
File (queue)	Tampon où les messages attendent un consommateur	Une file par rôle, une par hôte, une par diffusion, une par réponse
Liaison (binding)	Règle reliant un échange à une file selon la clé de routage	Créée par le service lui-même à son démarrage
Clé de routage	Étiquette posée sur le message par l'émetteur	Le nom du rôle (`scheduler`) ou rôle + hôte (`compute.cmp01`)
Consommateur	Processus abonné à une file ; il acquitte chaque message traité	Plusieurs consommateurs sur une même file = répartition de charge

oslo.messaging : call, cast, fanout

call — appel avec réponse



A attend la réponse, au plus rpc_response_timeout (60 s par défaut).

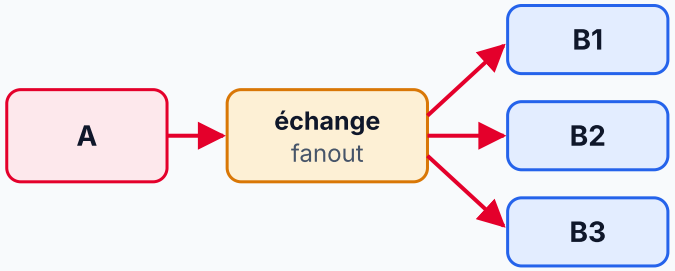
cast — ordre sans réponse



A poursuit aussitôt. Le résultat se lira plus tard dans l'état de la ressource (BUILD → ACTIVE ou ERROR).

C'est le mode des opérations longues : création, migration, attachement.

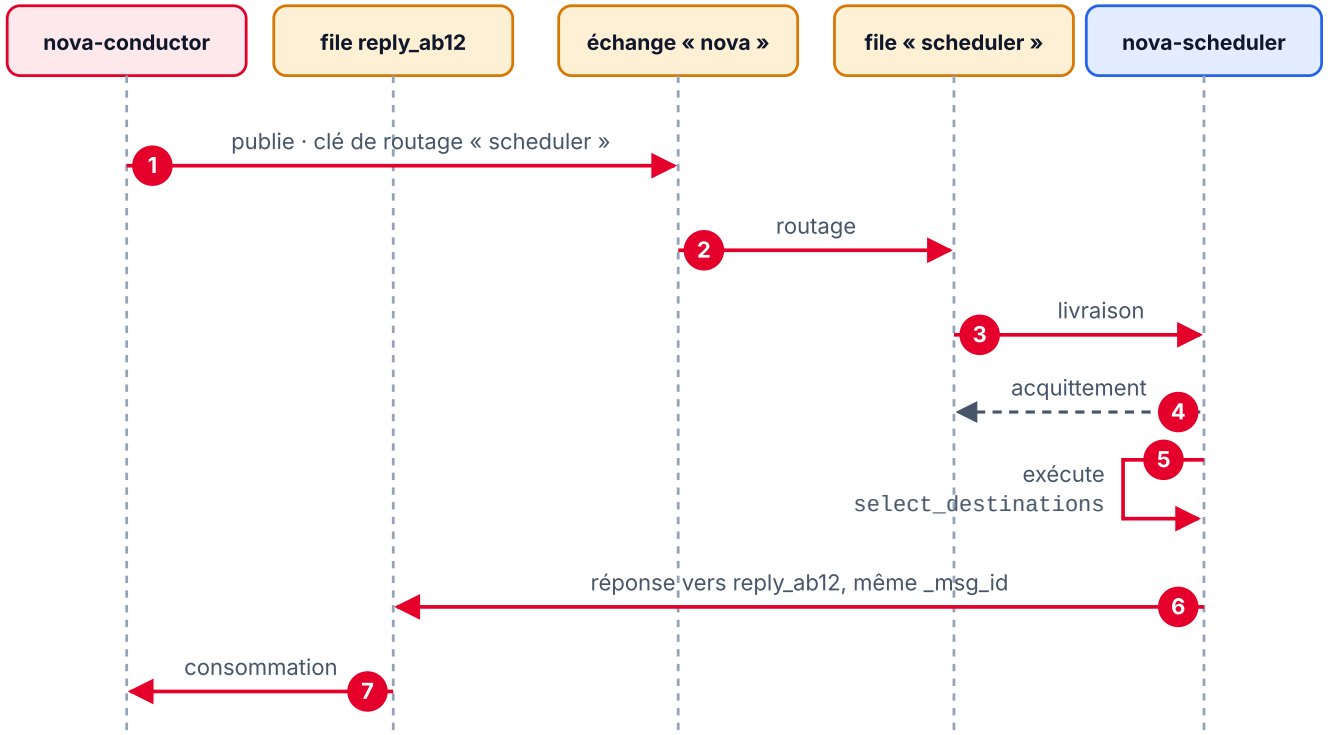
fanout — diffusion à tous



Chaque abonné reçoit sa propre copie, dans une file qui lui est réservée.

Mode	Bloquant ?	Destinataires	En cas de destinataire absent	Exemple
call	Oui	Un seul, pris à tour de rôle ou ciblé par hôte	Exception `MessagingTimeout` chez l'appelant	conductor → scheduler : `select_destinations`
cast	Non	Un seul	Le message attend dans la file ; l'opération paraît figée	conductor → compute : `build_and_run_instance`
fanout	Non	Tous les abonnés du moment	Les absents ne recevront jamais le message	cinder-volume → tous les cinder-scheduler : capacités du backend

Séquence d'un appel RPC « call »

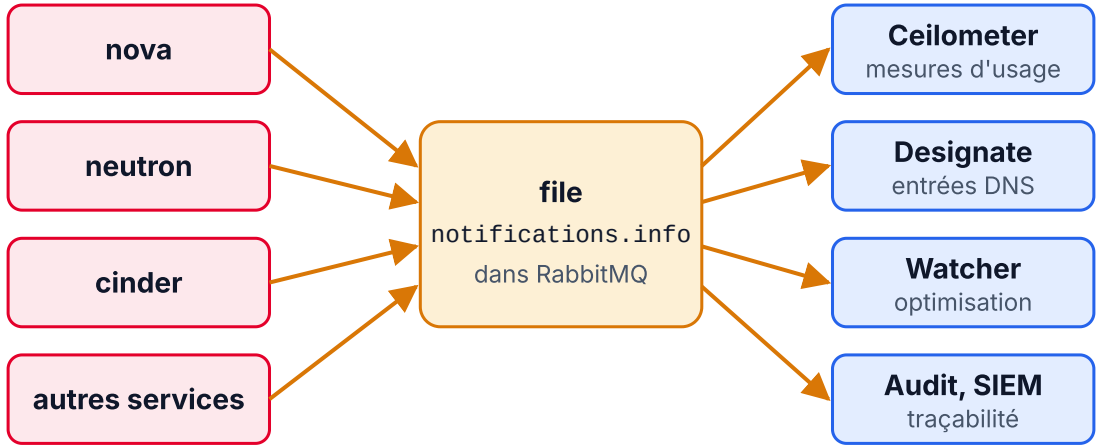


Sans réponse sous rpc_response_timeout : exception MessagingTimeout chez l'appelant.
Trois nova-scheduler abonnés à la même file : RabbitMQ les sert à tour de rôle.

```
# Message publié à l'étape 1 (simplifié)
{
  "method": "select_destinations",
  "args": { "spec_obj": {…}, … },
  "version": "4.5",
  "_msg_id": "c0a1f3…",
  "_reply_q": "reply_ab12…",
  "_context_request_id": "req-5f1c2b7e…",
  "_context_project_id": "c3d4…",
  "_context_user_id": "a1b2…"
}
```

- `method` et `args` : la fonction distante à exécuter.
- `version` : version de l'interface RPC ; elle permet à des services de versions différentes de cohabiter pendant une montée de version.
- `_msg_id` et `_reply_q` : de quoi renvoyer la réponse au bon appelant.
- `_context_*` : identité et identifiant de requête, transportés d'un processus à l'autre.

Les notifications : un second flux, à sens unique



Exemple d'événement émis par Nova

```
event_type : compute.instance.create.end
publisher_id : compute.cmp01
payload : instance_id, tenant_id, flavor, host, state…
```

Chaque étape d'une opération produit un événement « .start » puis « .end » (ou « .error »). L'émetteur n'attend rien en retour et ignore qui écoute.

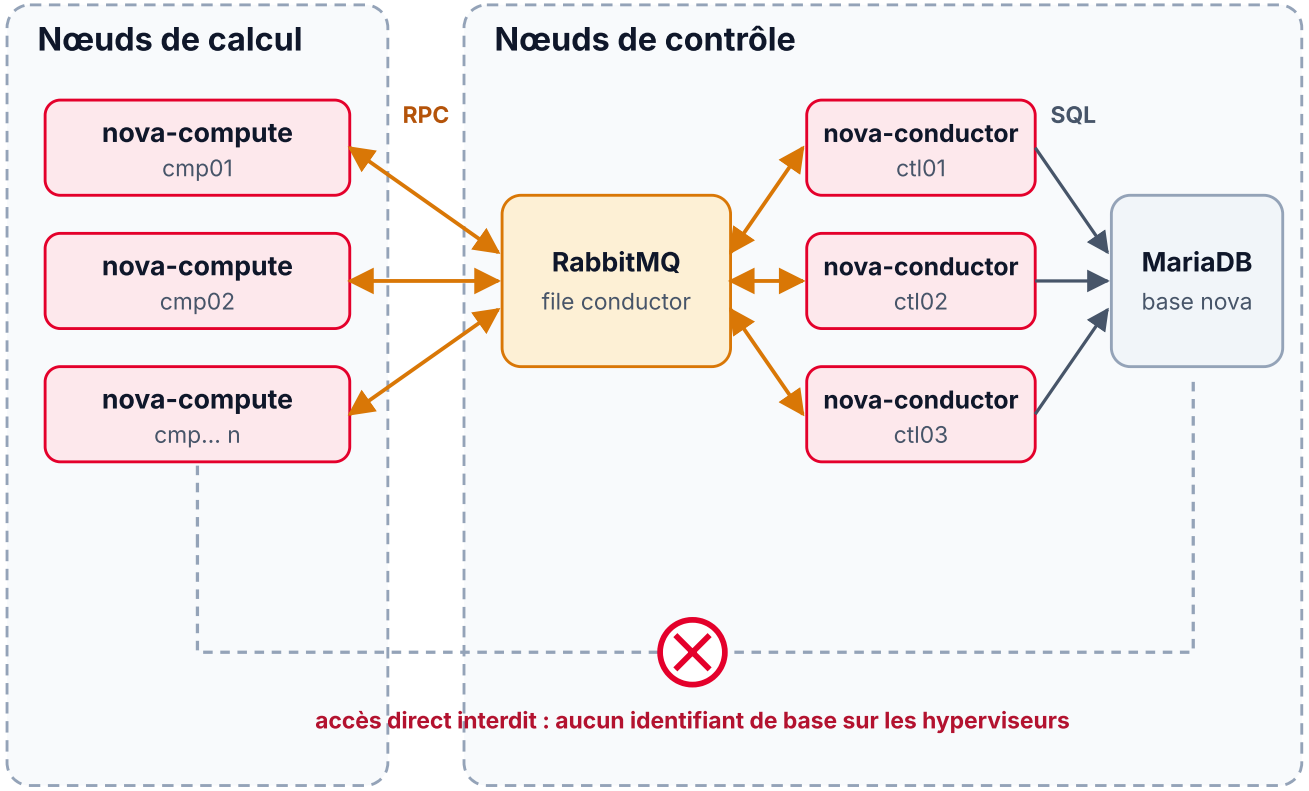
	RPC	Notification
But	Faire exécuter une action	Informer qu'une action a eu lieu
Destinataire	Un processus du même service	Tout consommateur externe
Si personne n'écoute	L'opération échoue ou attend	Les messages s'empilent

```
[oslo_messaging_notifications]
driver = messagingv2   # noop pour ne rien émettre
topics = notifications
# bus distinct possible, pour isoler la charge :
transport_url = rabbit://notif:***@mq-notif:5672//

[notifications]         # nova.conf
notification_format = unversioned
```

Piège : activer les notifications sans consommateur remplit RabbitMQ jusqu'à saturer sa mémoire, et c'est alors le RPC qui s'arrête. Sans Ceilometer ni équivalent : `driver = noop`.

Le motif « conductor » : aucun hyperviseur ne touche à la base



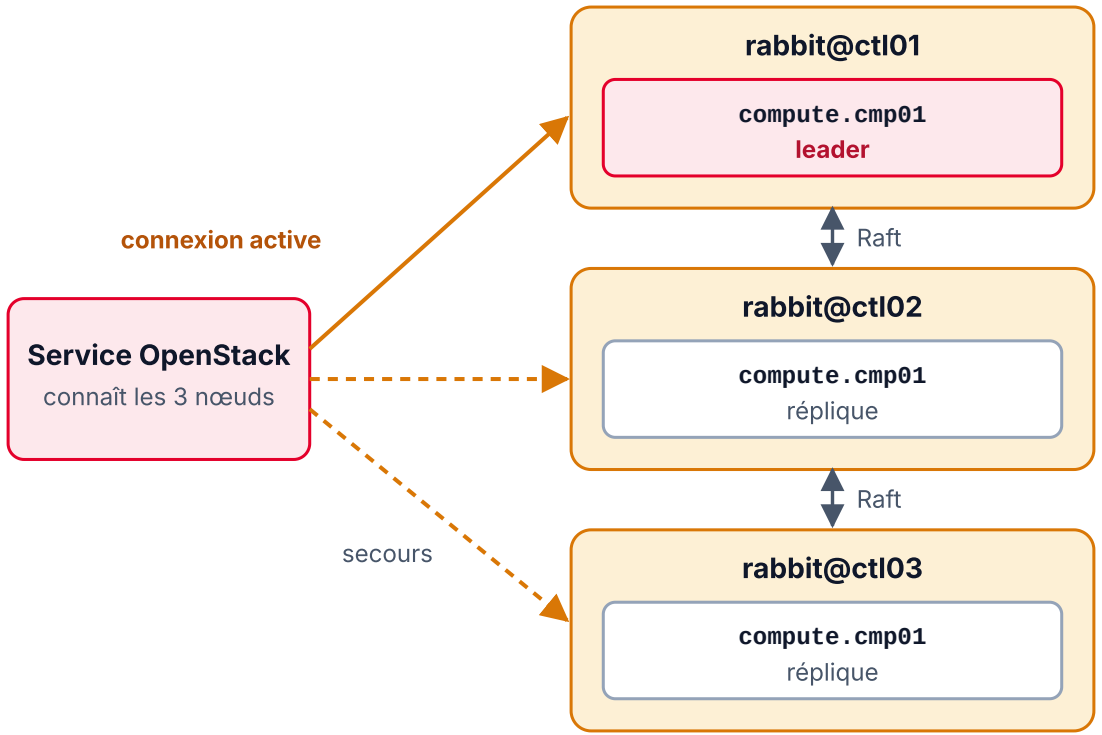
Quatre raisons d'être

- **Sécurité :** un hyperviseur exécute du code de locataires ; s'il est compromis, il ne détient aucun mot de passe de base de données.
- **Montées de version :** le conductor traduit les objets entre versions ; des nova-compute anciens cohabitent avec un contrôle déjà mis à jour.
- **Mise à l'échelle :** quelques conductors mutualisent les connexions SQL de milliers d'hyperviseurs.
- **Orchestration :** il enchaîne les étapes des opérations longues (création, redimensionnement, migration) entre scheduler et computes.

Avec les cellules : un « super-conductor » au niveau de l'API répartit vers le conductor de chaque cellule, qui possède sa base et son bus.

Cinder n'applique pas ce motif : `cinder-volume` accède directement à sa base. Il tourne donc sur des nœuds de confiance.

RabbitMQ en cluster : files à quorum et connexions directes



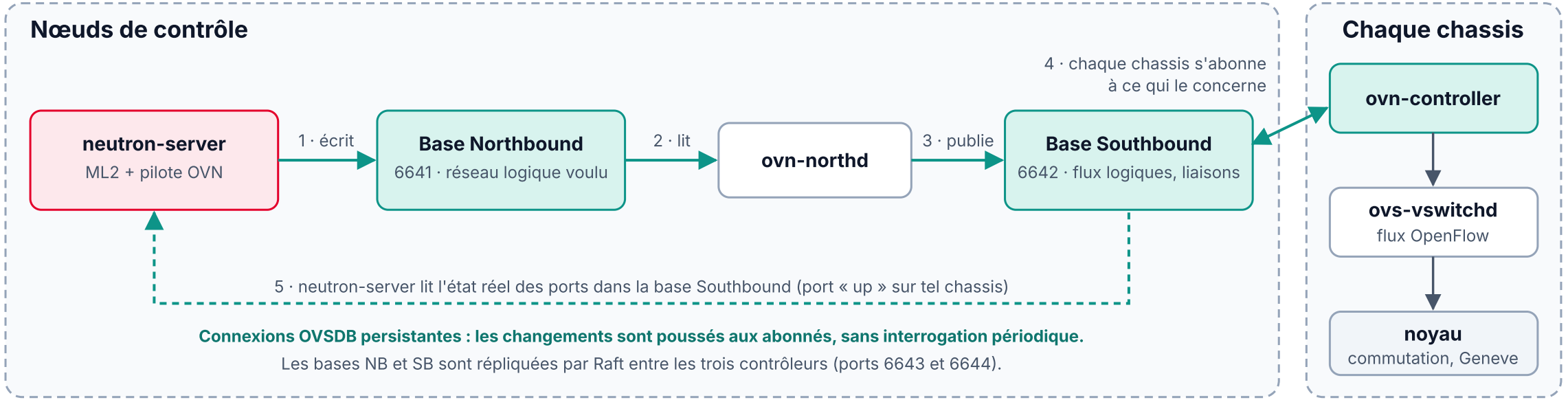
Un message publié n'est confirmé qu'une fois écrit sur la majorité des nœuds.
Si le leader disparaît, une réplique est élue : la file survit sans perte.

```
[DEFAULT]
transport_url = rabbit://openstack:***@10.20.0.11:5672,
    openstack:***@10.20.0.12:5672,
    openstack:***@10.20.0.13:5672//

[oslo_messaging_rabbit]
rabbit_quorum_queue = true
heartbeat_timeout_threshold = 60
ssl = true
```

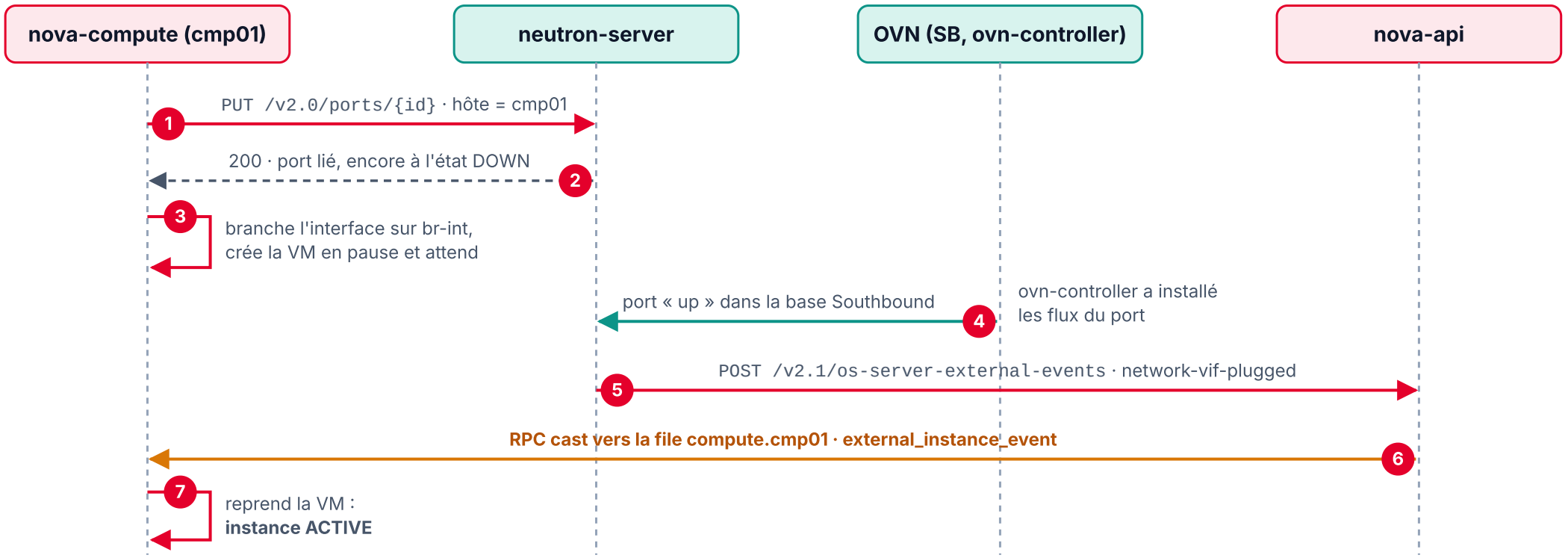
- Les services se connectent **directement** aux nœuds RabbitMQ, sans passer par HAProxy : la bibliothèque gère elle-même la reconnexion.
- **Battements AMQP :** une connexion muette au-delà du seuil est fermée puis rétablie ; un pare-feu qui coupe les connexions inactives produit les mêmes symptômes.

Neutron avec OVN : OVSDB plutôt qu'AMQP



	ML2/OVN (référence)	ML2/Open vSwitch (historique)
Transport des ordres	OVSDB, de base en base ; chaque chassis s'abonne à la base Southbound	RPC AMQP : neutron-server ↔ agents, à travers RabbitMQ
Processus sur les hôtes	ovn-controller, ovn-metadata-agent	Agents OVS, L3, DHCP, métadonnées
Où vit l'état réseau	Base Neutron (référence) + bases NB / SB (traduction)	Base Neutron + état local de chaque agent
Dépendance à RabbitMQ	Faible : un bus en panne ne fige pas le réseau	Forte : sans bus, plus aucune mise à jour des agents
Point de vigilance	Cohérence entre base Neutron et base NB (outil de synchronisation)	Charge RPC qui croît avec le nombre d'agents et de ports

Nova ↔ Neutron : l'événement qui libère le démarrage

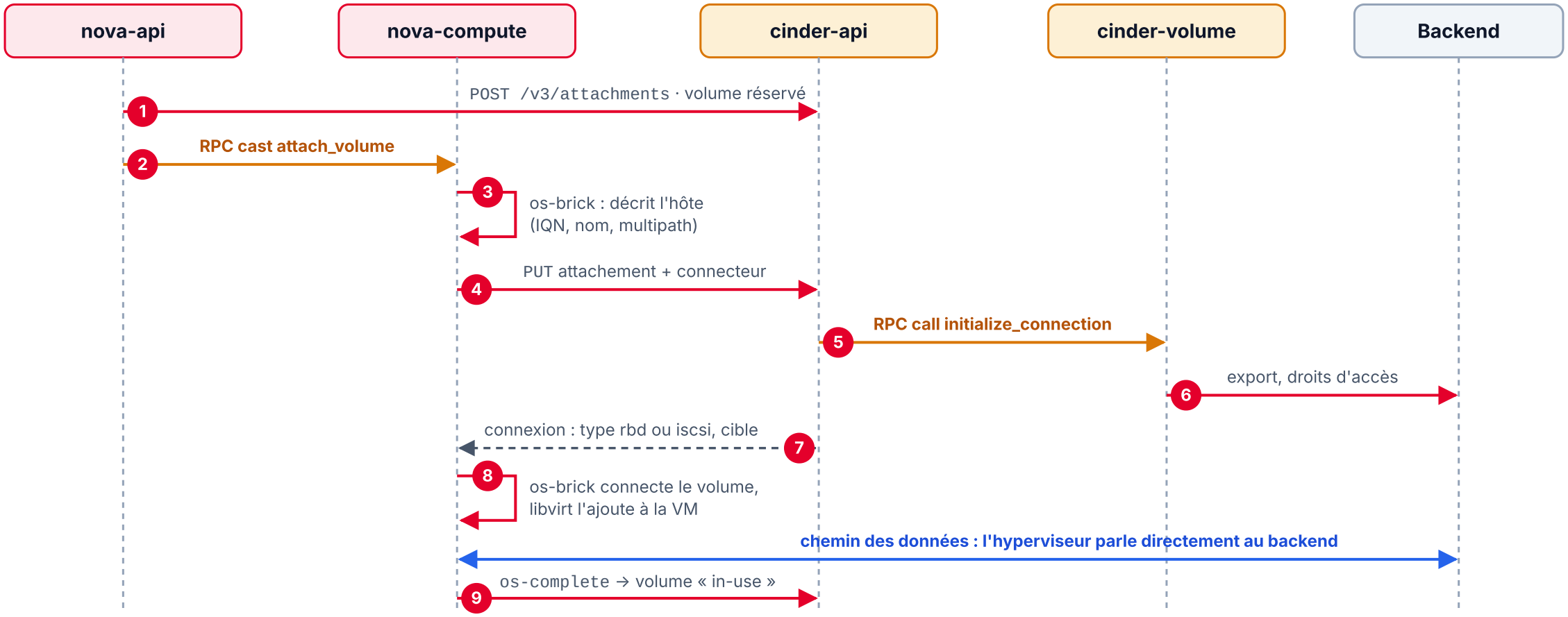


Trois protocoles en une seule opération : REST (1, 2, 5), OVSDB (4), RPC (6).

Si l'événement n'arrive pas dans le délai `vif_plugging_timeout` (300 s par défaut), l'instance passe en ERROR avec « Virtual Interface creation failed ». Causes habituelles : ovn-controller arrêté sur l'hôte, Neutron incapable de joindre l'API Nova.

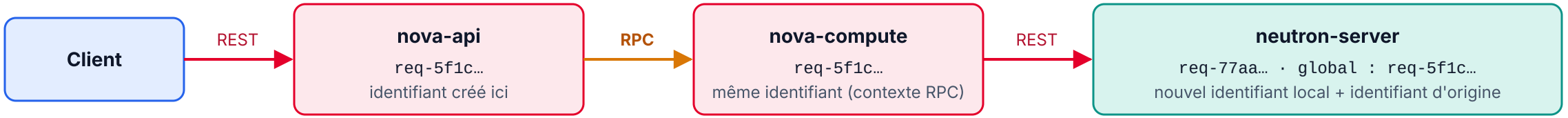
Neutron a donc besoin d'un compte et d'un accès à Nova : section `[nova]` de `neutron.conf`, interface `internal`. C'est l'un des rares appels « à rebours » de l'architecture.

Nova ↔ Cinder : attacher un volume



Lecture : Nova ne sait rien du stockage. Il décrit l'hôte à Cinder (étape 4), Cinder répond comment s'y connecter (étape 7), puis s'efface. Entre services : REST ; à l'intérieur de chaque service : RPC ; pour les données : le protocole du backend, sans intermédiaire.

Suivre une requête d'un service à l'autre



À l'intérieur d'un service, l'identifiant ne change pas. En changeant de service, il devient l'« identifiant global ».

```
# 1. Récupérer l'identifiant renvoyé au client
$ openstack --debug server create … 2>&1 | grep -i x-openstack-request-id
x-openstack-request-id: req-5f1c2b7e-9c41-4d0a-b7c2-3a1e6d0f8a11

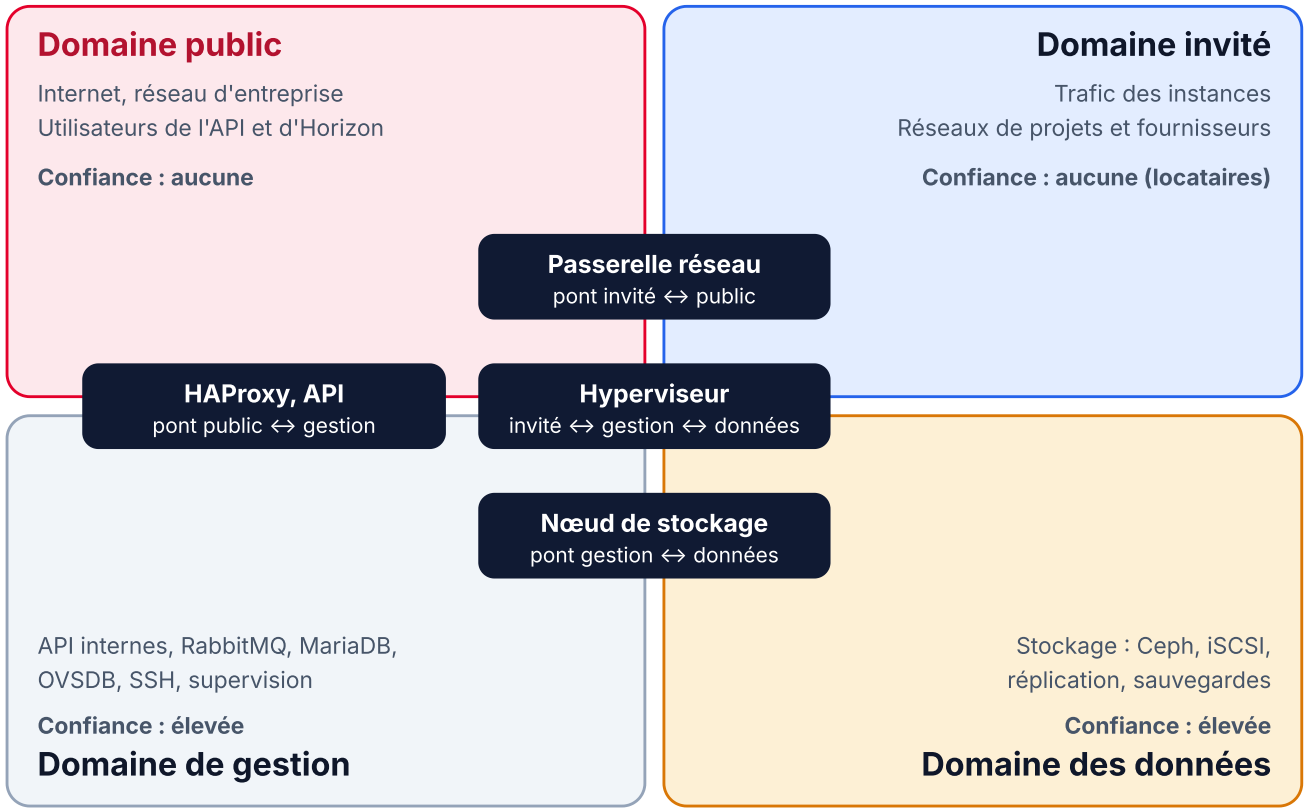
# 2. nova-api.log, sur un contrôleur
INFO nova.osapi_compute.wsgi.server [req-5f1c2b7e-… - a1b2… c3d4…] "POST /v2.1/servers" status: 202

# 3. nova-compute.log, sur l'hyperviseur choisi — même identifiant
INFO nova.compute.manager [req-5f1c2b7e-… - a1b2… c3d4…] [instance: 9d0e…] Creating image(s)

# 4. neutron-server.log — identifiant local, puis identifiant global
INFO neutron.wsgi [req-77aa0c5d-… req-5f1c2b7e-… a1b2… c3d4…] "POST /v2.0/ports" status: 201
```

Méthode : centraliser les journaux de tous les nœuds, puis filtrer sur l'identifiant de requête. On reconstitue ainsi tout le parcours d'une opération, protocole après protocole, sans se connecter à chaque machine.

Pourquoi segmenter : quatre domaines de sécurité

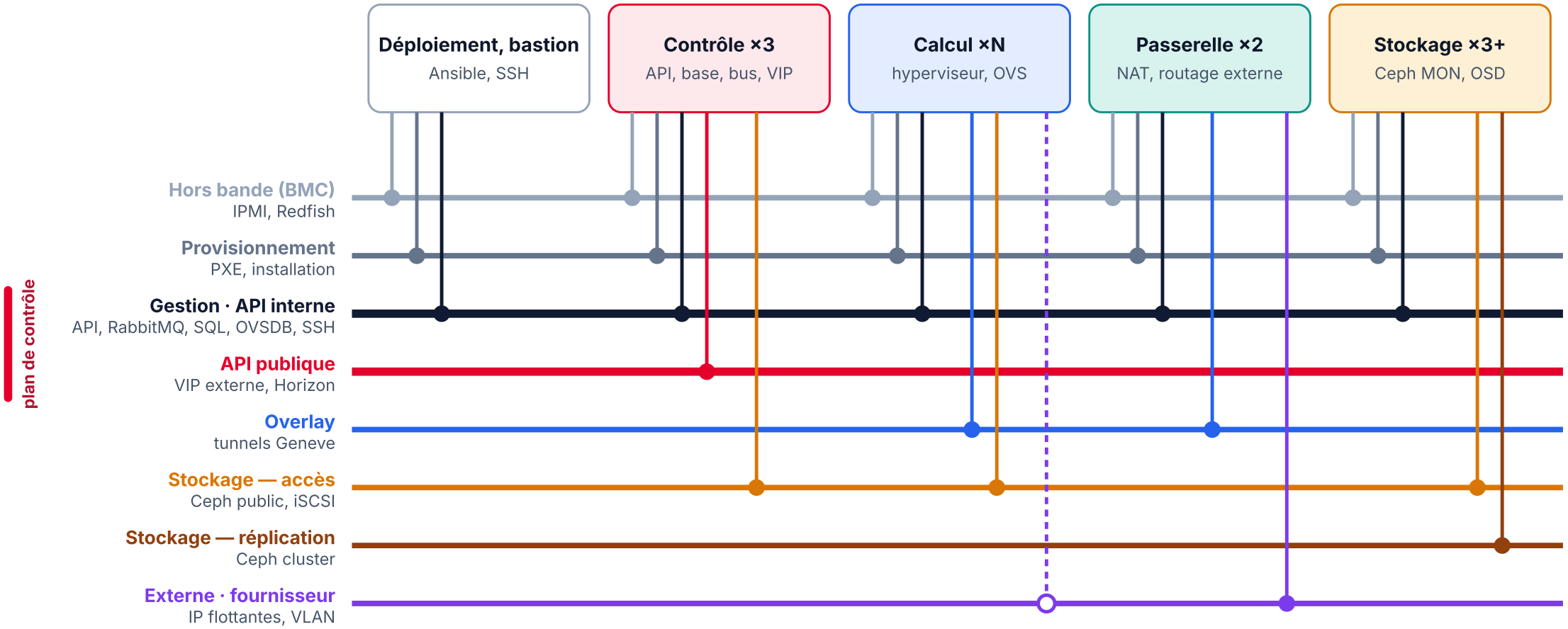


Le guide de sécurité d'OpenStack distingue quatre **domaines**, de niveaux de confiance différents. Les composants à cheval sur deux domaines — les **ponts** — sont les points à durcir en priorité.

- **Sécurité :** une instance compromise ne doit jamais atteindre RabbitMQ, MariaDB ou le SSH des nœuds. Sur un réseau unique, rien ne l'en empêcherait.
- **Performance :** une reconstruction Ceph ou une migration à chaud sature un lien ; les battements de Galera et de RabbitMQ ne doivent pas en pâtir.
- **Limitation des pannes :** une tempête de diffusion sur un réseau de projet ne doit pas faire tomber le plan de contrôle.
- **Exploitation :** un réseau par usage rend les règles de pare-feu lisibles et les captures de trafic exploitables.

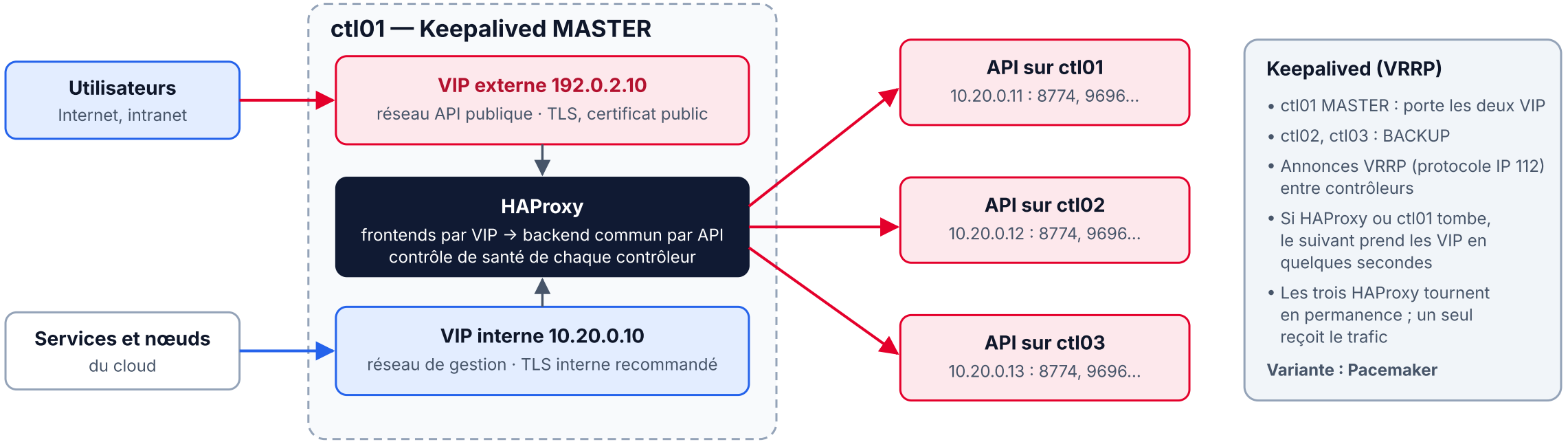
> Segmenter, c'est attribuer un **VLAN et un sous-réseau par famille de flux**, puis ne raccorder chaque nœud qu'aux réseaux dont son rôle a besoin.

Quel nœud sur quel réseau



Un nœud n'est raccordé qu'aux réseaux nécessaires à son rôle : un nœud de calcul n'a **aucune patte** sur l'API publique ni sur la réplication du stockage, et ne rejoint l'externe (pointillés) que si les IP flottantes sont distribuées. Un nœud de stockage ne voit ni l'overlay ni l'extérieur. Seuls les contrôleurs portent l'API publique.

VIP interne et VIP externe : deux entrées pour les mêmes API



VIP externe

Seule adresse du cloud visible des utilisateurs. Elle correspond aux points d'accès `public` du catalogue. On y applique certificat reconnu, limitation de débit, éventuel pare-feu applicatif.

VIP interne

Adresse utilisée par les services entre eux (points d'accès `internal`) et pour MariaDB. Jamais routée hors du réseau de gestion.

Pourquoi deux

Le trafic interne ne sort pas vers le domaine public ; on peut filtrer ou couper l'accès externe sans casser le cloud ; les certificats et les règles diffèrent.

MTU : le coût de l'encapsulation sur le réseau overlay

Trame émise par l'hyperviseur sur le réseau overlay (Geneve, IPv4)



MTU physique	Geneve	VXLAN
1500	1442	1450
9000	8942	8950

MTU annoncée aux instances d'un réseau de projet, en IPv4. Neutron la calcule et la diffuse par DHCP.

```
# neutron.conf
[DEFAULT]
global_physnet_mtu = 9000

# ml2_conf.ini
[ml2]
path_mtu = 9000
```

- **Symptôme d'une MTU incohérente :** le ping et l'ouverture SSH passent, les gros transferts et certaines négociations TLS se figent.
- La MTU doit être **identique de bout en bout** : cartes, agrégats, VLAN, ports des commutateurs.
- Test : `ping -M do -s 8972` entre deux hôtes du réseau overlay ou stockage.

> Le réseau de gestion reste en général à 1500 : il porte de petits messages. Les trames géantes profitent à l'overlay (pour offrir 1500 aux instances) et au stockage (débit de réplication).

03

Les composants système d'OpenStack

File de messages, base de données, serveur d'API : rôle, exposition, éléments de diagnostic

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Expliquer ce que font RabbitMQ, MariaDB Galera et le serveur d'API, et ce qu'ils ne font pas.
- Relier chaque option de configuration d'un service au composant qu'elle pilote.
- Reconnaître dans un journal le message qui désigne le composant fautif.
- Citer leurs **ports**, leurs comptes et les réseaux sur lesquels ils doivent — ou ne doivent pas — être joignables.
- Lancer les **commandes de diagnostic** de chacun et interpréter leur sortie.
- Conduire les reprises courantes : partition du bus, perte de quorum de la base, backend d'API hors service.

Trois composants sous tous les services



Conséquence pratique : savoir diagnostiquer ces trois composants, c'est savoir diagnostiquer la moitié des incidents de **n'importe quel** service — Nova, Neutron, Cinder ou Heat échouent de la même façon quand le bus ou la base ne répond plus.

Dans le lab : ces composants avec Sunbeam

	Exemples de ce module (Kolla-Ansible)	Canonical OpenStack (Sunbeam)
Plan de contrôle	Conteneurs Docker ou Podman sur les contrôleurs	Pods de Canonical Kubernetes, gérés par des charms Juju (modèle `openstack`)
File de messages	RabbitMQ, un nœud par contrôleur	RabbitMQ, charm `rabbitmq-k8s`
Base de données	MariaDB + Galera, derrière HAProxy	MySQL InnoDB Cluster (`mysql-k8s`), joint par MySQL Router (`mysql-router-k8s`)
Entrée des API	HAProxy + Keepalived, deux VIP	Traefik (`traefik-k8s`), sur les adresses du répartiteur de Kubernetes
Bases OVN	Conteneurs sur les contrôleurs	`ovn-central-k8s`, `ovn-relay-k8s` ; MicroOVN sur les nœuds réseau
Hyperviseur	Conteneurs `nova_compute`, `nova_libvirt`, `ovn_controller`	Snap `openstack-hypervisor`
Lancer un outil	`docker exec rabbitmq rabbitmqctl …`	`sudo k8s kubectl exec -n openstack <pod> -c <conteneur> -- …`
Journaux	`/var/log/kolla/<service>/`	`sudo k8s kubectl logs -n openstack <pod> -c <conteneur>` · `juju debug-log -m openstack`
État d'ensemble	État des conteneurs sur chaque nœud	`juju status -m openstack` · `sunbeam cluster list`
Base : état, sauvegarde	`wsrep_%` · `kolla-ansible mariadb-backup`	Actions du charm MySQL : `get-cluster-status`, `create-backup`

Les commandes de ce module suivent Kolla-Ansible. Ce qui ne change pas avec Sunbeam : le rôle de chaque composant, les protocoles (REST, AMQP, SQL), les options oslo et leurs messages d'erreur, les commandes `openstack` et `rabbitmqctl`.

Ce qui change pour la base : réplication de groupe MySQL au lieu de Galera. Mêmes principes (majorité, nœud primaire unique), mais variables `group_replication_%` au lieu de `wsrep_%`.

Rôle : ce que le bus transporte, et pour qui

Trois services rendus

- **Appels RPC :** un processus demande à un autre processus du même service d'exécuter une fonction (créer une instance, attacher un volume).
- **Notifications :** un service annonce qu'un événement a eu lieu, pour qui voudra l'entendre (mesure d'usage, audit).
- **Découverte implicite :** un nova-compute devient joignable dès qu'il s'abonne à sa file. Il n'existe aucun registre d'adresses des processus.

Ce qu'il n'est pas

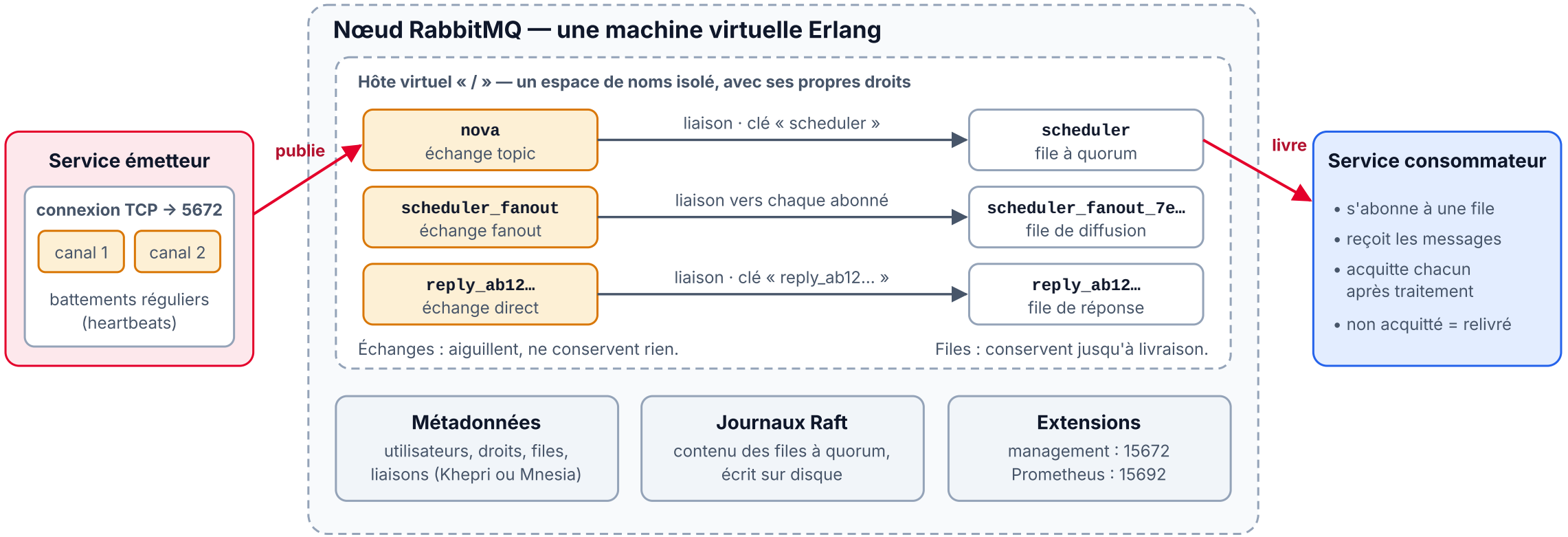
- Un stockage : un message consommé disparaît. L'état durable est en base.
- Un canal **entre** services : Nova parle à Neutron en REST, jamais par le bus.
- Un chemin de données : aucun paquet ni bloc d'instance n'y transite.

Découplage : l'émetteur ignore où tourne le destinataire et combien d'exemplaires existent. C'est ce qui permet d'ajouter un hyperviseur ou un scheduler sans rien reconfigurer.

Service	RPC	Notifications	Remarque
Nova	●	●	Le plus gros utilisateur du bus
Cinder	●	●	
Neutron	○	●	RPC réduit au minimum avec OVN
Heat, Octavia, Magnum	●	●	API ↔ moteur ou contrôleur
Ironic, Manila, Designate	●	●	
Keystone	—	○	Notifications optionnelles (audit)
Glance	—	○	Notifications optionnelles
Placement	—	—	REST et SQL uniquement
Swift	—	—	N'utilise ni bus ni base SQL

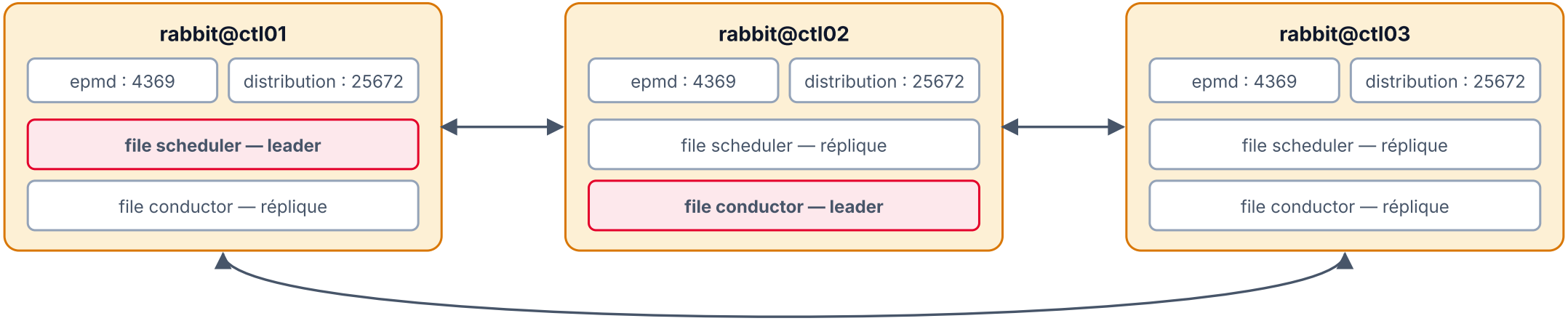
● utilisé · ○ partiel ou optionnel. Tous partagent en général le même cluster RabbitMQ ; un cluster ou un hôte virtuel distinct par service est possible à grande échelle.

Anatomie d'un nœud RabbitMQ



Lecture du schéma : l'émetteur ouvre une connexion TCP et publie sur un échange ; l'échange aiguille le message, selon sa clé, vers une ou plusieurs files, qui le conservent jusqu'à ce qu'un consommateur l'acquitte. Un message non acquitté est relivré ; seules les files gardent des données, sur disque pour les files à quorum.

Le cluster : trois nœuds, un cookie, une majorité



Distribution Erlang entre les trois nœuds : même secret partagé (fichier .erlang.cookie), noms d'hôtes résolus de façon identique partout.

Réaction à une partition	Comportement	Usage
`pause_minority`	Le côté minoritaire se met en pause et refuse les clients	Recommandé à trois nœuds
`autoheal`	Les deux côtés continuent, puis le perdant est redémarré	Privilégie la disponibilité, au prix de pertes
`ignore`	Rien : les deux côtés divergent	À éviter

- Les **leaders** des files à quorum se répartissent sur les trois nœuds : aucun n'est « le maître ».
- Une file à quorum reste utilisable tant que **deux** de ses trois répliques se voient.
- Le cluster se forme par nom : un nœud dont le nom d'hôte change est vu comme un inconnu.
- Les services listent les trois nœuds et se reconnectent seuls : pas de VIP devant RabbitMQ.

Exposition : comptes, hôtes virtuels et chaîne de connexion

Anatomie de l'option transport_url




```
# Créer un hôte virtuel, un compte et ses droits
rabbitmqctl add_vhost openstack
rabbitmqctl add_user nova 'S3cr3t'
rabbitmqctl set_permissions -p openstack nova ".*" ".*" ".*"
#                              configurer  écrire  lire

# Vérifier
rabbitmqctl list_users
rabbitmqctl list_permissions -p openstack
```

> Le compte `guest` par défaut n'est accepté que depuis l'hôte local ; il doit être supprimé ou désactivé en production.

Droit	Autorise
configurer	Créer et supprimer échanges et files (les services déclarent les leurs)
écrire	Publier dans un échange
lire	Consommer une file

```
[oslo_messaging_rabbit]     # chiffrement
ssl = true
ssl_ca_file = /etc/ssl/certs/ca-interne.pem
```

- Un hôte virtuel ou un compte par service limite l'effet d'un secret compromis.
- Le mot de passe figure en clair dans le fichier de configuration de chaque service : fichiers lisibles par le seul compte du service.

Diagnostic : les commandes à connaître

```
# 1. Le cluster est-il complet et sans alarme ?
rabbitmqctl cluster_status
rabbitmq-diagnostics check_running
rabbitmq-diagnostics alarms

# 2. Les files : messages en attente, consommateurs
rabbitmqctl list_queues name type messages_ready \
    messages_unacknowledged consumers

# 3. Seulement celles qui posent question
rabbitmqctl list_queues name messages consumers \
    | awk '$2 > 0 || $3 == 0'

# 4. Qui est connecté, depuis où, dans quel état
rabbitmqctl list_connections user peer_host state
rabbitmqctl list_consumers
```

Avec Kolla-Ansible, RabbitMQ tourne en conteneur : préfixer par `docker exec rabbitmq` (ou `podman exec`). Avec Sunbeam : `sudo k8s kubectl exec` dans le pod RabbitMQ (p. 55).

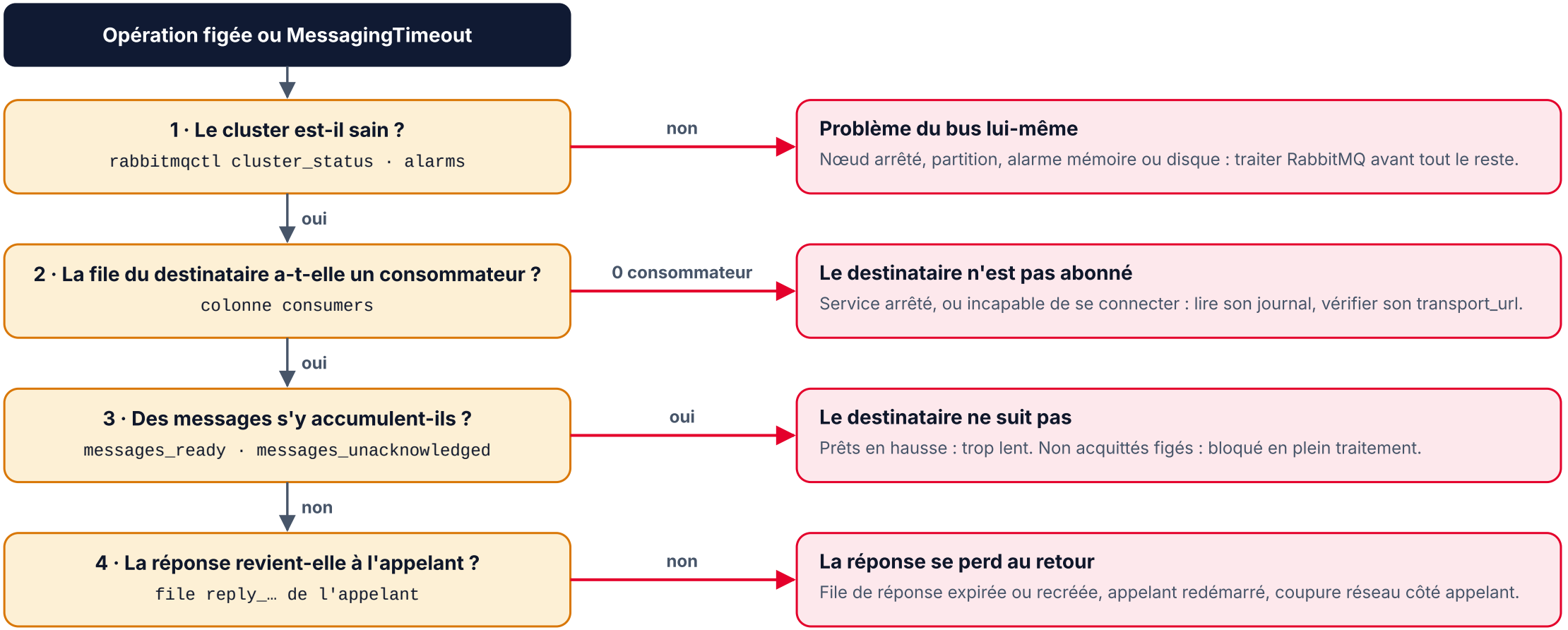
```
# 5. Ressources du nœud
rabbitmq-diagnostics memory_breakdown
rabbitmq-diagnostics status
rabbitmq-diagnostics listeners

# 6. Files à quorum : leader, membres, retard
rabbitmq-queues quorum_status scheduler
rabbitmq-queues check_if_node_is_quorum_critical
```

Ce que l'on cherche	Sortie saine
Nœuds en service	Trois « Running Nodes », aucune partition listée
Alarmes	Aucune alarme mémoire ni disque
Files RPC	0 message prêt, au moins un consommateur
Connexions	État `running` ; jamais `blocked`
Files à quorum	Trois membres en ligne, un leader

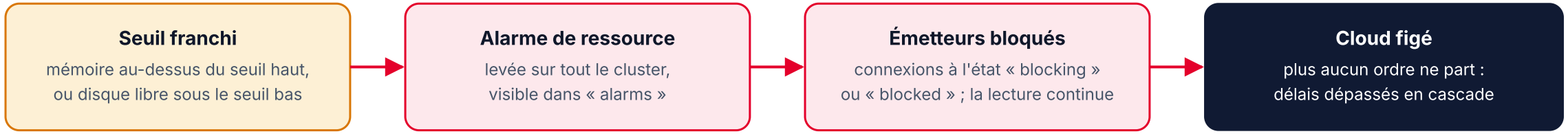
`check_if_node_is_quorum_critical` répond à la question à se poser **avant** d'arrêter un nœud : son arrêt ferait-il perdre le quorum à certaines files ?

Diagnostic : lire l'état des files pas à pas



Si les quatre réponses sont rassurantes, le bus est hors de cause : le traitement est simplement plus long que `rpc_response_timeout`, ou la panne se trouve derrière le destinataire (base, hyperviseur, stockage).

Alarmes de ressources : quand RabbitMQ bloque les émetteurs



Causes habituelles

Notifications sans consommateur
une file qui grossit sans fin

Destinataires arrêtés
leurs files ciblées se remplissent

Files et connexions orphelines
redémarrages en boucle d'un service

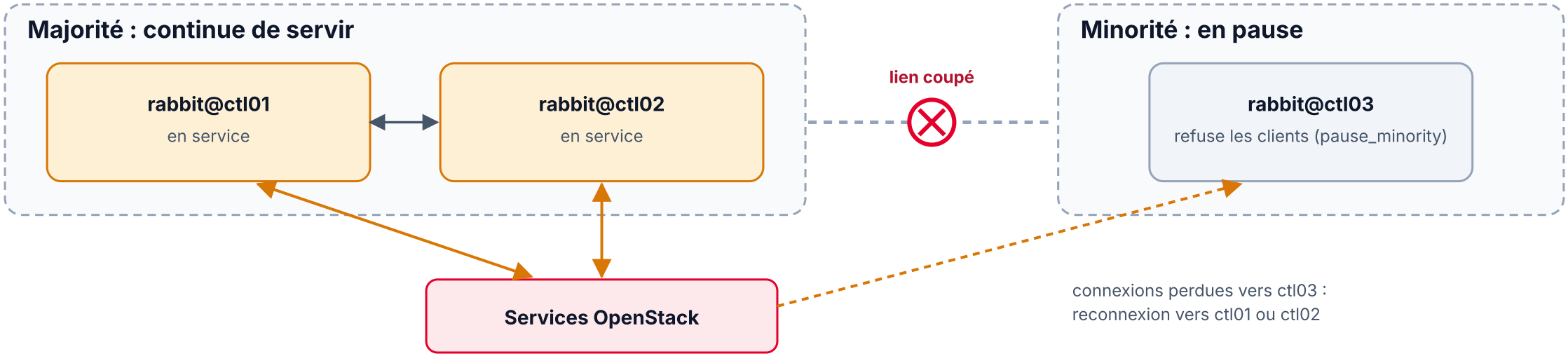
Paramètre RabbitMQ	Effet
`vm_memory_high_watermark`	Part de la mémoire au-delà de laquelle l'alarme se lève (0,4 à 0,6 selon la version)
`disk_free_limit`	Espace disque minimal ; la valeur par défaut, très basse, est à relever

```
rabbitmq-diagnostics alarms
rabbitmqctl list_connections name state | grep -i block
rabbitmq-diagnostics memory_breakdown
```

- Le blocage est un **mécanisme de protection** : RabbitMQ préfère ralentir les émetteurs plutôt que tomber faute de mémoire.
- Vu des services, rien n'échoue tout de suite : les envois attendent, puis les appelants dépassent leur délai.
- Dans un conteneur, le seuil se calcule sur la mémoire de l'hôte si la limite du conteneur n'est pas déclarée à RabbitMQ.

Sortie de crise : trouver la file qui enfle (`list_queues` triée par taille), rétablir son consommateur ou, s'il s'agit de notifications inutiles, la purger et passer le pilote à `noop` .

Partition réseau : symptômes et reprise



Ce que l'on observe

- Dans les journaux des services : vagues de « AMQP server ... is unreachable », puis reconnexions.
- `cluster_status` : section « Network Partitions » non vide, ou un nœud absent des « Running Nodes ».

Reprise après retour du lien

- Avec `pause_minority`, le nœud isolé rejoint seul le cluster.
- S'il reste à l'écart : `rabbitmqctl stop_app` puis `start_app` sur ce nœud.

Arrêt complet du cluster

- Le **dernier nœud arrêté** doit redémarrer en premier : lui seul connaît le dernier état.
- Si cet ordre est inconnu ou impossible : `rabbitmqctl force_boot` sur un nœud, puis démarrage des autres.

Après toute reprise, contrôler les services eux-mêmes : certains processus restent connectés à une file qui n'existe plus et ne s'en aperçoivent pas. Vérifier la colonne `consumers`, redémarrer ceux qui manquent.

Rôle : la mémoire du cloud

Ce qu'elle contient

- L'**inventaire** : instances, ports, réseaux, volumes, images, utilisateurs, projets.
- L'**état** de chaque ressource et de chaque opération en cours (`vm_state`, `task_state`, `status`).
- Les **affectations** : quelle instance sur quel hôte, quel volume attaché où, quelles allocations dans Placement.
- Les **battements** des services et la configuration dynamique : quotas, gabarits, agrégats, rôles.

Ce qu'elle ne contient pas

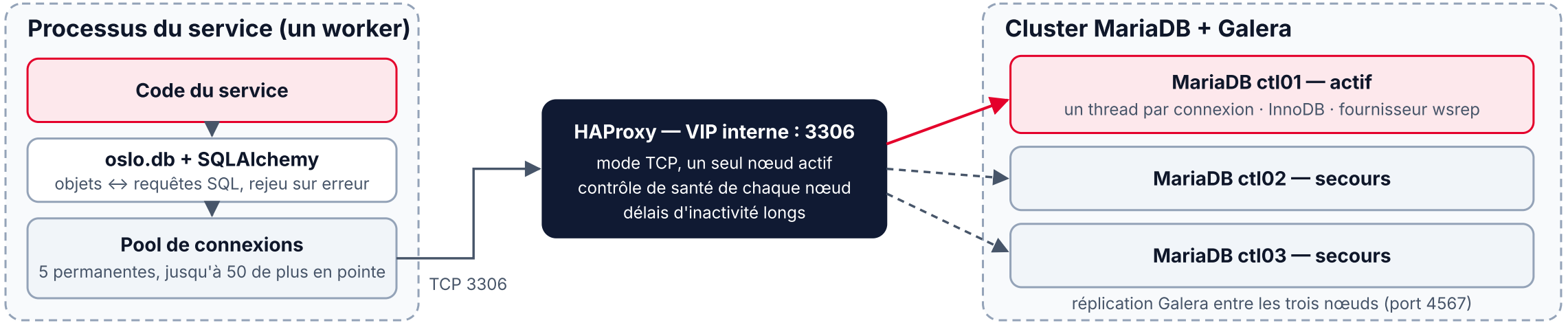
- Les données des utilisateurs : disques, objets et images vivent dans le stockage.
- Les jetons Keystone : au format Fernet, ils ne sont pas enregistrés.
- L'état réel des hyperviseurs : la base dit ce qui **devrait** être, pas ce qui est.

> La base est la **référence**. En cas de divergence avec la réalité (instance disparue de l'hyperviseur, port orphelin), ce sont les services qui se resynchronisent sur elle — ou l'administrateur qui la corrige par les API, jamais par SQL.

Schéma	Tables caractéristiques	Croissance
`keystone`	`project`, `user`, `assignment`, `endpoint`	Faible
`nova_api`	`flavors`, `request_specs`, `cell_mappings`, `quotas`	Modérée
`nova` **(cellule)**	`instances`, `instance_actions`, `block_device_mapping`, `services`	Forte
`nova_cell0`	Instances jamais placées sur un hôte	Faible
`placement`	`resource_providers`, `inventories`, `allocations`	Modérée
`neutron`	`networks`, `ports`, `ipallocations`, `securitygrouprules`	Modérée
`cinder`	`volumes`, `snapshots`, `volume_attachment`	Modérée
`glance`	`images`, `image_locations`, `image_properties`	Faible
`heat`	`stack`, `resource`, `event`	Forte

> « Forte » : des lignes s'ajoutent à chaque opération et les suppressions ne font que les marquer. Sans purge, ces schémas grossissent sans limite.

La chaîne d'accès : du service à la ligne écrite

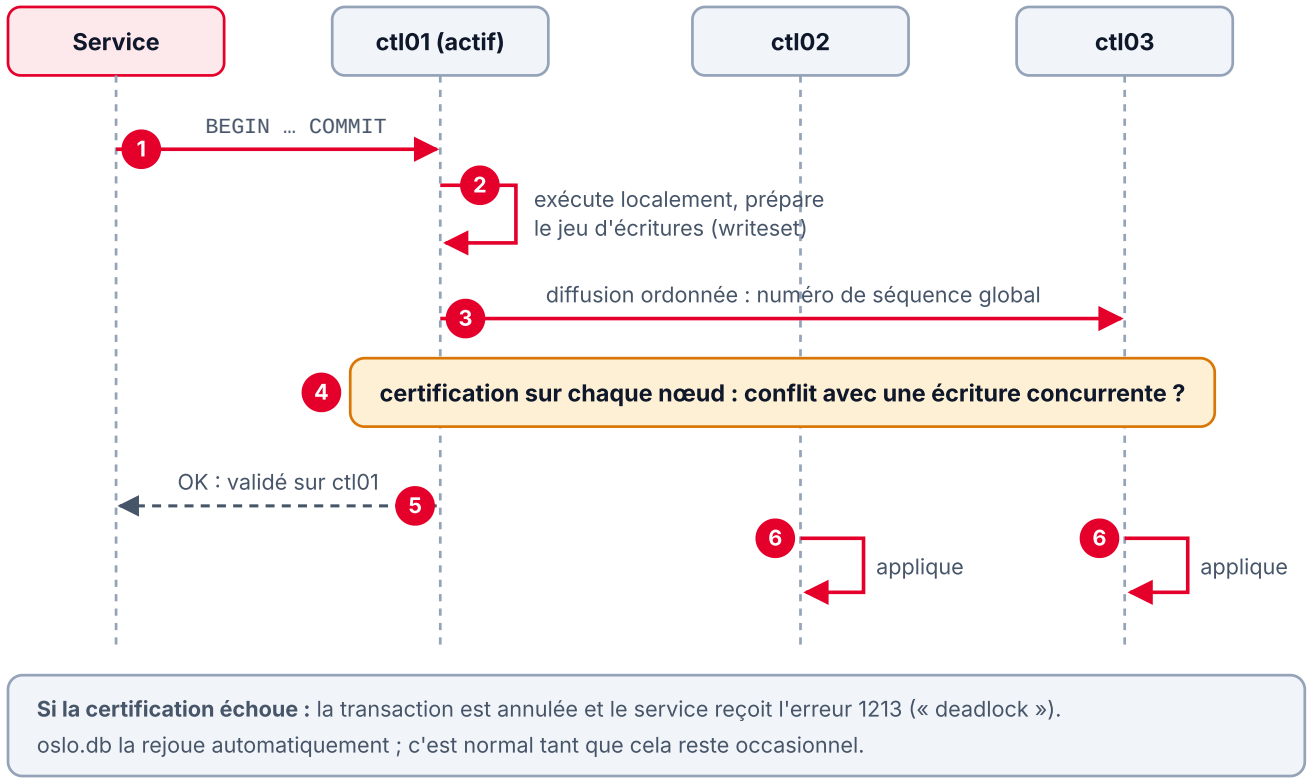


Délai d'inactivité	Réglé où	Ordre de grandeur
`connection_recycle_time`	Service, section `[database]`	600 s (défaut 3600 : à abaisser)
`timeout client` / `server`	HAProxy, bloc MariaDB	3600 s
`wait_timeout`	MariaDB	28800 s

Règle : recyclage côté service < délai HAProxy < délai MariaDB. Sinon un équipement ferme une connexion que le pool croit vivante : « MySQL server has gone away ».

- Les services ne connaissent que la **VIP** : ils ignorent quel nœud est actif.
- HAProxy envoie tout vers **un seul nœud** ; les deux autres ne servent qu'en cas de panne du premier.
- Une bascule coupe toutes les connexions ouvertes : oslo.db les rouvre et rejoue la requête en cours.
- Chaque connexion occupe un thread et de la mémoire côté MariaDB : la somme de tous les pools doit rester sous `max_connections`.

Galera : la réplication par certification



Si la certification échoue : la transaction est annulée et le service reçoit l'erreur 1213 (« deadlock »).
oslo.db la rejoue automatiquement ; c'est normal tant que cela reste occasionnel.

- **Synchrone à la validation :** quand le service reçoit « OK », la transaction est connue et acceptée par la majorité. Aucune écriture validée ne se perd à la panne d'un nœud.
- **Appliquée ensuite :** les autres nœuds l'écrivent avec un léger décalage. Lire sur un autre nœud juste après peut renvoyer l'ancienne valeur.
- **Tous les nœuds acceptent les écritures**, mais deux transactions concurrentes sur les mêmes lignes via deux nœuds se heurtent à la certification. D'où le nœud d'écriture unique.
- **Contrôle de flux :** un nœud trop lent à appliquer freine tout le cluster. Les trois nœuds doivent avoir des disques équivalents.

Contraintes de Galera : tables InnoDB uniquement, clé primaire sur chaque table, grosses transactions à éviter. Les schémas d'OpenStack les respectent.

Quorum et états d'un nœud Galera

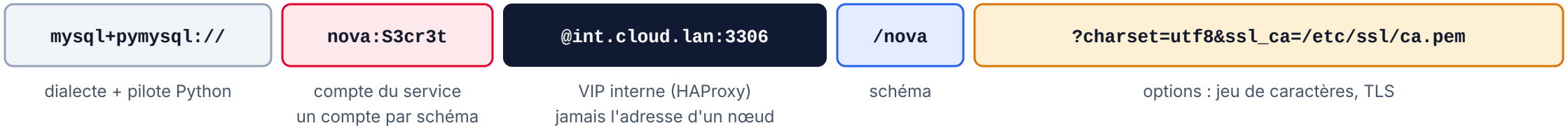


Transfert	Principe	Quand	Durée
IST (incrémental)	Rejoue les transactions manquantes depuis le cache d'un donneur	Absence courte : le retard tient dans le cache (`gcache`)	Secondes à minutes
SST (complet)	Copie intégrale des données depuis un donneur (mariabackup)	Nœud neuf, absence longue, données corrompues	Proportionnelle à la taille de la base

- **Majorité stricte :** à trois nœuds, deux doivent se voir. À deux, la moindre coupure entre eux arrête tout.
- **Nombre pair :** ajouter un arbitre (`garbd`) qui vote sans stocker de données.
- **Arrêt propre** d'un nœud : il quitte le groupe et la majorité se recalcule sur les nœuds restants.
- **Arrêt brutal** des trois : plus de composant primaire, aucun nœud ne redémarre seul en service — il faut réamorcer le cluster.
- Pendant un SST, le donneur est mis à l'écart : à trois nœuds, un seul reste pleinement disponible.

Exposition : comptes, privilèges et chaîne de connexion

Anatomie de l'option connection




```
-- Ce que l'outil de déploiement crée pour chaque service
CREATE DATABASE nova;
CREATE USER 'nova'@'10.20.0.%' IDENTIFIED BY 'S3cr3t';
GRANT ALL PRIVILEGES ON nova.* TO 'nova'@'10.20.0.%';

-- Vérifier
SELECT user, host FROM mysql.user;
SHOW GRANTS FOR 'nova'@'10.20.0.%';
```

> Avec HAProxy en mode TCP, MariaDB voit l'adresse du **répartiteur**, pas celle du service : la restriction par hôte porte donc sur les contrôleurs.

Compte	Portée	Utilisé par
Un par service	Tous les droits sur son seul schéma	Les processus du service
Contrôle de santé	Connexion sans privilège	Script interrogé par HAProxy
Transfert d'état	Lecture complète, rechargement	mariabackup pendant un SST
Supervision	Lecture des variables d'état	Exporteur de métriques
root	Tout	Administrateur, en local seulement

- Un service compromis ne lit que son schéma : Glance n'accède pas aux tables de Keystone.

Diagnostic : l'état du cluster Galera

```
MariaDB> SHOW GLOBAL STATUS LIKE 'wsrep_%';
+----------------------------+--------------+
| wsrep_cluster_size         | 3            |
| wsrep_cluster_status       | Primary      |
| wsrep_connected            | ON           |
| wsrep_ready                | ON           |
| wsrep_local_state_comment  | Synced       |
| wsrep_incoming_addresses   | 10.20.0.11…  |
| wsrep_last_committed       | 48211937     |
| wsrep_local_recv_queue_avg | 0.02         |
| wsrep_flow_control_paused  | 0.00         |
| wsrep_local_cert_failures  | 117          |
+----------------------------+--------------+
```

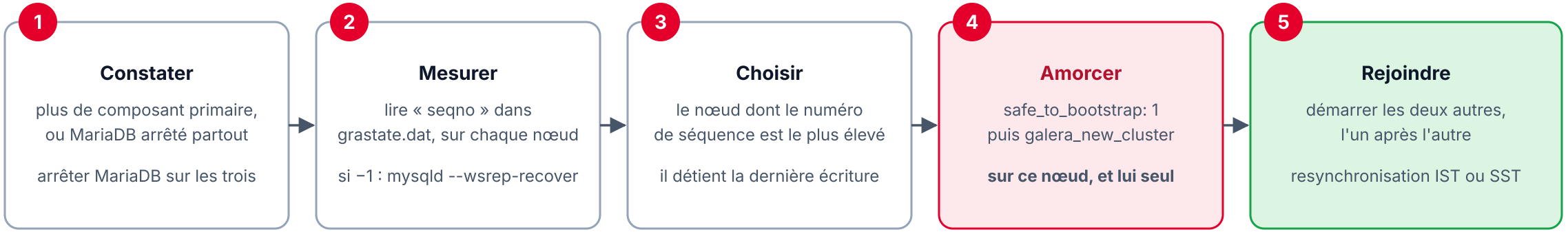
```
# En une ligne, sur chaque contrôleur
mariadb -e "SHOW STATUS WHERE Variable_name IN
  ('wsrep_cluster_size','wsrep_cluster_status',
   'wsrep_local_state_comment','wsrep_ready')"
```

Variable	Sain	Sinon
`wsrep_cluster_size`	3	Un nœud manque : redondance réduite
`wsrep_cluster_status`	Primary	Ce nœud est hors quorum et refuse les requêtes
`wsrep_ready`	ON	Le nœud n'accepte pas de requêtes
`wsrep_local_state_comment`	Synced	Joining, Donor… : nœud en transfert
`wsrep_last_committed`	Identique sur les trois	Écart durable : un nœud décroche
`wsrep_local_recv_queue_avg`	Proche de 0	Ce nœud applique trop lentement
`wsrep_flow_control_paused`	Proche de 0	Part du temps où le cluster a été freiné
`wsrep_local_cert_failures`	Croissance lente	Hausse rapide : écritures sur plusieurs nœuds

Interroger les trois nœuds, pas seulement la VIP : chacun donne sa propre vision. Trois nœuds qui annoncent chacun `cluster_size = 1` forment trois clusters distincts — la pire situation.

 Sunbeam : MySQL InnoDB Cluster, p. 55

Perte de quorum : réamorcer le cluster




```
# /var/lib/mysql/grastate.dat
# GALERA saved state
version: 2.1
uuid:    7f3c2a91-5c1e-11ef-9a41-b7d0c5a8e2f1
seqno:   48211937
safe_to_bootstrap: 0
```

```
# Avec Kolla-Ansible : toute la procédure en une commande
kolla-ansible mariadb-recovery -i inventaire
```

- **Pourquoi le plus avancé :** amorcer depuis un nœud en retard ferait perdre les transactions que seul un autre avait reçues. Les autres nœuds s'aligneraient sur lui.
- **seqno = −1** signifie que le nœud ne s'est pas arrêté proprement : la position réelle se retrouve en rejouant le journal InnoDB.
- **Ne jamais amorcer deux nœuds :** on obtiendrait deux clusters indépendants, qui divergent derrière la même VIP.
- Cas plus simple — nœuds vivants mais tous « non-Primary » : sur le plus avancé, `SET GLOBAL wsrep_provider_options='pc.bootstrap=YES'`.

> Pendant toute l'opération, les API sont indisponibles et les instances continuent de fonctionner. Rien ne presse au point de sauter l'étape 2.

 Sunbeam : action create-backup, p. 55

Hygiène : purger et sauvegarder

La plupart des services ne suppriment pas les lignes : ils les marquent (`deleted` , `deleted_at`). Sans traitement périodique, les tables grossissent, les requêtes ralentissent et les transferts d'état Galera s'allongent.

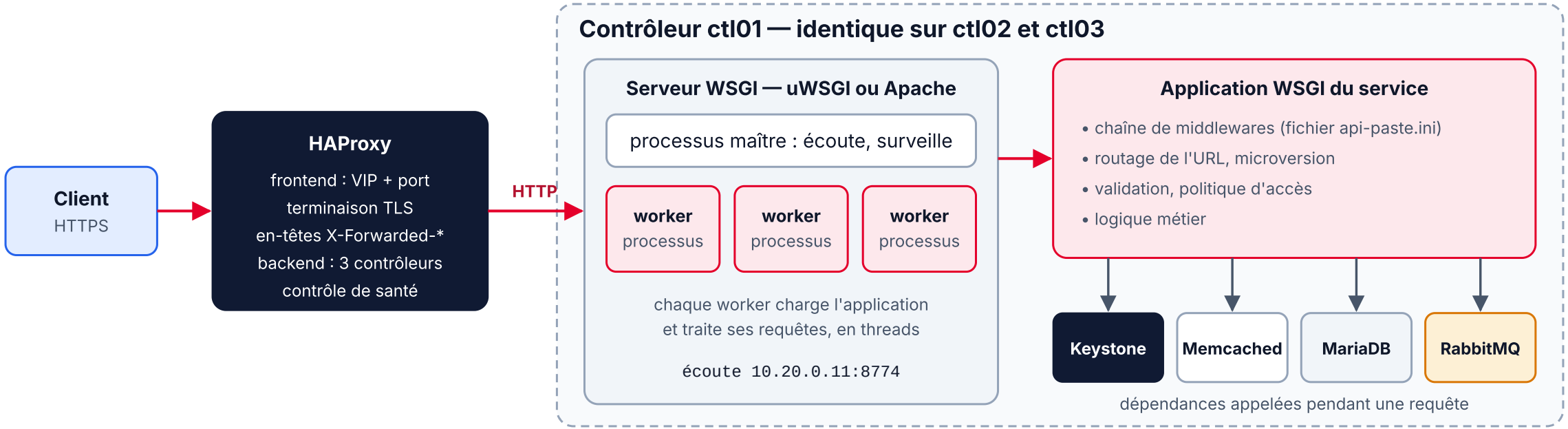
Service	Commande de purge	Effet
Nova	`nova-manage db archive_deleted_rows --until-complete --all-cells`	Déplace les lignes supprimées vers des tables « shadow »
Nova	`nova-manage db purge --before <date> --all-cells`	Efface définitivement les lignes archivées
Cinder	`cinder-manage db purge <jours>`	Efface les lignes supprimées depuis plus de N jours
Glance	`glance-manage db purge --age_in_days <jours>`	Idem pour les images
Heat	`heat-manage purge_deleted -g days <jours>`	Efface les piles supprimées et leurs événements
Keystone	`keystone-manage trust_flush`	Efface les délégations expirées

Sauvegarder

- **mariabackup** : copie physique à chaud, complète ou incrémentale ; avec Kolla-Ansible : `kolla-ansible mariadb-backup -i inventaire` .
- **mariadb-dump** : export logique, schéma par schéma ; lent sur les gros volumes mais lisible et portable.
- Sauvegarder sur **un** nœud, stocker **hors** du cluster, et tester la restauration.

Trois copies Galera ne sont pas une sauvegarde : une suppression par erreur ou une migration ratée est répliquée instantanément sur les trois nœuds. Une sauvegarde est indispensable **avant chaque montée de version** — c'est la seule voie de retour arrière.

La pile : du répartiteur au code du service



HAProxy

Reçoit toutes les connexions, déchiffre, choisit un contrôleur sain. Ne connaît rien d'OpenStack : il ne voit que du HTTP.

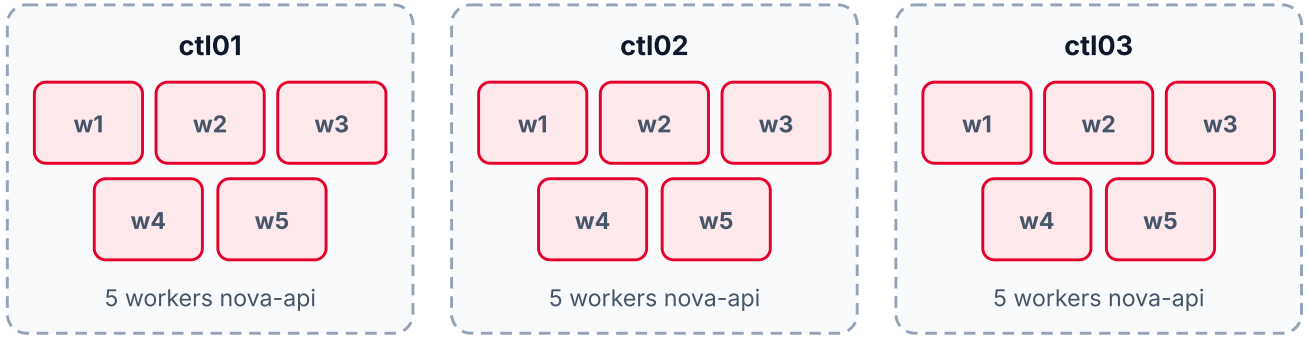
Serveur WSGI

Fait tourner le code Python : gère les processus, les threads, les redémarrages. C'est lui qui fixe la capacité de traitement.

Application

Le service lui-même. Une requête n'aboutit que si **toutes** ses dépendances répondent : la disponibilité de l'API est le produit des leurs.

Workers : la capacité, et ce qu'elle coûte en aval



Ce que ces 15 processus peuvent ouvrir

3 contrôleurs × **5** workers = **15** processus nova-api

× jusqu'à **55** connexions SQL chacun (5 + 50) = **825 connexions possibles**

× plusieurs connexions RabbitMQ chacun

× quelques centaines de Mo de mémoire chacun

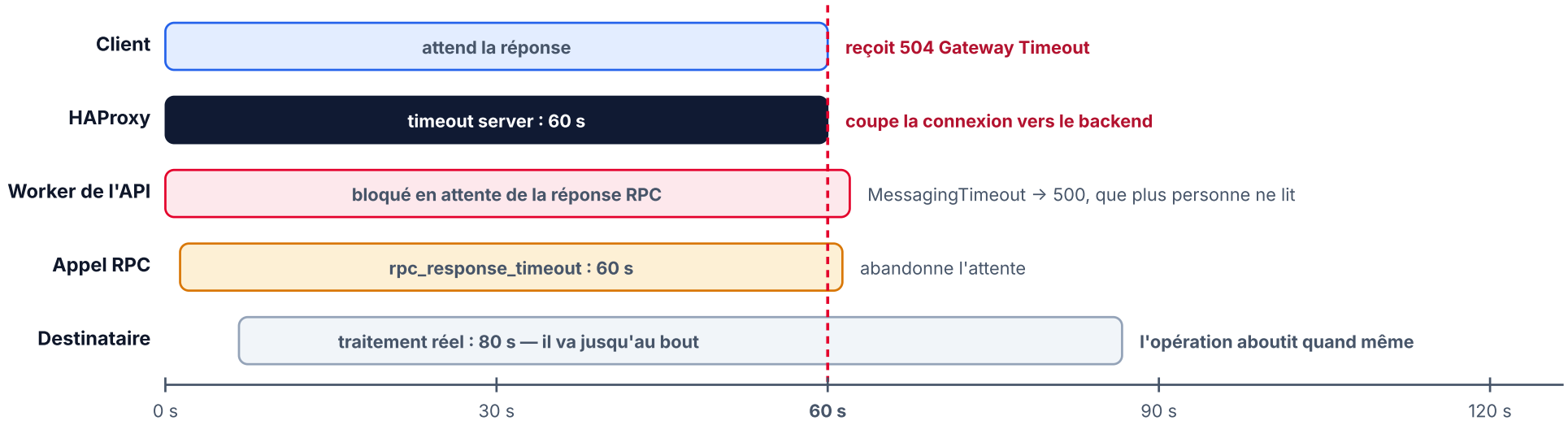
Et autant pour chaque autre API : Neutron, Cinder, Keystone, Glance, Placement, Heat…

- Un **worker** traite une requête à la fois par thread. Trop peu de workers : les requêtes attendent, HAProxy renvoie des 504.
- Trop de workers : mémoire épuisée sur les contrôleurs et plafond de connexions de la base dépassé.
- La bonne valeur se **mesure** : temps de réponse, file d'attente HAProxy, `Threads_connected`.

Où se règle le nombre de workers
uWSGI : `processes` et `threads`
mod_wsgi : directive `WSGIDaemonProcess`
Kolla-Ansible : variable `openstack_service_workers`

Piège du défaut « un worker par cœur » : sur un contrôleur de 64 cœurs, dix services à 64 workers font 640 processus et des dizaines de milliers de connexions potentielles. Toujours borner.

Délais en cascade : pourquoi un 504 n'annule rien



Étage	Délai	Réglé où
Client	Propre à l'outil	Option `--timeout` de la CLI, SDK
HAProxy	`timeout client` , `timeout server`	`haproxy.cfg`
Serveur WSGI	Durée maximale d'une requête	uWSGI (`harakiri`), Apache
RPC	`rpc_response_timeout`	Fichier du service
Base	`pool_timeout` , verrous	Service et MariaDB

- Le **premier** délai qui expire décide de ce que voit le client ; les étages suivants continuent de travailler.
- Un **504** signifie « pas de réponse à temps », pas « opération échouée ». Relancer aveuglément crée des doublons.
- Après un 504, **vérifier l'état** de la ressource avant toute nouvelle tentative.
- Allonger les délais repousse le symptôme sans traiter la lenteur. La bonne question : pourquoi 80 s ?

> C'est aussi pourquoi les opérations longues sont des « cast » : l'API répond 202 tout de suite et n'attend rien.

Un arbre de diagnostic commun aux trois composants

Un appel API échoue ou n'aboutit pas

Serveur d'API

Signes

- 502, 503, 504, connexion refusée
- 401 par intermittence
- latence en hausse sur toutes les requêtes

Vérifier, dans l'ordre

1. curl sur la VIP, puis sur chaque backend
2. état des backends dans HAProxy
3. journal HAProxy : durées, état de fin
4. journal de l'API à l'identifiant req-…
5. Keystone, Memcached, horloges

Reprise type : retirer le backend fautif

File de messages

Signes

- 202 reçu, mais ressource figée (BUILD…)
- MessagingTimeout dans un journal
- services « down » en série

Vérifier, dans l'ordre

1. cluster_status, alarmes
2. list_queues : messages, consommateurs
3. journal du service destinataire
4. connexions : état, nombre
5. réseau entre le nœud et les contrôleurs

Reprise type : rétablir le consommateur

Base de données

Signes

- 500 avec DBConnectionError, DBDeadlock
- toutes les API en erreur en même temps
- erreur 1047 : nœud hors quorum

Vérifier, dans l'ordre

1. wsrep_cluster_size, wsrep_cluster_status
2. nœud actif vu par HAProxy
3. Threads_connected / max_connections
4. requêtes lentes, verrous
5. latence disque des contrôleurs

Reprise type : rétablir le quorum

Dans tous les cas : partir de l'identifiant de requête et suivre l'opération dans les journaux, étage par étage.

Lecture du schéma : partir du symptôme observé pour choisir la colonne, puis dérouler les vérifications dans l'ordre, du plus extérieur au plus profond. La reprise type, en bas de colonne, n'intervient qu'une fois la cause localisée.

04

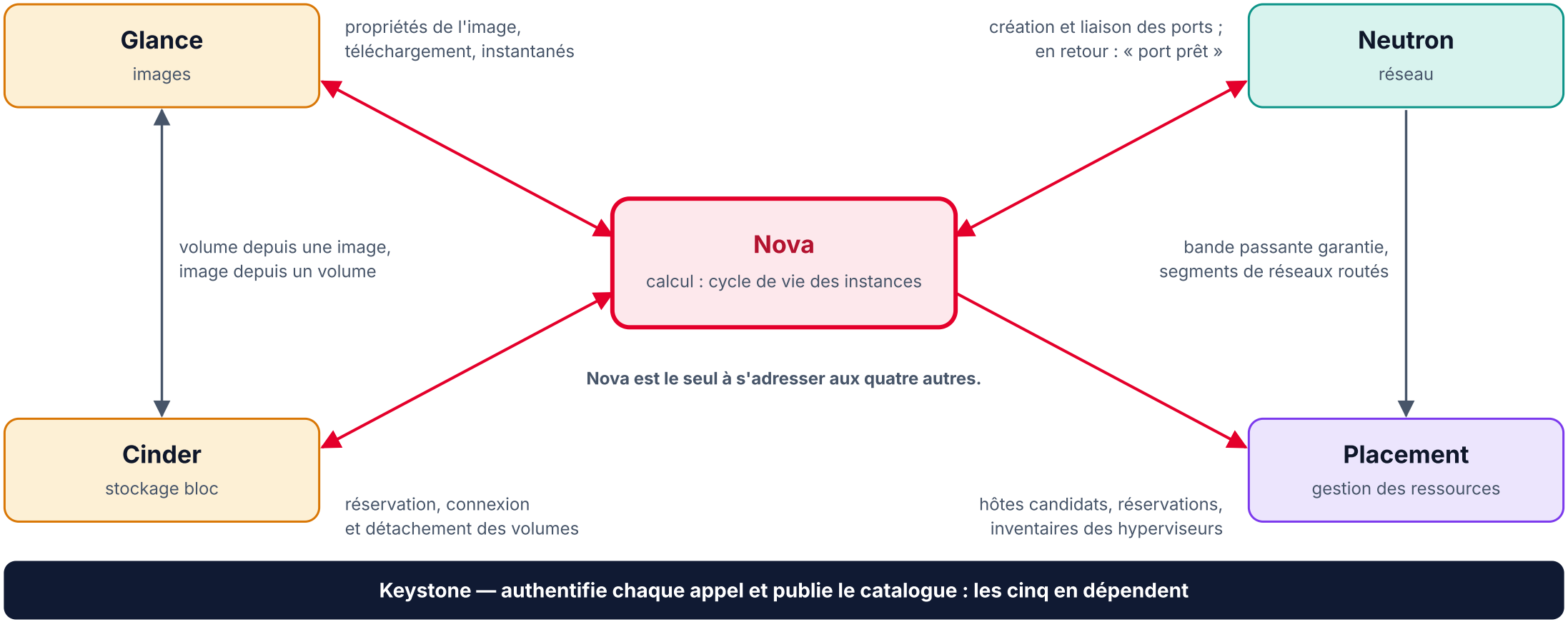
Tour des principaux services

Images, gestion des ressources, calcul, stockage bloc, réseau : rôle, architecture, interactions

À L'ISSUE DE CE MODULE, VOUS SAUREZ

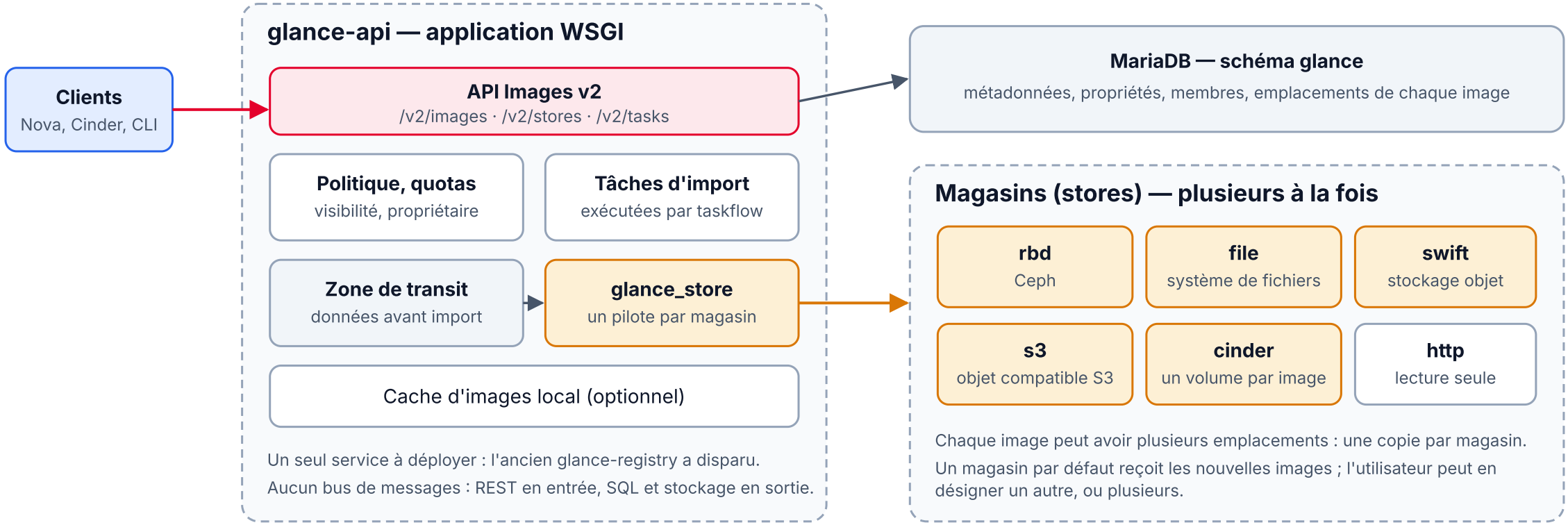
- Décrire le **rôle** de Glance, Placement, Nova, Cinder et Neutron, et la frontière de chacun.
- Lire le **cycle de vie** d'une image, d'une instance, d'un volume et d'un port à travers leurs états.
- Choisir entre les variantes de fonctionnement (téléchargement ou clone, disque local ou volume, routage distribué ou centralisé).
- Dessiner leur **architecture** : processus, pilotes, bases, backends.
- Expliquer les **interactions** : qui appelle qui, à quel moment, pour obtenir quoi.
- Lancer les commandes qui montrent l'état de chaque service.

Cinq services, et ce qu'ils se demandent

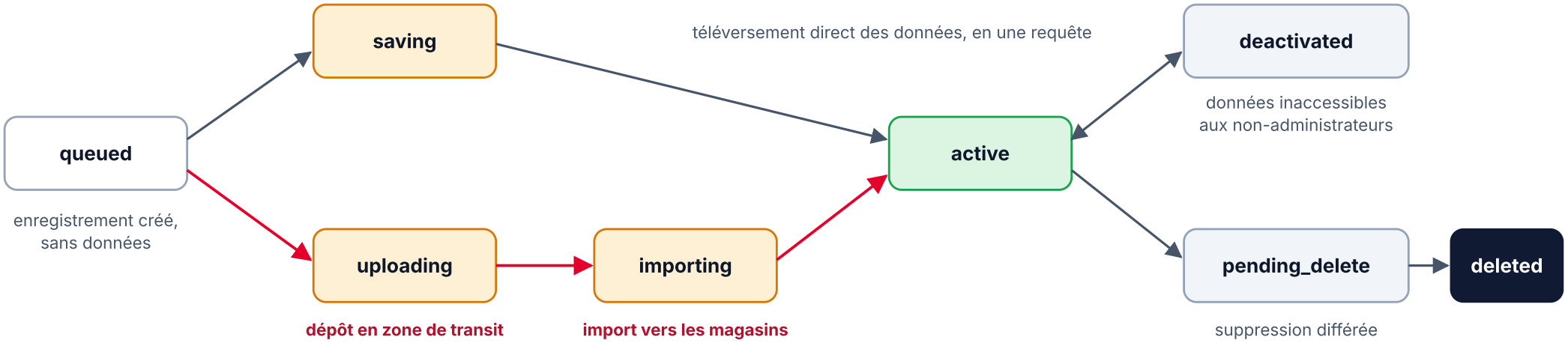


Lecture du schéma : chaque flèche est une question posée par un service à un autre, par son API REST. Nova est au centre parce qu'il est le seul à interroger les quatre autres ; Keystone, en bas, est sollicité par tous pour valider les jetons et trouver les adresses.

Architecture : une API, une base, des magasins



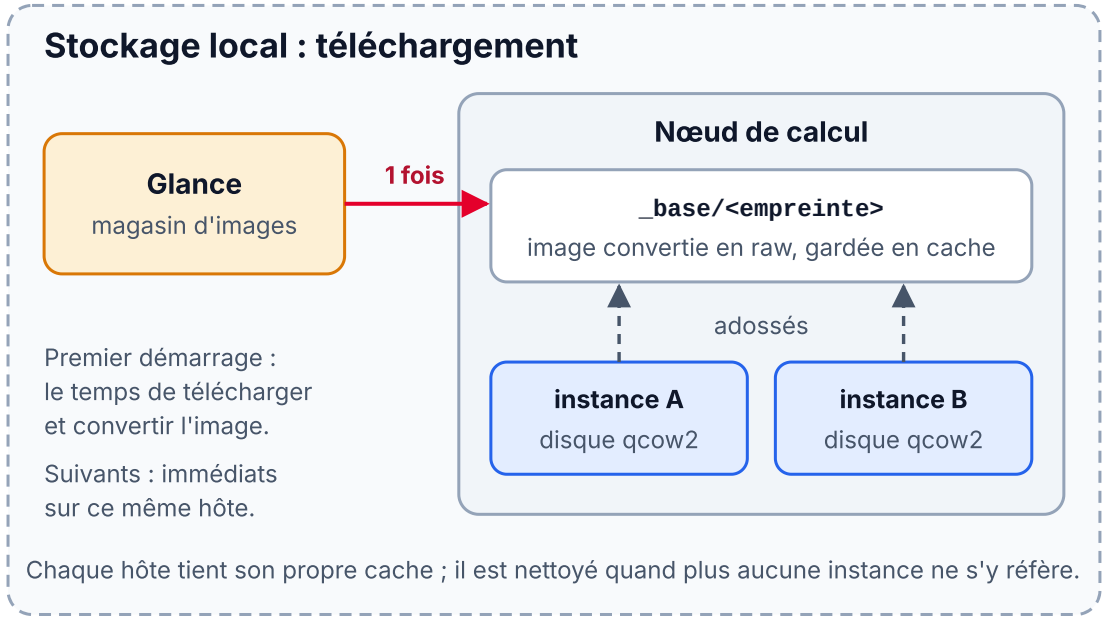
Cycle de vie d'une image

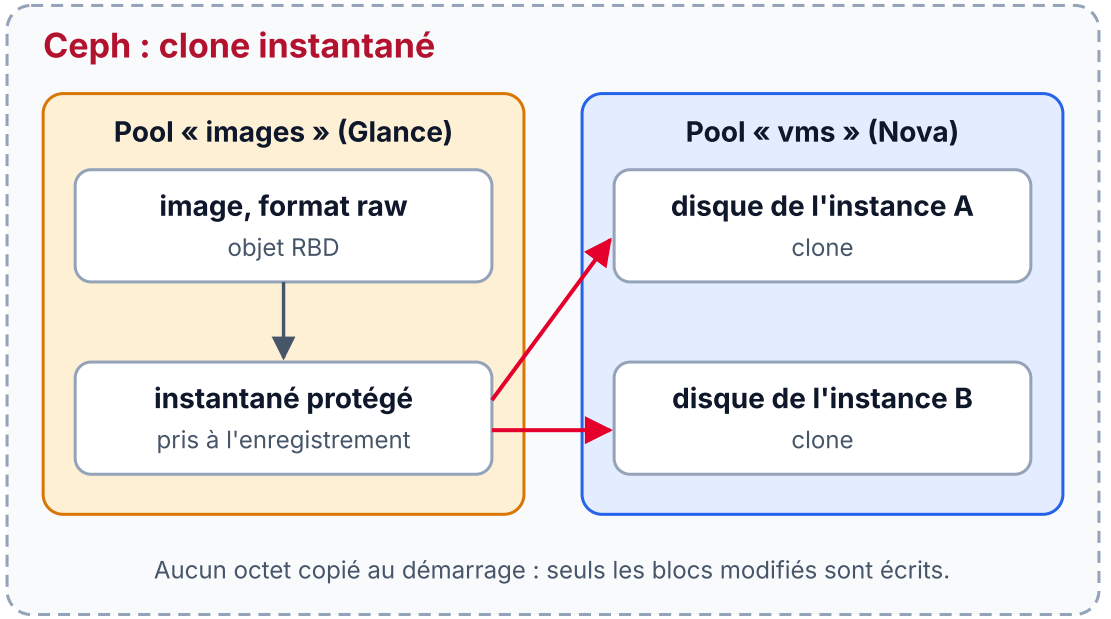


Statut	Signification
queued	L'image existe en base, ses données ne sont pas encore fournies
saving	Données en cours de réception, par l'ancien mode de téléversement direct
uploading, importing	Données en transit, puis en cours de traitement et d'écriture dans les magasins
active	Utilisable : somme de contrôle calculée, au moins un emplacement

- Seule une image **active** peut servir à démarrer une instance ou créer un volume.
- Un import en échec laisse en général l'image à l'état `importing` ou la ramène à `queued` : on corrige, puis on relance.
- Supprimer une image n'affecte pas les instances déjà créées ; si leurs disques en sont des clones Ceph, Glance refuse la suppression.

Interaction avec Nova : télécharger ou cloner





Condition du clone Ceph	Réglage
Image au format raw	Conversion à l'import, côté Glance
Même cluster Ceph	Glance et Nova sur des pools du même cluster
Emplacement visible de Nova	Glance : `show_image_direct_url` ; Nova : `images_type = rbd`

- Si une condition manque, Nova **retombe sans prévenir** sur le téléchargement complet : démarrages lents, espace consommé en double.
- Dans les deux cas, Nova vérifie la somme de contrôle de ce qu'il reçoit.
- **Instantané d'instance :** chemin inverse — Nova fige le disque et le téléverse dans Glance comme nouvelle image ; avec Ceph, c'est encore un clone.

Le modèle de données



Quantités d'un côté, qualités de l'autre : les **classes de ressources** se comptent et se consomment (VCPU, MEMORY_MB, DISK_GB, VGPU, SRIOV_NET_VF) ; les **caractéristiques** sont présentes ou absentes et ne s'épuisent pas. Une requête combine toujours les deux.

Le calcul de la capacité

VCPU sur cmp01 — capacité = (total − réservé) × ratio = (64 − 0) × 4,0 = 256



MEMORY_MB sur cmp01 — capacité = (262 144 − 8 192) × 1,0 = 253 952

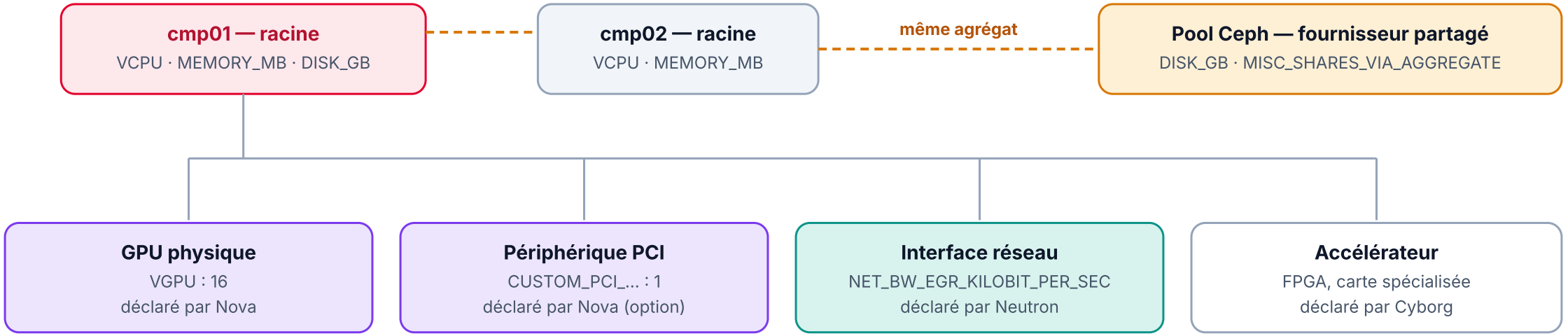


Champ d'un inventaire	Rôle
`total`	Quantité physique déclarée par le fournisseur
`reserved`	Part gardée pour l'hôte lui-même, jamais attribuée
`allocation_ratio`	Facteur de surallocation ; 1,0 = aucune
`min_unit` , `max_unit`	Plus petite et plus grande quantité qu'une seule allocation peut prendre
`step_size`	Pas : les quantités demandées en sont des multiples

- Une demande est acceptée si **alloué + demandé ≤ capacité**, pour chaque classe, et si elle respecte `max_unit` .
- Avec 76 VCPU libres mais `max_unit = 64` , une instance de 72 VCPU est refusée : elle ne tiendrait pas sur les cœurs réels.
- Les ratios viennent de `nova.conf` sur chaque hyperviseur (`cpu_allocation_ratio` …) ou sont fixés par l'API de Placement.
- La mémoire se suralloue rarement : au-delà de 1,0, l'hôte risque de tuer des machines faute de mémoire.

> Ce sont des **promesses**, pas des mesures : Placement compte ce qui est réservé par les gabarits, pas ce que les instances consomment réellement.

Arbres de fournisseurs et ressources partagées



Une même requête peut être servie par plusieurs fournisseurs d'un arbre : VCPU et mémoire sur la racine, VGPU sur un enfant, disque sur le pool partagé.

Fournisseurs imbriqués

Un hyperviseur n'est pas un bloc uniforme : un GPU, une carte SR-IOV ont leur propre inventaire et leurs propres caractéristiques. Ils sont déclarés comme enfants du nœud de calcul.

Fournisseurs partagés

Un pool de stockage commun à plusieurs hôtes n'est compté qu'**une fois**. Il est rattaché à ces hôtes par un agrégat ; sans cela, chaque hôte déclarerait le pool entier comme sien.

Plusieurs déclarants

Nova n'est pas seul à écrire : Neutron et Cyborg ajoutent leurs fournisseurs sous l'arbre du même hôte. Une caractéristique récente, `OWNER_NOVA`, distingue ceux que Nova gère.

Rôle et processus

Rôle : créer, démarrer, arrêter, déplacer et supprimer des **instances** — machines virtuelles le plus souvent, serveurs physiques avec Ironic.

- **Ce qu'il possède :** les instances, les gabarits, les paires de clés, les groupes de serveurs, les agrégats d'hôtes.
- **Ce qu'il emprunte :** l'image (Glance), les ports (Neutron), les volumes (Cinder), la place (Placement).
- **Ce qu'il ne fait pas :** surveiller ce qui tourne dans l'instance, garantir sa disponibilité, sauvegarder ses données.

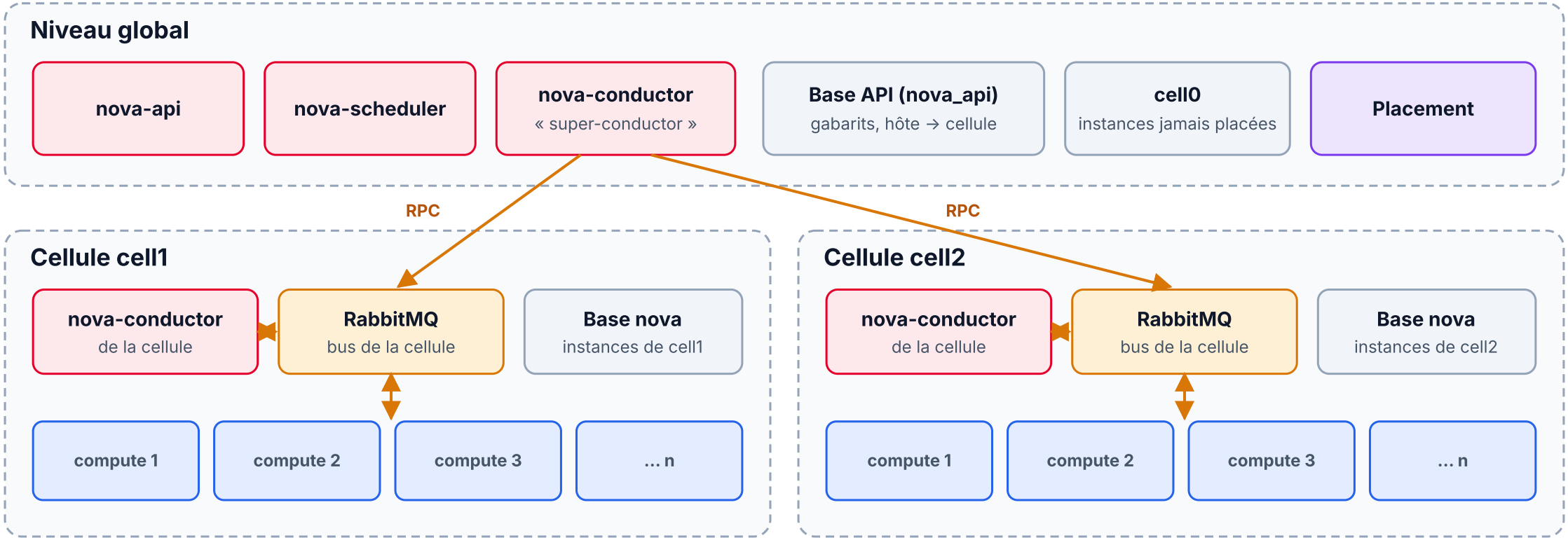
Notion	Définition
Gabarit (flavor)	Quantités de ressources et réglages matériels (`hw:…`)
Groupe de serveurs	Politique d'affinité ou d'anti-affinité, stricte ou souple
Agrégat d'hôtes	Regroupement d'hyperviseurs par l'administrateur, avec métadonnées
Zone de disponibilité	Un agrégat rendu visible des utilisateurs
Correspondance de périphériques bloc	Liste des disques de l'instance : image, volume, éphémère, swap

Processus	Où	Fonction
nova-api	Contrôle	API REST ; valide, vérifie les quotas, enregistre, délègue
nova-scheduler	Contrôle	Choisit l'hôte : Placement, filtres, pondérations
nova-conductor	Contrôle	Orchestre les opérations longues ; accès base pour les hôtes
nova-compute	Chaque hyperviseur	Exécute : pilote libvirt, disques, interfaces, volumes
API de métadonnées	Contrôle	Sert `169.254.169.254` : données utilisateur, clés
nova-novncproxy	Contrôle	Relaie les consoles vers le navigateur

Deux bases, pas une : la base « API » (`nova_api`) pour ce qui est global — gabarits, quotas, cartographie des cellules ; une base par **cellule** (`nova`) pour les instances et leurs hôtes.

Les quotas peuvent être comptés à partir de Placement et définis dans Keystone (« limites unifiées ») : Nova n'a alors plus de table de quotas propre.

Cellules : une architecture à deux niveaux



Pourquoi

Une base et un bus uniques finissent par saturer. Chaque cellule isole les siens : la charge et les pannes restent locales.

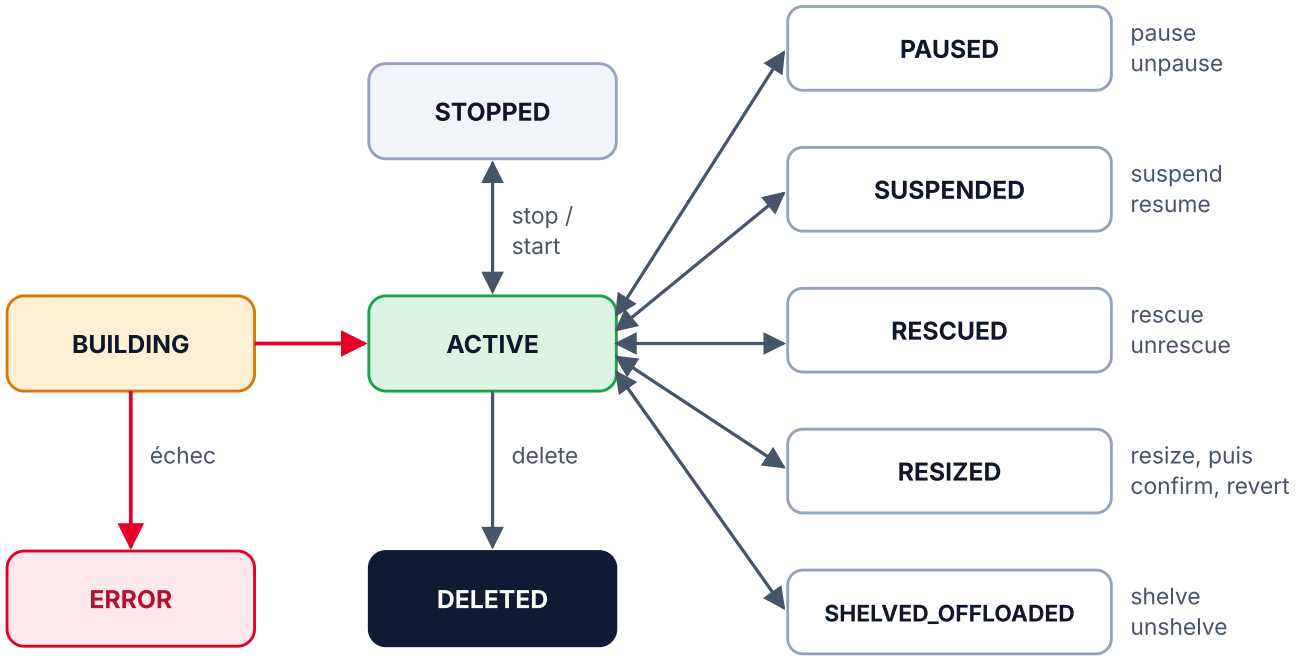
Comment

La base API sait dans quelle cellule vit chaque instance et chaque hôte. L'API s'adresse ensuite directement à la base et au bus de cette cellule.

À savoir

Tout déploiement a au moins `cell0` et `cell1`. Un nouvel hyperviseur doit être rattaché : `nova-manage cell_v2 discover_hosts`.

Les états d'une instance



ERROR est atteignable depuis presque tous les états, à la suite d'une opération en échec.

Champ	Ce qu'il décrit
`vm_state`	État stable de l'instance, du point de vue de Nova
`task_state`	Opération en cours ; vide quand rien ne se passe
`power_state`	Ce que l'hyperviseur constate réellement
`status`	Synthèse des deux premiers, montrée par l'API

- Pendant une création, `task_state` passe par `scheduling`, `networking`, `block_device_mapping`, `spawning` : il dit **à quelle étape** l'instance est restée bloquée.
- Une instance en cours d'opération refuse toute autre action : erreur 409 tant que `task_state` n'est pas vide.
- **SHELVED_OFFLOADED :** disque sauvegardé dans Glance, hôte et ressources libérés ; au retour, l'instance est replacée.
- Si `power_state` contredit `vm_state`, une tâche périodique réaligne la base sur la réalité.

Remettre une instance en état : `openstack server set --state active` corrige la **base**, pas l'hyperviseur. À n'utiliser qu'après avoir vérifié la réalité.

Ordonnancement : préfiltres, filtres, pondérations

1 · Préfiltres (via Placement)

- **Quantités** du gabarit : VCPU, mémoire, disque
- **Caractéristiques** exigées par le gabarit ou l'image
- **Zone de disponibilité** demandée → agrégat
- **Hôtes désactivés** exclus (`COMPUTE_STATUS_DISABLED`)
- **Format d'image** pris en charge par l'hôte (option)
- **Isolation** de projets ou d'agrégats (option)

> Tout ce qui s'exprime en quantité, caractéristique ou agrégat est traité **dans la base de Placement**, en une requête.

2 · Filtres (nova-scheduler)

- `ComputeFilter` : service actif
- `ImagePropertiesFilter` : architecture, hyperviseur
- `ComputeCapabilitiesFilter` : capacités exigées par le gabarit
- `ServerGroupAntiAffinityFilter` , `ServerGroupAffinityFilter`
- `NUMATopologyFilter` : CPU dédiés, pages larges
- `PciPassthroughFilter` : périphériques attribués
- `AggregateInstanceExtraSpecsFilter` : gabarit ↔ agrégat

> Un filtre répond oui ou non, hôte par hôte. Liste active : `[filter_scheduler] enabled_filters` .

3 · Pondérations

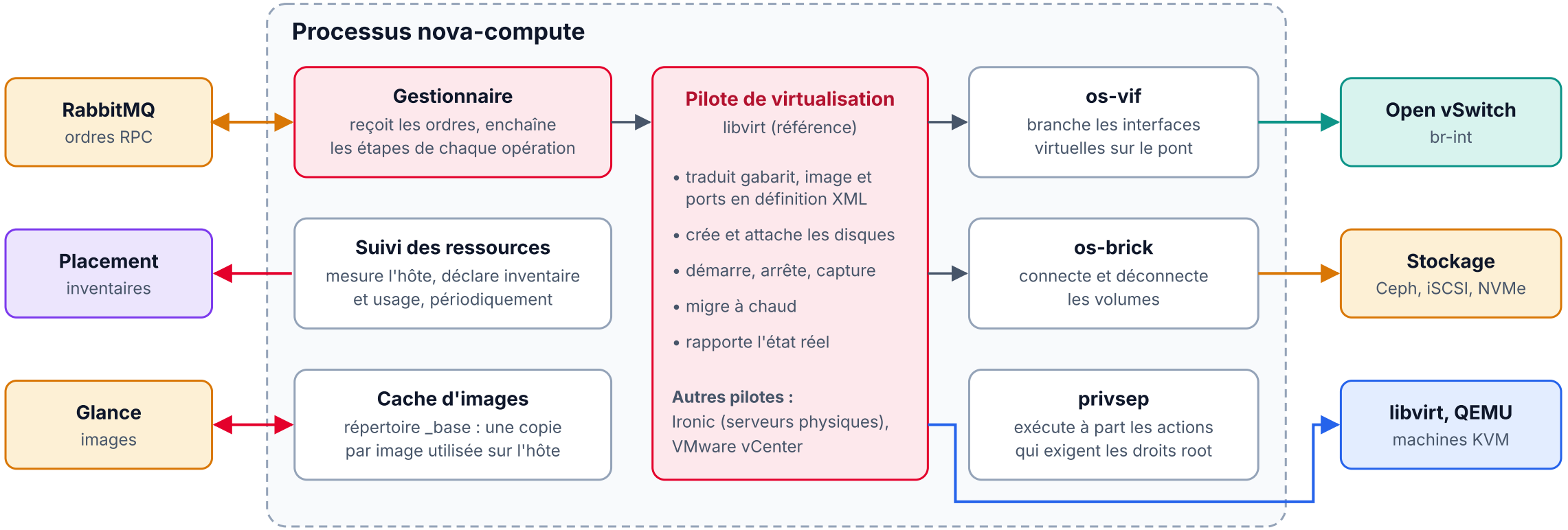
- `RAMWeigher` , `CPUWeigher` , `DiskWeigher` : ressources libres
- `IoOpsWeigher` : opérations en cours sur l'hôte
- `BuildFailureWeigher` : échecs récents
- `ServerGroupSoft…Weigher` : affinité souple
- `NumInstancesWeigher` , `PCIWeigher` …

> Chaque pondération note les hôtes restants ; un multiplicateur fixe son poids. Positif : **répartir**. Négatif : **tasser**.

Après le classement : le premier hôte est retenu, sa place réservée dans Placement, et quelques **suppléants** sont transmis avec la demande. Si la construction échoue sur l'hôte, le conductor réessaie sur le suivant, sans repasser par le scheduler (`max_attempts`).

Lire un refus : en mode `debug` , nova-scheduler journalise, filtre par filtre, combien d'hôtes restent. Le filtre qui fait tomber le compte à zéro désigne la contrainte impossible à satisfaire.

nova-compute : ce qui se passe sur l'hyperviseur



Un seul nova-compute par hyperviseur, et c'est lui qui « possède » les instances de cet hôte : s'il s'arrête, elles continuent de tourner mais plus aucune opération sur elles n'est possible.

Où vivent les disques d'une instance



	Fichier local	Ceph RBD	Démarrage sur volume
Réglage	`[libvirt] images_type = qcow2`	`images_type = rbd`, `images_rbd_pool = vms`	Demandé à la création de l'instance
Création	Rapide si l'image est déjà en cache	Instantanée (clone)	Selon le backend : clone ou copie
Migration à chaud	Avec copie des disques entre hôtes	Oui, sans copie	Oui, sans copie
Perte de l'hyperviseur	Disque perdu avec l'hôte	Instance relançable ailleurs	Instance relançable ailleurs
Suppression de l'instance	Disque supprimé	Disque supprimé	Volume conservé si demandé
Instantané	Téléversé dans Glance	Clone dans Glance	Instantané de volume, dans Cinder

> Dans tous les cas, une instance peut recevoir en plus un **disque de configuration** (données de démarrage en lecture seule), un disque éphémère supplémentaire et un disque d'échange, selon le gabarit.

Déplacer une instance : cinq opérations, cinq usages

Opération	Instance pendant l'opération	Hôte source	Choix de la cible	Condition principale	Usage
Migration à chaud	En marche ; pause de quelques instants en fin de transfert	En service	Scheduler ou hôte imposé	Processeurs compatibles ; disque partagé, ou copie des disques	Vider un hôte avant maintenance
Migration à froid	Arrêtée, puis redémarrée	En service	Scheduler	Accès entre hôtes pour copier le disque local	Déplacement sans exigence de continuité
Redimensionnement	Arrêtée, puis redémarrée	En service	Scheduler (souvent un autre hôte)	Nouveau gabarit ; confirmation ou retour arrière	Changer de taille
Évacuation	Déjà arrêtée par la panne	Hors service	Scheduler ou hôte imposé	Hôte déclaré « down » ; disque partagé ou sur volume pour garder les données	Reprise après panne d'hôte
Mise à l'écart (shelve)	Arrêtée, disque sauvegardé	Libéré	Au retour, par le scheduler	—	Libérer des ressources sans perdre l'instance

```
openstack server migrate --live-migration web-01  # à chaud
openstack server migrate web-01                   # à froid
openstack server resize --flavor m1.large web-01
openstack server resize confirm web-01          # ou revert
openstack server evacuate web-01
openstack server shelve web-01
```

Migration à chaud, échange par échange : p. 100.

Évacuer n'est pas migrer : l'évacuation suppose l'hôte mort. Lancée sur un hôte encore vivant mais injoignable, elle démarre un second exemplaire de l'instance sur le même disque — d'où l'intérêt d'isoler l'hôte (coupure électrique commandée) avant d'évacuer.

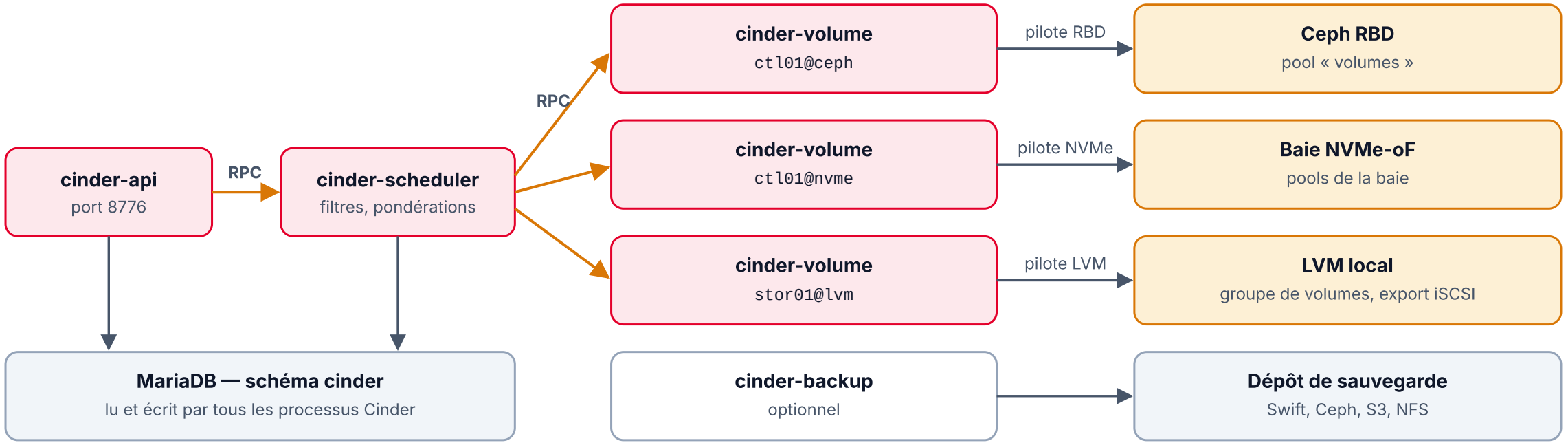
Suivre un déplacement : `openstack server migration list`, et l'état `VERIFY_RESIZE` qui attend `resize confirm` ou `resize revert`.

Interactions : ce que Nova demande, et à quel moment

	Validation (nova-api)	Ordonnancement	Construction sur l'hôte	Finalisation
Keystone	valide le jeton de l'utilisateur			
Glance	**l'image existe-t-elle ?** propriétés, taille minimale		**télécharge ou clone l'image** vérifie la somme de contrôle	
Placement		**hôtes candidats,** puis réservation sur l'hôte retenu	**en cas d'échec : libération,** nouvel essai sur un suppléant	
Neutron	**réseaux et ports demandés ?** quota de ports		**crée ou complète les ports,** désigne l'hôte de liaison	**événement « port prêt »** attendu avant de démarrer
Cinder	**volumes demandés disponibles ?**		**crée les volumes, obtient** les informations de connexion	
État vu par l'API	**BUILD** task_state : scheduling	**BUILD** task_state : scheduling	**BUILD** networking→block_device_mapping→spawning	**ACTIVE** task_state vide

Lecture du schéma : les colonnes suivent le temps, de la validation à la finalisation ; chaque ligne montre ce que Nova demande à un service à cette étape. La dernière ligne donne l'état visible par l'API : une instance bloquée en BUILD désigne l'étape, donc le service, à examiner.

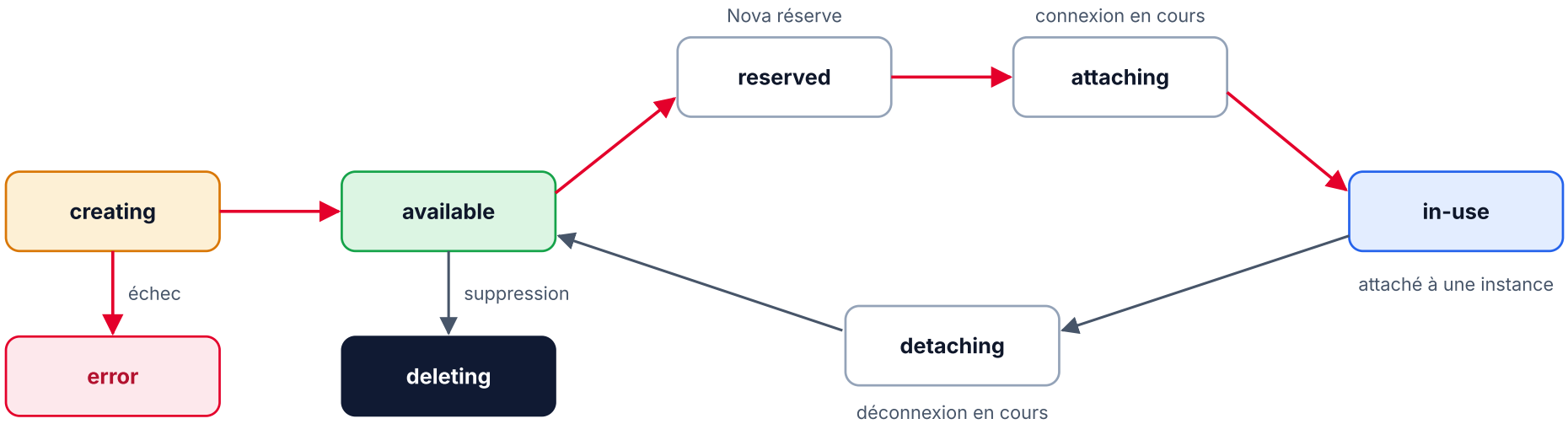
Architecture : un service de volume par backend




```
# /etc/cinder/cinder.conf
[DEFAULT]
enabled_backends = ceph, nvme
[ceph]
volume_driver = cinder.volume.drivers.rbd.RBDDriver
volume_backend_name = ceph
rbd_pool = volumes
rbd_user = cinder
```

- Chaque backend est une **section** du fichier et donne naissance à un service de volume nommé `hôte@backend`.
- Un backend peut exposer plusieurs **pools** : la désignation complète est `hôte@backend#pool`.
- Backend distant (Ceph, baie) : cinder-volume tourne sur les contrôleurs. Backend local (LVM) : il tourne sur le nœud qui porte les disques.

Les états d'un volume

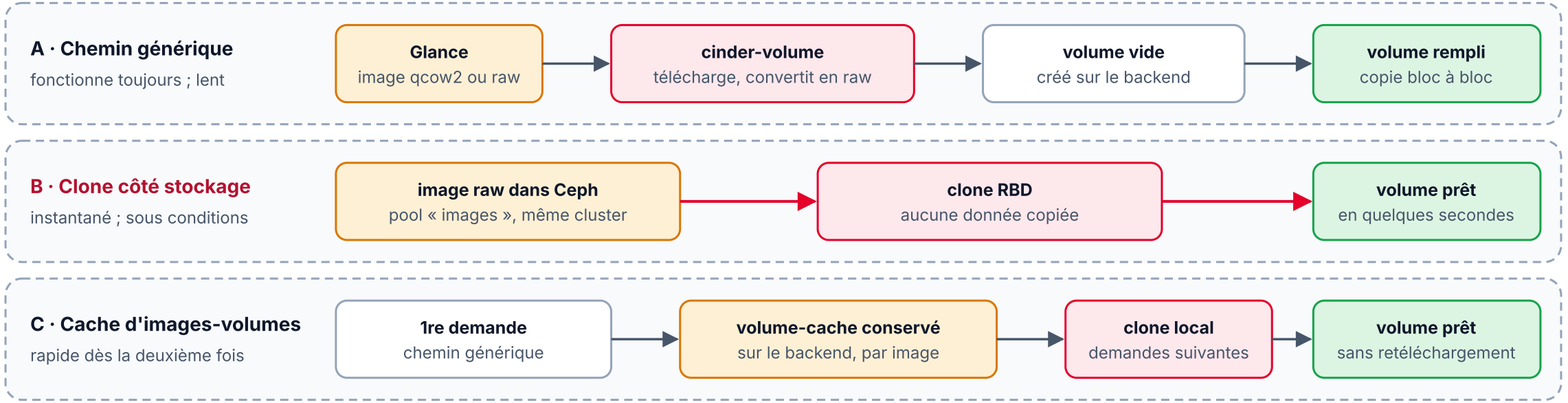


État transitoire	Opération en cours
extending	Agrandissement ; possible volume attaché si le backend le permet
backing-up, restoring-backup	Sauvegarde ou restauration
uploading, downloading	Export vers une image Glance ; remplissage depuis une image
retyping, migrating	Changement de type ou de backend
error_extending, error_deleting…	L'opération a échoué ; le volume reste dans cet état

- Un volume ne s'attache que s'il est **available** (ou déjà *in-use* avec l'attachement multiple).
- Un volume bloqué dans un état transitoire refuse toute action : c'est le symptôme d'un ordre RPC perdu ou d'un cinder-volume arrêté.
- Remise en état par l'administrateur : `openstack volume set --state available` — après avoir vérifié la réalité côté backend et côté Nova.

> Un volume a aussi un **état d'attachement** et un **état de migration** distincts : les trois doivent être cohérents avant de forcer quoi que ce soit.

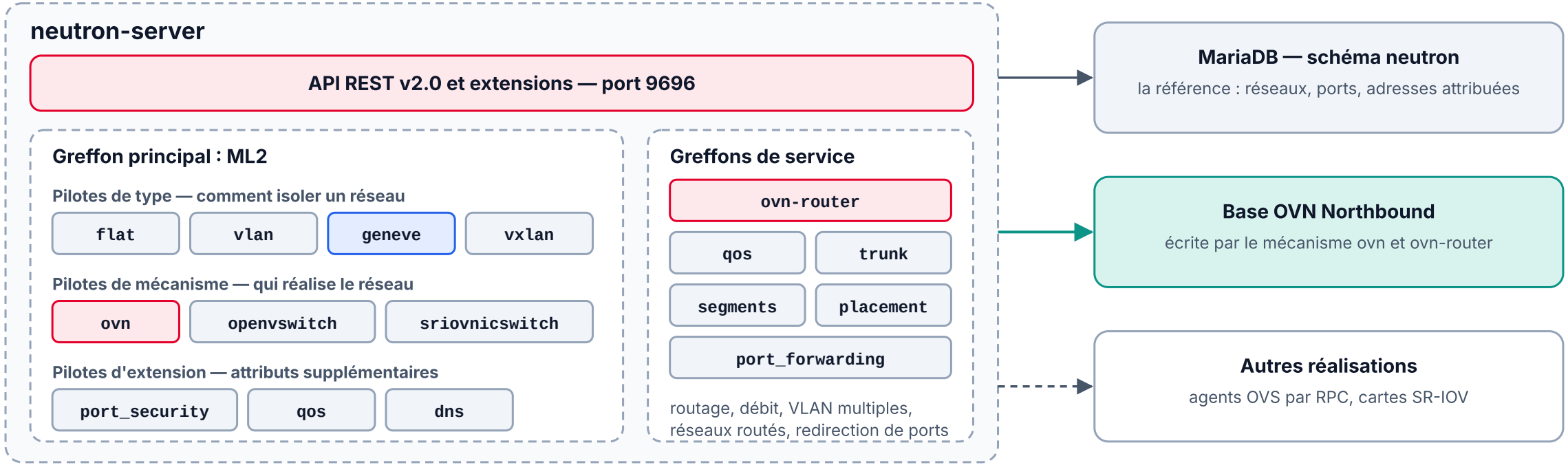
Créer un volume depuis une image : trois chemins



Chemin	Conditions	Coût
A · Générique	Aucune	Temps et espace temporaire, selon la taille
B · Clone	Image raw, même cluster Ceph, emplacement exposé	Quasi nul
C · Cache	`image_volume_cache_enabled` sur le backend	Un volume caché par image

- Le chemin A exige de l'**espace disque temporaire** là où tourne cinder-volume : une image de 40 Go qui échoue « sans raison » a souvent rempli ce répertoire.
- Le chemin B crée une **dépendance** entre le volume et l'image : tant que le clone existe, l'image ne peut pas être supprimée de Ceph.
- Dans tous les cas, le volume reçoit les métadonnées de l'image (`volume_image_metadata`) : Nova les relira au démarrage.

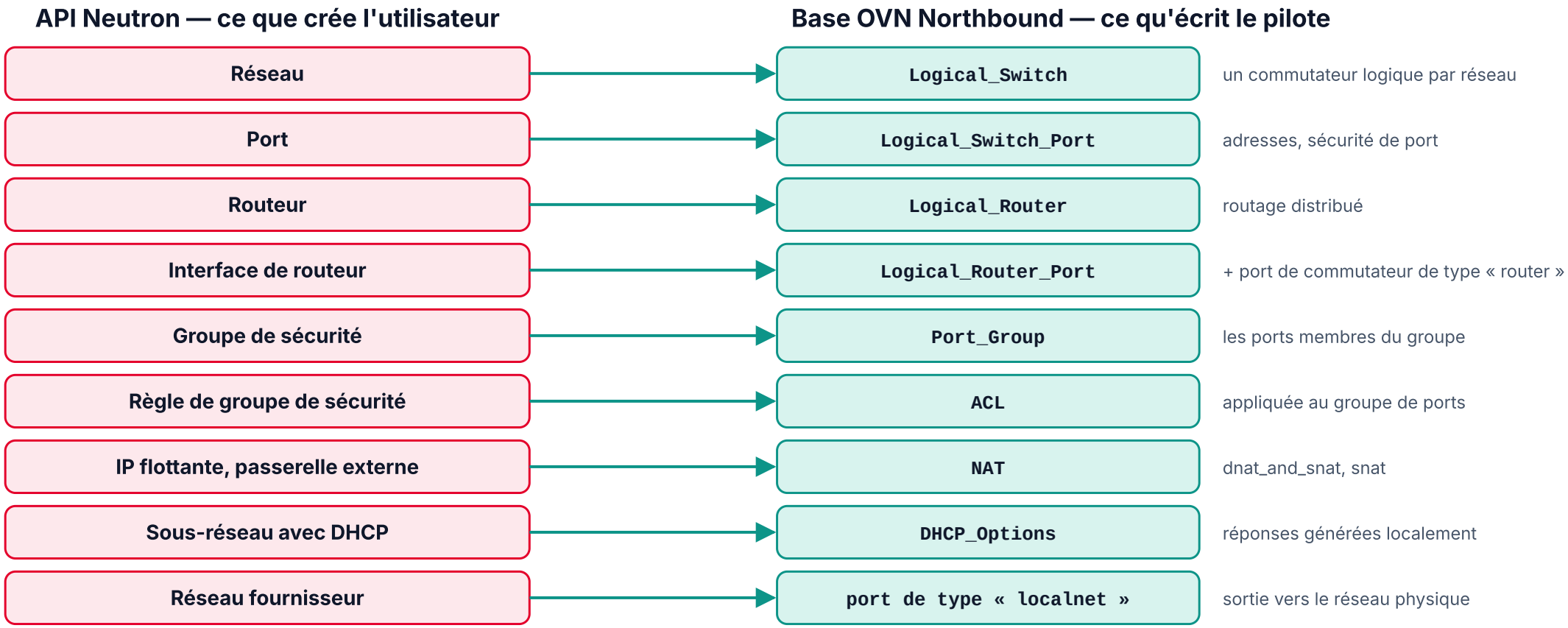
Architecture : neutron-server, ML2 et ses pilotes




```
# /etc/neutron/plugins/ml2/ml2_conf.ini
[ml2]
type_drivers = flat,vlan,geneve
tenant_network_types = geneve
mechanism_drivers = ovn
extension_drivers = port_security,qos,dns
[ml2_type_vlan]
network_vlan_ranges = physnet1:100:199
```

- **ML2 sépare deux questions :** le *type* (comment un réseau est isolé) et le *mécanisme* (qui le met en œuvre). Plusieurs mécanismes peuvent cohabiter.
- `tenant_network_types` : type attribué aux réseaux que créent les projets — ici des tunnels Geneve.
- `network_vlan_ranges` : VLAN utilisables pour les réseaux fournisseurs, par réseau physique nommé.

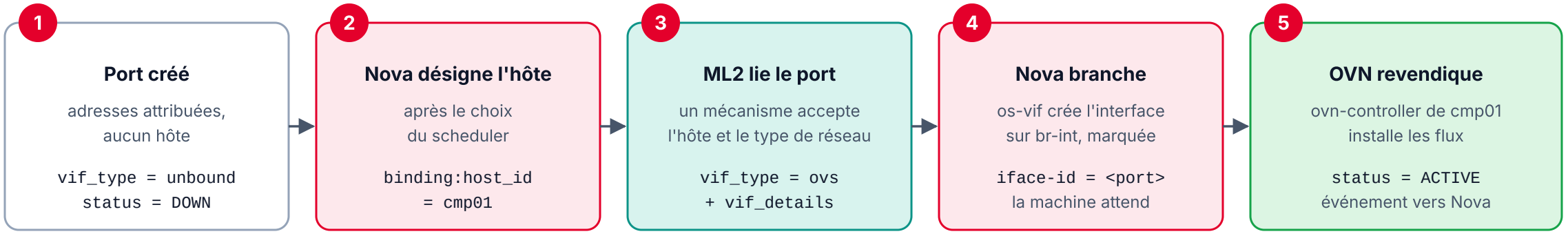
Du modèle Neutron au modèle OVN



Lecture : le pilote `ovn` traduit chaque objet de l'API en une ou plusieurs lignes de la base Northbound. `ovn-northd` en déduit ensuite les flux logiques, que chaque ovn-controller convertit en règles Open vSwitch.

En cas d'écart entre les deux bases (panne pendant une écriture, intervention manuelle), l'outil `neutron-ovn-db-sync-util` compare et, en mode réparation, réaligne OVN sur Neutron.

La liaison de port : relier un port à un hôte

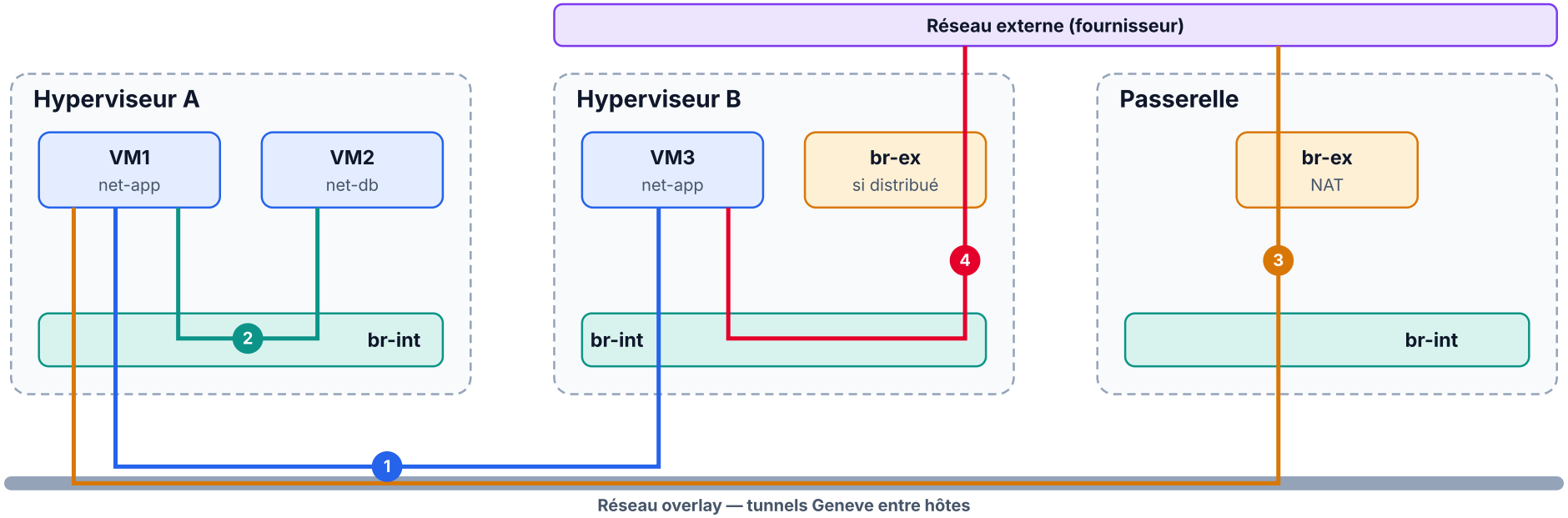


Attribut du port	Sens
`binding:host_id`	Hôte sur lequel le port doit vivre ; renseigné par Nova
`binding:vnic_type`	Nature de l'interface : `normal` (virtuelle), `direct` (SR-IOV), `baremetal`
`binding:vif_type`	Résultat : `ovs`, `unbound`, ou `binding_failed`
`binding:vif_details`	Ce que Nova doit savoir pour brancher : type de pont, filtrage
`binding:profile`	Informations fournies par Nova : adresse PCI, données de migration
`device_owner`, `device_id`	Qui utilise le port : instance, routeur, DHCP, IP flottante

- `binding_failed` : aucun mécanisme n'a pu lier le port à cet hôte — hôte inconnu d'OVN, réseau physique non déclaré sur l'hôte (`bridge-mappings`), type de réseau non pris en charge.
- Le lien entre l'interface locale et le port logique se fait par l'identifiant du port, posé sur l'interface Open vSwitch.
- **Migration à chaud :** un port peut porter deux liaisons, source et cible ; la seconde est activée au moment de la bascule.

> Un port DOWN sur une instance ACTIVE mérite toujours un regard : `openstack port show <id> -c binding_vif_type -c binding_host_id -c status` .

Les chemins du trafic



	Cas	Chemin	Où se fait le traitement
1	Même réseau, hôtes différents	VM1 → br-int A → tunnel → br-int B → VM3	Commutation logique sur A ; aucun équipement central
2	Deux réseaux reliés par un routeur	VM1 → br-int A → VM2	Routage **distribué** : exécuté sur l'hyperviseur source
3	Sortie sans IP flottante	VM1 → tunnel → passerelle → br-ex → externe	Traduction d'adresse source (SNAT) **centralisée** sur la passerelle
4	IP flottante, mode distribué	VM3 → br-int B → br-ex B → externe	NAT sur l'hyperviseur lui-même, s'il est raccordé au réseau externe

> Le mode distribué des IP flottantes (`enable_distributed_floating_ip`) suppose que **chaque hyperviseur** ait une patte sur le réseau externe. Sinon, les IP flottantes passent elles aussi par la passerelle, comme le cas 3.

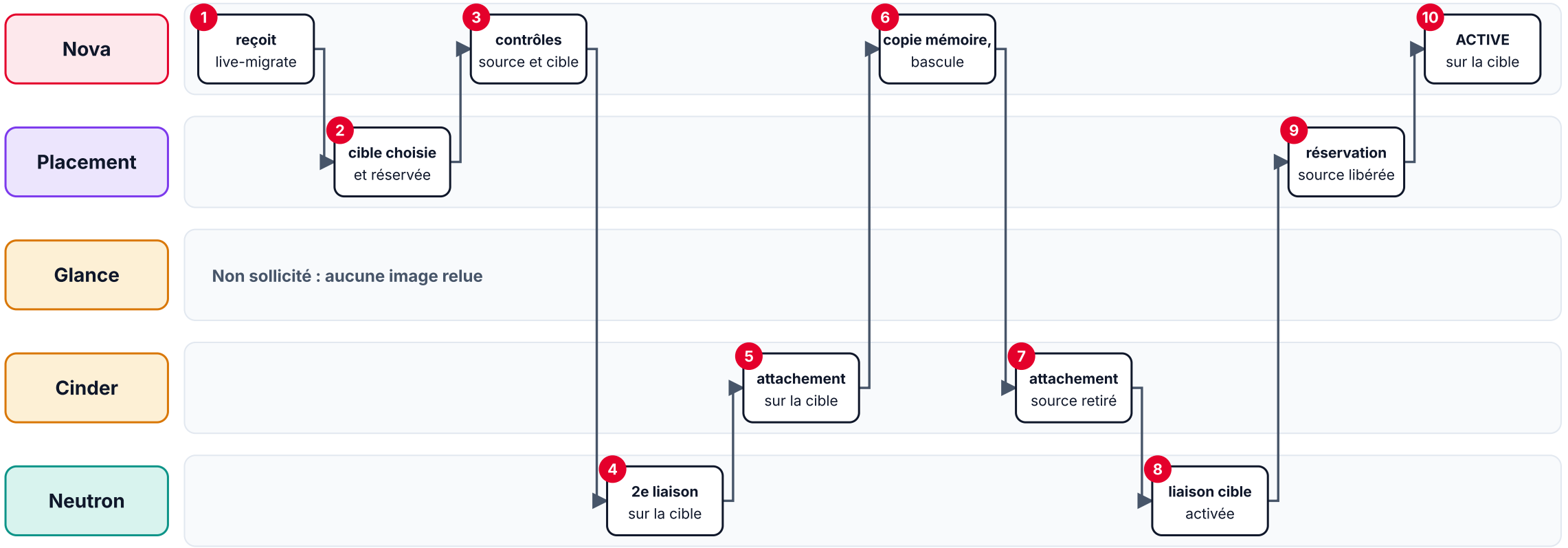
Adresses, DHCP, DNS, métadonnées : les services intégrés

Service	Qui le rend	Comment	Point d'attention
Attribution d'adresses (IPAM)	neutron-server	À la création du port, une adresse est prise dans la plage du sous-réseau et enregistrée en base	L'adresse appartient au **port**, pas à l'instance : elle survit à un redémarrage, pas à la suppression du port
DHCP	OVN, sur chaque hyperviseur	ovn-controller répond lui-même aux requêtes, d'après les options inscrites en base	Aucun serveur ni agent DHCP ; les options du sous-réseau (DNS, routes, MTU) y sont reprises
DNS interne	OVN	Les noms des ports d'un réseau sont résolus localement	Requiert le pilote d'extension `dns` ; publication externe via Designate
Métadonnées	ovn-metadata-agent, sur chaque hyperviseur	Intercepte `169.254.169.254`, ajoute l'identité de l'instance, relaie vers l'API de Nova	Un espace de noms et un relais par réseau présent sur l'hôte
Routage, NAT	OVN	Routeurs logiques exécutés par les flux Open vSwitch	Pas d'espace de noms de routeur ; SNAT sur les passerelles
Répartition de charge	Octavia, avec le fournisseur OVN ou des amphores	Service distinct qui s'appuie sur les ports Neutron	Hors de Neutron lui-même

Tout est local à l'hyperviseur : avec OVN, une instance obtient son adresse, résout un nom et lit ses métadonnées sans qu'aucun paquet ne quitte son hôte.

Vérifier depuis l'instance : bail DHCP reçu ? `curl http://169.254.169.254/openstack/latest/meta_data.json` répond ? Deux tests qui valident d'un coup le port, le filtrage et les relais.

Les services dans une même opération : la migration à chaud



Scénario : migrer à chaud une instance démarrée sur un volume Ceph. **Placement** garde la réservation des deux côtés jusqu'à la fin (celle de la source passe au nom de la migration) ; **Neutron** et **Cinder** préparent la cible avant la bascule, puis la source est nettoyée. **Glance** n'intervient pas : le disque n'est pas recréé.

05

Déployer OpenStack : panorama et critères de choix

Outils communautaires, distributions d'éditeurs, offres gérées — comment ils fonctionnent et comment choisir

À L'ISSUE DE CE MODULE, VOUS SAUREZ

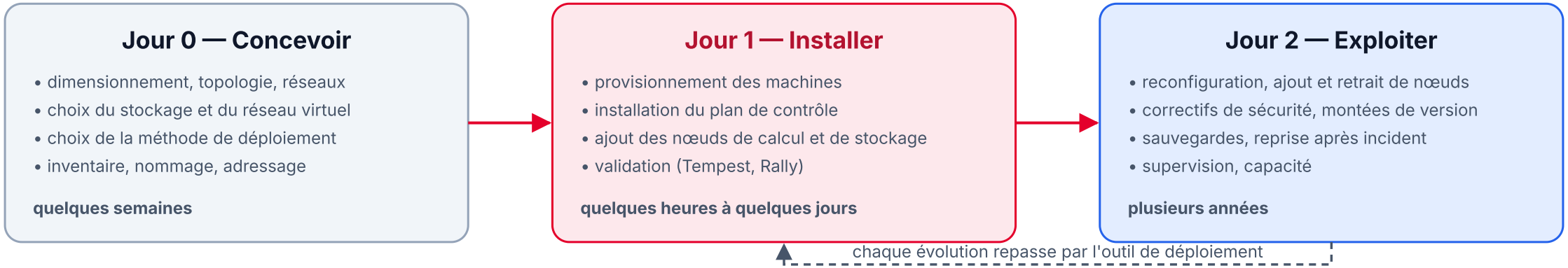
- Décomposer un déploiement en **couches** et dire quel outil couvre laquelle.
- Décrire le fonctionnement de DevStack, Kolla-Ansible, OpenStack-Ansible, OpenStack-Helm, Sunbeam, RHOSO et MOSK.
- **Comparer** les méthodes sur l'exploitation : montée de version, personnalisation, fonctionnement déconnecté.
- Classer les méthodes selon trois axes : **forme de livraison**, **orchestrateur**, **socle d'exécution**.
- Distinguer outil communautaire, distribution supportée et offre gérée.
- Appliquer une **grille de critères** et un arbre de décision à un contexte donné.

Les six couches d'un déploiement

Couche	Ce qu'il faut faire	Qui s'en charge, selon la méthode
6 · Configuration du cloud ce que voient les utilisateurs	réseaux externes, gabarits, images, quotas, projets, types de volumes	vous, avec Terraform ou la CLI ; amorcée par certains outils (sunbeam configure)
5 · Services OpenStack Keystone, Nova, Neutron, Cinder, Glance…	installer, configurer, initialiser les bases, enregistrer les points d'accès, démarrer	**cœur de métier de tous les outils** Kolla-Ansible, OSA, Helm, Sunbeam, RHOSO…
4 · Services d'infrastructure base de données, bus, répartiteur, cache, stockage	MariaDB Galera, RabbitMQ, HAProxy, Memcached, Ceph, OVN — en haute disponibilité	inclus dans la plupart des outils ; Ceph souvent déployé à part
3 · Préparation des hôtes système prêt à recevoir les services	paquets, noyau, horloge, interfaces et ponts, moteur de conteneurs ou Kubernetes	en partie : bootstrap-servers (Kolla), setup-hosts (OSA), prepare-node (Sunbeam)
2 · Provisionnement des machines du serveur nu au système installé	amorçage PXE, installation du système, configuration des disques et du réseau	outil séparé : MAAS, Ironic / Bifrost, Metal3, Kayobe, ou à la main
1 · Matériel et réseau physique baies, câblage, commutateurs, adressage	conception, achat, mise en baie — toujours à votre charge, sauf offre hébergée	aucun outil de déploiement

Première question à poser à une méthode : à quelle couche commence-t-elle, à laquelle s'arrête-t-elle ? Deux outils « de déploiement » peuvent ne couvrir que la couche 5 pour l'un, les couches 2 à 6 pour l'autre.

Avant, pendant, après : le déploiement est un cycle



Ce que cela change dans le choix

- Une démonstration juge le **jour 1** ; l'exploitation se joue au **jour 2**. Une installation en vingt minutes ne dit rien de la montée de version dans deux ans.
- L'outil choisi au départ devient le **seul chemin** pour modifier le cloud : toute intervention manuelle hors outil sera écrasée à la prochaine exécution.
- Changer de méthode en cours de vie revient presque toujours à **reconstruire** un cloud à côté et à migrer les charges.

Opération du jour 2	Question à poser à l'outil
Changer une option	Une commande, ou un redéploiement complet ?
Ajouter un hyperviseur	Sans toucher aux nœuds existants ?
Monter de version	Sans interruption des API ? Saut de version possible ?
Remplacer un contrôleur	Procédure documentée et testée ?
Revenir en arrière	Prévu, ou restauration de sauvegarde ?

Trois axes pour classer les méthodes

Axe 1 · Forme de livraison

Sous quelle forme le code d'OpenStack arrive sur les machines.

Options : paquets de la distribution · sources dans un environnement Python · images de conteneurs · snaps

Axe 2 · Orchestrateur

Quel moteur enchaîne les étapes, applique la configuration et la maintient.

Options : scripts shell · Ansible · Puppet · Juju · opérateurs Kubernetes et Helm

Axe 3 · Socle d'exécution

Sur quoi tourne le plan de contrôle une fois installé.

Options : directement sur l'hôte · conteneurs Docker ou Podman · conteneurs LXC · pods Kubernetes

Méthode	Forme de livraison	Orchestrateur	Socle du plan de contrôle
DevStack	Sources (git)	Scripts shell	Hôte, services systemd
Installation manuelle	Paquets de la distribution	L'administrateur	Hôte
Kolla-Ansible	Images de conteneurs Kolla	Ansible	Conteneurs Docker ou Podman
OpenStack-Ansible	Sources dans des environnements Python	Ansible	Conteneurs LXC, ou hôte
OpenStack-Helm et dérivés	Images de conteneurs (LOCI)	Helm, opérateurs	Pods Kubernetes
Sunbeam (Canonical)	Images OCI (rocks) et snaps	Juju, Terraform	Pods Kubernetes ; snaps pour le calcul
RHOSO (Red Hat)	Images de conteneurs	Opérateurs OpenShift, Ansible	Pods OpenShift ; RHEL pour le calcul
MOSK (Mirantis)	Images de conteneurs	Opérateur, Helm	Pods sur le Kubernetes de Mirantis

Ces trois choix déterminent presque tout le reste : compétences nécessaires, façon de monter de version, facilité à corriger un service, comportement en environnement déconnecté.

Trois générations d'outils

du début des années 2010 à aujourd'hui

1 · Gestion de configuration

paquets + Puppet, Chef, scripts

- **Puppet OpenStack**
- **Charms (Juju)**
- Packstack
- Chef OpenStack
- Fuel
- TripleO (débuts)

Un état voulu décrit dans des manifestes ;
des services installés directement sur les hôtes.

2 · Conteneurs pilotés par Ansible

une image ou un conteneur par service

- **Kolla-Ansible**
- **OpenStack-Ansible**
- **Kayobe**
- TripleO (conteneurs)

Des playbooks idempotents ; chaque service
isolé, remplaçable par changement d'image.

3 · Kubernetes comme socle

le plan de contrôle devient une application

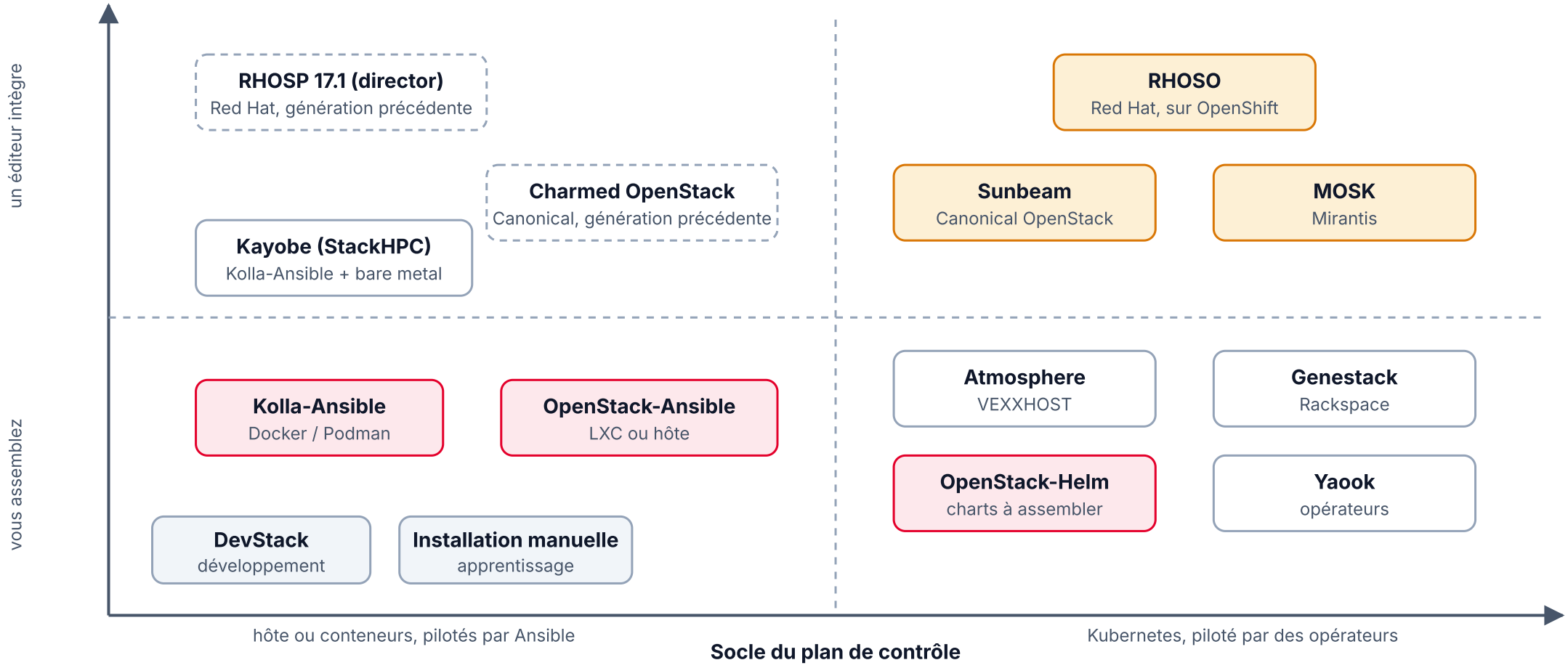
- **OpenStack-Helm**
- **Sunbeam**
- **RHOSO**
- **MOSK**
- **Atmosphere**
- **Genestack · Yaook**

Des opérateurs qui réconcilient en continu ;
Kubernetes gère santé et replanification.

Cadres en pointillés : projets retirés ou en fin de vie. TripleO et Chef OpenStack ne figurent plus parmi les projets officiels ; Fuel a disparu depuis longtemps.

Les générations coexistent : Kolla-Ansible et OpenStack-Ansible restent très utilisés et activement maintenus. La troisième génération domine chez les éditeurs, pas encore dans toutes les équipes d'exploitation.

Carte du paysage



Lecture : en bas, des outils libres que l'on assemble et exploite soi-même ; en haut, des distributions intégrées et supportées. À gauche, la filière Ansible ; à droite, la filière Kubernetes, vers laquelle les éditeurs ont tous basculé.

Statut des projets en 2026

Outil	Porté par	Statut	À retenir
DevStack	Communauté OpenStack (assurance qualité)	**Actif**	Outil de développement et de test ; sert à l'intégration continue du projet
Kolla, Kolla-Ansible	Projet officiel OpenStack	**Actif**	Suit chaque version ; hôtes Ubuntu 24.04, Debian 13, Rocky Linux 10, CentOS Stream 10 (sans images Kolla)
OpenStack-Ansible	Projet officiel OpenStack	**Actif**	Suit chaque version ; Ubuntu, Debian, Rocky, CentOS Stream
OpenStack-Helm	Projet officiel OpenStack	**Actif**	Collection de charts ; base de plusieurs distributions sur Kubernetes
Sunbeam	Projet officiel OpenStack, mené par Canonical	**Actif**	Base de Canonical OpenStack ; succède à MicroStack
OpenStack Charms	Projet officiel OpenStack, mené par Canonical	Maintenu	Génération précédente de Canonical ; les nouveaux déploiements vont vers Sunbeam
Puppet OpenStack	Projet officiel OpenStack	Maintenu	Modules de configuration, sans outil d'orchestration associé
TripleO	Communauté, Red Hat	**Retiré**	Remplacé chez Red Hat par des opérateurs sur OpenShift (RHOSO)
Chef OpenStack	Communauté	**Retiré**	Dépôts vidés ; plus aucun mainteneur
Packstack	Communauté RDO	Fin de vie	À ne pas retenir pour un nouveau projet ; vérifier son état avant tout usage

Enseignement : un outil de déploiement vit et meurt. La santé du projet — nombre de contributeurs, régularité des versions, diversité des entreprises qui le portent — est un critère de choix à part entière.

Où vérifier : la liste des équipes officielles sur `governance.openstack.org`, l'activité des dépôts sur `opendev.org`, la date de la dernière version sur `releases.openstack.org`.

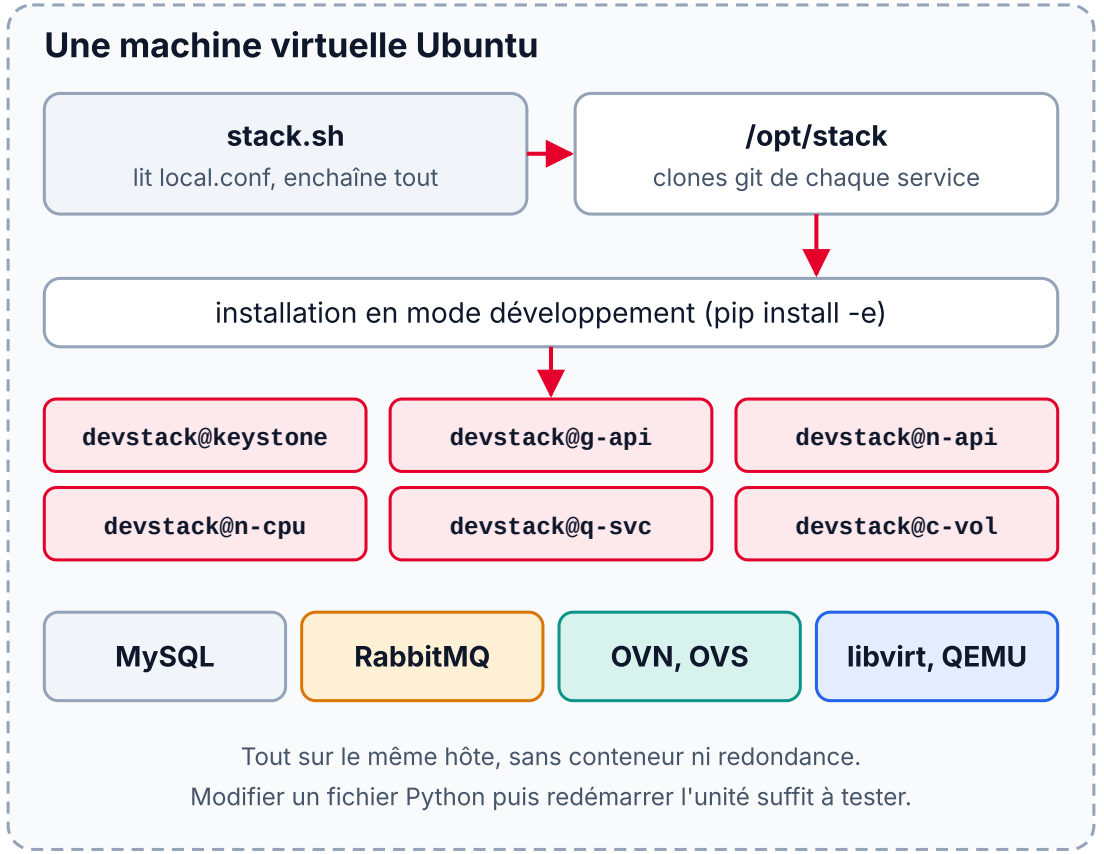
DevStack : le cloud du développeur

Principe : un ensemble de scripts shell qui clone les dépôts de chaque service, installe leurs dépendances et lance tout sur **une seule machine**, à partir des sources.

- **Pour qui :** contributeurs au code d'OpenStack, tests d'intégration, découverte des API.
- **Ce qu'il installe :** Keystone, Glance, Placement, Nova, Neutron (OVN), Cinder (LVM sur fichier), Horizon ; d'autres services par greffons.
- **Comment :** chaque service devient une unité systemd `devstack@<service>` ; les journaux se lisent avec `journalctl`.
- **Atout :** n'importe quelle branche, n'importe quel correctif en cours de revue, en une variable.

Jamais en production : pas de haute disponibilité, mots de passe simples, système hôte profondément modifié, aucune procédure de montée de version. Un redémarrage de la machine peut laisser le cloud inutilisable.

À lancer sur une machine **jetable** : une machine virtuelle dédiée, jamais son poste de travail.



Les installations tout-en-un

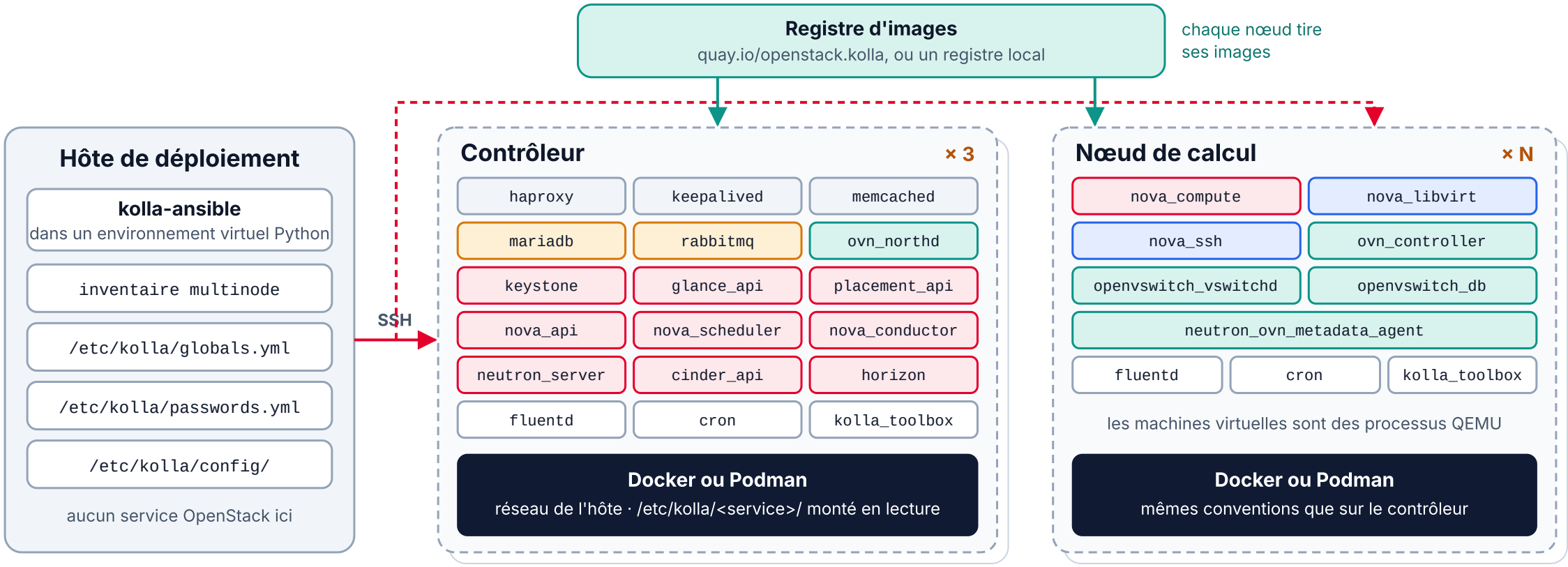
La plupart des outils de production proposent un mode « une seule machine ». On y retrouve les mêmes conteneurs, les mêmes fichiers et les mêmes commandes qu'en production : c'est le meilleur terrain d'apprentissage d'un outil donné.

Méthode	Ressources minimales	Mise en route	Fidélité à la production	Survit à un redémarrage	Usage conseillé
DevStack	8 Go de RAM, 60 Go de disque	`./stack.sh`	Faible : services sur l'hôte, sources	Non garanti	Développement, lecture du code
Kolla-Ansible « all-in-one »	8 Go de RAM, 40 Go, 2 interfaces	Inventaire `all-in-one`, trois commandes	Élevée : mêmes images, mêmes playbooks	Oui	Découvrir Kolla, lab de formation
OpenStack-Ansible « AIO »	16 Go de RAM, 80 Go ou plus	Scripts d'amorçage AIO, puis playbooks	Élevée : mêmes rôles, conteneurs LXC	Oui	Découvrir OSA, tester une configuration
Sunbeam, nœud unique	16 Go de RAM, 4 cœurs, 100 Go ou plus	`sunbeam cluster bootstrap`	Élevée : Kubernetes, Juju, snaps	Oui	Découvrir Canonical OpenStack
Installation manuelle	2 machines virtuelles, 4 à 8 Go chacune	Guide d'installation, service par service	Moyenne : paquets, sans haute disponibilité	Oui	Comprendre chaque composant

Limite commune : aucune haute disponibilité, un seul hyperviseur, stockage sur fichier ou sur disque local. On y apprend l'outil et les API, pas le comportement en panne.

Pour aller plus loin sans matériel : trois à cinq machines virtuelles avec virtualisation imbriquée permettent de monter un vrai plan de contrôle redondant et de simuler la perte d'un nœud.

Kolla-Ansible : un conteneur par processus



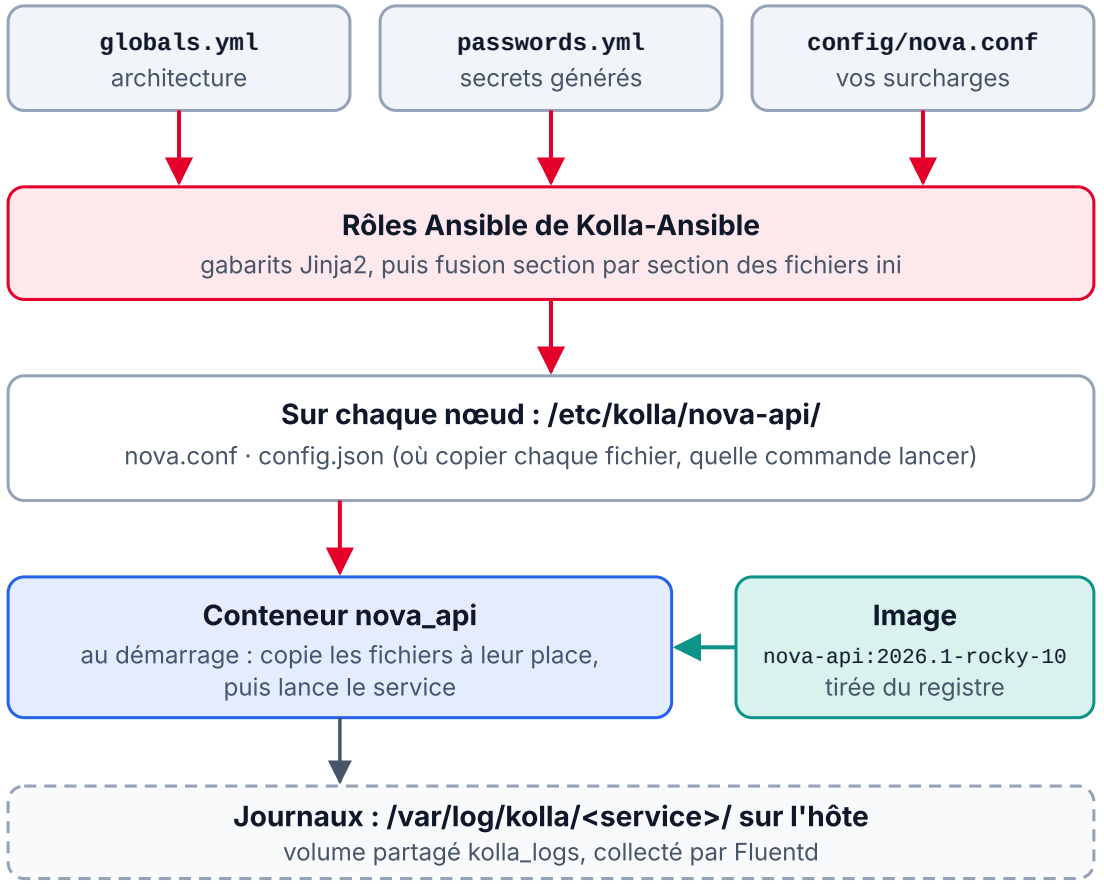
À retenir : les images sont construites à l'avance par le projet **Kolla** ; **Kolla-Ansible** se contente de générer la configuration, de tirer les images et de démarrer les conteneurs. Mettre à jour un service, c'est remplacer son image.

Kolla-Ansible : le modèle de configuration

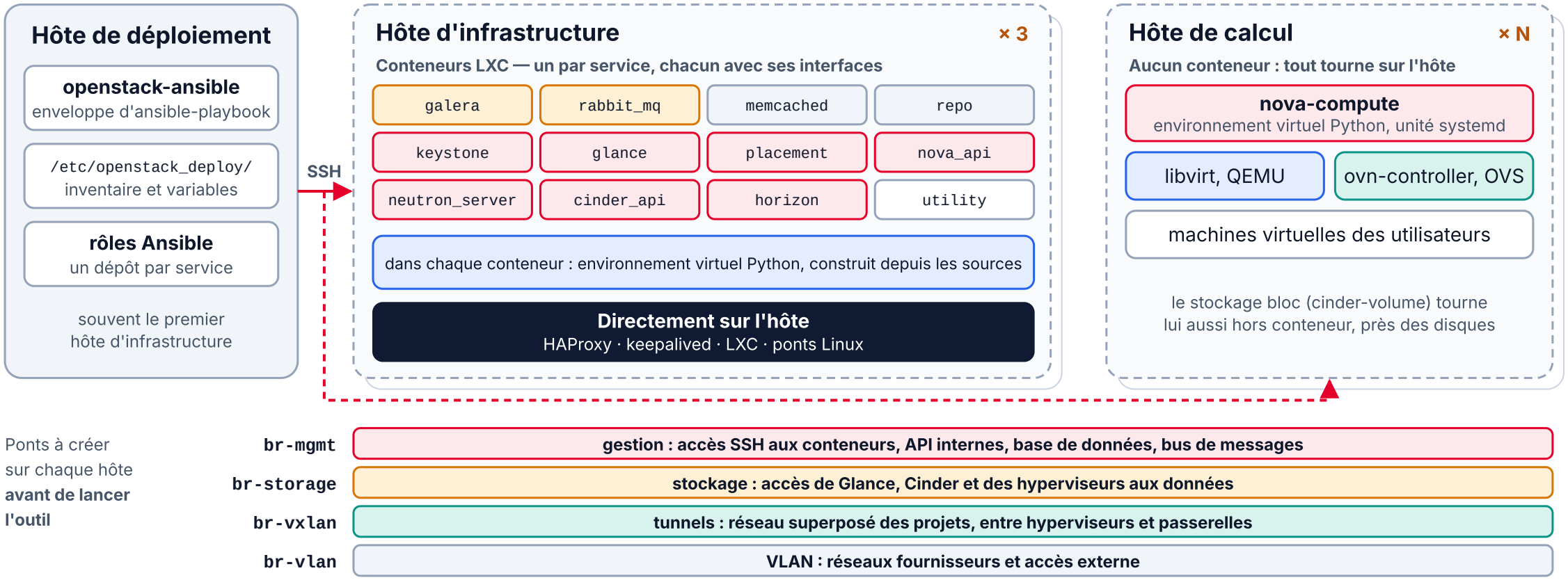
```
# /etc/kolla/globals.yml — extrait
kolla_base_distro: "rocky"
kolla_internal_vip_address: "10.20.0.10"
network_interface: "bond0.20"
neutron_external_interface: "bond1"
neutron_plugin_agent: "ovn"
kolla_enable_tls_external: "yes"
enable_cinder: "yes"
cinder_backend_ceph: "yes"
enable_central_logging: "yes"
```

Trois niveaux de réglage

- `globals.yml` — les choix d'architecture : interfaces, adresse virtuelle, services activés. Quelques dizaines de lignes suffisent.
- `/etc/kolla/config/` — n'importe quelle option d'un service. Un fichier `nova.conf` déposé ici est **fusionné** avec celui que génère l'outil ; un sous-dossier au nom d'un hôte limite la surcharge à cet hôte.
- **Images** — pour un correctif de code ou un pilote tiers, on reconstruit l'image avec `kolla-build` et on la pousse dans son registre.



OpenStack-Ansible : des services isolés dans des conteneurs LXC



À retenir : OpenStack-Ansible ne télécharge pas d'images. Il **construit** les services depuis les sources au moment du déploiement, dans des conteneurs système qui se comportent comme de petites machines ; un mode sans conteneur (« metal ») existe aussi.

 Réglages, mises à jour, dépannage : p. 123

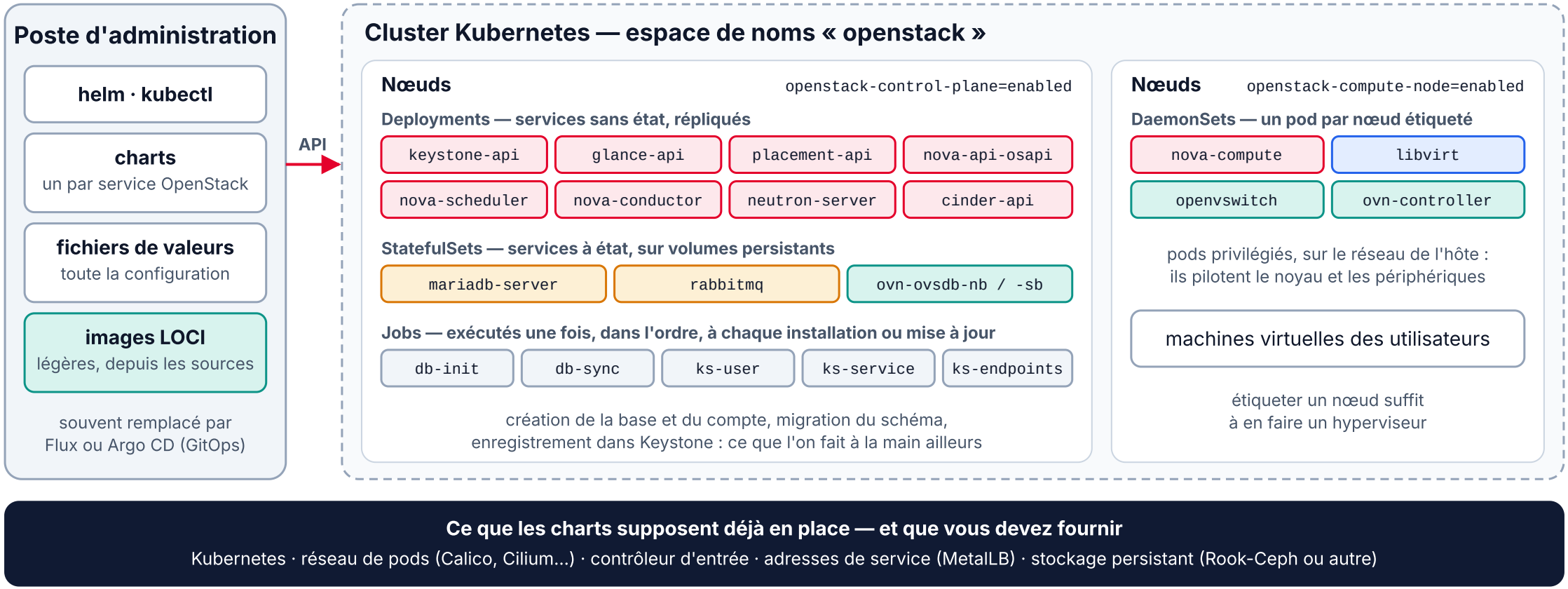
Kolla-Ansible et OpenStack-Ansible face à face

Critère	Kolla-Ansible	OpenStack-Ansible
Forme de livraison	**images de conteneurs construites à l'avance**	code source installé dans des environnements virtuels Python
Isolation	**conteneurs applicatifs Docker ou Podman, réseau de l'hôte**	conteneurs système LXC reliés à des ponts, ou directement sur l'hôte
Un service =	**un conteneur par processus (`nova_api`, `nova_scheduler` …)**	un conteneur par service, plusieurs unités systemd à l'intérieur
Fichiers à écrire	`globals.yml`, `passwords.yml`, **inventaire** `multinode`	`openstack_user_config.yml`, `user_variables.yml`, `user_secrets.yml`
Haute disponibilité	**HAProxy, keepalived, Galera, RabbitMQ en cluster**	mêmes briques ; Galera et RabbitMQ en conteneurs LXC, HAProxy sur l'hôte
Réseau à préparer	**deux interfaces et une adresse virtuelle**	quatre ponts créés et adressés sur chaque hôte
Durée d'un déploiement	**courte : les images sont déjà prêtes**	plus longue : chaque service est construit sur place
Machine nue	**par Kayobe, projet frère**	hors périmètre

Kolla-Ansible convient à qui veut un résultat reproductible et rapide, accepte les choix d'architecture de l'outil et maîtrise Docker ou Podman.

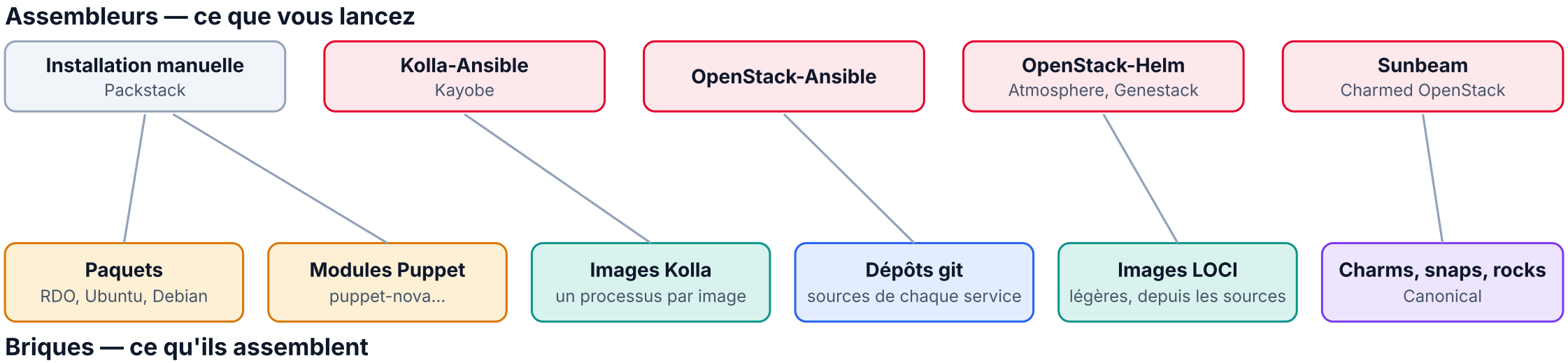
OpenStack-Ansible convient à qui veut garder la main sur chaque service, suivre des branches du code de près et raisonner en « serveurs » plutôt qu'en images.

OpenStack-Helm : OpenStack comme application Kubernetes



Lecture du schéma : tout ce qui compose OpenStack devient un objet Kubernetes — Deployments pour les services sans état, StatefulSets pour la base et le bus, Jobs pour les tâches d'installation, DaemonSets pour les hyperviseurs. L'étiquette d'un nœud décide de son rôle ; le bandeau du bas liste ce que le cluster doit déjà fournir.

Briques et assembleurs : qui réutilise quoi



Puppet OpenStack

Un module par service, maintenu par une équipe officielle. Sert de fondation à Packstack et a longtemps servi de fondation à TripleO. Pertinent si Puppet gère déjà tout votre parc ; peu de nouveaux déploiements partent de là.

Images Kolla et LOCI

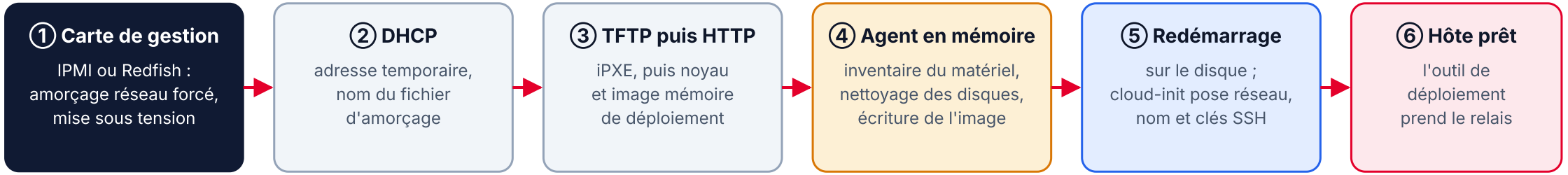
Deux projets officiels qui ne font **que** des images. Kolla : images complètes, par distribution. LOCI : images minimales construites depuis les sources. Réutilisables par votre propre orchestrateur.

Construire son propre outil ?

Possible à partir de ces briques, et pratiqué par de très grands opérateurs. À réserver aux équipes capables de maintenir l'outil pendant dix ans : c'est un produit interne à part entière.

MAAS en détail : module 6

Avant les services : provisionner les machines



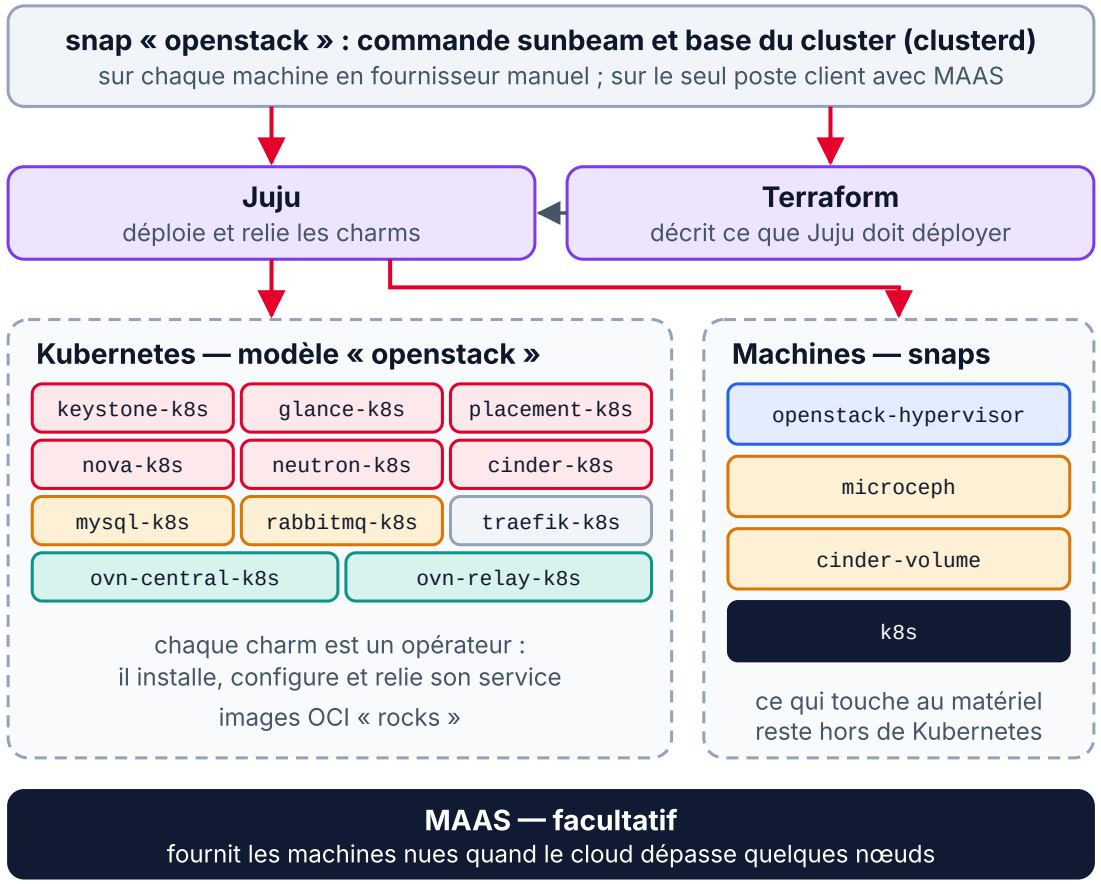
Réseau dédié au provisionnement, non routé, sur lequel le serveur d'amorçage est le seul serveur DHCP.

Outil	Principe	Utilisé par	Remarque
MAAS	**service autonome de Canonical : découverte, inventaire, installation, réseau**	Sunbeam, Charmed OpenStack	interface web, API ; Ubuntu d'abord
Ironic autonome, Bifrost	**le service bare metal d'OpenStack, utilisé seul, sans Keystone ni Nova**	Kayobe	même code que celui qu'offrira ensuite le cloud
Metal3	**Ironic piloté par des ressources Kubernetes (`BareMetalHost`)**	RHOSO, plateformes Kubernetes	cohérent si tout est déjà déclaré dans Kubernetes
Installateur existant	**Kickstart, autoinstall, Foreman, images maison**	Kolla-Ansible, OSA, OpenStack-Helm	souvent déjà en place dans l'entreprise : à réutiliser

À trancher tôt : un outil qui « commence au système installé » suppose que vous savez réinstaller quarante serveurs à l'identique. Sans provisionnement automatisé, le remplacement d'un nœud devient une opération manuelle.

 Approfondi au module 6

Canonical : Sunbeam et Charmed OpenStack



	Sunbeam	Charmed OpenStack
Plan de contrôle	**pods Kubernetes**	conteneurs LXD sur des machines
Pilotage	**commande `sunbeam`, qui orchestre Juju**	Juju directement, par lots de charms
Machines nues	**MAAS facultatif**	MAAS dans la pratique
Taille minimale	**une seule machine**	plusieurs machines
Stockage	**MicroCeph**	Ceph, par charms
Réseau	**OVN**	OVN
Position	**voie recommandée pour un nouveau cloud**	maintenu pour les clouds existants

- **« Canonical OpenStack »** est le nom commercial de la distribution fondée sur Sunbeam ; le code de Sunbeam est un projet officiel d'OpenStack.
- **Gratuit sans contrat** : le support s'ajoute par abonnement Ubuntu Pro, sans changer de logiciel.
- **Peu de réglages exposés** : l'outil fait les choix d'architecture à votre place. C'est sa force pour démarrer et sa limite pour les cas atypiques.

 Approfondi au module 6

Sunbeam : déroulé et cycle de vie

Prérequis : Ubuntu Server LTS, un compte avec `sudo`, un nom de machine pleinement qualifié, une plage d'adresses libre pour les points d'accès.

```
# Première machine — le canal fixe la version d'OpenStack
sudo snap install openstack --channel 2026.1/stable
sunbeam prepare-node-script --bootstrap | bash -x
newgrp snap_daemon
sunbeam cluster bootstrap \
  --role control,compute,storage
sunbeam configure --openrc demo-openrc

# Machines suivantes (snap et prepare-node-script déjà passés)
sunbeam cluster add noeud2.exemple.lan \
  --output noeud2.jeton      # sur la première
cat noeud2.jeton | sunbeam cluster join \
  --role control,compute,storage -   # sur la nouvelle
sunbeam cluster resize                  # passe en HA à 3

# Vie du cloud
sunbeam enable observability
sunbeam cluster refresh          # correctifs de la piste
```

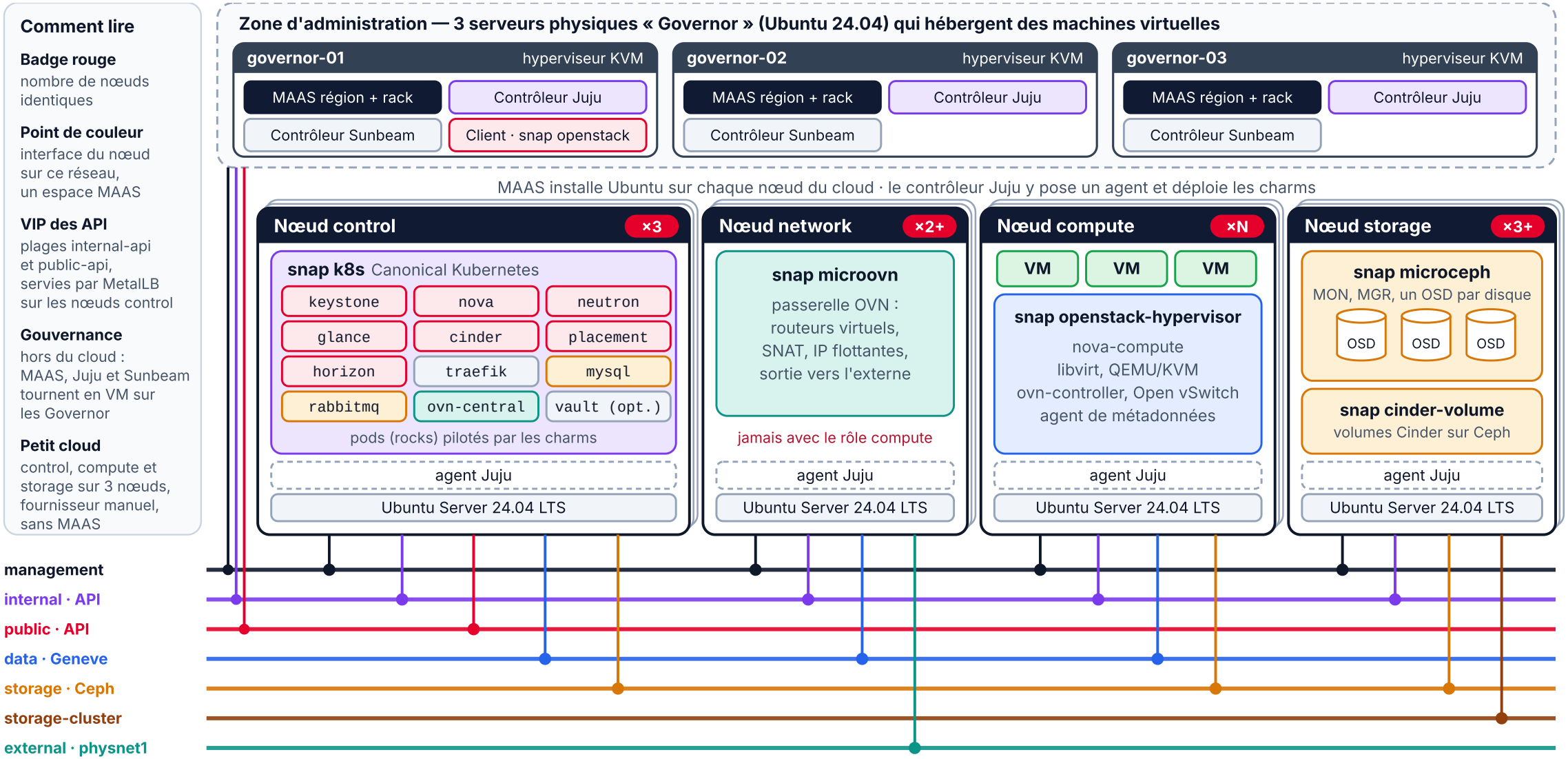
Sujet	Ce que propose Canonical
Versions à long terme	2024.1 et 2026.1, alignées sur Ubuntu LTS
Durée de maintenance	5 ans en standard ; jusqu'à 12 avec Ubuntu Pro (+5) et Legacy Support (+2)
Support	abonnement Ubuntu Pro, par nœud
Offre gérée	exploitation par Canonical, sur votre matériel
Fonctions optionnelles	`sunbeam enable` : supervision, équilibrage de charge, orchestration, validation…

- **Même outil de un à plusieurs centaines de nœuds :** on commence sur une machine, on ajoute des nœuds sans réinstaller.
- **Mises à jour dans une piste :** `cluster refresh` applique les révisions du canal ; passer d'une piste à l'autre (2024.1 → 2026.1) n'est pas pris en charge par cette procédure.
- **Diagnostic :** `juju status` donne l'état de chaque charm, `kubectl` celui des pods, `snap logs` celui des services sur l'hôte.

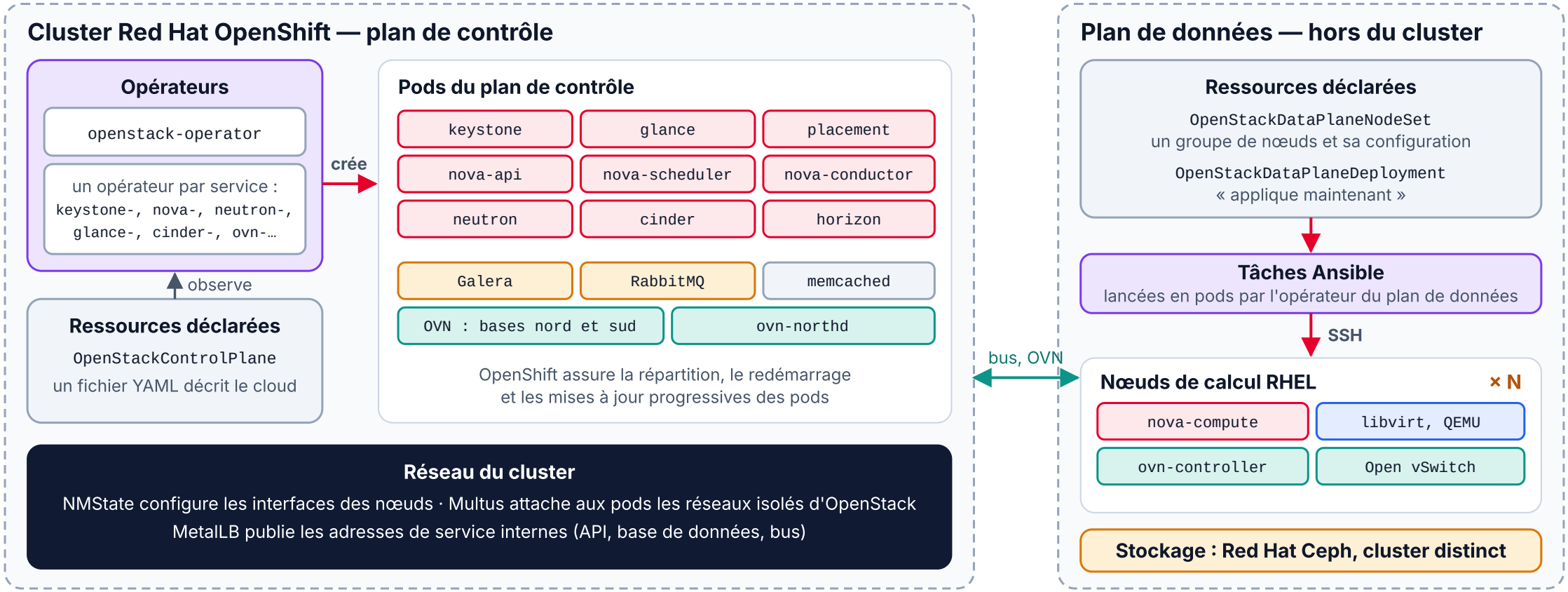
> Les durées et le contenu des offres évoluent : à confirmer sur le site de l'éditeur au moment du choix.

 Approfondi au module 6

Sunbeam en production : l'infrastructure physique

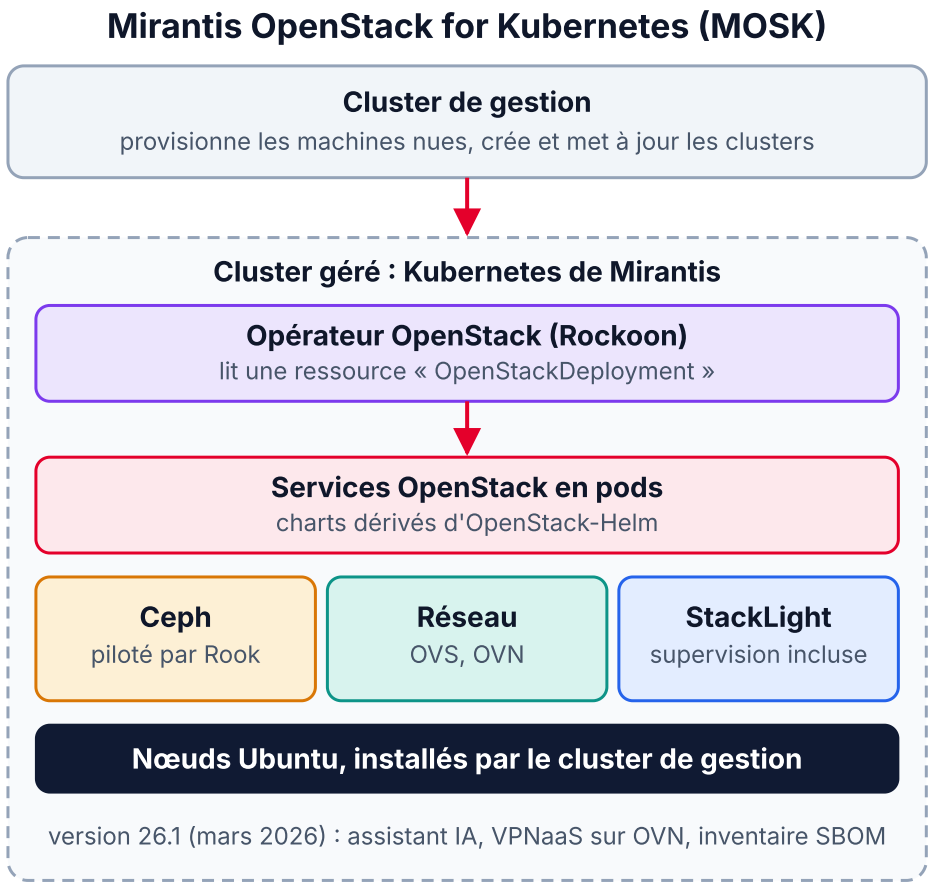


Red Hat OpenStack Services on OpenShift (RHOSO)



À retenir : le plan de contrôle est une application OpenShift décrite par des ressources Kubernetes ; les hyperviseurs restent des serveurs RHEL classiques, configurés par Ansible — mais Ansible est déclenché par un opérateur, plus par un administrateur.

Mirantis et les autres éditeurs



Éditeur	Offre	Socle technique
Canonical	Canonical OpenStack, Charmed OpenStack	Juju, Kubernetes, snaps
Red Hat	RHOSO	opérateurs sur OpenShift
Mirantis	MOSK	Kubernetes, opérateur, charts Helm
VEXXHOST	Atmosphere, avec support ou hébergé	Kubernetes, charts Helm
Rackspace	OpenStack exploité par Rackspace	Genestack
Wind River	plateforme pour la périphérie	StarlingX
Platform9	plan de contrôle fourni en service	propre à l'éditeur

- **Tendance commune :** tous les éditeurs ont déplacé le plan de contrôle sur Kubernetes ces dernières années.
- **Intégrateurs :** beaucoup de clouds en production reposent sur Kolla-Ansible ou OpenStack-Ansible, avec le support d'une société de services spécialisée plutôt que d'un éditeur.
- **Éditeurs sortis du marché :** SUSE a arrêté sa distribution ; un éditeur peut se retirer, d'où l'intérêt d'un socle ouvert.

> Catalogue et noms d'offres changeants : à vérifier auprès de chaque éditeur.

Qui fait quoi : cinq modes de consommation

	Outil communautaire Kolla-Ansible, OSA, Helm	**Distribution supportée** Canonical, Red Hat, Mirantis	**Cloud privé géré** sur votre site, exploité par un tiers	**Cloud privé hébergé** matériel dédié chez un tiers	**Cloud public OpenStack** OVHcloud, Infomaniak, Cleura…
Matériel, centre de données	vous	vous	vous	fournisseur	fournisseur
Conception de l'architecture	vous	vous, guidé par l'éditeur	fournisseur	fournisseur	fournisseur
Déploiement	vous	vous, outil de l'éditeur	fournisseur	fournisseur	fournisseur
Correctifs du code	vous, depuis l'amont	éditeur	fournisseur	fournisseur	fournisseur
Montées de version	vous	vous, procédure de l'éditeur	fournisseur	fournisseur	fournisseur
Supervision, astreinte	vous	vous	fournisseur	fournisseur	fournisseur
Projets, quotas, images	vous	vous	vous	vous	fournisseur
Applications, données	vous	vous	vous	vous	vous

Lecture : de gauche à droite, on échange de la maîtrise et du temps d'ingénieur contre une facture et une dépendance. Le logiciel, lui, reste le même : les API sont identiques dans les cinq colonnes.

Avant de choisir un outil, choisir une colonne. Une équipe de trois personnes qui prend la première colonne consacrera l'essentiel de son temps à l'exploitation de la plateforme, pas aux utilisateurs.

M5 · COMPARAISON TRANSVERSALE Sunbeam face aux autres : module 6

Fiche technique comparée

	Kolla-Ansible	OpenStack-Ansible	OpenStack-Helm	Sunbeam	RHOSO	MOSK
Statut	projet officiel, communautaire	projet officiel, communautaire	projet officiel, communautaire	projet officiel ; support Canonical	produit Red Hat, sous souscription	produit Mirantis, sous souscription
Système hôte	Ubuntu, Debian, Rocky ; CentOS Stream sans images	Ubuntu, Debian, CentOS Stream, Rocky	Ubuntu 24.04 testé	Ubuntu LTS	RHEL, RHCOS	Ubuntu
Réseau virtuel	OVN ou OVS, au choix	OVN par défaut	OVN ou OVS	OVN	OVN	OVN ou OVS
Ceph	externe, à déployer à part	externe, ou rôles intégrés	Rook, dans le cluster	MicroCeph, inclus ; quelques baies tierces	Red Hat Ceph, externe	Rook, inclus
Machines nues	avec Kayobe	non	non	avec MAAS	Metal3	inclus
Haute disponibilité	HAProxy, keepalived, Galera	HAProxy, keepalived, Galera	réplicas, services Kubernetes	réplicas, Traefik	réplicas, MetalLB, Galera	réplicas, services Kubernetes
Taille minimale	une machine (tout-en-un)	une machine (tout-en-un)	un cluster à un nœud	une machine	cluster OpenShift compact et un nœud de calcul	cluster de gestion et cluster géré

Lecture : les trois colonnes de gauche sont des outils que l'on assemble soi-même ; les trois de droite sont des ensembles intégrés où l'outil choisit pour vous le système, le réseau et le stockage. Orchestrateur, socle et forme de livraison : tableau des trois axes, p. 104. Valeurs relevées dans les documentations en octobre 2026.

Montées de version : le vrai test d'une méthode



Une version sort tous les six mois. Depuis 2023.1, les versions « .1 » sont dites SLURP : on peut passer de l'une à la suivante sans s'arrêter sur la version intermédiaire.

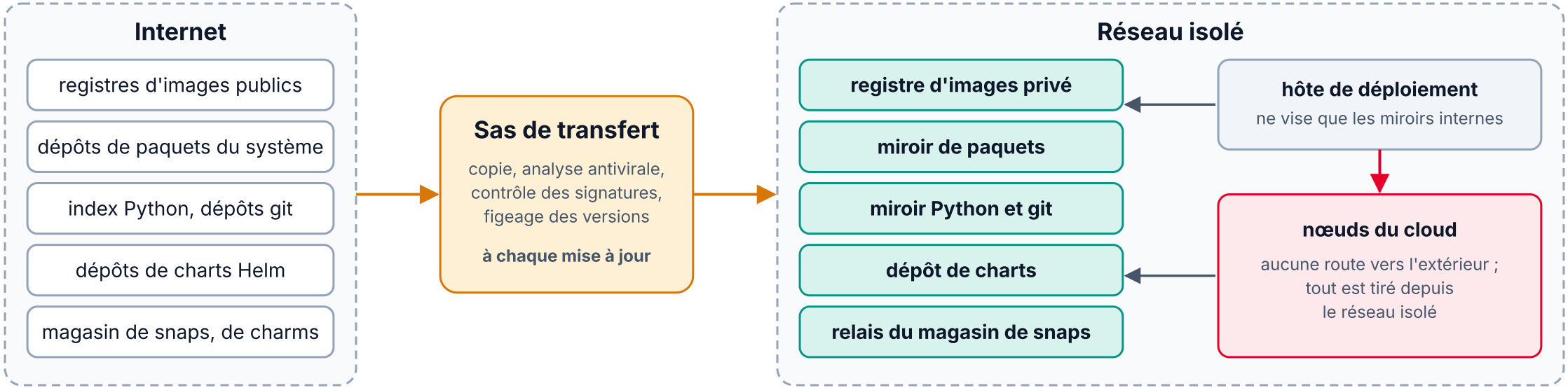
Ce que fait toute montée de version

- **Plan de contrôle d'abord :** nouveau code, migration du schéma des bases, redémarrage des API une à une.
- **Nœuds de calcul ensuite**, par lots : les agents acceptent de dialoguer avec un contrôleur plus récent qu'eux.
- **Les machines virtuelles continuent de tourner :** seule l'API est brièvement perturbée, pas le trafic des instances.
- **Pas de retour en arrière simple :** une fois le schéma migré, revenir exige de restaurer la sauvegarde.

Méthode	Ce qui change concrètement
Kolla-Ansible	nouvelle version de l'outil, nouvelles images, `upgrade`
OpenStack-Ansible	nouvelle étiquette git, environnements reconstruits
OpenStack-Helm	nouveaux charts et images, `helm upgrade` ordonné
Sunbeam	`cluster refresh` dans une piste ; pas de passage d'une piste à l'autre à ce jour
RHOSO, MOSK	l'opérateur enchaîne les étapes après validation

À vérifier pour chaque outil : le saut SLURP est-il supporté par **l'outil**, et la version du système hôte doit-elle changer en même temps ?

Déployer sans accès à Internet



Méthode	Ce qu'il faut rendre disponible en interne	Difficulté
Kolla-Ansible	les images Kolla, les paquets du système, les dépendances Python de l'outil	modérée : une seule variable désigne le registre
OpenStack-Ansible	miroirs de paquets, de l'index Python et des dépôts git de chaque service	élevée : de nombreuses sources à refléter
OpenStack-Helm	images, charts, et tout ce qu'exige déjà le cluster Kubernetes	modérée si le cluster est déjà déconnecté
Sunbeam	relais du magasin de snaps, registre pour les rocks, charms	mandataire pris en charge ; hors ligne complet pas encore disponible
RHOSO, MOSK	registre miroir alimenté par les outils de l'éditeur	procédure documentée et supportée

Durée de vie et support : combien de temps une version tient-elle ?

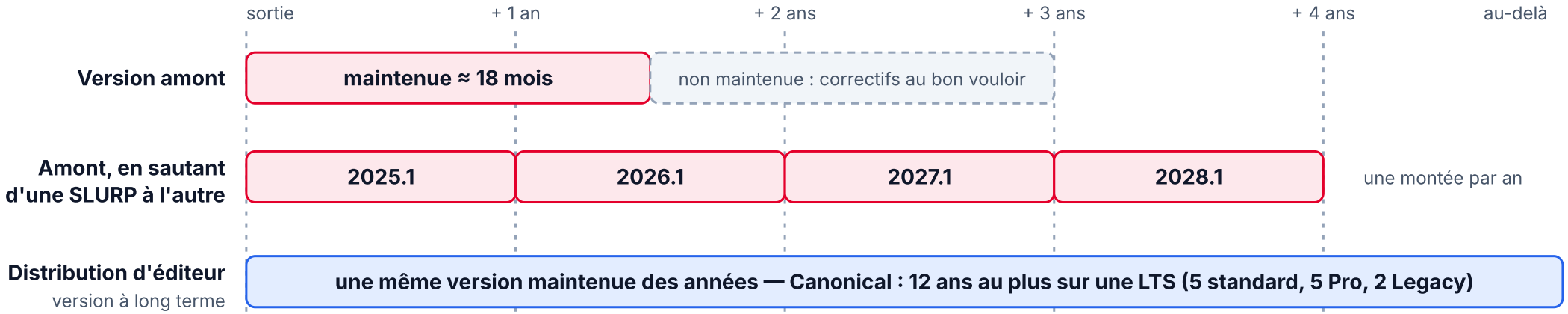


Schéma de principe : les durées exactes dépendent de chaque version et de chaque contrat.

Suivre l'amont

Une montée de version tous les six mois, ou une par an en ne prenant que les versions SLURP. Exige une chaîne de test et une équipe rodée. En retour : les nouveautés tout de suite, aucun coût de licence.

S'appuyer sur un éditeur

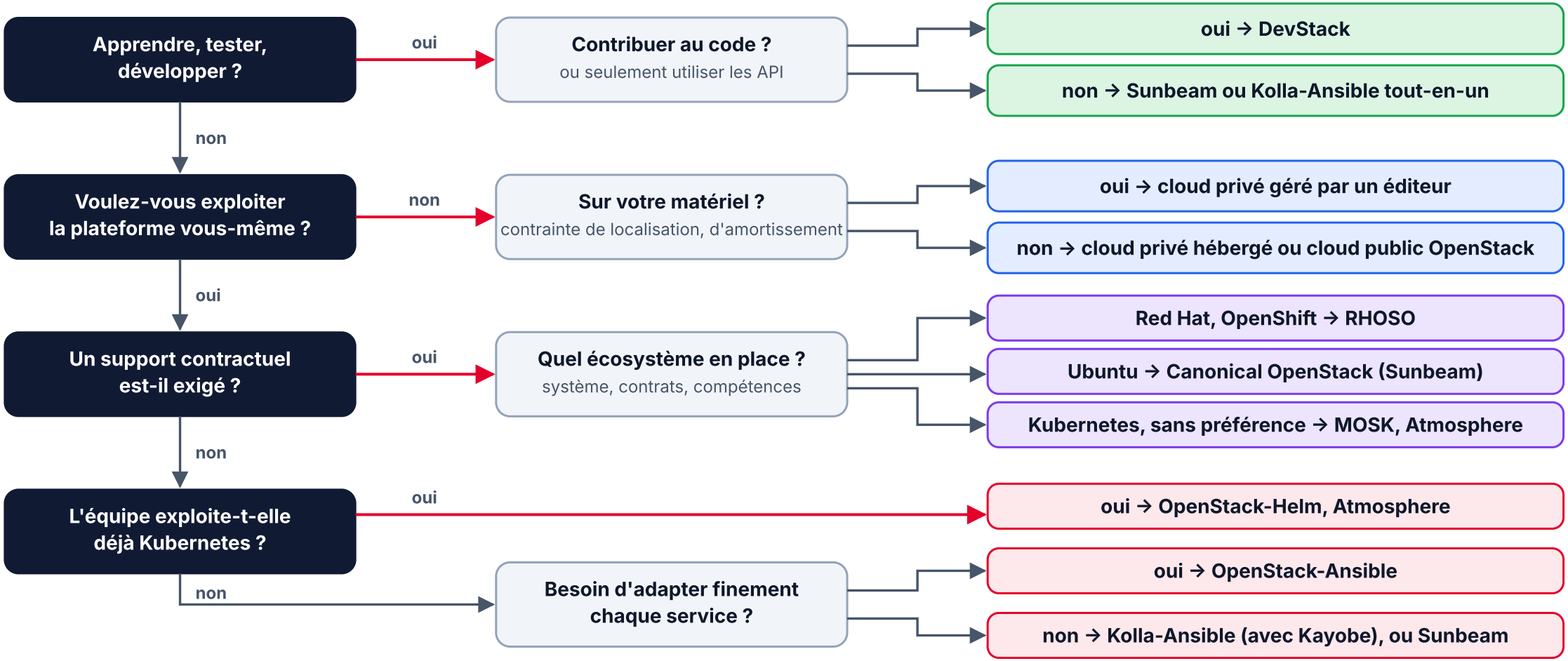
Une version garde ses correctifs de sécurité pendant des années. Les montées de version deviennent rares, mais chacune franchit plusieurs versions amont : à préparer comme un projet.

Le piège : ne rien faire

Un cloud communautaire laissé trois ans sans mise à jour n'a plus de correctifs et ne peut plus monter que version par version. Rattraper le retard coûte plus cher que de le prévenir.

Question à poser en comité : « Qui, nommément, fera la prochaine montée de version, et quand ? » Sans réponse, privilégier une version à long terme sous contrat.

Arbre de décision



Mode d'emploi : l'arbre donne une **première** piste en moins d'une minute. Il ne remplace ni la grille de notation ni la maquette : une contrainte forte (réseau isolé, matériel imposé) peut invalider la branche obtenue.

06

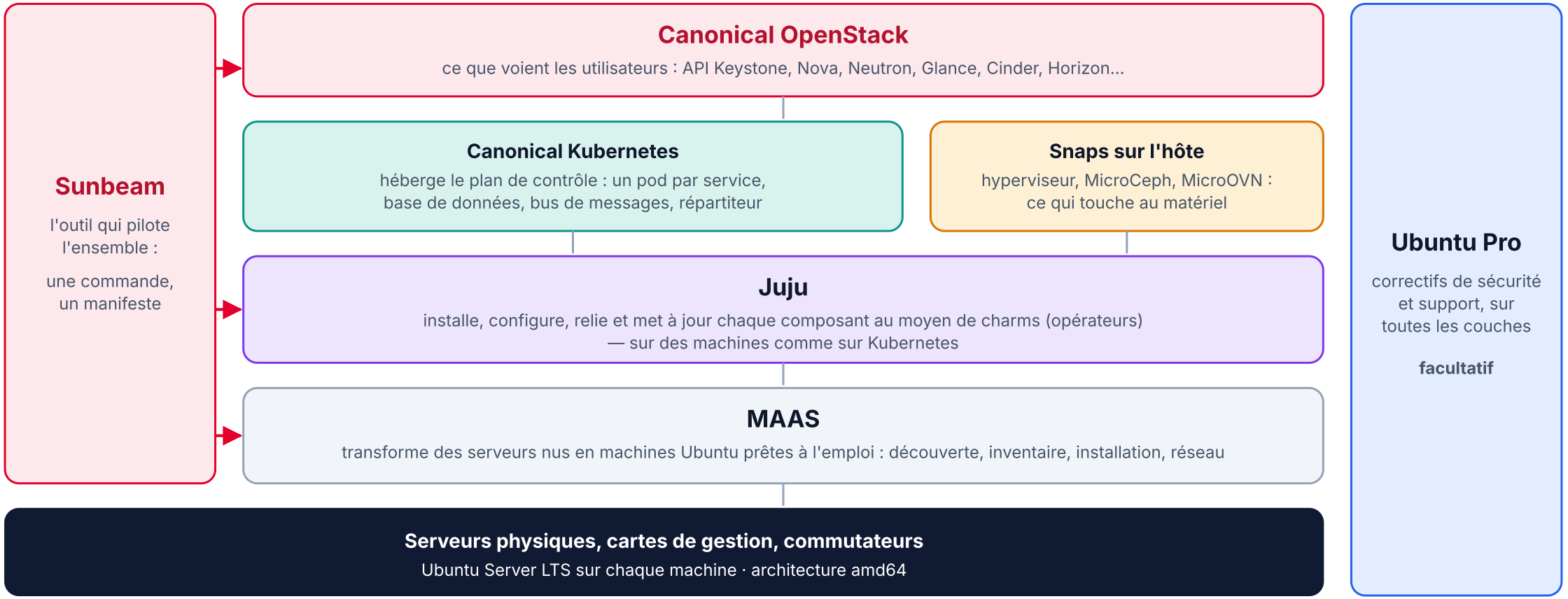
La stack Canonical : MAAS, Juju, Canonical OpenStack

Du serveur nu au cloud privé — architecture, fonctionnalités, avantages, limites et modèle de support

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Situer **MAAS**, **Juju** et **Sunbeam** dans les couches d'un déploiement.
- Expliquer le modèle de Juju : contrôleur, modèles, applications, unités, **charms** et **relations**.
- Dérouler une installation, avec ou sans MAAS.
- Décrire l'architecture de MAAS et le **cycle de vie** d'une machine, de la découverte à la mise en service.
- Décrire l'architecture de **Canonical OpenStack** : rôles des machines, réseaux, composants.
- Énoncer les **avantages**, les **limites** et le **modèle de support** de la stack.

La stack en une image



Lecture : chaque brique est utilisable seule — MAAS sans Juju, Juju sans MAAS ni OpenStack. Sunbeam les enchaîne pour vous ; sur un petit cloud, MAAS est facultatif et les machines sont installées à la main.

Qui fait quoi

Brique	En une phrase	Ce qu'elle prend en entrée	Ce qu'elle rend	Équivalents ailleurs
MAAS	**le « cloud » des serveurs physiques**	des serveurs nus, câblés, avec carte de gestion	des machines Ubuntu installées, à la demande, par API	Ironic, Foreman, Metal3
Juju	**l'orchestrateur d'applications**	des machines ou un cluster Kubernetes, et des charms	des applications installées, configurées et reliées entre elles	Ansible, Helm et opérateurs Kubernetes
Charms	**le savoir-faire d'exploitation, emballé**	des événements envoyés par Juju	les actions à mener : installer, configurer, migrer, mettre à jour	rôles Ansible, charts Helm, opérateurs
Canonical Kubernetes	**le socle du plan de contrôle**	les machines de contrôle	un cluster où tournent les services d'OpenStack	OpenShift, toute distribution Kubernetes
Sunbeam	**l'assembleur du cloud**	des machines, quelques réponses ou un manifeste	un cloud OpenStack complet et ses mises à jour	Kolla-Ansible, RHOSO, MOSK
Ubuntu Pro	**l'abonnement**	un jeton attaché à chaque machine	correctifs étendus, conformité, support en option	souscriptions Red Hat, Mirantis

Tout est libre et gratuit à l'usage : MAAS, Juju, les charms, Sunbeam et Ubuntu se téléchargent et s'utilisent en production sans contrat. L'abonnement ajoute des correctifs sur une durée plus longue et un interlocuteur.

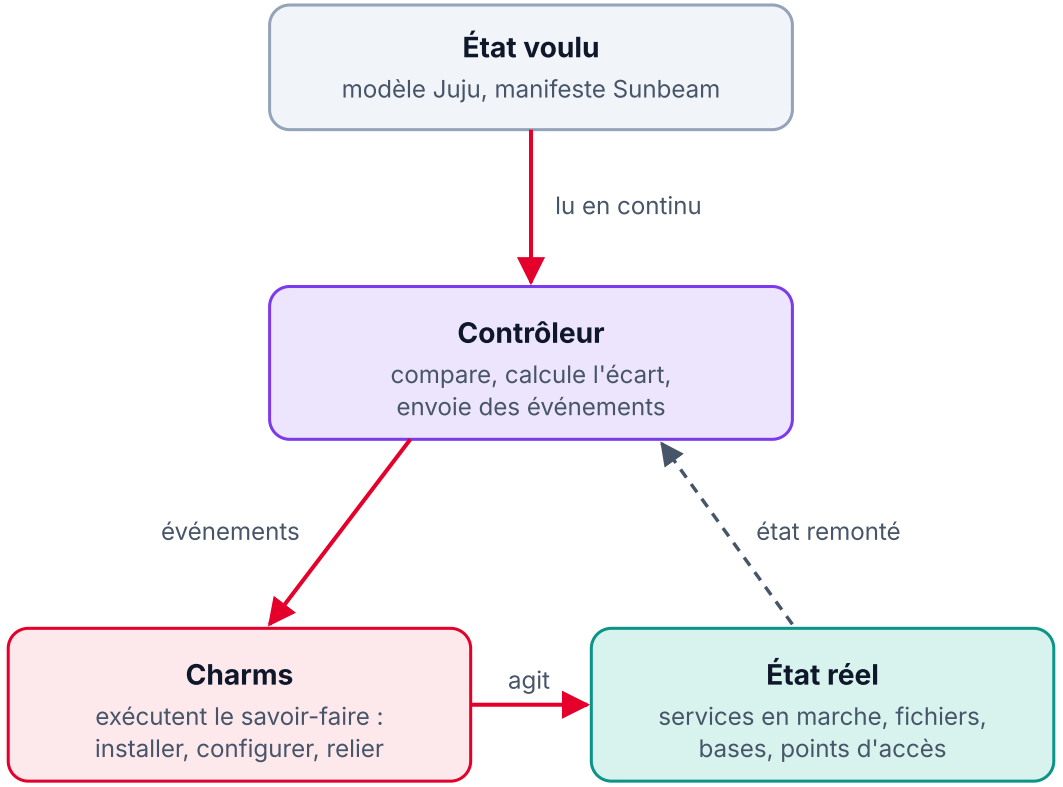
« Canonical OpenStack » désigne aujourd'hui la distribution fondée sur **Sunbeam**. L'ancienne génération, fondée sur les charms « machine », s'appelle **Charmed OpenStack** et reste maintenue pour les clouds existants.

Des opérations pilotées par un modèle

Le point commun des trois briques : on ne décrit pas **les étapes** à exécuter, on décrit **l'état voulu**, et un contrôleur se charge d'y arriver puis de s'y maintenir.

Brique	Ce que l'on déclare	Qui réalise
MAAS	« cette machine doit être déployée en Ubuntu 24.04, avec ce partitionnement et ces interfaces »	les contrôleurs de région et de rack
Juju	« le modèle contient Keystone en trois unités, relié à MySQL »	le contrôleur Juju et les charms
Sunbeam	« ce cloud a ces nœuds, ces rôles, ces fonctionnalités »	Sunbeam, qui pilote Juju

- **Ce que cela change :** pas de fichier de configuration édité à la main sur un serveur ; toute modification passe par le modèle.
- **Ce que cela exige :** faire confiance aux choix intégrés dans les charms, et savoir lire l'état du modèle pour diagnostiquer.



La boucle tourne en permanence, pas seulement à l'installation.

MAAS : rôle et principe

Le problème résolu : installer un système sur cent serveurs, à l'identique, sans clé USB ni console, et pouvoir recommencer à volonté.

- **Découvre** les serveurs qui démarrent sur le réseau et dresse leur **inventaire matériel** : processeurs, mémoire, disques, cartes réseau, micrologiciels.
- **Pilote l'alimentation** par la carte de gestion de chaque serveur (IPMI, Redfish).
- **Installe** un système d'exploitation en quelques minutes : Ubuntu, mais aussi CentOS, RHEL, Oracle Linux, Windows, VMware ESXi.
- **Configure** le stockage (partitions, LVM, RAID logiciel) et le réseau (agrégats, VLAN, ponts) **avant** l'installation.
- **Fournit les services réseau** nécessaires : DHCP, DNS, NTP, amorçage, mandataire de paquets.
- **Expose tout par API** : interface web, ligne de commande, Juju, Terraform.

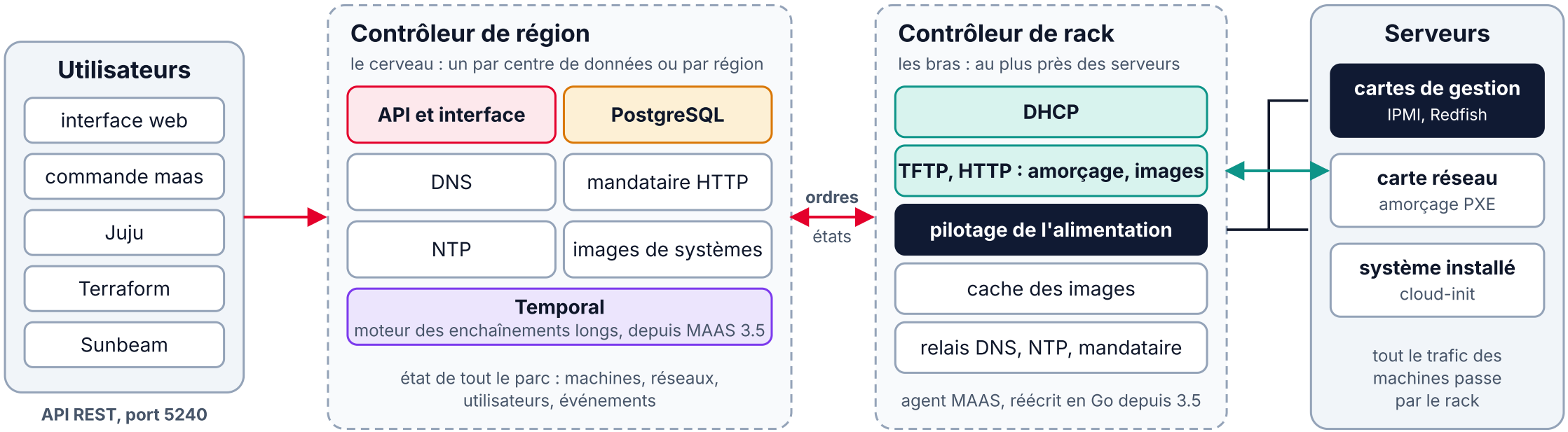
> **Ce que MAAS n'est pas :** un outil de configuration d'applications. Il s'arrête au système installé et joignable en SSH.

Dans un cloud public	Dans MAAS
créer une instance	allouer puis déployer une machine
type d'instance	contraintes : mémoire, cœurs, étiquettes
image	image de système synchronisée
zone de disponibilité	zone
projet, quota	groupe de ressources
données utilisateur	cloud-init
supprimer l'instance	libérer la machine, effacer les disques

> **Pourquoi c'est la base de la stack :** Juju sait parler à MAAS comme à un cloud. Demander « trois machines étiquetées stockage » devient une ligne de commande, et le remplacement d'un nœud en panne une opération banale.

> **Prérequis à vérifier avant tout :** un réseau où MAAS est le **seul** serveur DHCP, l'amorçage réseau activé dans chaque serveur, et un accès aux cartes de gestion.

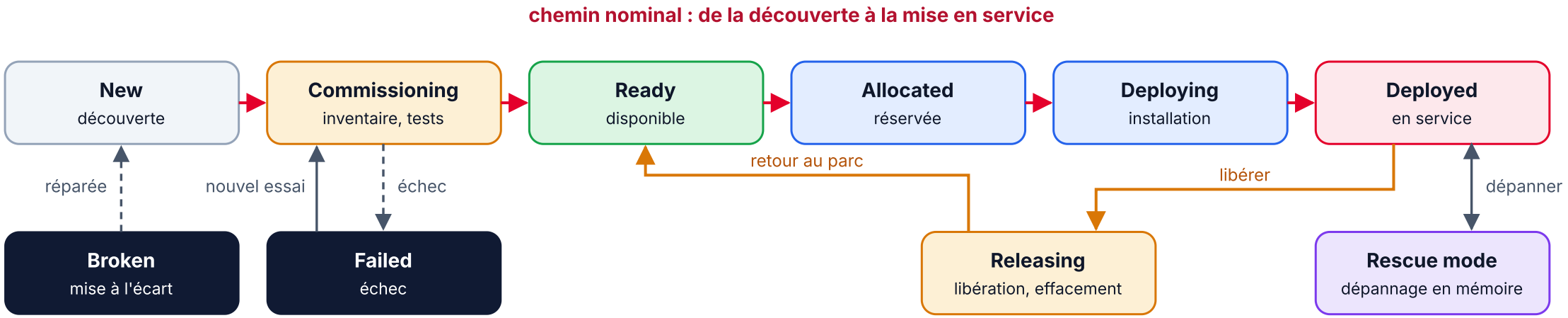
Architecture : contrôleur de région et contrôleurs de rack



Répartition selon la taille

Maquette ou petit site : région et rack sur la même machine (mode « region+rack »), base de test intégrée.
Production : plusieurs régions derrière une base PostgreSQL redondante, et deux contrôleurs de rack par réseau d'amorçage.

Cycle de vie d'une machine

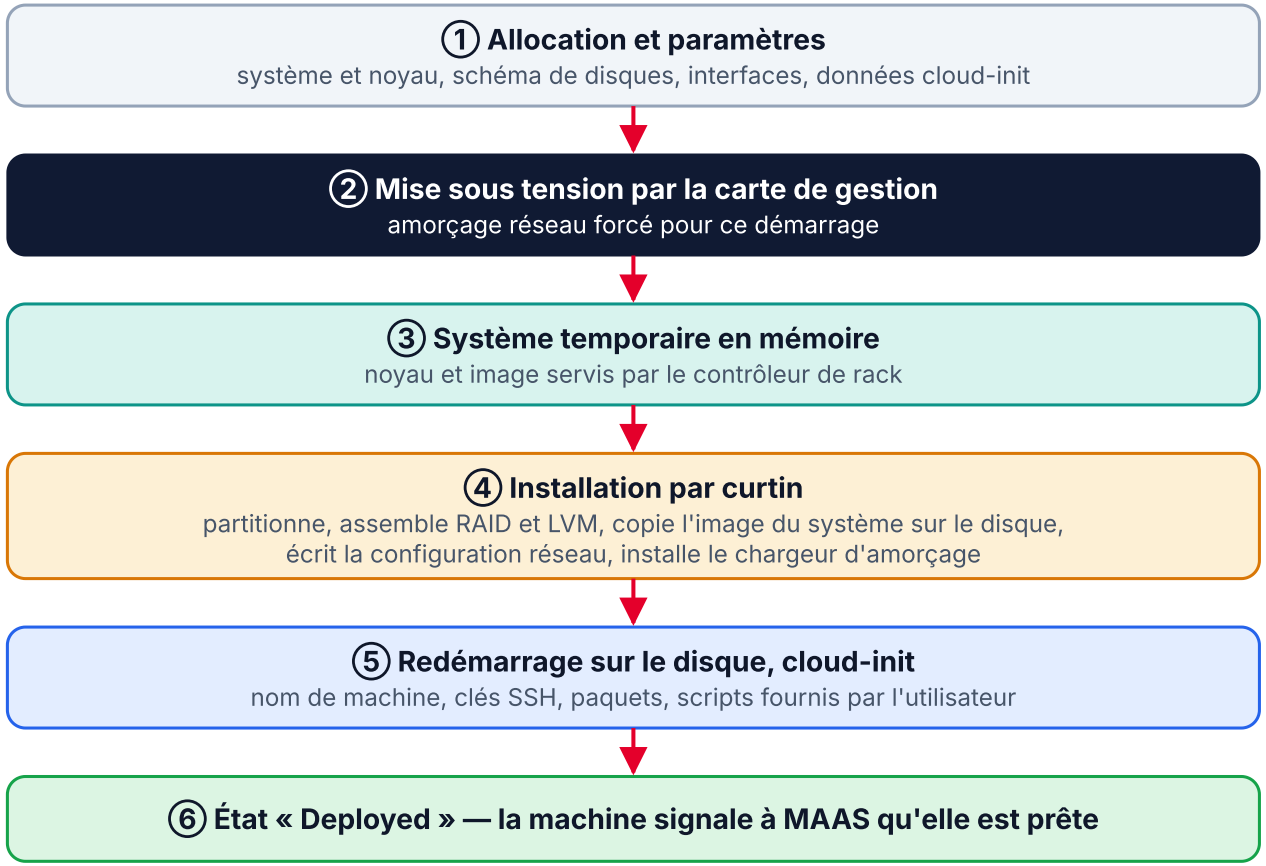


Une machine en panne est marquée « Broken » par l'administrateur ; réparée, elle repasse par l'inventaire.

État	Signification
New	découverte, visible des administrateurs, pas encore de confiance
Commissioning	inventaire matériel et tests sur un système temporaire en mémoire
Ready	inventoriée, inactive, disponible
Allocated	réservée par un utilisateur ; toujours éteinte ou inactive

État	Signification
Deploying, Deployed	installation en cours, puis système en service
Releasing	rendue au parc ; effacement des disques si demandé
Failed	échec d'inventaire, de test ou d'installation : à reprendre
Broken, Rescue mode	mise à l'écart pour panne ; démarrage de dépannage en mémoire

Déploiement : de « Ready » à un système en service



Ce qu'il faut retenir

- **Une copie d'image, pas une installation paquet par paquet :** c'est ce qui rend le déploiement rapide et identique d'une machine à l'autre.
- **Le réseau et les disques se décident dans MAAS**, avant le déploiement ; le système arrive déjà configuré.
- **cloud-init** est le point d'entrée de la personnalisation : c'est par lui que Juju installe son agent.
- **Variantes :** déploiement en mémoire seule, sans toucher aux disques (depuis 3.5) ; enregistrement d'une machine déjà installée, sans la réinstaller.

Script	Moment
d'inventaire	pendant « Commissioning » : micrologiciel, RAID matériel
de test	disques, mémoire, réseau
de déploiement	avant ou après l'installation
de libération	au retour dans le parc (depuis 3.5)

Le modèle réseau de MAAS

Fabric « prod » — un ensemble de commutateurs reliés

VLAN non étiqueté

`10.10.0.0/24`

amorçage, gestion

oam

VLAN 100

`10.10.100.0/24`

API internes, base

interne

VLAN 200

`10.10.200.0/24`

accès à Ceph

stockage

Fabric « externe »

VLAN non étiqueté

`192.0.2.0/24`

sortie vers l'extérieur

sous-réseau non géré par MAAS

Vocabulaire

Fabric : domaine de niveau 2

VLAN : segment, avec ou sans DHCP

Sous-réseau : plage IP d'un VLAN

Espace : étiquette d'usage regroupant des sous-réseaux, lue par Juju

Plage réservée : adresses que MAAS ne distribuera jamais

Découpage d'un sous-réseau géré

.1 – .9	.10 – .49	.50 – .99	.100 – .254
passerelle, équipements	plage réservée (adresses virtuelles)	plage dynamique (DHCP)	adresses fixes attribuées aux machines déployées

La plage dynamique sert à l'enrôlement et à l'inventaire. Une machine déployée reçoit en général une adresse **fixe**, choisie par MAAS hors de cette plage et inscrite dans le DNS.

Les espaces sont la clé de l'intégration : Juju et Sunbeam ne manipulent pas des VLAN mais des espaces. « Place ce service sur l'espace interne » fonctionne quel que soit l'adressage du site.

Juju : rôle et principe

Le problème résolu : installer un logiciel est facile ; le **relier** aux autres, le faire évoluer et le maintenir pendant des années ne l'est pas.

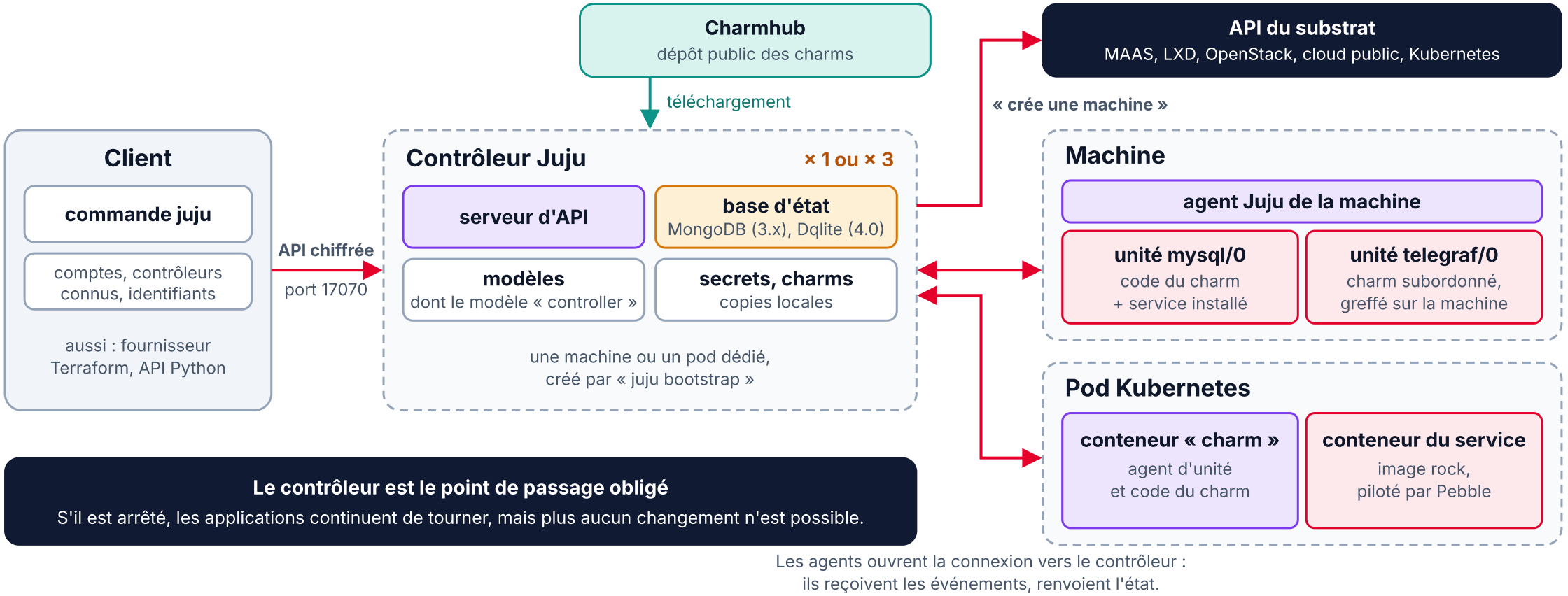
- **Un charm par application :** il contient tout ce qu'un spécialiste saurait faire — installer, configurer, initialiser la base, mettre à jour, sauvegarder.
- **Des relations entre applications :** « Keystone a besoin d'une base » se dit en une commande ; les deux charms échangent adresses et identifiants sans intervention.
- **Un modèle, plusieurs substrats :** la même commande déploie sur MAAS, sur LXD, sur un cloud public ou sur Kubernetes.
- **Tout le cycle de vie :** mise à l'échelle, changement d'option, montée de version, actions ponctuelles.

> **Idée centrale :** Juju ne sait rien d'OpenStack ni de PostgreSQL. Il transmet des **événements** ; c'est le charm qui sait quoi faire.

Question	Ansible	Helm	Juju
Unité de réutilisation	rôle	chart	charm
Nature	tâches exécutées à la demande	gabarits de manifestes	programme réagissant à des événements
Agent permanent	non	non	oui, sur chaque unité
Liaison entre applications	variables à tenir cohérentes	valeurs à tenir cohérentes	relations négociées par les charms
Cibles	machines	Kubernetes	machines et Kubernetes
État connu	non conservé	historique des versions	modèle tenu par le contrôleur

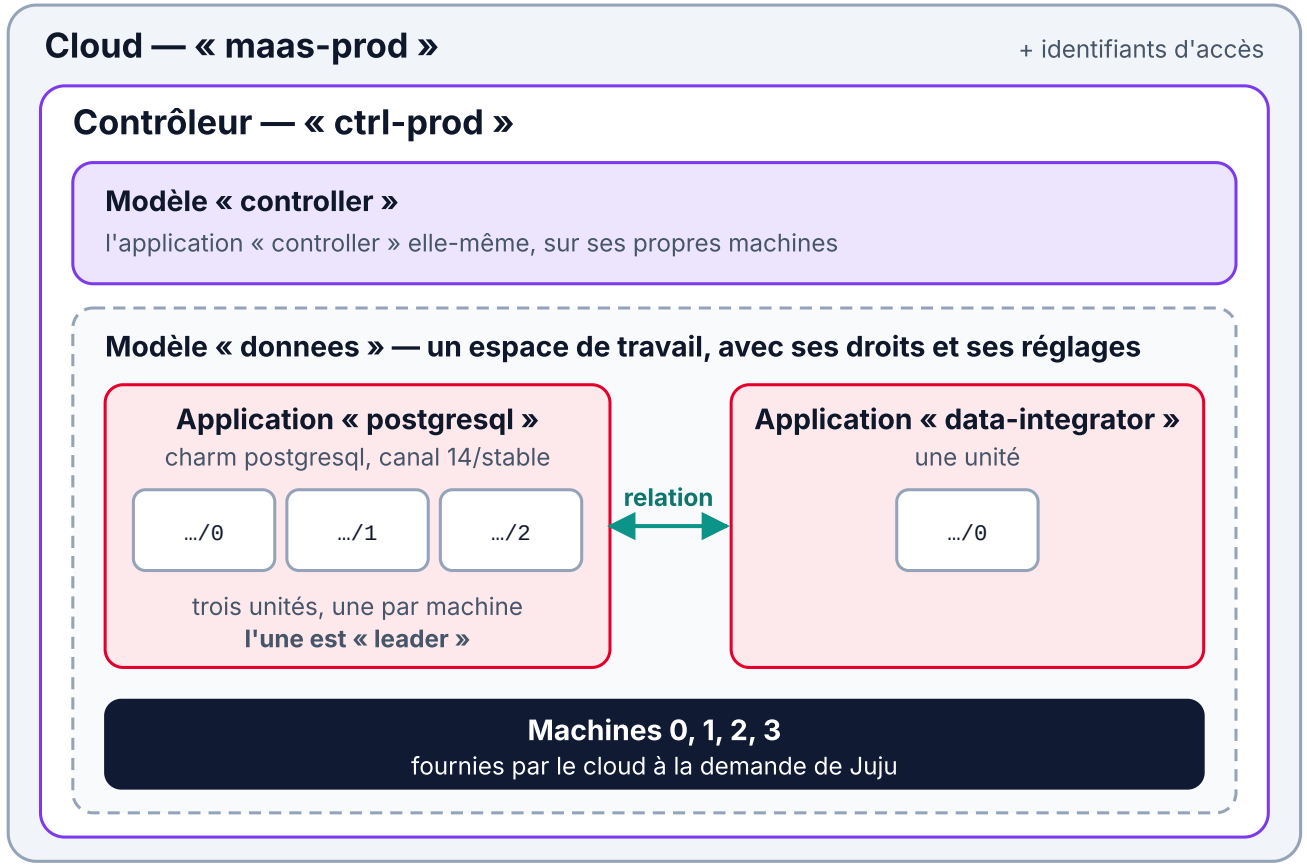
> **Contrepartie :** on dépend de la **qualité du charm**. Ce qu'il ne prévoit pas ne se fait pas proprement ; le contourner à la main sera défait par le prochain événement.

Architecture : client, contrôleur, agents



Lecture du schéma : le client ne parle qu'au contrôleur ; le contrôleur demande les machines au substrat — MAAS, LXD, Kubernetes… —, puis ses agents, installés sur chaque machine ou dans chaque pod, exécutent le code des charms et lui renvoient l'état.

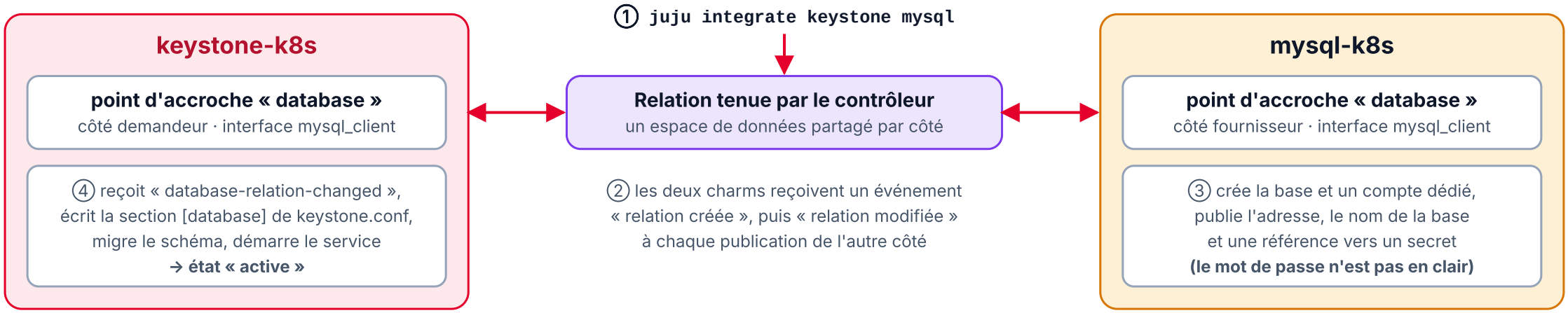
Le vocabulaire du modèle



Terme	Définition
Cloud	un fournisseur de ressources : MAAS, LXD, Kubernetes…
Contrôleur	le service qui tient l'état et pilote un ou plusieurs modèles
Modèle	un ensemble d'applications gérées ensemble ; un espace de noms sur Kubernetes
Application	un charm déployé, avec sa configuration
Unité	une instance en marche de l'application
Leader	l'unité élue pour les opérations à ne faire qu'une fois
Machine	le serveur ou la machine virtuelle qui porte des unités
Relation	un lien déclaré entre deux applications

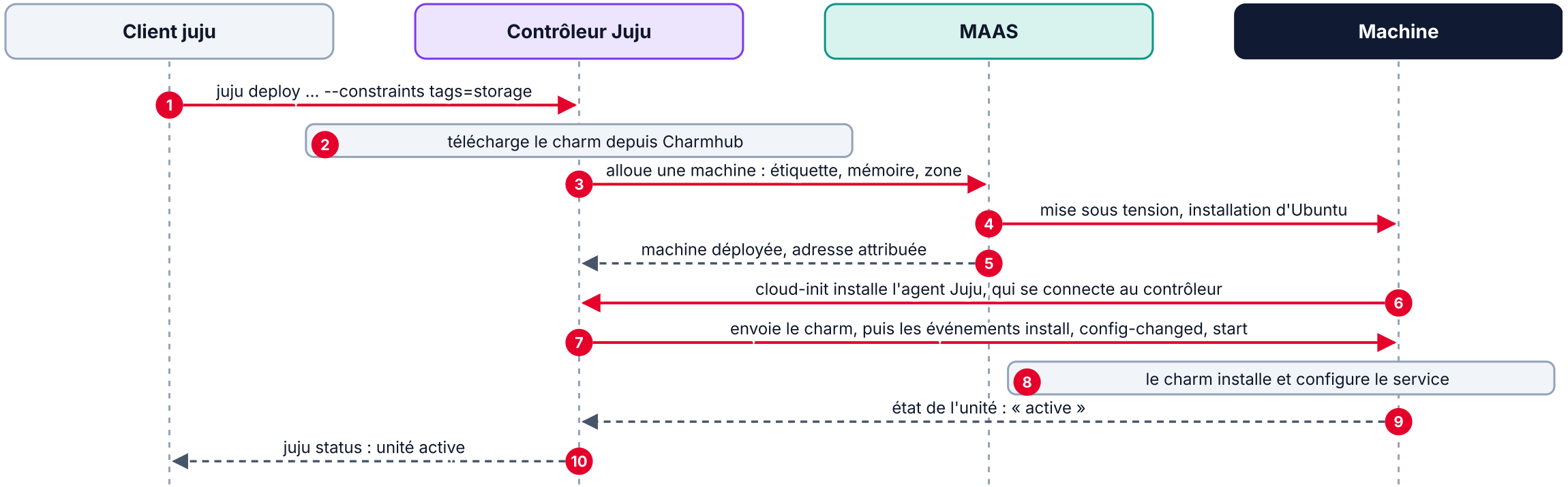
> **Lire `juju status`, c'est lire ce schéma :** le modèle, ses applications, leurs unités, les machines, puis les relations.

Les relations : le cœur de Juju



- **Une relation n'est possible qu'entre interfaces identiques :** un demandeur `mysql_client` ne se branche que sur un fournisseur `mysql_client`. C'est un contrat.
- **Elle vit dans le temps :** si l'adresse de la base change ou si une unité s'ajoute, l'autre côté est prévenu et se reconfigure seul.
- **La retirer** (`juju remove-relation`) déclenche le nettoyage : compte supprimé, service reconfiguré.
- **Relation entre modèles :** une application peut être **offerte** à un autre modèle, voire à un autre contrôleur — la supervision ou un Vault partagé, par exemple.
- **Relation « pair » :** entre les unités d'une même application, pour former une grappe.
- **Dans Canonical OpenStack :** plusieurs dizaines de relations sont créées pour vous ; `juju status --relations` les montre toutes.

Juju au-dessus de MAAS : ce que déclenche « juju deploy »



Les contraintes font le lien : `tags=` , `mem=` , `cores=` , `zones=` , `spaces=` sont traduites en critères de sélection dans l'inventaire de MAAS.

Retirer l'unité rend la machine : `juju remove-unit` puis `juju remove-machine` la font repasser à l'état « Ready » dans MAAS, disques effacés si la politique le prévoit.

Juju 3.6 LTS et Juju 4.0 : où en est-on ?

	Juju 3.6	Juju 4.0
Statut	**version à long terme**	version majeure, sortie le 14 novembre 2025
Base d'état	**MongoDB**	Dqlite : SQLite répliqué par Raft
Contrôleur redondant	`juju enable-ha`	`juju add-unit` sur le modèle « controller »
Lots de charms	**encore gérés**	`export-bundle` retiré ; Terraform conseillé
Fournisseur « manual »	**« manual »**	renommé « unmanaged »
Clés SSH	**ajoutées aux nouveaux modèles**	plus ajoutées automatiquement

Retiré en 4.0

- Les « séries » (remplacées par les bases), les charms Kubernetes d'ancienne génération, le fournisseur KVM.
- `juju wait-for`, l'option `--watch` de `juju status`, les réglages de leader.

Politique de versions

- Une version mineure environ **tous les trois mois**, des correctifs mensuels.
- Une version mineure reçoit des correctifs pendant quatre mois, puis deux mois de correctifs de sécurité seulement.
- À chaque version majeure, la dernière mineure de la série précédente devient **version à long terme**, avec des correctifs de sécurité annoncés sur quinze ans.

Passer de 3.6 à 4.0

- Pas de mise à jour en place du contrôleur : on **crée un contrôleur 4.0** et on y **migre les modèles**.
- Un guide « 3.6 vers 4.0 » est publié (4.0.15 en octobre 2026). Il liste surtout les ruptures à traiter avant : `enable-ha` et `upgrade-machine` retirés, espaces MAAS explicites. À dérouler d'abord en maquette.

> **Pour Canonical OpenStack :** la version de Juju est celle qu'installe Sunbeam. Ne pas la remplacer de sa propre initiative ; suivre la documentation de la version du cloud.

Architecture : quatre fonctions, trois types d'artefacts

Gouvernance
déployer, opérer, observer

- **Sunbeam** — client et service de grappe
- **contrôleur Juju** — modèles, charms, relations
- **MAAS** — si fournisseur MAAS
- **supervision** — optionnelle

Contrôle
API, ordonnanceurs, conducteurs

- **services OpenStack** — Keystone, Nova, Neutron, Glance…
- **MySQL, RabbitMQ** — état et messages
- **Traefik** — points d'accès publics et internes
- **bases OVN** — nord et sud

Calcul et réseau
faire tourner les instances

- **nova-compute** — libvirt, QEMU, KVM
- **OVN et Open vSwitch** — réseau virtuel sur chaque hôte
- **service de métadonnées** — pour les instances
- **passerelles OVN** — nœuds « network » (MicroOVN)

Stockage
conserver les données

- **MicroCeph** — moniteurs, gestionnaires, OSD
- **cinder-volume** — pilote les volumes
- **passerelle objet** — optionnelle

Ceph par défaut ; baies Pure Storage, Dell et Hitachi (option) en plus

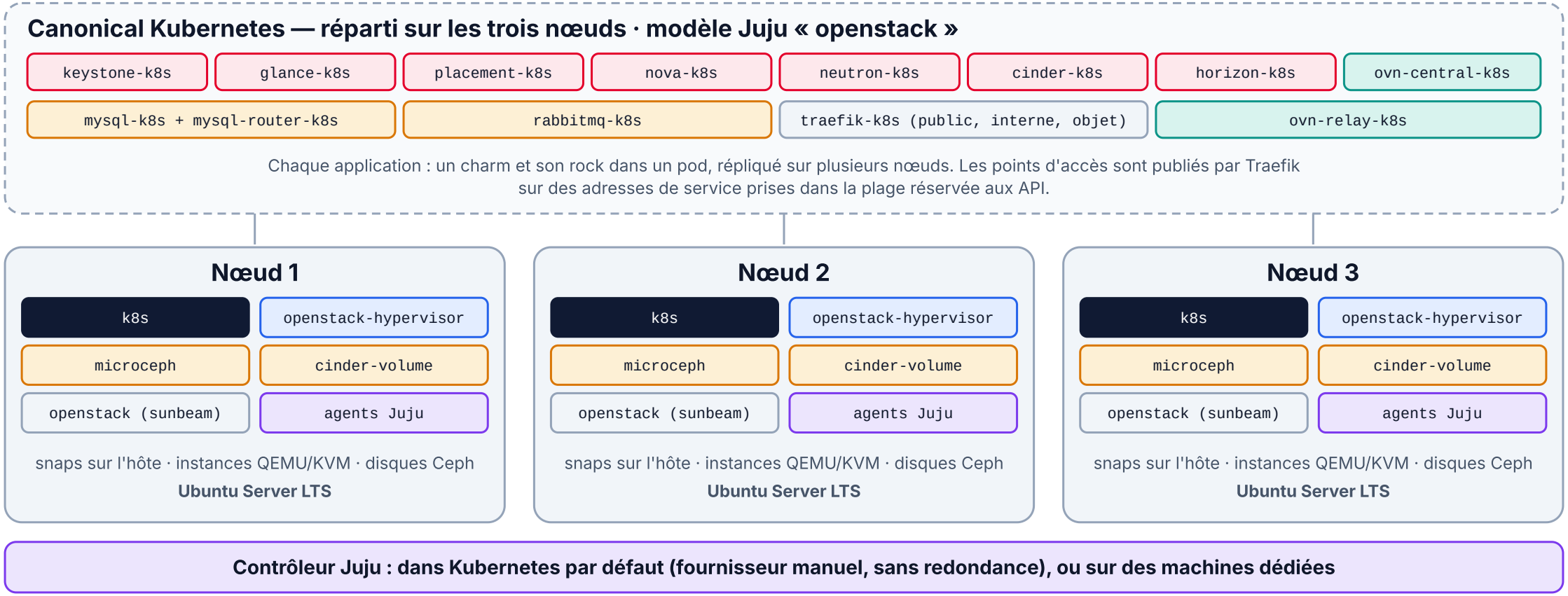
Rocks
images OCI : services du plan de contrôle et pile de supervision
→ fonction Contrôle

Snaps
applications confinées : processus du plan de données et beaucoup d'autres composants
→ Calcul, Réseau, Stockage, Gouvernance

Charms
opérateurs qui déploient, configurent, mettent à l'échelle et relient rocks et snaps
→ partout

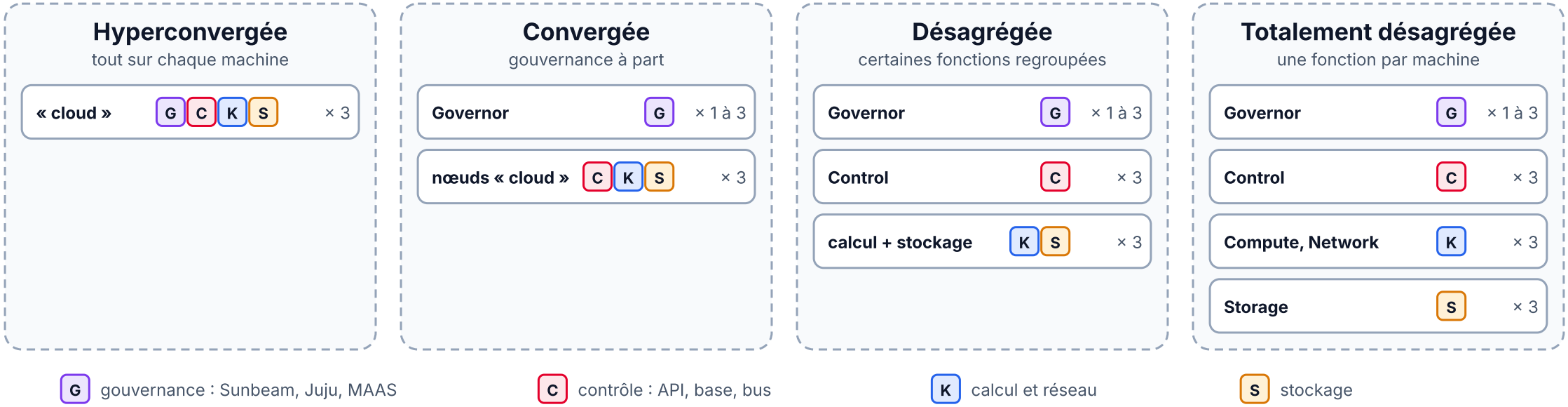
Lecture du schéma : les quatre colonnes sont les fonctions du cloud ; les trois bandeaux du bas disent sous quelle forme chaque composant est livré : rocks, images OCI dans Kubernetes, pour le contrôle ; snaps sur les machines pour le calcul, le réseau et le stockage ; charms partout, pour les piloter.

Ce qui tourne où : un cluster de trois nœuds « cloud »



Deux modèles Juju à connaître : « openstack » pour les charms Kubernetes du plan de contrôle, et « admin/controller » pour les charms qui pilotent les snaps des hôtes. Premières commandes de diagnostic : `juju status -m openstack`, puis `juju status -m admin/controller`.

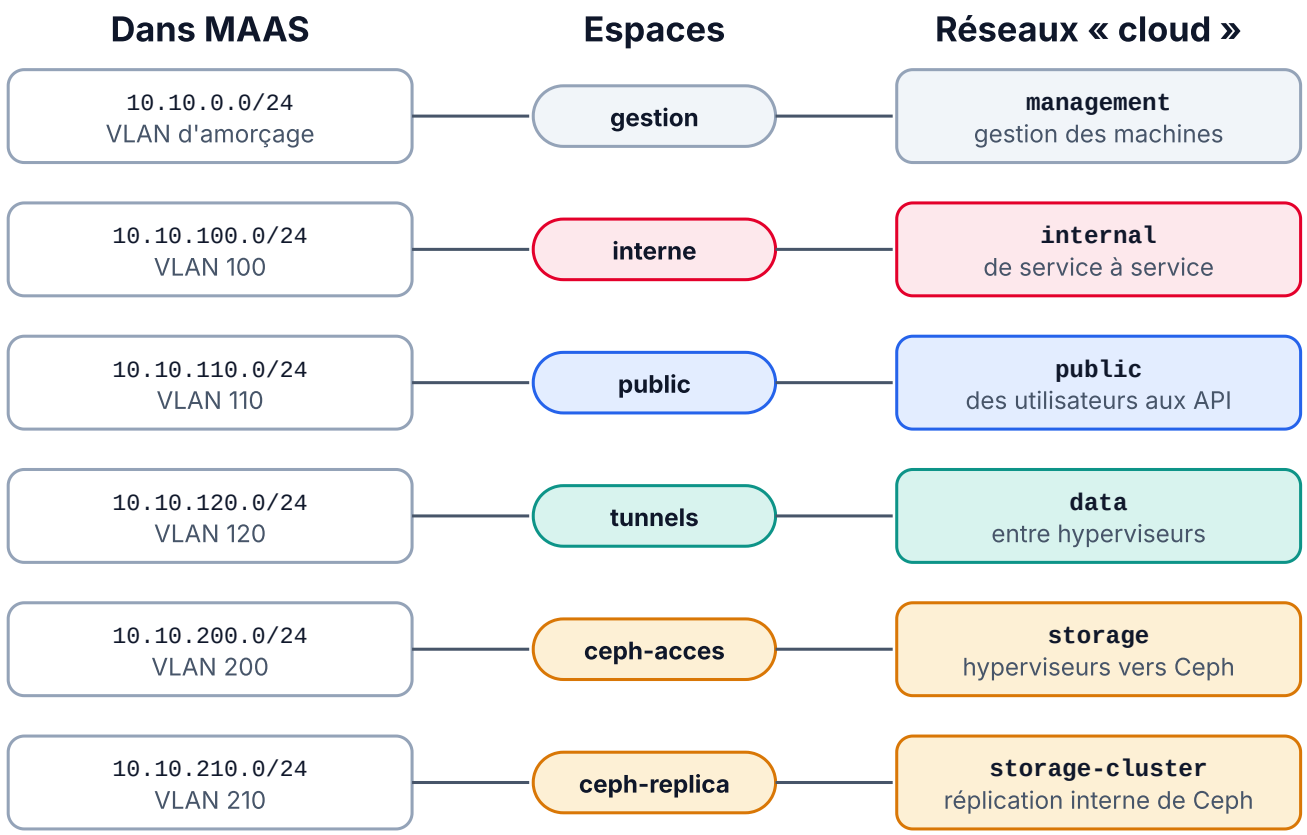
Quatre architectures de référence



Taille	Réseau physique	Architecture	Machines nues
1 à 9 machines	un seul niveau	hyperconvergée	fournisseur manuel
10 machines et plus	deux niveaux, *spine-leaf*	convergée, désagrégée ou totalement désagrégée	MAAS

Haute disponibilité : facultative mais recommandée ; elle exige **au moins trois machines de chaque type**, quelle que soit l'architecture retenue. Recommandations issues de la documentation de conception de Canonical.

Réseaux : deux réseaux physiques, six réseaux « cloud »



Adressage et noms d'espaces donnés en exemple.

Le minimum : deux réseaux physiques

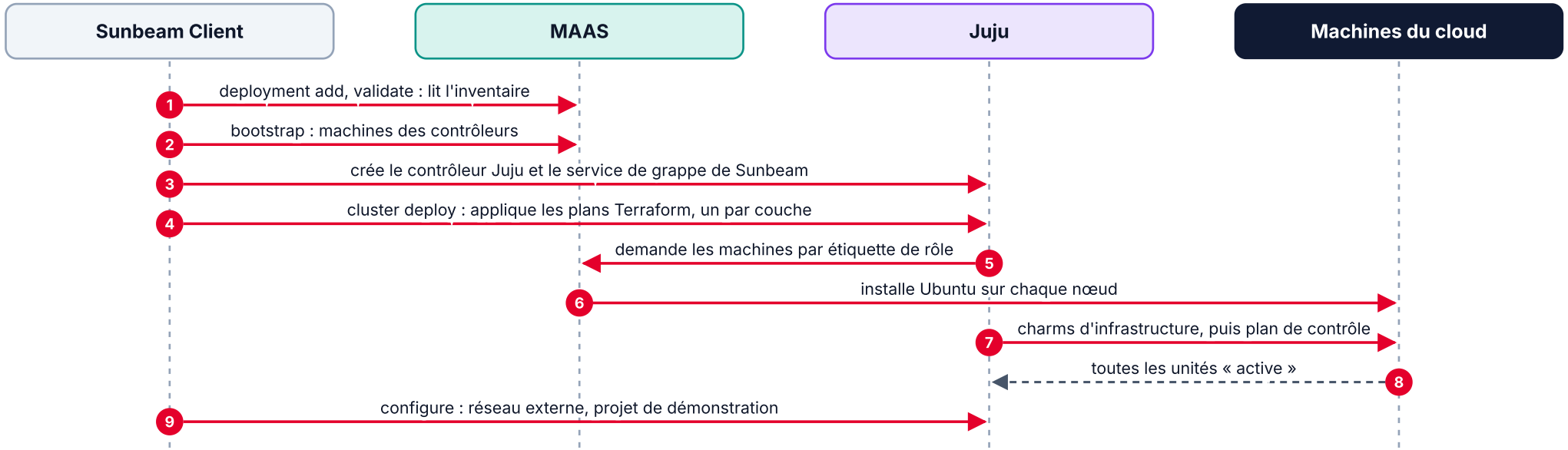
- **Générique :** provisionnement, gestion et accès aux API.
- **Externe :** adresses flottantes et sortie des instances vers l'extérieur.

Avec MAAS : une isolation plus fine

- Chaque sous-réseau est rangé dans un **espace** ; chaque espace est associé à un **réseau « cloud »** par `sunbeam deployment space map`.
- Un seul espace pour tout : aucune isolation. Six espaces : séparation complète.
- **Plages d'adresses à réserver** dans MAAS, nommées `<nom>-internal-api` et `<nom>-public-api` : dix à vingt adresses chacune, pour les points d'accès.

Réseau externe : sur l'interface qui y est branchée, laisser le sous-réseau « non configuré » dans MAAS et poser l'étiquette réseau `neutron:physnet1`. C'est OVN qui s'en servira, pas le système hôte.

Installation avec MAAS : l'enchaînement




```
# Depuis la machine « Sunbeam Client »
sudo snap install openstack --channel 2026.1/stable
sunbeam prepare-node-script --client | bash -x
sunbeam deployment add maas prod $CLE_API http://10.10.0.2:5240/MAAS
sunbeam deployment space map interne:internal   # un par réseau
sunbeam deployment validate
```

```
sunbeam cluster bootstrap   # couche d'orchestration
sunbeam cluster deploy      # le cloud lui-même
sunbeam configure --openrc demo-openrc

# Plus tard, après avoir étiqueté de nouvelles machines :
sunbeam cluster deploy
```

Lecture du schéma : les flèches numérotées se lisent dans l'ordre. Le client Sunbeam ne touche jamais les machines : il passe par Juju, qui les demande à MAAS par étiquette de rôle, puis y déploie les charms. Les commandes correspondent aux étapes 1 (deployment add, validate), 2 (bootstrap), 4 (cluster deploy) et 9 (configure).

Le manifeste : décrire le cloud dans un fichier

```yaml
core:
  config:
    bootstrap:
      management_cidr: 10.10.0.0/24
    database: single            # ou multi
    region: RegionOne
    k8s-addons:
      loadbalancer: 10.10.0.100-10.10.0.120
    external-networks:
      physnet1:
        cidr: 192.0.2.0/24
        gateway: 192.0.2.1
        range: 192.0.2.10-192.0.2.50
        network_type: flat
    microceph_config:
      cloud-1.exemple.lan:
        osd_devices: /dev/sdb,/dev/sdc
  software:
    charms:
      keystone-k8s:
        channel: 2026.1/stable
features:
  loadbalancer:
    config:
      amphora_enabled: false
```

Section	Ce qu'elle porte
`core.config`	les réponses aux questions de l'installation : réseaux, région, base, utilisateur de démonstration, disques de Ceph, noms des points d'accès, mandataire
`core.software`	versions : arguments de Juju, canal ou révision et options de chaque charm, source des plans Terraform
`features`	réglages des fonctionnalités optionnelles : supervision, équilibrage de charge, TLS
`storage`	baies tierces (Pure Storage ; Dell, Hitachi en option), par type puis par instance

- **Intérêt :** une installation **sans question**, reproductible, que l'on versionne dans git et que l'on relit.
- **Usage :** le manifeste se passe aux commandes d'installation et de mise à jour par l'option `--manifest`.
- **Point d'ajustement fin :** c'est ici que l'on épingle la révision d'un charm ou que l'on règle une option qu'aucune question ne propose.

> Extrait simplifié, à valeurs d'exemple : la liste complète des clés est dans la référence du manifeste de la documentation.

Exploitation : les opérations du quotidien

Besoin	Commande
État des nœuds	`sunbeam cluster list`
État des services	`juju status -m openstack`
État des machines	`juju status -m admin/controller`
Journal des charms	`juju debug-log -m openstack`
Pods du plan de contrôle	`sudo k8s kubectl get pods -n openstack`
Services d'un hôte	`sudo snap logs openstack-hypervisor`
État de Ceph	`sudo microceph status`
Identifiants d'administration	`sunbeam openrc > admin-openrc`
Adresse du tableau de bord	`sunbeam dashboard-url`
Retirer un nœud	`sunbeam cluster remove <nom>`

Trois niveaux de lecture : Sunbeam pour les nœuds, Juju pour les applications, Kubernetes ou snap pour les processus.

Mettre à jour, dans cet ordre

```
sudo snap refresh openstack    # d'abord le snap
sunbeam cluster refresh k8s    # 1 · Kubernetes
sunbeam cluster refresh vault  # 2 · Vault, s'il est activé
sunbeam cluster refresh mysql  # 3 · bases de données
sunbeam cluster refresh        # 4 · le reste des charms
```

- Rafraîchir le snap **ne met pas à jour** le cloud : ce sont les commandes `cluster refresh` qui le font.
- Fournisseur manuel : rafraîchir le snap sur **tous** les nœuds ; avec MAAS, sur le client seulement.
- Vault reste **scellé** après sa mise à jour : à desceller à la main.

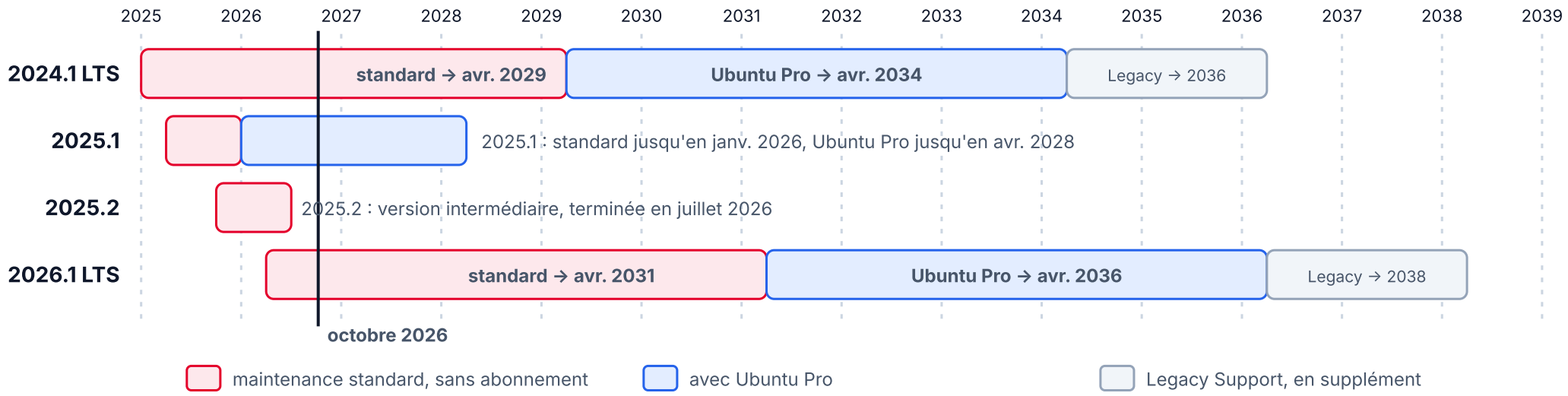
Limite actuelle, écrite dans la documentation : le rafraîchissement **d'une piste à l'autre** — de 2024.1 à 2025.1, par exemple — n'est pas pris en charge par cette procédure, pas plus que le changement de version mineure de Kubernetes. On met à jour **à l'intérieur** d'une piste.

Limites à connaître avant de s'engager

Limite	Détail	Conséquence pratique
Choix techniques imposés	**QEMU/KVM, OVN, Ceph ; en plus, baies Pure Storage, Dell et Hitachi (option)**	une autre baie ou un autre réseau virtuel ne s'intègrent pas aujourd'hui
Écosystème fermé sur Ubuntu	**hôtes Ubuntu, Kubernetes de Canonical, snaps**	inadapté si la politique impose un autre système ou un Kubernetes existant
Architecture amd64 seulement	**pas d'autre architecture pour les nœuds**	à vérifier avant tout achat de matériel
Accès à Internet	**déploiement possible derrière un mandataire ; hors ligne complet pas encore disponible**	frein pour les réseaux strictement isolés
Fournisseur de machines figé	**manuel ou MAAS : choix à la conception, sans retour**	démarrer en manuel puis passer à MAAS impose de redéployer
Montées de version	**la procédure de rafraîchissement ne franchit pas les pistes**	vérifier le chemin de mise à niveau entre versions avant de choisir la version de départ
Peu de réglages exposés	**seules les options prévues par les charms et le manifeste**	les besoins atypiques passent par une prestation ou une contribution
Compétences Juju	**moins répandues qu'Ansible ou Kubernetes**	formation à prévoir ; recrutement plus difficile
Produit jeune	**première version LTS de Sunbeam : 2024.1 ; rôles et procédures évoluent encore**	lire les notes de version et les limites connues à chaque mise à jour

Plusieurs couches à diagnostiquer : une API qui ne répond pas peut venir d'OpenStack, du charm, de Juju, de Kubernetes, de Traefik ou du snap sous-jacent. Savoir descendre les couches une à une est la compétence clé de l'exploitant.

Cycle de vie des versions de Canonical OpenStack



- **Deux versions par an**, en général en avril et en octobre, alignées sur le calendrier d'OpenStack.
- **Versions LTS de Sunbeam :** 2024.1 et 2026.1 ; la prochaine est 2028.1, 2027.1 étant une version intermédiaire.
- **Système hôte :** 2024.1 et 2025.1 sur Ubuntu 24.04 ; 2026.1 sur Ubuntu 24.04 ou 26.04.
- **Maintenance standard :** cinq ans sur une LTS, sans abonnement.
- **Ubuntu Pro :** cinq ans de plus ; **Legacy Support :** deux ans encore, soit douze au total.
- **Versions intermédiaires :** quelques mois seulement. Une production se pose sur une LTS.

> Dates relevées dans la page « Release cycle and supported versions » de la documentation 2026.1 ; la page commerciale de Canonical évoque jusqu'à quinze ans de correctifs de sécurité. Vérifier la disponibilité d'un canal avec `snap info openstack`.

La stack Canonical face aux alternatives

	Canonical OpenStack (Sunbeam)	Kolla-Ansible	Red Hat RHOSO	Mirantis MOSK
Nature	distribution libre, support optionnel	outil communautaire	produit sous souscription	produit sous souscription
Machines nues	MAAS, facultatif	Kayobe, projet frère	Metal3, dans OpenShift	inclus
Orchestrateur	Juju, derrière `sunbeam`	Ansible	opérateurs OpenShift, Ansible	opérateur, Helm
Socle du plan de contrôle	Kubernetes de Canonical	Docker ou Podman	OpenShift	Kubernetes de Mirantis
Système hôte	Ubuntu	CentOS Stream, Debian, Rocky, Ubuntu	RHEL	Ubuntu
Stockage	Ceph (MicroCeph) ; baies Pure Storage, Dell, Hitachi	Ceph externe, LVM, pilotes tiers	Red Hat Ceph, pilotes certifiés	Ceph (Rook), pilotes tiers
Taille minimale	une machine	une machine	un cluster OpenShift et un nœud de calcul	cluster de gestion et cluster géré
Utilisable sans contrat	oui, intégralement	oui	non	non
Liberté de réglage	limitée aux options des charms	élevée	encadrée	encadrée
Compétence clé	Juju, snaps, un peu de Kubernetes	Ansible, conteneurs	OpenShift	Kubernetes
Durée d'une version	5 ans sur LTS ; 12 avec Ubuntu Pro et Legacy	celle de l'amont, environ 18 mois	plusieurs années, selon contrat	selon contrat

Ce qui distingue Canonical : c'est la seule offre d'éditeur dont le produit complet s'utilise sans contrat, et qui démarre sur une seule machine.

Ce qui la pénalise : le moins de choix techniques et la technologie d'orchestration la moins répandue des quatre.

07

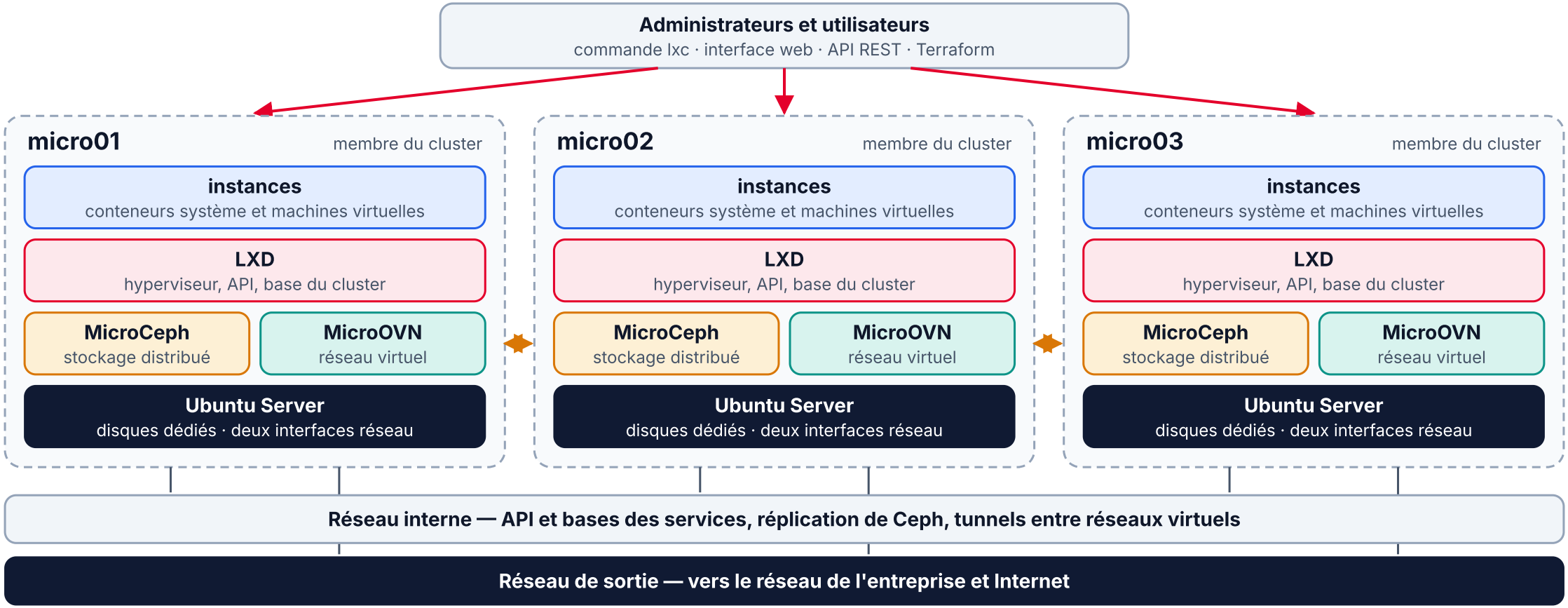
MicroCloud : virtualisation, stockage et réseau intégrés

Fonctionnalités, cas d'usage, avantages, limitations et positionnement par rapport à Canonical OpenStack

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Décrire le rôle de **LXD**, **MicroCeph** et **MicroOVN** dans MicroCloud.
- Dérouler une **initialisation** et les opérations courantes.
- Énoncer les **cas d'usage**, les **avantages** et les **limitations**.
- Expliquer le modèle de **stockage** (local, distribué) et le modèle **réseau** (intra-cluster, sortie, réseaux virtuels).
- Dire ce qui se passe lors d'une **panne** d'un membre, d'un disque, d'un lien.
- **Positionner** MicroCloud face à Canonical OpenStack et choisir.

MicroCloud en une image



À retenir : chaque membre porte les trois fonctions, sans base de données externe ni bus de messages.

Les principes de conception

Hyperconvergé par construction

Calcul, stockage et réseau sur les mêmes machines. On ajoute de la capacité en ajoutant un membre identique aux autres.

Automatisé de bout en bout

Une commande pose les questions, puis forme trois grappes et les relie. Un fichier de préconfiguration rend l'opération reproductible.

Petit d'abord

Un membre pour essayer, trois pour la haute disponibilité, cinquante au maximum. La cible est le petit cloud, pas le très grand.

Sans plan de contrôle séparé

Chaque service réplique son état entre les membres. N'importe quel membre répond aux requêtes.

Conteneurs et machines virtuelles

Les deux types d'instances, avec les mêmes commandes, les mêmes réseaux et les mêmes volumes.

Libre, support en option

Les quatre snaps sont libres et gratuits. Ubuntu Pro ajoute durée de maintenance et assistance.

Ce que cela donne en pratique

- Un cluster opérationnel en **quelques dizaines de minutes**, par une personne.
- Très peu de concepts à apprendre : instance, image, profil, projet, pool, réseau.
- Une exploitation proche de celle d'un hyperviseur classique, pas de celle d'un cloud à services multiples.

Ce que cela implique

- Des choix imposés : LXD, Ceph, OVN, Ubuntu, snaps.
- Pas de catalogue de services au-dessus des instances.
- Une taille maximale connue à l'avance.

Versions et cycle de vie

Combinaisons supportées

Ubuntu LTS	MicroCloud	LXD	MicroCeph	MicroOVN
22.04	2.1	5.21	Squid	24.03
24.04	2.1	5.21	Squid	24.03
26.04	3.4	6.10	Tentacle	26.03

Durée de vie des versions LTS

Version	Phase actuelle	Supportée jusqu'en
3.4 LTS	support complet	juin 2031
2.1 LTS	maintenance	juin 2029

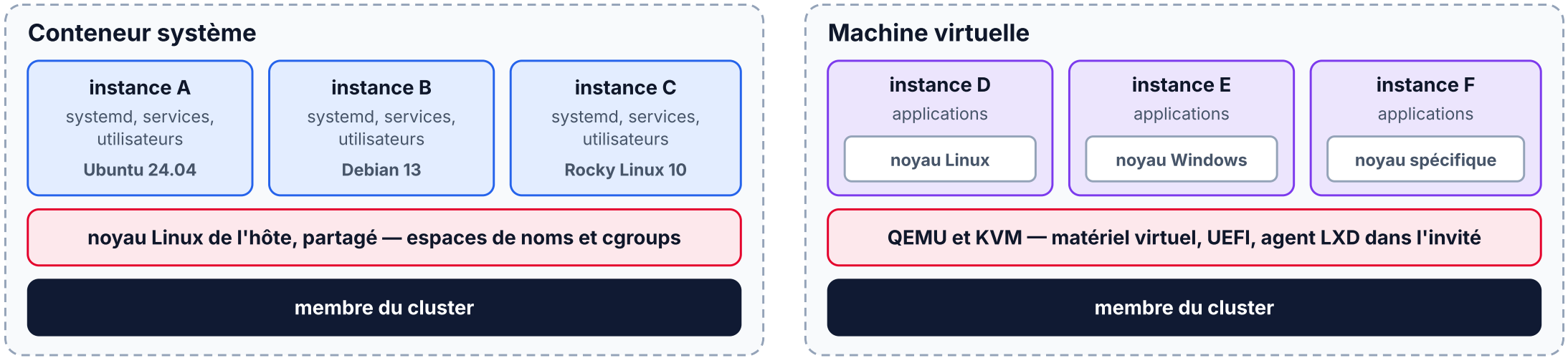
> Une LTS : **cinq ans** de support standard — deux de support complet, trois de maintenance. Ubuntu Pro en ajoute **cinq**.

Pistes du snap `microcloud`

Piste	Statut	Usage
1	fin de vie depuis avril 2025	à quitter
2	LTS, en maintenance	clusters existants
3	LTS, piste par défaut	production
4	versions de fonctionnalités	essais, pas la production

- **Canal** = piste + risque : `3/stable`. Risques : `stable`, `candidate`, `edge`.
- **Règle absolue :** tous les membres exécutent la **même version** de chaque snap, et les quatre versions doivent être compatibles entre elles.
- La version de Ceph est supportée pendant toute la vie de la version d'Ubuntu qui l'a livrée.

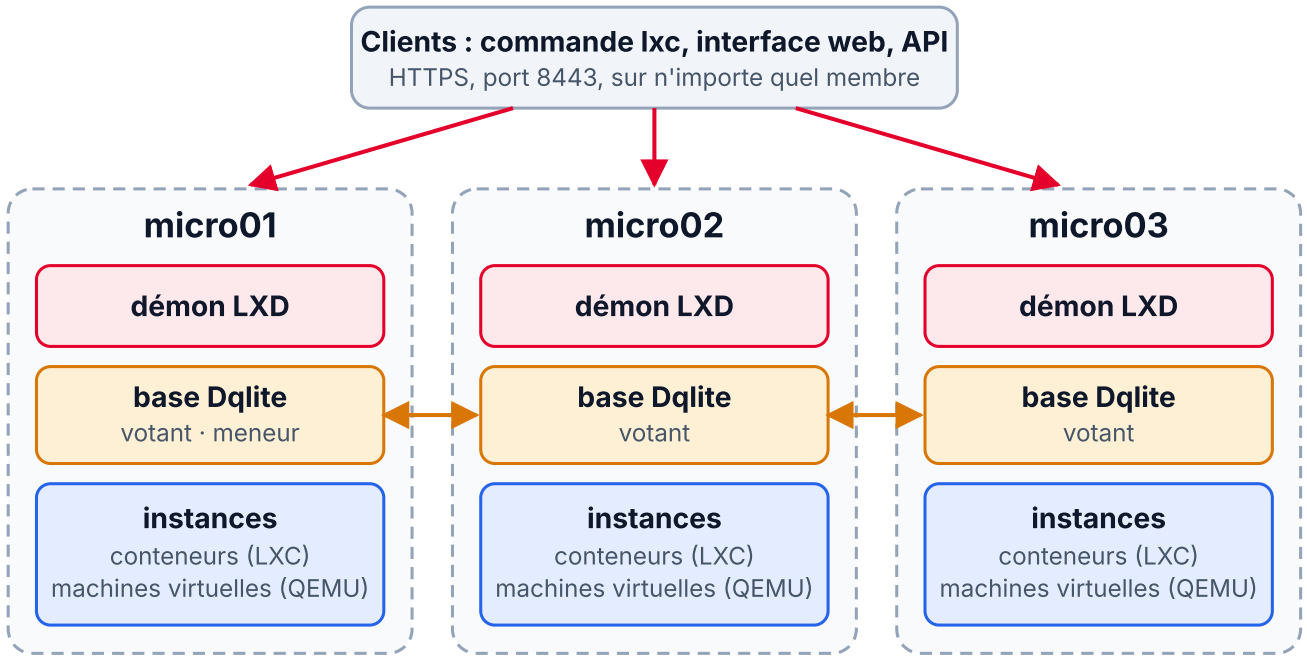
Conteneurs système et machines virtuelles



	Conteneur système	Machine virtuelle
Contenu	**un système Linux complet, sans noyau propre**	un système quelconque, avec son noyau
Démarrage, densité	**quelques secondes ; très forte densité**	plus long ; mémoire réservée par instance
Isolation	**bonne, mais le noyau est partagé**	forte, par la virtualisation matérielle
Migration à chaud	**non : arrêt, déplacement, redémarrage**	oui, si l'instance y est préparée
Cas typiques	**services Linux, environnements de développement, intégration continue**	Windows, noyaux particuliers, charges non fiables, Kubernetes

Même commande, un seul drapeau d'écart : `lxc launch ubuntu:24.04 web1` crée un conteneur ; `lxc launch ubuntu:24.04 web1 --vm` crée une machine virtuelle.

Le cluster LXD



La base est répliquée par consensus : la majorité des votants doit être joignable.

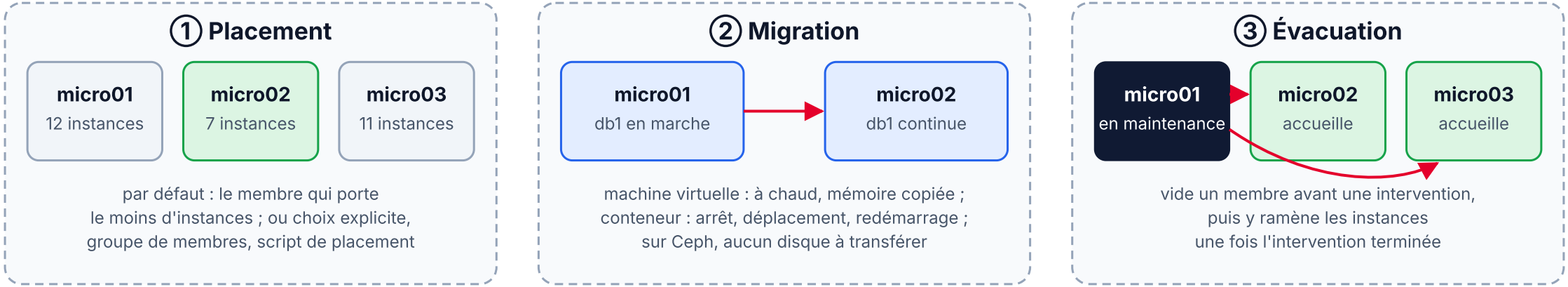
Trois votants : une panne tolérée
au-delà de trois membres, les autres sont en attente et remplacent un votant défaillant

- **Pas de nœud maître :** chaque membre expose la même API et peut traiter n'importe quelle requête.
- **L'état du cluster** — instances, profils, réseaux, pools, droits — vit dans une base **Dqlite** (SQLite répliqué) portée par les membres eux-mêmes.
- **La configuration est commune :** un pool ou un réseau se définit une fois pour tout le cluster, avec des valeurs propres à chaque membre quand il le faut (quel disque, quelle interface).
- **Les instances, elles, tournent sur un membre précis** ; leur disponibilité dépend du stockage choisi.

Besoin	Commande
Membres et rôles	`lxc cluster list`
Détail d'un membre	`lxc cluster show micro02`
Où tournent les instances	`lxc list` — colonne LOCATION

Perte de la majorité (deux membres sur trois) : les instances en marche continuent, mais l'API n'accepte plus aucun changement tant que le quorum n'est pas revenu.

Placement, migration, évacuation



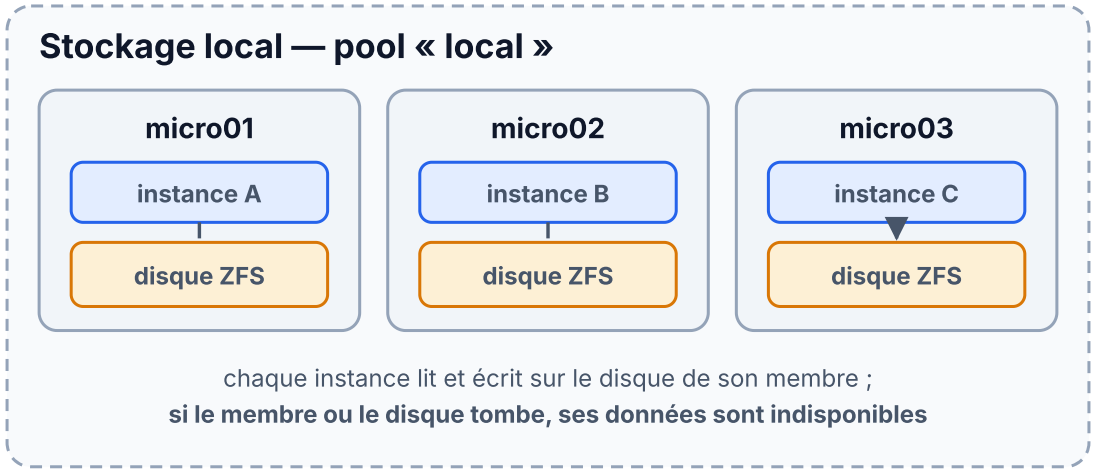

```
# Préparer une machine virtuelle à la migration à chaud
lxc config set db1 migration.stateful=true

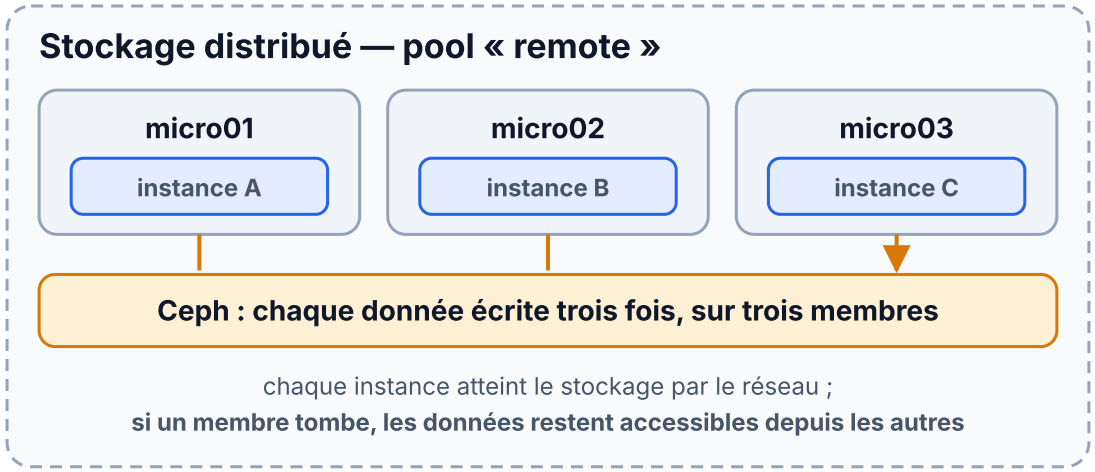
# Déplacer une instance
lxc move db1 --target micro02

# Maintenance d'un membre
lxc cluster evacuate micro01
lxc cluster restore micro01
```

- **Le stockage décide de tout :** sur le pool `remote`, une instance repart sur un autre membre sans copie ; sur `local`, son disque doit être transféré par le réseau.
- **Reprise automatique :** LXD peut évacuer seul les instances d'un membre resté hors ligne au-delà d'un délai (`cluster.healing_threshold`) — réservé aux instances sur stockage distribué, et à activer explicitement.
- **Groupes de membres :** réserver des machines à un usage — cartes graphiques, architecture, licence.

Local ou distribué : le choix structurant

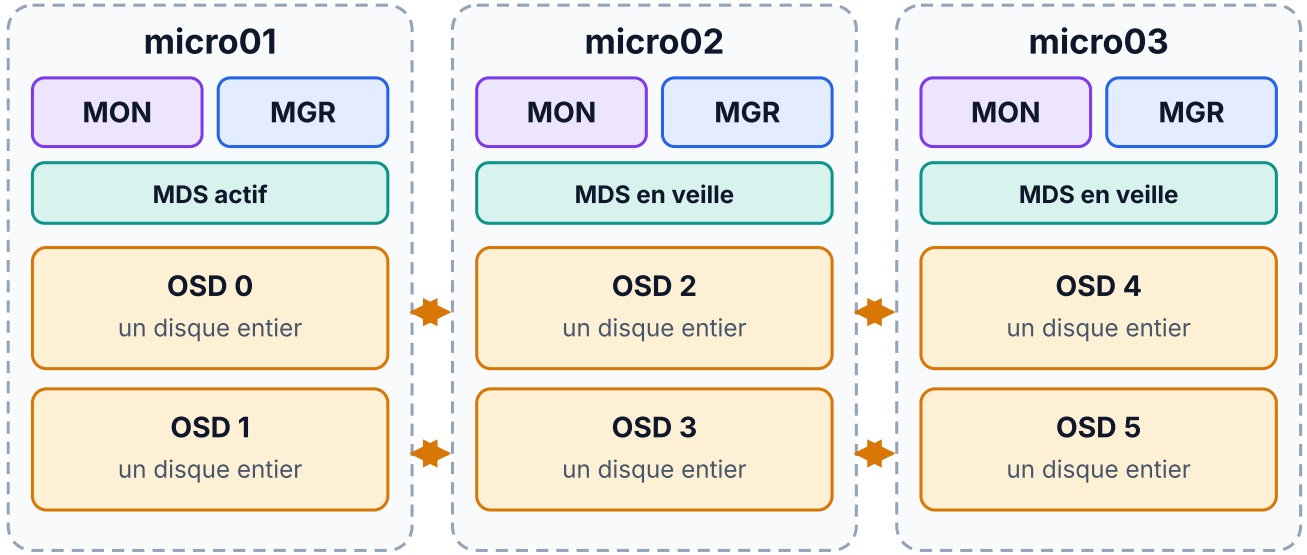




	Local	Distribué
Redondance	**aucune ; repose sur les sauvegardes**	trois copies par défaut ; réparation automatique
Performances	**souvent meilleures : tout reste sur le disque local**	latence du réseau et coût de la réplication, sensibles en écriture
Ressources consommées	**faibles**	processeur, mémoire et bande passante pour Ceph
Déplacement d'une instance	**le disque est copié par le réseau**	rien à copier : rapide
Après la perte d'un membre	**restauration depuis une sauvegarde**	redémarrage de l'instance sur un autre membre
Capacité utile	**celle du disque**	environ un tiers de la capacité brute

Recommandation pour la production : les deux. Le local pour les charges temporaires qui veulent des disques rapides et tolèrent la perte ; le distribué pour tout ce qui doit durer et rester disponible.

MicroCeph : Ceph en grappe, sans la complexité d'installation



Un objet écrit = trois copies, sur trois membres différents

l'algorithme CRUSH choisit les disques ; le client calcule lui-même où lire et écrire

Domaine de panne ajusté automatiquement

à partir de trois membres, MicroCeph répartit les copies par machine et non plus par disque

Démon	Rôle
MON	tient la carte du cluster ; quorum de trois
MGR	métriques, modules de gestion
OSD	un par disque : stocke, réplique, répare
MDS	métadonnées du système de fichiers CephFS

- **Ce que fait MicroCeph :** installe ces démons, les met en grappe, ajoute les disques, répartit les rôles quand des membres arrivent.
- **Ce que fait MicroCloud :** lui indique les disques choisis, puis déclare les pools dans LXD.
- **Ce que fait LXD :** crée un volume par disque d'instance et le branche sur le membre où tourne l'instance.

```
sudo microceph status
sudo microceph disk list
sudo microceph.ceph status      # santé détaillée
sudo microceph disk add /dev/sdd --wipe
```

Les trois pools créés par MicroCloud

Pool	Pilote	Ce qu'il contient	Usage	Particularité
local	**zfs**	un jeu de données ZFS par instance, sur un disque de chaque membre	charges temporaires, disques rapides, caches, intégration continue	pool commun au cluster, mais contenu propre à chaque membre
remote	**ceph**	un volume bloc Ceph (RBD) par disque d'instance ou volume de données	tout ce qui doit survivre à la perte d'un membre ; pool du profil par défaut	un volume bloc s'attache à une seule instance à la fois
remote-fs	**cephfs**	un système de fichiers partagé, découpé en volumes	données lues et écrites par **plusieurs** instances en même temps	facultatif ; ne porte pas de disque racine d'instance

```
# Voir les pools et leur occupation
lxc storage list
lxc storage info remote

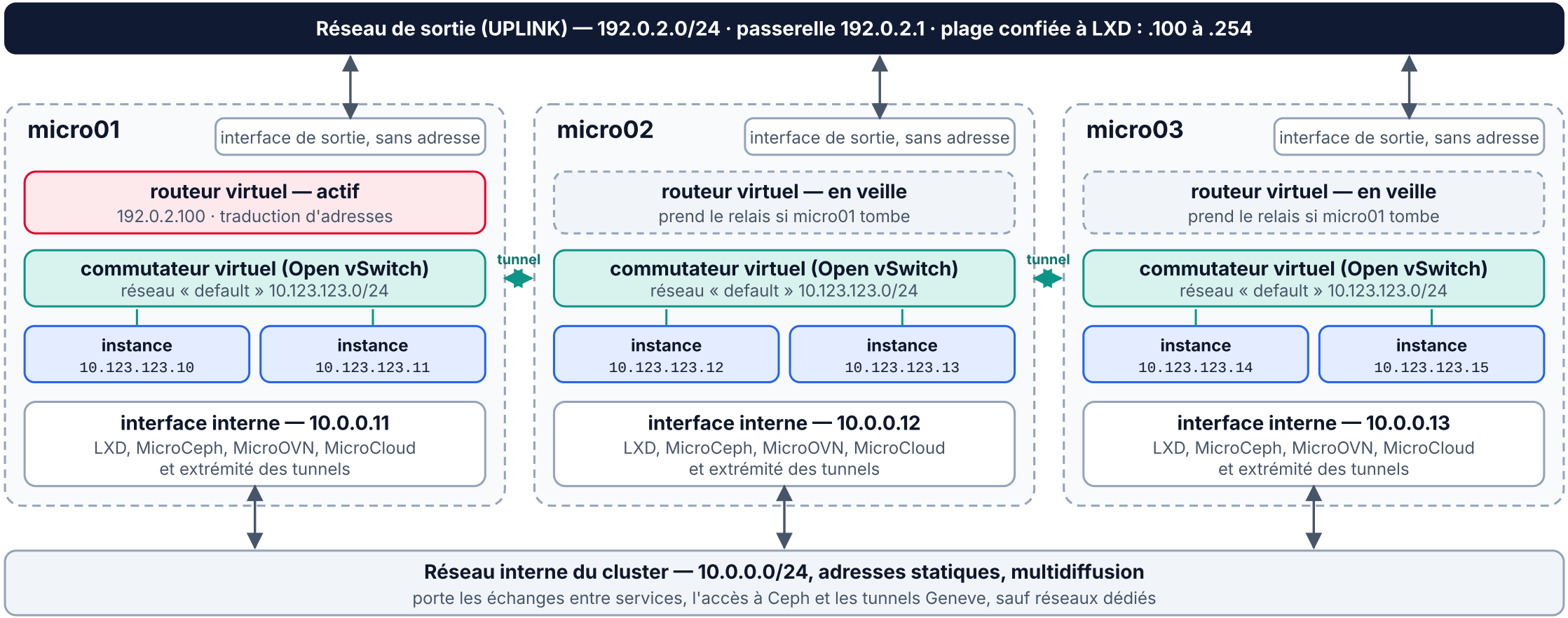
# Choisir le pool à la création
lxc launch ubuntu:24.04 cache1 --storage local

# Volume partagé entre deux instances
lxc storage volume create remote-fs partage
lxc storage volume attach remote-fs partage web1 /data
lxc storage volume attach remote-fs partage web2 /data
```

- **Images :** mises en cache dans le pool à la première utilisation ; les instances suivantes sont créées par clonage, en quelques secondes.
- **Instantanés :** natifs sur ZFS et sur Ceph, donc rapides et économes.
- **Déplacer une instance d'un pool à l'autre :** possible, instance arrêtée (`lxc move web1 --storage remote`).
- **Dimensionner `remote` :** capacité utile ≈ capacité brute ÷ 3, moins une marge pour la réparation — ne pas dépasser environ 80 % d'occupation.

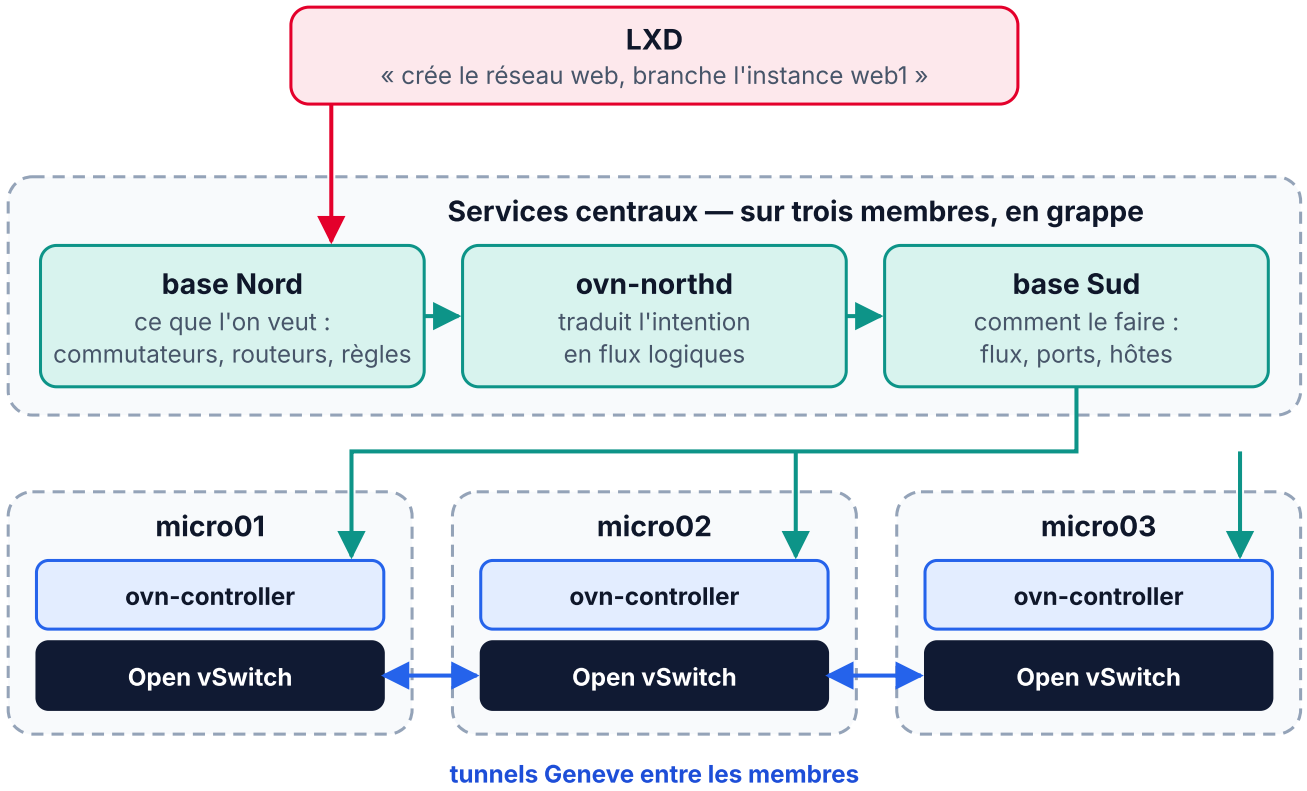
> Sans disque pour le stockage distribué, MicroCloud ne crée que `local` : le cluster fonctionne, mais sans redondance des données ni reprise sur un autre membre.

Les réseaux d'un MicroCloud



À retenir : deux interfaces par membre au minimum. L'une, adressée, porte tout le trafic interne du cluster ; l'autre, **sans adresse**, est confiée à OVN pour relier les réseaux virtuels au monde extérieur.

MicroOVN : architecture



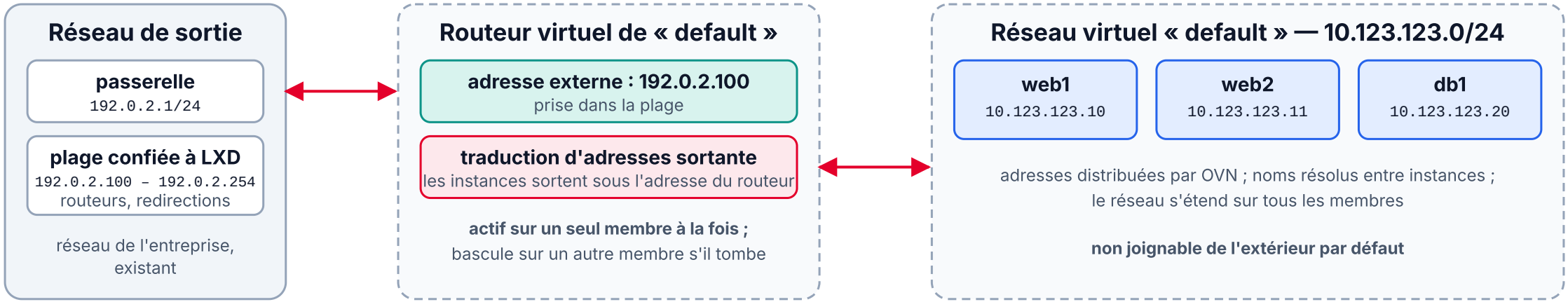
Chaque membre programme son propre commutateur : pas de point de passage central pour le trafic.

- **MicroOVN** est une enveloppe minimale autour d'OVN : il installe les services, les met en grappe et gère leurs certificats.
- **Commutation et routage distribués :** chaque membre héberge son commutateur virtuel. Le trafic entre instances continue même si d'autres membres sont hors ligne.
- **DHCP et DNS** des réseaux virtuels sont rendus par OVN lui-même, sur chaque membre.
- **Sécurité :** les points d'accès d'OVN sont protégés par TLS 1.2 ou plus, avec des clés à courbe elliptique P-384.
- **On ne configure rien dans OVN à la main :** c'est LXD qui écrit dans la base Nord.

```
sudo microovn status
sudo microovn.ovn-nbctl show   # vue logique
sudo microovn.ovn-sbctl show   # hôtes et tunnels
```

> Ces deux dernières commandes servent à **lire** pour diagnostiquer, jamais à modifier.

Sortir et entrer : le routeur virtuel

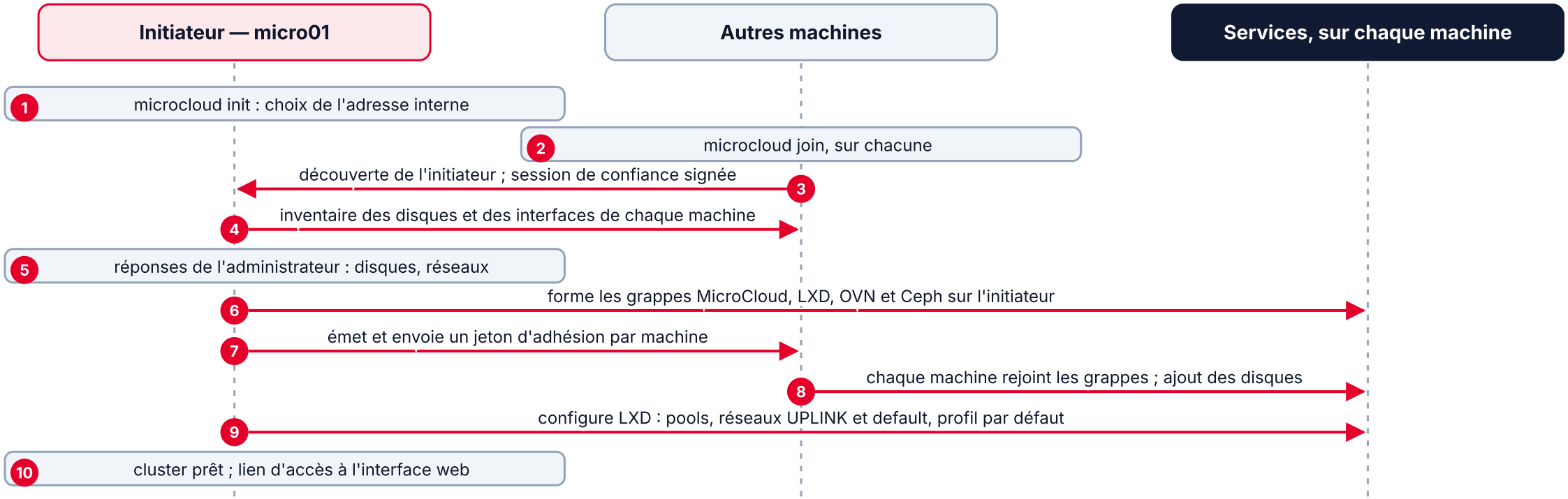


Pour exposer…	Mécanisme
un port d'une instance	redirection : une adresse de la plage, des ports renvoyés vers l'instance
plusieurs instances derrière une adresse	répartiteur de charge de niveau 4, rendu par OVN
une instance entière	adresse externe routée vers l'instance

```
# Rediriger 192.0.2.110:443 vers web1
lxc network forward create default 192.0.2.110
lxc network forward port add default 192.0.2.110 \
  tcp 443 10.123.123.10

# Répartir sur web1 et web2 : même principe, avec
lxc network load-balancer create default 192.0.2.111
```

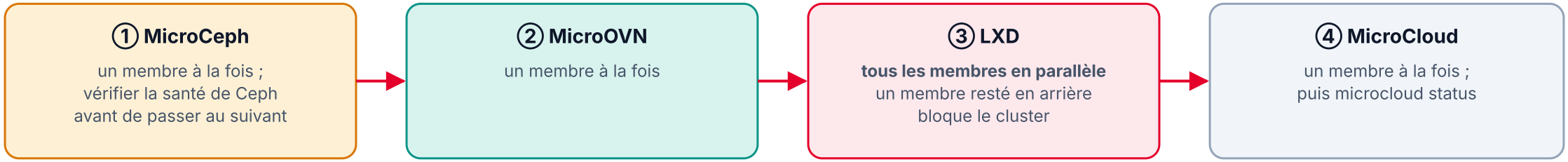
L'initialisation, vue de l'intérieur



Confiance : les échanges entre l'initiateur et les machines qui le rejoignent sont signés à partir d'un secret partagé qui ne circule jamais sur le réseau ; ensuite, tout passe en TLS mutuel, par certificats.

Découverte : par multidiffusion, limitée au sous-réseau de l'interface interne. Là où elle est filtrée — chez un hébergeur, par exemple — on indique l'adresse de l'initiateur à la main.

Mises à jour : un ordre à respecter



Chaque composant est traité sur tous les membres avant de passer au suivant.

```
# Mise à jour dans la piste courante
sudo snap refresh microceph --cohort="+"
sudo snap refresh microovn  --cohort="+"
sudo snap refresh lxd       --cohort="+"
sudo snap refresh microcloud --cohort="+"

# Montée de version : même ordre, avec le nouveau canal
sudo snap refresh microceph --channel=<canal> --cohort="+"
```

Vérifier l'alignement : `snap list lxd` doit afficher `in-cohort` sur chaque membre.

- **Avant :** sauvegarder LXD et MicroCeph ; lire le guide de mise à niveau de chaque composant.
- **Impact :** LXD laisse tourner les instances ; les mises à jour de Ceph et d'OVN peuvent perturber brièvement stockage et réseau.
- **Pour ne rien interrompre :** évacuer chaque membre avant, restaurer après — les machines virtuelles migrent à chaud, les conteneurs sont arrêtés puis relancés.
- **Snaps désalignés :** les instances continuent, mais on ne peut plus ajouter ni retirer de membre ou d'instance.
- **Grand parc :** un mandataire de magasin d'entreprise permet de figer les révisions distribuées.

Disponibilité : que se passe-t-il quand un membre tombe ?

Couche	Mécanisme de redondance	Un membre sur trois tombe	Deux membres sur trois tombent
API de LXD	**tout membre répond ; base Dqlite à trois votants**	l'API reste disponible sur les deux autres	plus de quorum : aucun changement possible
Instances du membre perdu	**dépend du stockage**	sur `remote` : à relancer ailleurs, à la main ou automatiquement ; sur `local` : indisponibles	idem, si le membre restant a la capacité
Instances des autres membres	**—**	continuent de tourner	continuent, mais leur stockage distribué se fige
Stockage distribué	**trois copies, quorum de moniteurs**	lectures et écritures continuent, en mode dégradé	figé pour protéger les données
Sortie vers l'extérieur	**routeur virtuel actif sur un membre, avec bascule**	bascule sur un autre membre ; brève coupure	dépend du membre restant

Trois membres

Tolère **une** panne. Pendant la panne, plus aucune marge : ni pour une seconde panne, ni pour reconstruire la troisième copie des données.

Quatre membres et plus

Recommandé pour un service critique : après une panne, Ceph retrouve trois copies et le cluster garde une marge.

Un seul site

Un MicroCloud vit sur un réseau local à faible latence. La protection contre la perte d'un site passe par un **second cluster** et des copies entre les deux.

Capacité à réserver : pour que les instances d'un membre perdu repartent ailleurs, garder libre l'équivalent d'un membre en mémoire et en processeurs.

M7 · BILAN

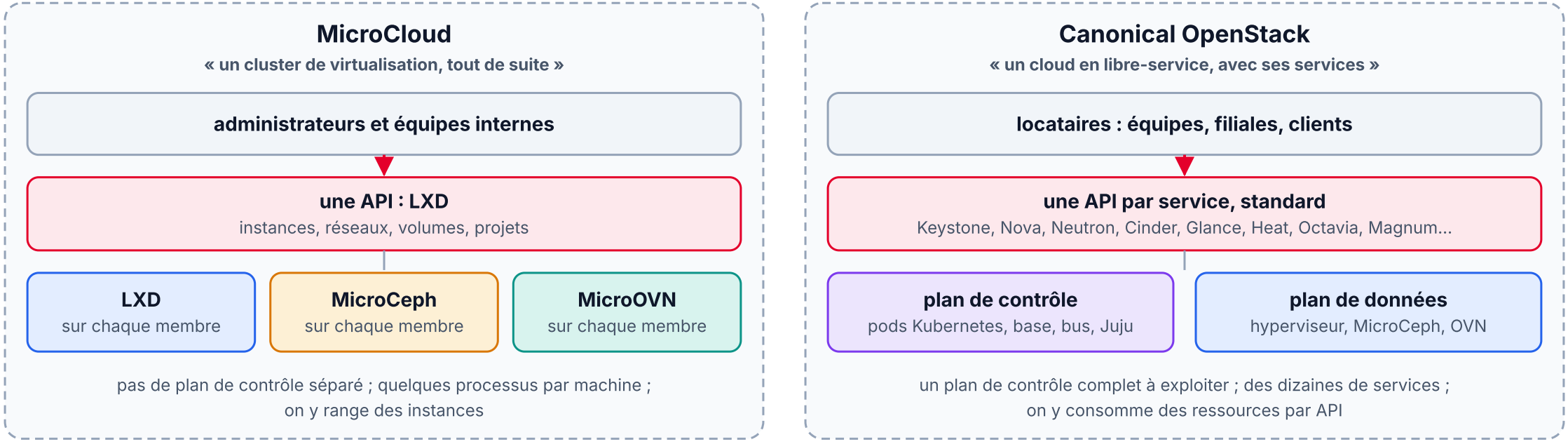
Limitations

Limitation	Détail	Conséquence pratique
Taille	**cinquante membres au plus par cluster**	au-delà : plusieurs clusters indépendants, ou une autre plateforme
Un seul site	**réseau local à faible latence entre membres ; adresses fixes**	pas de cluster étiré ; reprise d'activité par un second cluster
Pas de services de plateforme	**ni orchestration, ni bases ou Kubernetes à la demande, ni catalogue**	à construire au-dessus, avec Terraform, Juju ou Ansible
API propre à LXD	**aucune compatibilité avec OpenStack ou un cloud public**	outils et applications écrits pour ces API à adapter
Multi-locataire limité	**projets, limites et droits, mais pas de domaines, de facturation ni de portail en libre-service pour des clients**	adapté à des équipes internes plus qu'à des clients externes
Conteneurs	**pas de migration à chaud ; noyau partagé avec l'hôte**	machines virtuelles pour les charges critiques ou non fiables
Stockage local	**aucune redondance**	sauvegardes indispensables ; pas de reprise automatique
Choix techniques imposés	**LXD, Ceph, OVN, Ubuntu, snaps**	pas de baie de stockage ni de réseau virtuel tiers
Écosystème	**moins d'outils tiers certifiés que pour les hyperviseurs historiques**	vérifier sauvegarde, supervision et agents avant de migrer

Discipline exigée : snaps alignés et mis à jour dans un ordre précis, jamais moins de trois membres. Le produit est simple, pas indulgent.

La dernière ligne est une appréciation, pas une donnée de l'éditeur : à confronter à votre propre outillage.

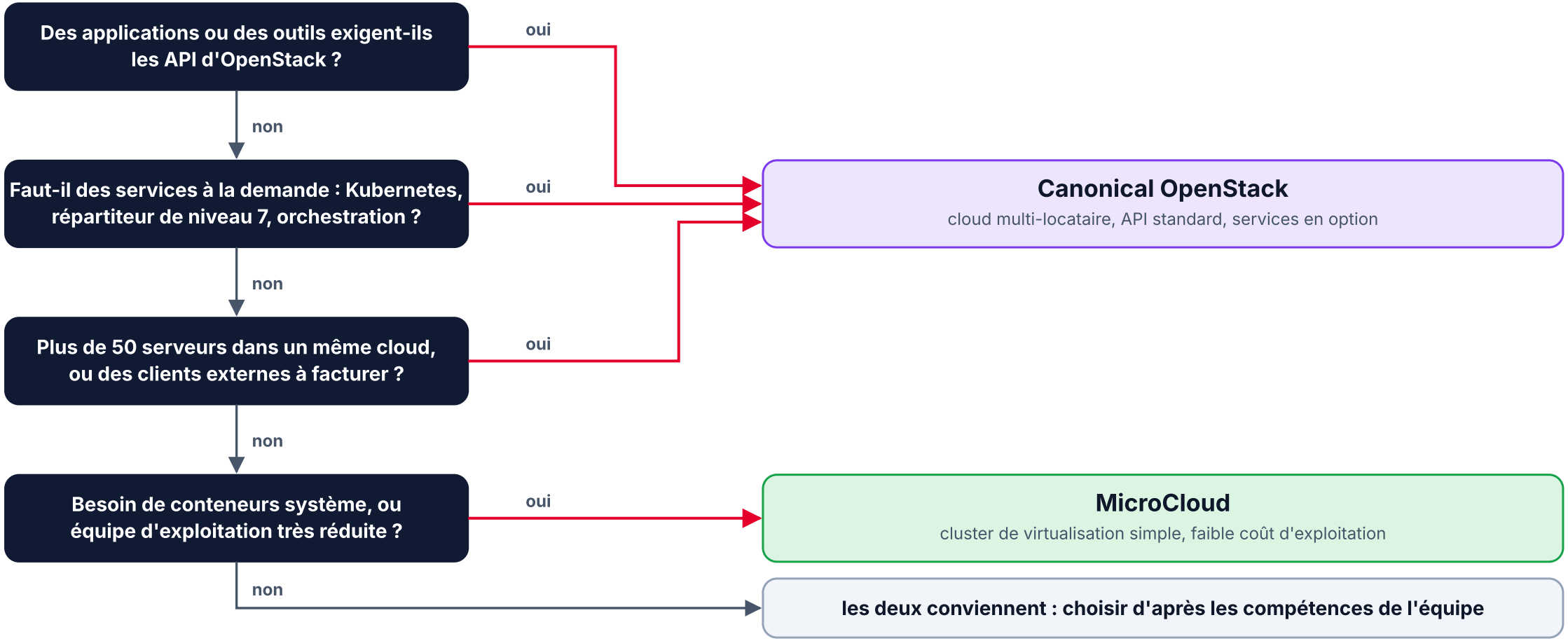
Deux philosophies



MicroCloud répond à « où faire tourner mes machines virtuelles et mes conteneurs, de façon fiable et simple ? ».

Canonical OpenStack répond à « comment offrir à de nombreux utilisateurs un cloud complet, avec des API standard et des services à la demande ? ».

Arbre de décision



Les trois premières questions éliminent MicroCloud ; la quatrième le désigne. Quand aucune ne tranche, la plateforme la plus simple que l'équipe sait exploiter est le bon choix.

08

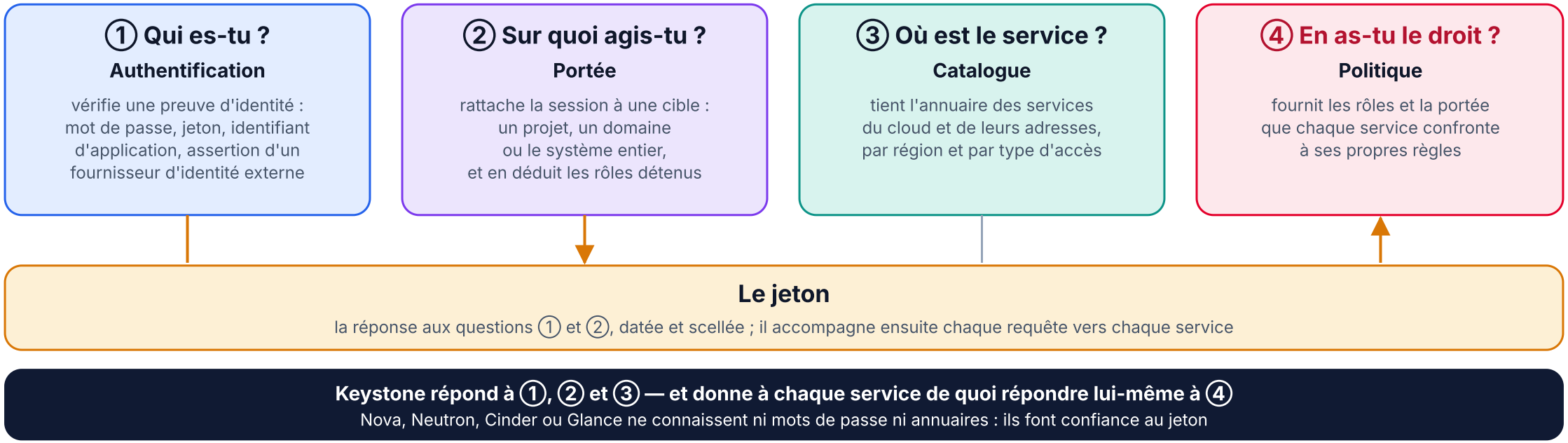
Keystone en profondeur

Entités, catalogue de services, jetons d'authentification, sources d'identité, politiques d'accès — sur Canonical OpenStack (Sunbeam)

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Décrire les **entités** de Keystone et leurs relations.
- Expliquer le cycle de vie d'un **jeton** : émission, contenu, validation, expiration, rotation des clés.
- Lire une **règle de politique** et prévoir qui peut faire quoi.
- Lire et exploiter le **catalogue de services**.
- Raccorder une **source d'identité** externe : annuaire LDAP ou fournisseur OIDC / SAML.
- Diagnostiquer un refus : **401**, **403**, service introuvable.

Quatre questions, un service

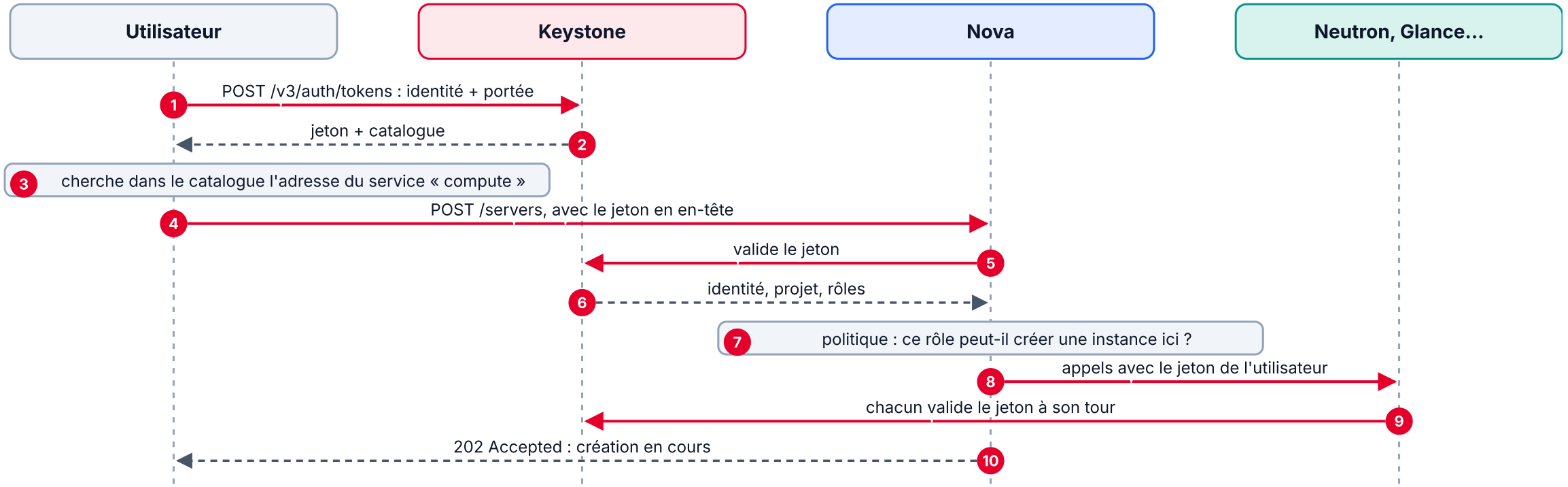


Ce que Keystone ne fait pas : il ne décide pas si l'on peut créer une instance ou supprimer un volume. Cette décision appartient à Nova ou à Cinder, à partir de ce que Keystone a certifié.

Version de l'API : une seule, la v3. Les notions de domaine, de groupe et de portée système n'existent que dans celle-ci.

 Parcours d'un jeton : module 1

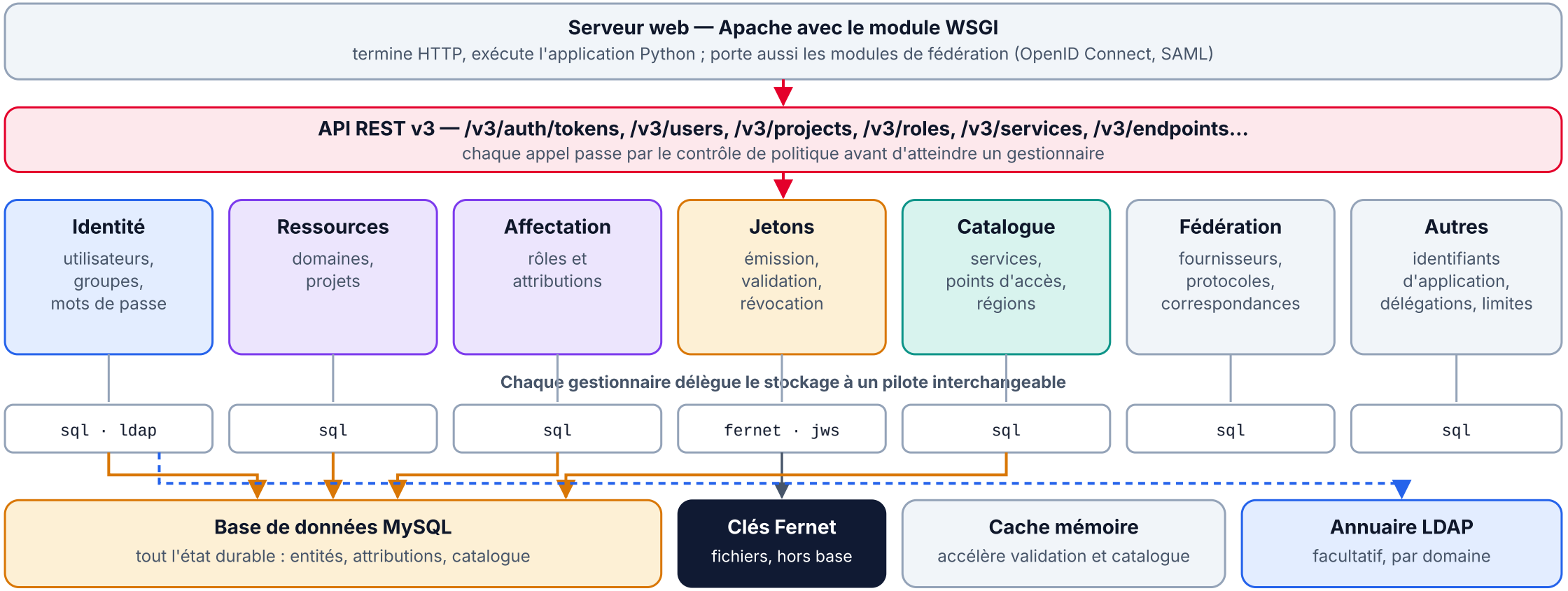
Une requête de bout en bout



Deux appels à Keystone par opération : l'un par l'utilisateur, pour obtenir le jeton ; l'autre par le service, pour le valider. Le second est mis en cache par le service pour ne pas être répété à chaque requête.

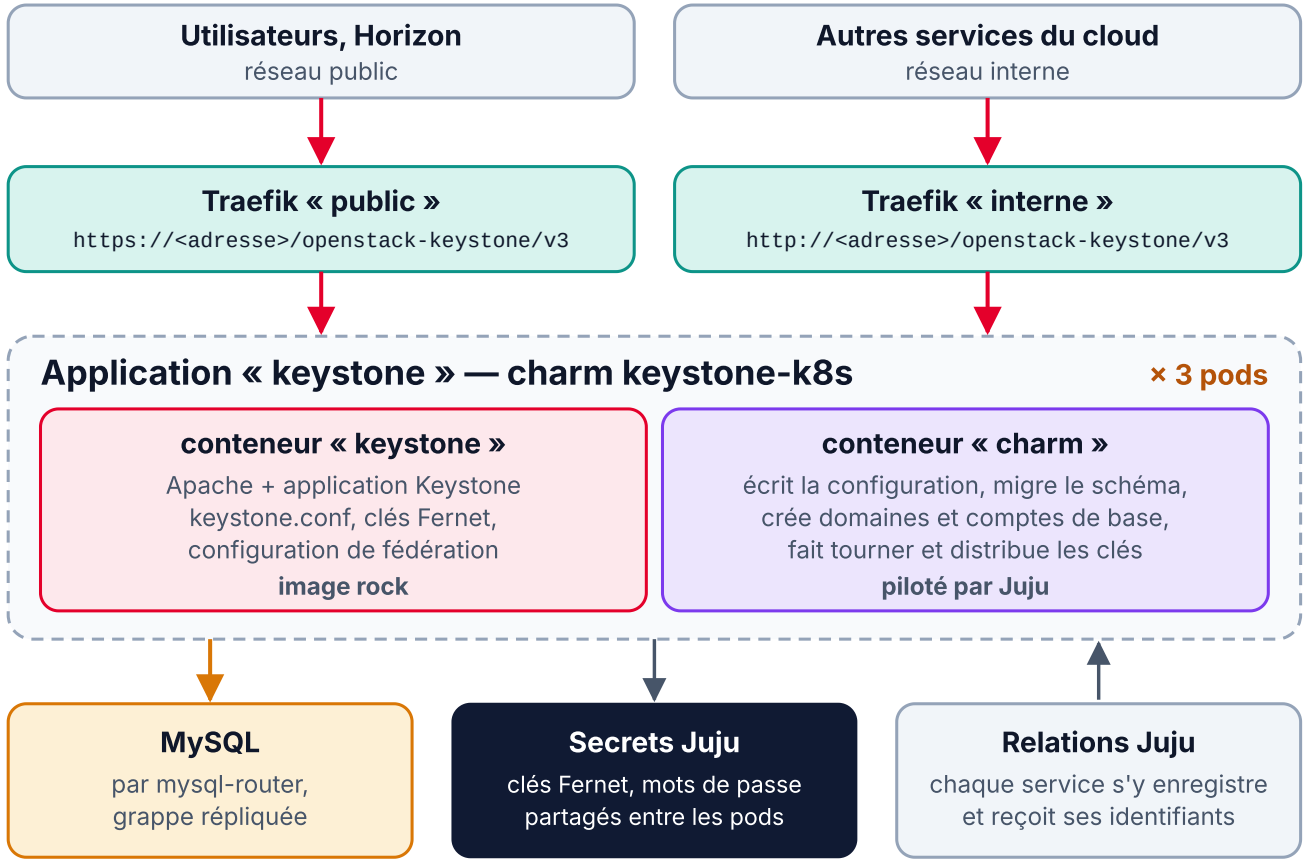
Conséquence d'exploitation : si Keystone est lent, tout le cloud est lent ; s'il est indisponible, les jetons déjà validés et encore en cache fonctionnent quelques minutes, puis tout s'arrête.

Architecture interne



Lecture du schéma : chaque requête traverse Apache puis le contrôle de politique avant d'atteindre l'un des gestionnaires ; chacun range ses données par un pilote — SQL presque partout, LDAP possible pour l'identité, fichiers pour les clés Fernet. Seule l'identité peut donc venir d'un annuaire : projets, rôles et attributions restent en base.

Keystone sur Sunbeam : où il tourne

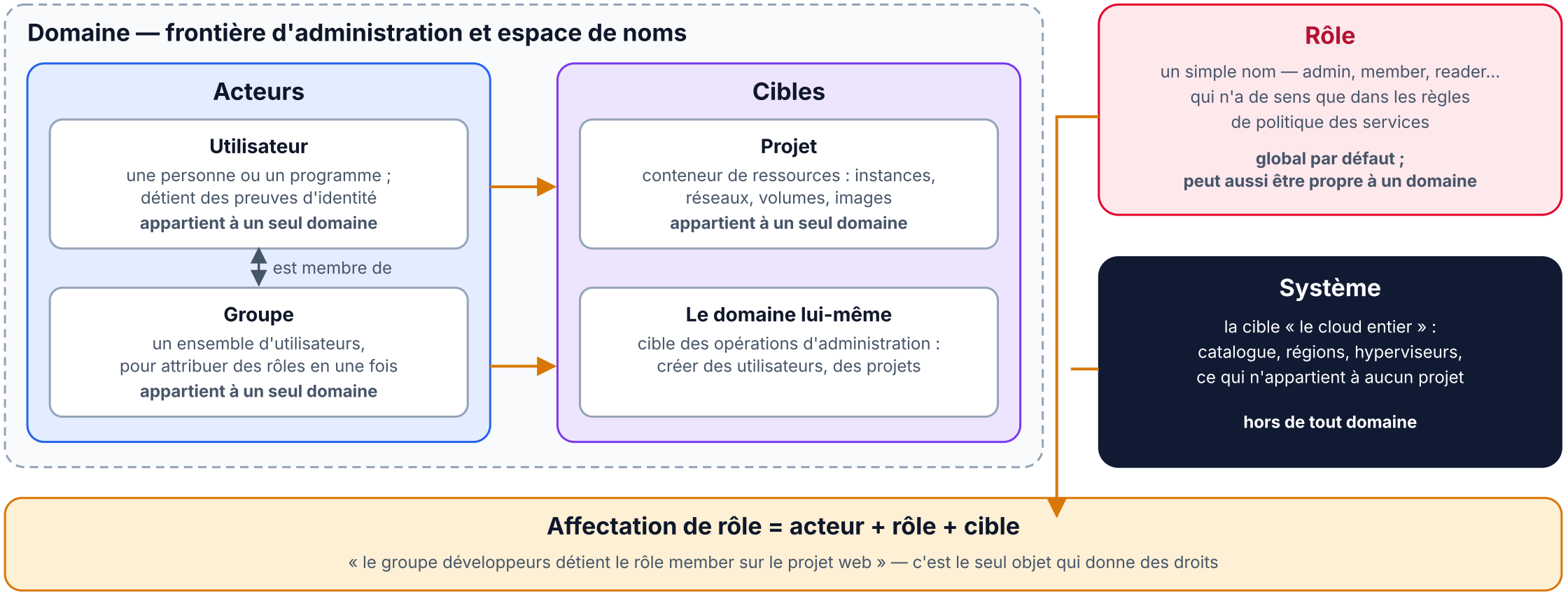


- **Un pod par unité**, trois unités en haute disponibilité : n'importe laquelle traite n'importe quelle requête, car l'état est dans la base et les clés sont identiques partout.
- **Deux entrées :** les points d'accès public et interne passent par deux Traefik distincts, sous un préfixe de chemin.
- **Aucun fichier édité à la main :** `keystone.conf` est produit par le charm à partir de ses options et de ses relations.
- **Les autres services** n'ont pas de compte créé à la main : la relation avec Keystone crée leur utilisateur de service et inscrit leurs points d'accès.

```
juju status -m openstack keystone
juju config -m openstack keystone
juju actions -m openstack keystone
sudo k8s kubectl -n openstack get pods \
  -l app.kubernetes.io/name=keystone
```

> Options, actions et noms exacts varient selon la révision du charm : ces quatre commandes les montrent pour votre cloud.

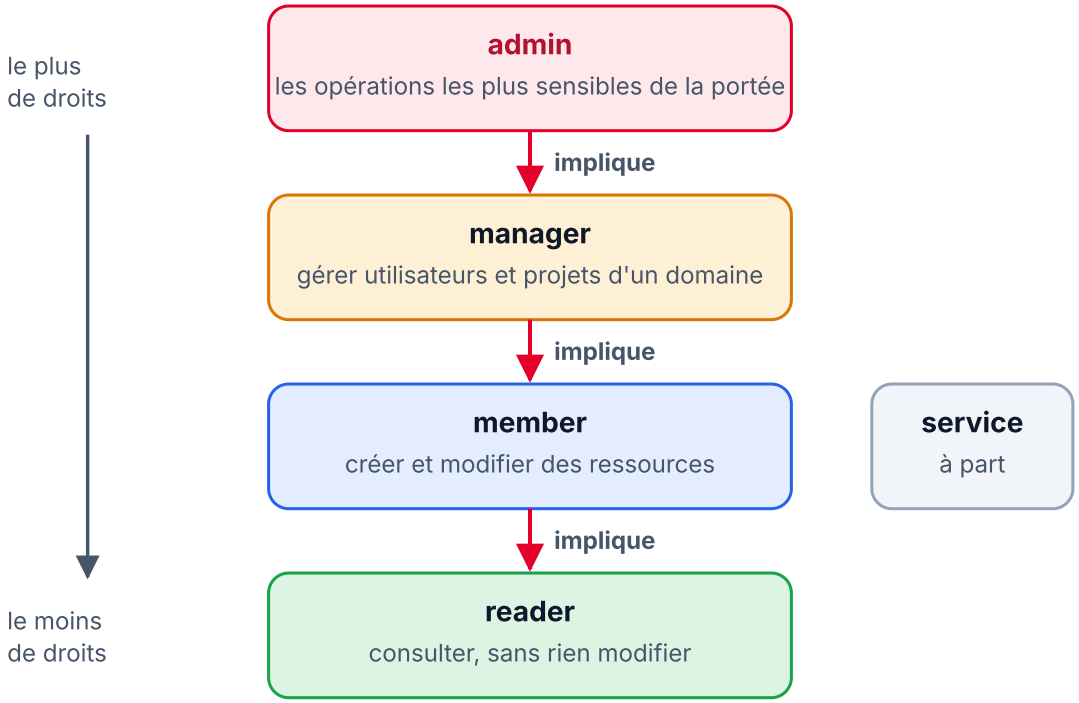
Carte des entités



Lecture du schéma : à gauche les acteurs, à droite les cibles, au centre le rôle. Aucun de ces objets ne donne de droit à lui seul : seule l'affectation, qui relie un acteur, un rôle et une cible, en crée un. Le système est la seule cible hors de tout domaine.

 Approfondi au module 12

Rôles et rôles impliqués



Détenir « admin » donne automatiquement manager, member et reader.
« service » est réservé aux échanges entre services et n'entre pas dans la chaîne.

- **Un rôle est une étiquette.** Keystone ne sait pas ce que « member » permet dans Nova : ce sont les règles de politique de Nova qui lui donnent un sens.
- **Cinq rôles fournis d'office :** `admin`, `member`, `reader` depuis longtemps ; `manager` et `service` depuis la version 2023.2.
- **Rôle impliqué :** une règle « tel rôle entraîne tel autre », résolue par Keystone à l'émission du jeton. Le jeton d'un administrateur contient donc aussi les rôles inférieurs.
- **Rôles supplémentaires :** certains services en définissent — par exemple pour l'équilibrage de charge ou l'orchestration.
- **Rôle propre à un domaine :** un alias local qui implique des rôles globaux ; rarement nécessaire.

```
openstack role list
openstack implied role list
openstack role create auditeur
openstack implied role create auditeur \
  --implied-role reader
```

Créer un rôle ne donne **aucun** droit tant qu'aucune règle de politique ne le mentionne : préférer les rôles par défaut, compris par tous les services.

Affectations de rôles : acteur, rôle, cible

Acteur	Rôle	Cible	Effet
utilisateur alice	**member**	projet « web »	alice crée et gère des ressources dans web
groupe developpeurs	**reader**	projet « production »	tous les développeurs consultent production, sans rien modifier
groupe exploitants	**member**	projet « production », **hérité**	le rôle vaut aussi pour tous les sous-projets, présents et futurs
utilisateur bob	**manager**	domaine « nord »	bob gère utilisateurs, groupes et projets du domaine
groupe plateforme	**admin**	système	administration du cloud entier
utilisateur nova	**service**	projet « services »	Nova s'identifie auprès des autres services

```
# Attribuer
openstack role add --user alice --user-domain nord \
  --project web --project-domain nord member
openstack role add --group exploitants \
  --group-domain nord --project production \
  --project-domain nord --inherited member
openstack role add --user bob --user-domain nord \
  --domain nord manager
openstack role add --group plateforme \
  --group-domain nord --system all admin
```

```
# Ce qui a été saisi
openstack role assignment list --names \
  --project web --project-domain nord

# Ce qui s'applique réellement : groupes,
# héritage et rôles impliqués résolus
openstack role assignment list --names \
  --user alice --user-domain nord --effective
```

> **Réflexe de diagnostic :** l'option `--effective` répond à la question « pourquoi cette personne a-t-elle, ou n'a-t-elle pas, ce droit ? ».

Domaines : module 9 · projets : module 10

Ce que Sunbeam a créé pour vous

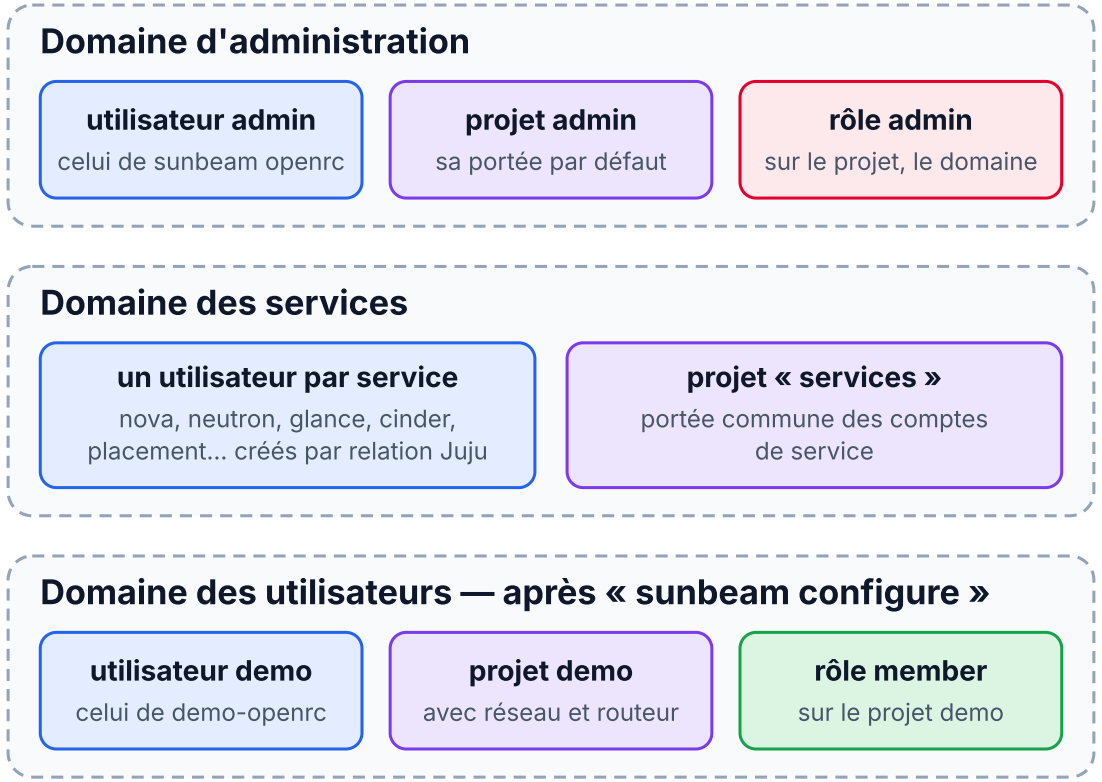


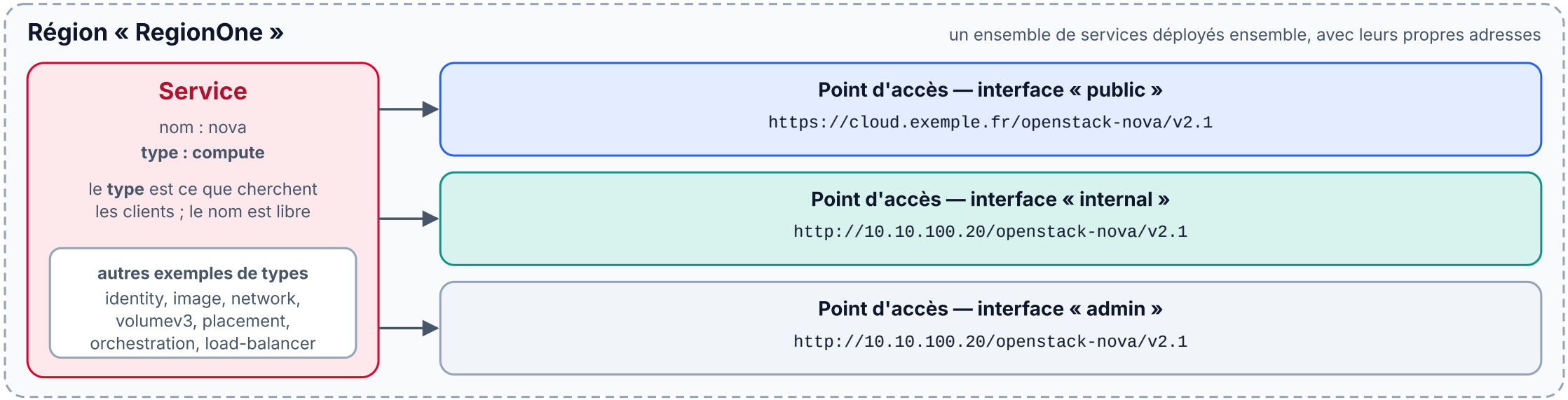
Schéma de principe : noms exacts à relever sur votre cloud.

```
# Relever ce qui existe réellement
openstack domain list
openstack project list --long
openstack user list --long
openstack role list
openstack role assignment list --names
```

- **Séparer humains et services :** les comptes des services vivent dans leur propre domaine, à ne jamais utiliser pour des personnes.
- **Ne pas travailler avec le compte `admin` initial :** créer un domaine pour l'organisation et des comptes nominatifs.
- **Le compte de démonstration** sert à valider l'installation ; il n'a pas vocation à rester sur un cloud de production.

> Le charm expose des actions pour retrouver le mot de passe de l'administrateur ou créer un compte de service : `juju actions keystone` en donne la liste.

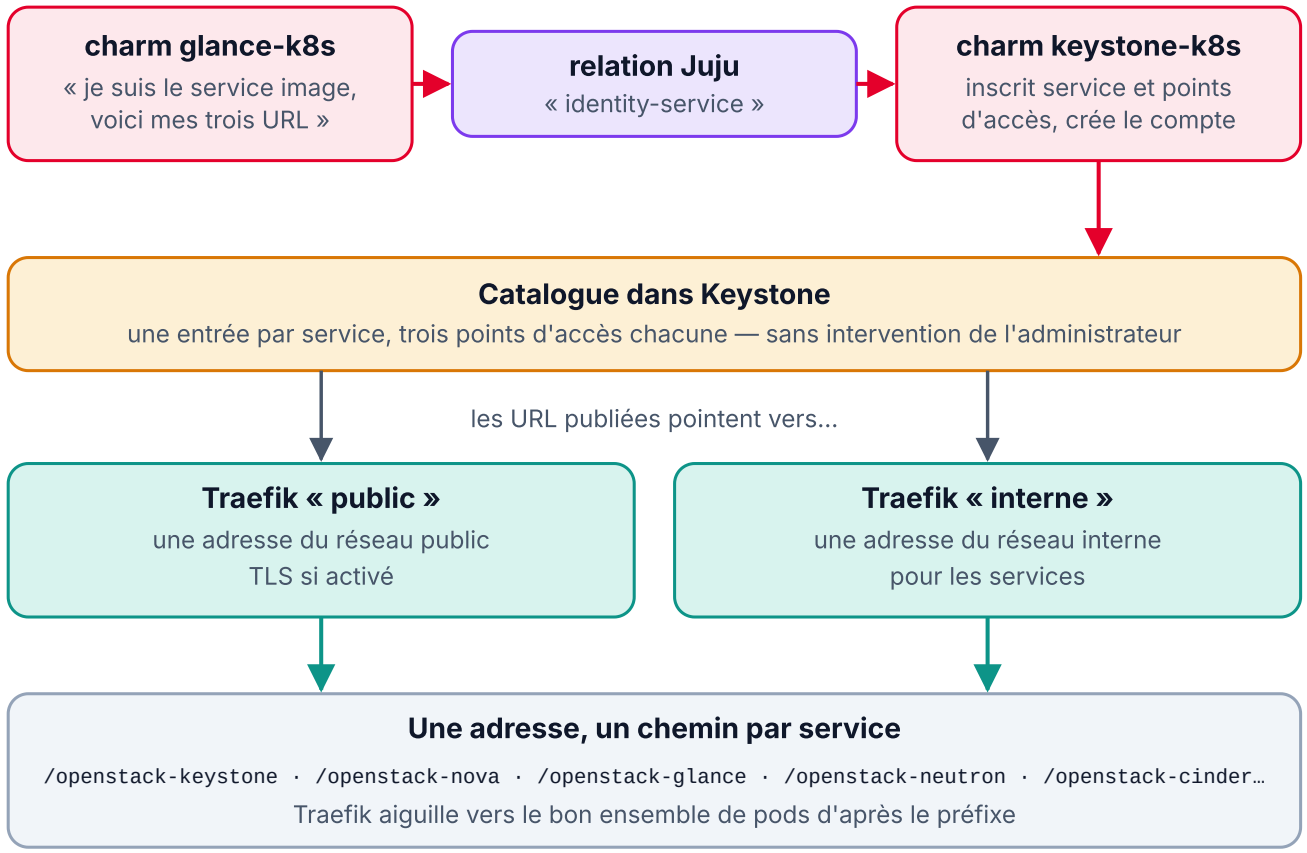
Services, points d'accès, régions, interfaces



Interface	Qui l'utilise	Sur quel réseau	Chiffrement
public	**utilisateurs, outils externes, Horizon côté navigateur**	réseau public du cloud, adresse ou nom publié	TLS attendu
internal	**les services entre eux**	réseau interne, non exposé	selon la configuration
admin	**historique ; rarement distingué aujourd'hui**	souvent identique à l'interface interne	selon la configuration

Adresses d'exemple. Un point d'accès est l'association d'un service, d'une région, d'une interface et d'une URL ; il peut être désactivé sans être supprimé.

Le catalogue sur Sunbeam



- **Le catalogue se remplit seul :** chaque charm de service déclare ses points d'accès à Keystone par une relation ; l'ajout d'une fonctionnalité Sunbeam ajoute ses entrées.
- **Adresses de service :** prises dans les plages réservées aux API, l'une sur le réseau public, l'autre sur le réseau interne.
- **Un seul port, des chemins :** tous les services partagent l'adresse de Traefik et se distinguent par leur préfixe.
- **TLS :** en activant la fonctionnalité correspondante, les URL publiques passent en `https` et le catalogue est mis à jour par les charms.
- **Nom d'hôte :** on peut publier un nom plutôt qu'une adresse IP ; le manifeste prévoit ce réglage pour chaque entrée.

Catalogue : il se règle par la configuration de Sunbeam ; un `openstack endpoint set` serait réécrit par le charm.

Le nom exact de la relation et des préfixes se lit dans `juju status --relations` et `openstack endpoint list`.

Jetons et portées

Type de jeton	Contient	Sert à	S'obtient en demandant
Sans portée	**l'identité seule : ni rôles, ni catalogue**	prouver qui l'on est ; lister ses projets ; obtenir ensuite des jetons à portée sans redonner son mot de passe	aucune portée
Portée projet	**identité, projet, rôles sur ce projet, catalogue**	le travail courant : instances, volumes, réseaux	un projet et son domaine
Portée domaine	**identité, domaine, rôles sur ce domaine, catalogue**	gérer utilisateurs, groupes et projets du domaine	un domaine
Portée système	**identité, rôles sur le système, catalogue**	opérations sur le cloud entier	le système (« all »)
Issu d'une délégation	**seulement ce que la délégation transmet**	agir au nom d'un autre utilisateur, sur un projet donné	une délégation existante

Un jeton est un « porteur »

Quiconque le détient peut s'en servir jusqu'à son expiration. Il se protège comme un mot de passe : jamais dans un journal, un ticket ou un dépôt.

Un jeton ne se modifie pas

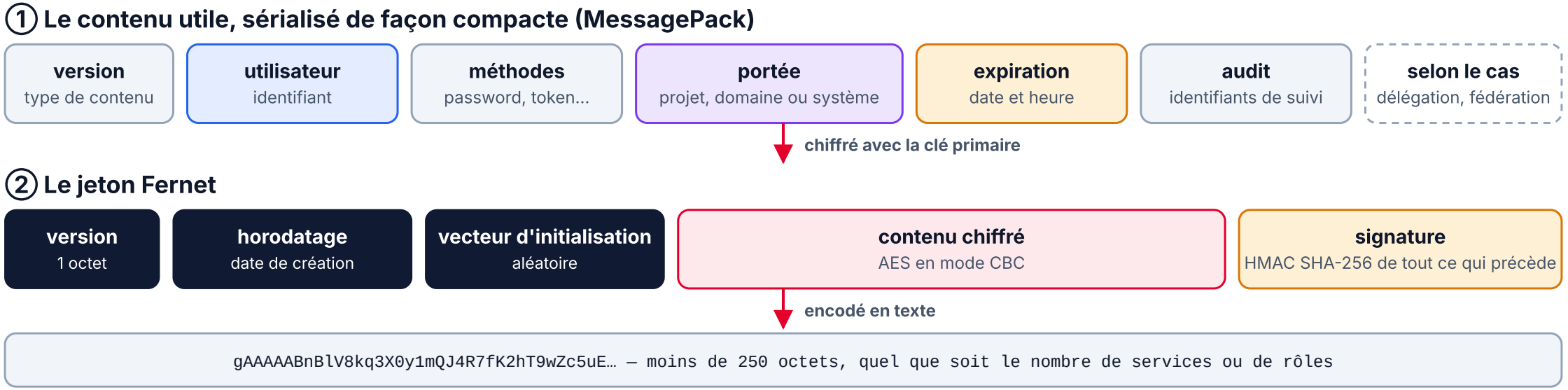
Si les rôles changent, le jeton déjà émis garde l'ancienne liste jusqu'à ce que le service le revalide ; un retrait de rôle provoque sa révocation.

Un jeton est court

Une heure par défaut dans Keystone. Les clients en redemandent un automatiquement à partir des identifiants.

Changer de projet = changer de jeton. C'est ce que fait Horizon quand on sélectionne un autre projet dans son menu : il échange son jeton contre un jeton de l'autre portée.

Anatomie d'un jeton Fernet

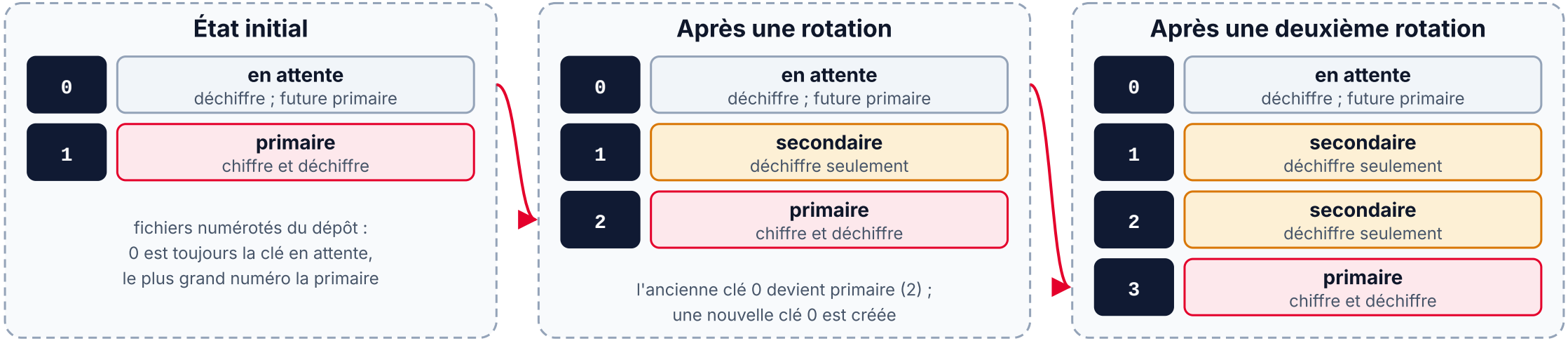


Ce que le jeton ne contient **pas** : les rôles et le catalogue. Ils sont recalculés par Keystone à chaque validation, ce qui explique qu'un jeton Fernet ne puisse être vérifié que par Keystone.

Rien n'est stocké : Keystone ne garde aucune trace des jetons émis. Il lui suffit de la clé pour déchiffrer et vérifier celui qu'on lui présente — d'où l'absence de table de jetons à purger.

Chiffré, donc opaque : impossible de lire ou de modifier un jeton sans les clés. Qui détient les clés, en revanche, peut **fabriquer** des jetons pour n'importe qui.

Les clés Fernet et leur rotation

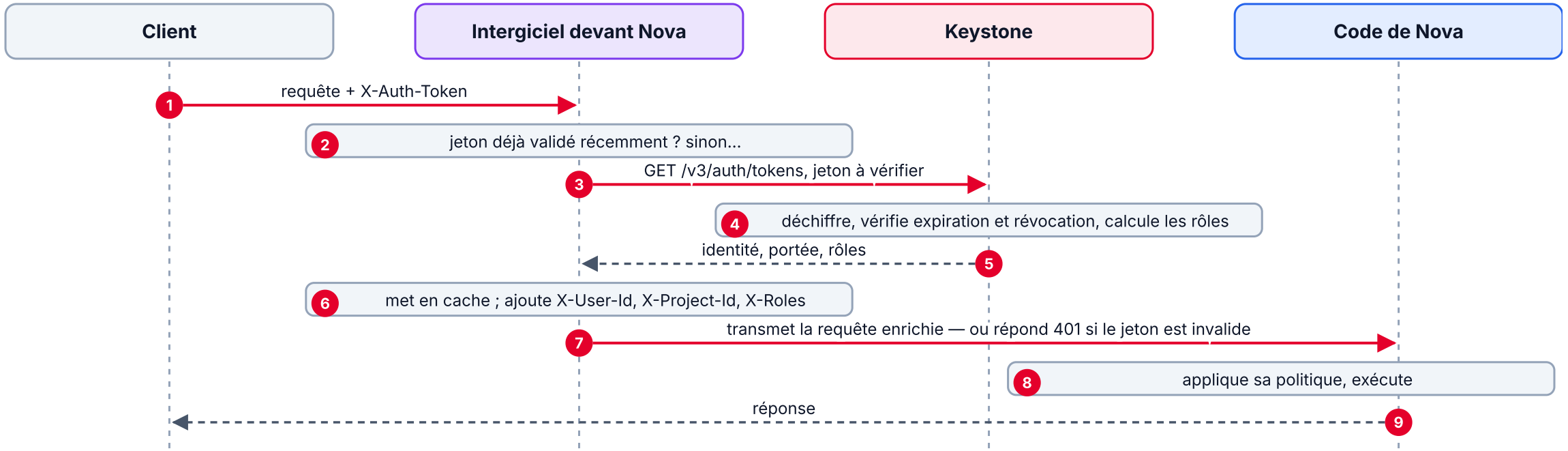


Clé	Numéro	Chiffre ?	Déchiffre ?	Rôle
primaire	le plus élevé	oui	oui	émet tous les nouveaux jetons
secondaire	entre les deux	non	oui	ancienne primaire : valide les jetons encore en circulation
en attente	0	non	oui	future primaire, déjà distribuée partout

- **Pourquoi une clé en attente :** quand elle devient primaire, tous les serveurs la connaissent déjà et savent valider les jetons qu'elle émet.
- **Dépôt de clés :** un répertoire de fichiers, hors base de données, qui doit être **identique** sur tous les serveurs Keystone.

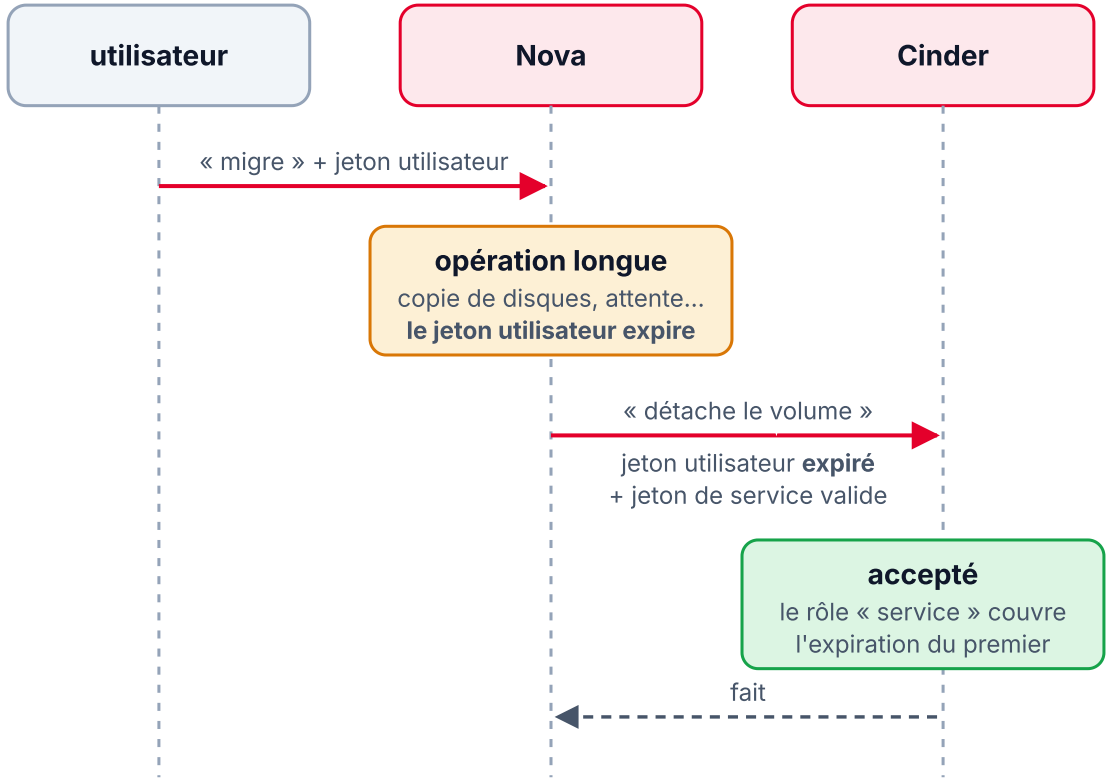
> Rotation et distribution vont ensemble : une rotation non distribuée, et une partie des serveurs refuse des jetons valides.

La validation par les services



- **Chaque service embarque le même composant** — `keystonemiddleware` — placé devant son API. C'est lui qui parle à Keystone ; le code du service ne voit que des en-têtes.
- **Le service a son propre compte** dans Keystone : il s'authentifie pour avoir le droit de valider les jetons des autres.
- **Cache :** le résultat d'une validation est gardé quelques minutes. C'est aussi le délai pendant lequel un jeton révoqué peut encore passer.
- **Section à connaître** dans la configuration de tout service : `[keystone_authtoken]` — adresse de Keystone, compte de service, cache.

Opérations longues et jetons de service



Le problème

- Un jeton dure une heure ; une migration à chaud ou la création d'une grosse image peut durer plus.
- En cours de route, Nova doit encore appeler Cinder, Neutron ou Glance **au nom de l'utilisateur**.

La solution : deux jetons dans la requête

- **Le jeton de l'utilisateur**, dans `X-Auth-Token` : il dit pour qui l'on agit.
- **Le jeton du service appelant**, dans `X-Service-Token` : il prouve que la requête vient bien d'un service du cloud.
- Si le second est valide et porte le rôle `service`, le service appelé accepte le premier **même expiré**, dans la limite de `allow_expired_window`.

> **C'est pour cela que le rôle `service` existe**, à part de la chaîne des autres : il ne donne aucun droit sur les ressources, seulement cette confiance entre services.

Symptôme d'une mauvaise configuration : des opérations longues qui échouent en 401 au bout d'une heure pile.

Durée de vie et révocation

Ce qui invalide un jeton avant son expiration

Événement	Jetons touchés
révocation explicite	**ce jeton, et ceux qui en sont issus**
changement de mot de passe	**tous ceux de l'utilisateur**
utilisateur désactivé ou supprimé	**tous les siens**
rôle retiré	**ceux qui portaient ce rôle sur cette cible**
projet ou domaine désactivé	**tous ceux de cette portée**
suppression des clés Fernet	**tous les jetons du cloud**

```
openstack token revoke gAAAAABn…
```

> **Mécanisme :** Keystone n'ayant pas de liste des jetons, il enregistre des **événements de révocation** — « tout ce qui concerne cet utilisateur avant telle heure » — consultés à chaque validation.

Régler la durée de vie

Durée	Avantage	Inconvénient
courte : 1 h	un jeton volé sert peu de temps	plus d'authentifications ; dépend des jetons de service
longue : 12 à 24 h	confort des sessions Horizon et des scripts	fenêtre d'abus plus grande ; plus de clés à garder

Bonnes pratiques

- **Garder la durée courte** ; pour l'automatisation, voir les identifiants d'application, p. 189.
- **Ne jamais copier un jeton** dans un script : y mettre de quoi en obtenir un.
- **Délai de prise d'effet :** une révocation n'est vue par un service qu'à l'expiration de son cache de validation.
- **Fuite des clés suspectée :** remplacer tout le dépôt de clés — cela déconnecte tout le monde, c'est le but.

Autres façons de s'authentifier

Méthode	Principe	Usage	À savoir
Mot de passe	**nom, domaine, mot de passe**	humains, base locale ou LDAP	à ne jamais mettre dans un script
Jeton	**échanger un jeton contre un autre, d'une autre portée**	changement de projet	le nouveau n'expire pas plus tard que l'ancien
Identifiant d'application	**un secret créé par l'utilisateur, limité à un projet et à un sous-ensemble de ses rôles**	scripts, CI, Terraform, applications	révocable sans toucher au mot de passe ; meurt avec l'utilisateur
Règles d'accès	**restreint un identifiant d'application à certains appels d'API**	moindre privilège	liste de couples méthode + chemin
Délégation	**un utilisateur confie certains de ses rôles sur un projet à un autre**	Heat agissant plus tard au nom de l'utilisateur	avec ou sans usurpation d'identité
Second facteur	**code à usage unique (TOTP) exigé en plus du mot de passe**	comptes sensibles	règles définies par utilisateur
Externe, fédérée	**le serveur web ou un fournisseur d'identité atteste l'identité**	connexion unique d'entreprise	voir p. 190

```
# Créer un identifiant d'application, en lecture seule, 90 jours
openstack application credential create ci-lecture \
  --role reader --expiration 2027-01-07T00:00:00
# → noter « id » et « secret » : le secret n'est montré qu'une fois
```

```
# S'en servir
export OS_AUTH_TYPE=v3applicationcredential
export OS_APPLICATION_CREDENTIAL_ID=<id>
export OS_APPLICATION_CREDENTIAL_SECRET=<secret>
openstack server list
```

 Un annuaire par domaine : module 9

LDAP sur Sunbeam

Prérequis : l'annuaire joignable depuis les nœuds de contrôle ; un compte de recherche en lecture ; le certificat de l'autorité si la liaison est en TLS.

```
# entreprise.yaml — les quatre clés obligatoires
url: ldaps://ldap.exemple.fr:636
user: cn=keystone,ou=services,dc=exemple,dc=fr
password: "••••••••"
suffix: dc=exemple,dc=fr
# puis, selon l'annuaire :
user_tree_dn: ou=personnes,dc=exemple,dc=fr
user_objectclass: inetOrgPerson
user_id_attribute: uid
user_name_attribute: uid
group_tree_dn: ou=groupes,dc=exemple,dc=fr
group_objectclass: groupOfNames
```

Le compte de recherche n'a besoin que de lire utilisateurs et groupes. Son mot de passe est dans ce fichier : le traiter comme un secret et le supprimer après usage.

```
# 1 · Activer la fonctionnalité, une fois
sunbeam enable ldap

# 2 · Raccorder un domaine à l'annuaire
sunbeam ldap add-domain \
  --domain-config-file ./entreprise.yaml \
  --ca-cert-file ./ca-entreprise.pem entreprise

# 3 · Vérifier
sunbeam ldap list-domains
openstack user list --domain entreprise
openstack group list --domain entreprise

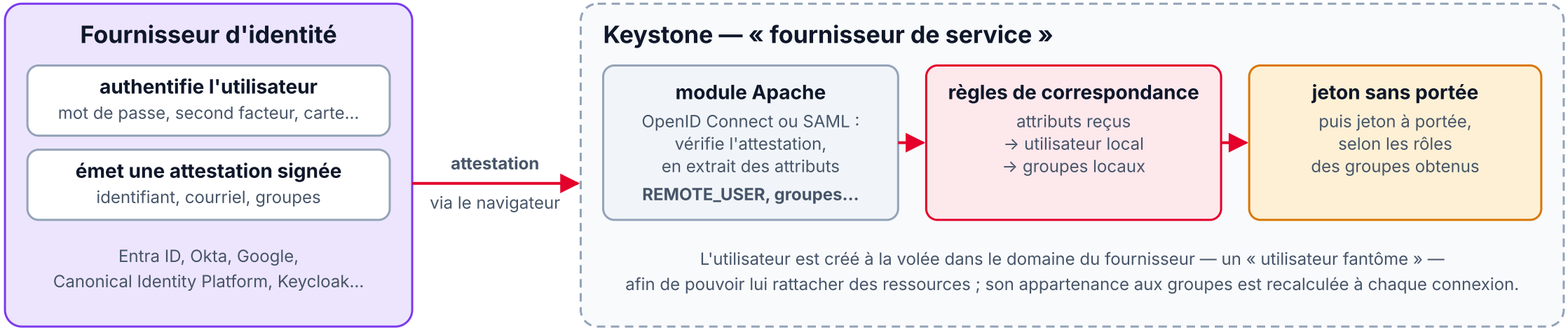
# 4 · Donner des droits — aux groupes de l'annuaire
openstack role add --group dev --group-domain entreprise \
  --project web --project-domain nord member

# Modifier, retirer
sunbeam ldap update-domain \
  --domain-config-file ./entreprise.yaml entreprise
sunbeam ldap remove-domain entreprise
```

Retirer ne supprime que la liaison avec l'annuaire : le domaine, ses projets et ses attributions restent. Pour tout enlever, `openstack domain delete .`

 Approfondi au module 11

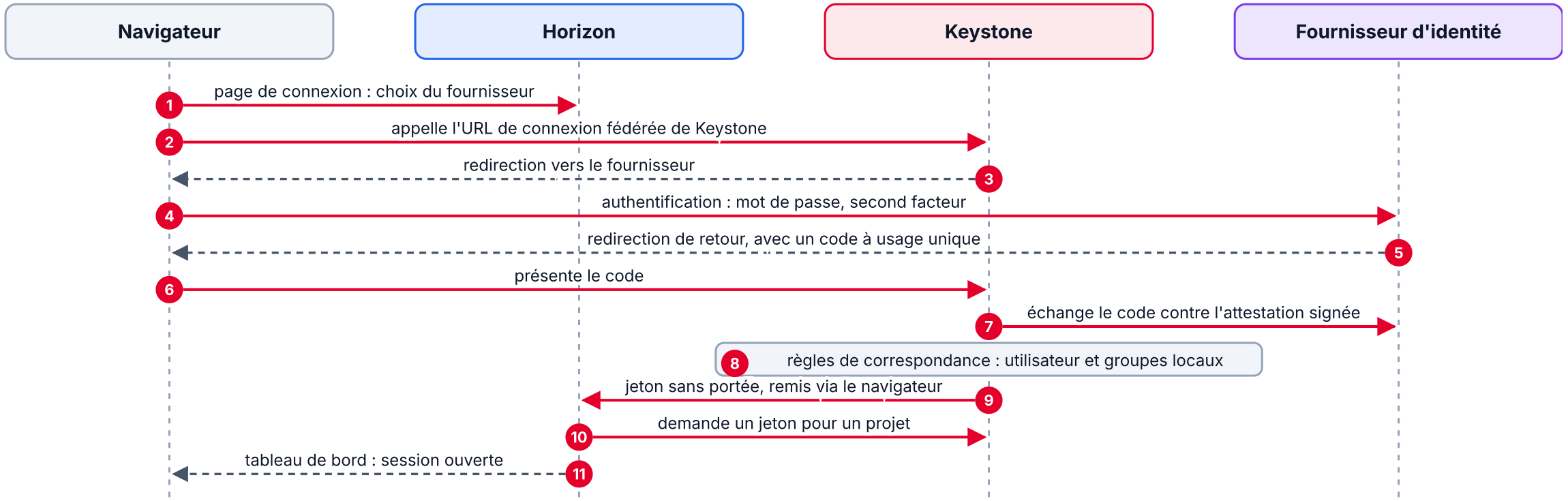
La fédération : principe



Objet Keystone	Ce qu'il représente	Exemple
Fournisseur d'identité	**la source externe, reconnue par son identifiant distant ; rattaché à un domaine**	« entra-groupe », domaine « partenaires »
Protocole	**la façon de dialoguer avec lui**	`openid` ou `saml2`
Correspondance	**les règles qui traduisent les attributs reçus en utilisateur et groupes locaux**	« groupe cloud-dev → groupe developpeurs »

Ce que l'on y gagne : connexion unique, second facteur et cycle de vie des comptes gérés par l'entreprise ; un départ coupe l'accès au cloud sans rien faire dans OpenStack.

Une connexion fédérée, pas à pas



Le mot de passe ne passe jamais par OpenStack : il est saisi chez le fournisseur d'identité, qui ne renvoie qu'une attestation signée, à durée de vie très courte.

Et en ligne de commande ? Le parcours par navigateur ne s'y prête pas : l'usage courant est de se connecter une fois à Horizon, d'y créer un **identifiant d'application**, et de s'en servir pour les scripts.

M8 · LES SOURCES D'IDENTITÉ Approfondi au module 11

La fédération sur Sunbeam

Fournisseur	Type Sunbeam	Protocole documenté
Canonical Identity Platform	`canonical`	OpenID Connect
Google	`google`	OpenID Connect
Okta	`okta`	OpenID Connect
Entra ID	`entra`	OpenID Connect
Autres, SAML2	non documentés	`--help` de votre version

Prérequis

- **TLS** sur les points d'accès publics — par la fonctionnalité TLS de Sunbeam (autorité fournie ou Vault) : la plupart des fournisseurs exigent une URL de retour en `https`.
- **Un nom d'hôte** pour le cloud : certains fournisseurs refusent une adresse IP.
- Côté fournisseur : une application déclarée, avec son identifiant client et son secret.

```
# entra.yaml
client-id: 5d1f…
client-secret: "••••••••"
microsoft-tenant: 8a2c…
```

```
# 1 · URL de retour à déclarer chez le fournisseur
sunbeam identity provider get-oidc-redirect-url \
https://cloud.exemple.fr/openstack-keystone/v3/\
OS-FEDERATION/protocols/openid/redirect_uri

# 2 · Déclarer le fournisseur : type, protocole, nom
sunbeam identity provider add \
  entra openid entra-groupe --config entra.yaml
sunbeam identity provider list

# 3 · Côté Keystone : domaine, fournisseur, groupe, rôle
openstack domain create partenaires
REMOTE_ID=$(sunbeam identity provider list --format=yaml \
  | yq -r '."entra-groupe".issuer_url')
openstack identity provider create \
  --remote-id $REMOTE_ID --domain partenaires entra-groupe
openstack group create --domain partenaires federes
openstack role add --group federes \
  --group-domain partenaires --project web \
  --project-domain nord member

# 4 · Règles de correspondance, puis protocole
openstack mapping create --rules regles.json entra_openid
openstack federation protocol create \
  --identity-provider entra-groupe \
  --mapping entra_openid openid
```

> Sunbeam règle Apache et les secrets ; les objets Keystone restent à créer par l'administrateur. Tant que la correspondance manque, le fournisseur apparaît dans Horizon mais personne ne peut se connecter.

Les règles de correspondance

```
# regles.json — règle simple : tout utilisateur authentifié
[
  {
    "local": [
      { "user":  { "name": "{0}" } },
      { "group": { "name": "federes",
                   "domain": { "name": "partenaires" } } }
    ],
    "remote": [
      { "type": "REMOTE_USER" }
    ]
  }
]
```

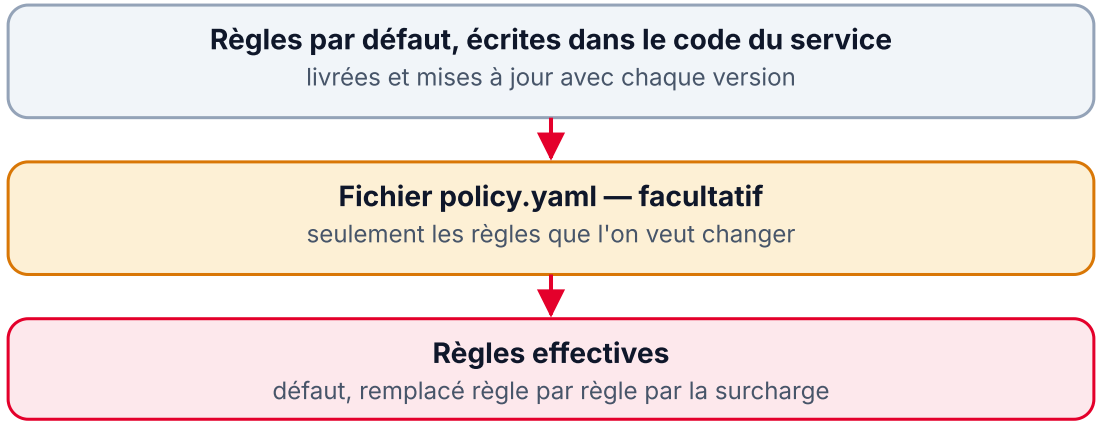
```
# Règle conditionnelle : seulement un groupe du fournisseur
"remote": [
  { "type": "OIDC-preferred_username" },
  { "type": "OIDC-groups", "any_one_of": ["cloud-dev"] }
]
```

Élément	Signification
`remote`	conditions sur les attributs transmis par le fournisseur ; toutes doivent être satisfaites
`local`	ce que l'on en fait : nom de l'utilisateur local, groupes d'affectation
`{0}`, `{1}`...	valeur du premier, du deuxième attribut de `remote`
`any_one_of`	la valeur doit figurer dans la liste
`not_any_of`	la valeur ne doit pas y figurer

- **Les règles sont évaluées dans l'ordre ;** un utilisateur peut cumuler les groupes de plusieurs règles satisfaites.
- **Aucune règle satisfaite :** connexion refusée, même si l'authentification chez le fournisseur a réussi.
- **Une seule correspondance** par couple fournisseur + protocole ; une même correspondance peut servir à plusieurs fournisseurs.
- **Noms des attributs :** ils dépendent du module et du fournisseur ; les journaux de Keystone en mode détaillé montrent ceux qui arrivent réellement.

> **Piège classique :** une règle trop large donne un rôle à **tout** compte du fournisseur. Toujours conditionner à un groupe dédié au cloud.

Règles par défaut et surcharges



- **Pas de fichier, pas de problème :** un service sans `policy.yaml` applique les règles de sa version. C'est l'état recommandé.
- **Format :** YAML ; l'ancien format JSON est déprécié depuis Wallaby.
- **Rechargement :** le fichier est relu à chaud, sans redémarrage.
- **Chaque surcharge est une dette :** à chaque montée de version, il faut vérifier qu'elle a encore un sens face aux nouveaux défauts.

```
# Voir les règles par défaut d'un service
oslopolicy-sample-generator --namespace keystone

# Voir les règles effectives, surcharges comprises
oslopolicy-policy-generator --namespace keystone

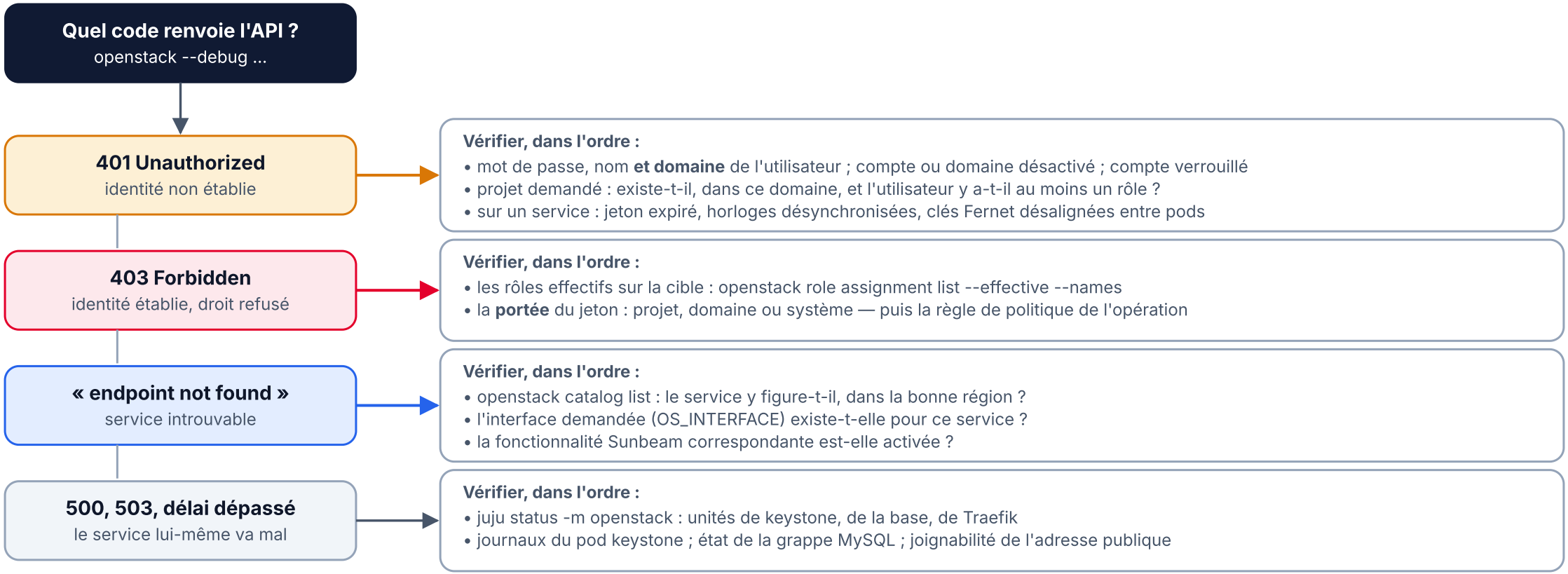
# Repérer les surcharges devenues inutiles
oslopolicy-list-redundant --namespace keystone
```

```
# policy.yaml — exemple de surcharge
# autoriser le rôle « auditeur » à lister les utilisateurs
"identity:list_users": "(rule:admin_required)
   or (role:reader and system_scope:all)
   or role:auditeur"
```

Sur Sunbeam

- Les fichiers de configuration sont dans les pods et **réécrits par les charms** : un `policy.yaml` déposé à la main ne survit pas.
- Une surcharge passe par ce que le charm expose ; à défaut, s'en tenir aux rôles et règles par défaut.
- Les outils `oslopolicy-*` s'exécutent dans le conteneur du service, par `kubectl exec`.

Diagnostiquer un refus



Où regarder sur Sunbeam

```
# 1 · L'application et ses relations
juju status -m openstack keystone --relations

# 2 · Les pods
sudo k8s kubectl -n openstack get pods | grep keystone

# 3 · Les journaux de Keystone
sudo k8s kubectl -n openstack logs keystone-0 \
  -c keystone --tail 100

# 4 · Les journaux du charm
juju debug-log -m openstack --include keystone/0

# 5 · La configuration réellement appliquée
sudo k8s kubectl -n openstack exec keystone-0 \
  -c keystone -- cat /etc/keystone/keystone.conf

# 6 · Les clés Fernet, pod par pod : mêmes fichiers ?
sudo k8s kubectl -n openstack exec keystone-0 \
  -c keystone -- ls /etc/keystone/fernet-keys
```

Constat	Piste
unité `blocked` ou `waiting`	lire le message d'état : relation manquante, base indisponible
401 intermittents	un pod n'a pas les mêmes clés que les autres ; horloge d'un nœud décalée
authentification lente	base de données ; annuaire LDAP lent à répondre
échec pour un seul domaine	configuration LDAP de ce domaine ; certificat de l'annuaire
connexion fédérée refusée	règles de correspondance ; URL de retour déclarée chez le fournisseur
tout le cloud en 401	Keystone injoignable par l'adresse interne ; Traefik interne

- **Journal détaillé :** l'option de débogage du charm, à activer le temps du diagnostic puis à retirer — les journaux détaillés sont volumineux.
- **Test de bout en bout le plus simple :** `openstack token issue`, d'abord depuis un nœud du cloud, puis depuis un poste extérieur.

> Noms des pods, des conteneurs et chemins à confirmer sur votre déploiement : ils dépendent de la révision des charms.

09

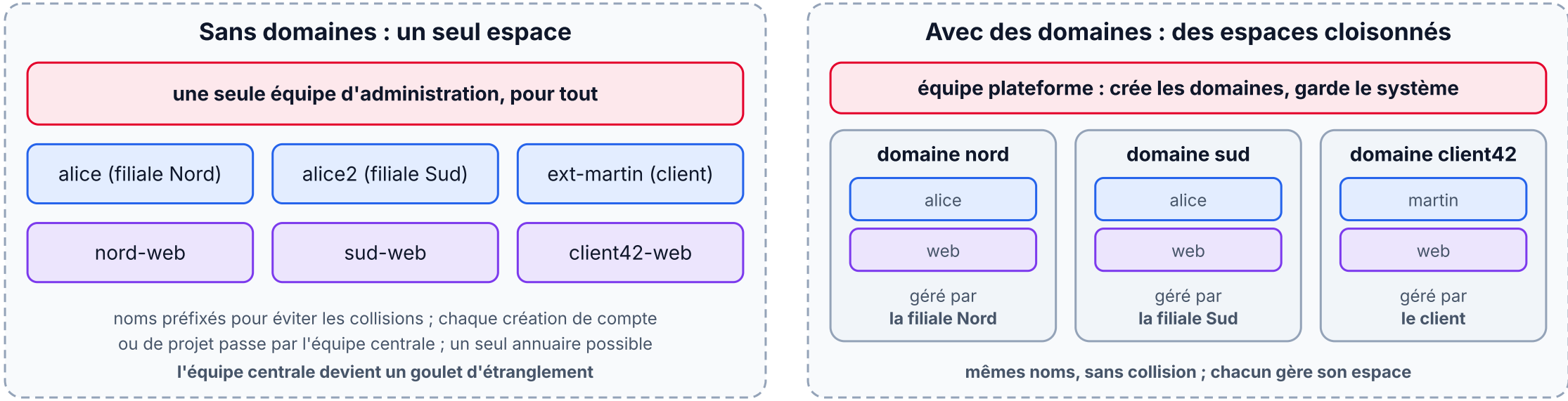
Les domaines dans OpenStack

Espaces d'identité indépendants, administration déléguée par domaine et administration globale — sur Canonical OpenStack (Sunbeam)

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Expliquer en quoi un domaine est un **espace d'identité indépendant**.
- Créer, raccorder, désactiver et supprimer un domaine sur Sunbeam.
- Mettre en place une **délégation** et en vérifier les limites.
- Dire ce qui **appartient à un domaine** et ce qui reste **global**.
- Distinguer **administrateur global**, **administrateur de domaine** et **gestionnaire de domaine**.
- Choisir un **découpage** adapté à une organisation.

Le problème d'un espace unique



Collision de noms

Dans un espace unique, deux équipes ne peuvent pas avoir chacune un projet « web » ni un utilisateur « alice ».

Administration tout ou rien

Pour créer un utilisateur, il faut être administrateur du cloud. Impossible de confier « seulement mon périmètre ».

Une seule source d'identité

Tous les comptes viennent du même endroit : pas d'annuaire par filiale, pas de fournisseur d'identité par client.

Les trois fonctions d'un domaine

① Espace de noms

les noms d'utilisateur, de groupe et de projet ne sont uniques qu'**à l'intérieur** du domaine

`alice@nord ≠ alice@sud`

Chaque organisation choisit ses noms sans se soucier des voisines.

répond à : « comment éviter les collisions ? »

② Frontière d'administration

un rôle détenu **sur le domaine** permet de gérer ses utilisateurs, groupes, projets

et rien au-dehors

La plateforme délègue la gestion courante sans céder le contrôle du cloud.

répond à : « qui administre quoi ? »

③ Point de raccordement

chaque domaine peut avoir sa propre source d'identité

base locale · annuaire LDAP · fédération

Les comptes de chaque organisation restent dans son propre référentiel.

répond à : « d'où viennent les utilisateurs ? »

Définition à retenir : un domaine est un conteneur d'utilisateurs, de groupes et de projets, qui délimite à la fois leurs noms, leur administration et leur origine.

Un peu d'histoire : les domaines sont arrivés avec la version 3 de l'API d'identité. Avant, il n'existait que des « tenants », tous au même niveau ; le domaine `Default` a été créé pour les accueillir.

Ce qu'un domaine n'isole pas

Point le plus mal compris : le domaine est une notion de **Keystone**. Nova, Neutron, Cinder et Glance ne raisonnent qu'en **projets** ; ils ignorent à quel domaine un projet appartient.

Sujet	Réalité
Isolation réseau	**par projet, pas par domaine : deux projets d'un même domaine sont aussi étanches que deux projets de domaines différents**
Quotas	**historiquement par projet ; pas de plafond « pour tout le domaine » dans les quotas classiques**
Images publiques, gabarits	**visibles de tous les domaines**
Réseaux partagés, réseau externe	**communs à tout le cloud**
Hyperviseurs, stockage	**mutualisés : les instances de deux domaines cohabitent sur les mêmes hôtes**
Catalogue, régions	**identiques pour tous**



- **Pour isoler du matériel** par organisation : agrégats d'hôtes, gabarits privés, zones de disponibilité — pas les domaines.
- **Pour plafonner une organisation entière :** les limites unifiées de Keystone, ou une somme de quotas par projet gérée par la plateforme — module 10.
- **Pour restreindre le partage :** images et gabarits en accès par projet ; réseaux partagés par règles d'accès ciblées.

Ce qui appartient à un domaine, ce qui est global

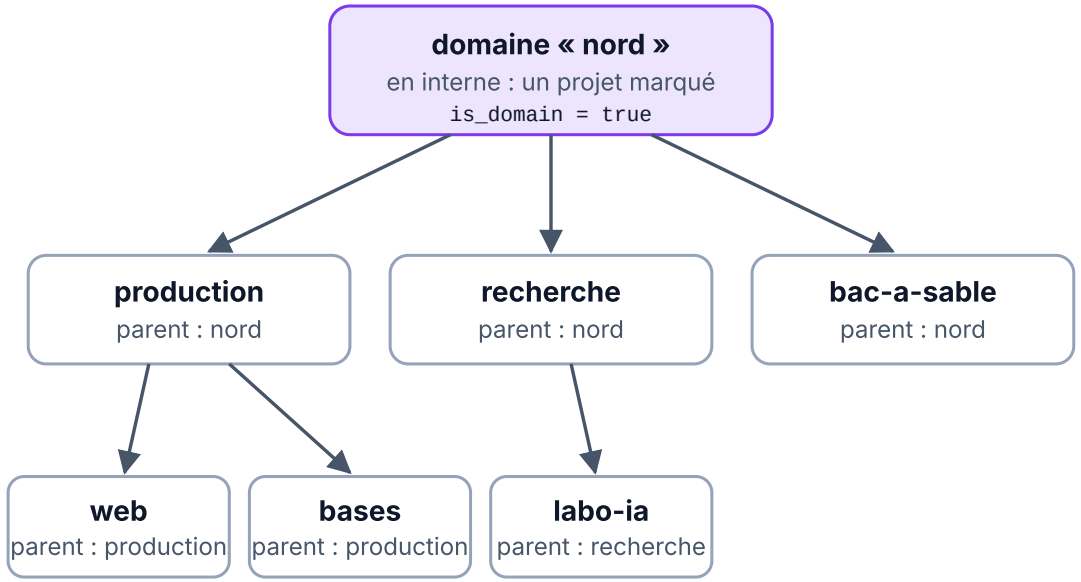


Appartenance exclusive : un utilisateur, un groupe ou un projet appartient à **un seul** domaine, pour toujours. On ne « déplace » pas un utilisateur d'un domaine à l'autre : on en crée un nouveau.

Les affectations de rôles n'appartiennent à aucun domaine : elles relient un acteur et une cible, qui peuvent être de domaines différents. C'est ce qui permet le travail entre organisations.

 Hiérarchie détaillée : module 10

Le domaine, racine d'un arbre de projets



Rôle hérité, posé sur le domaine

« exploitants : reader, hérité, sur le domaine nord »

→ lecture sur tous les projets de l'arbre, présents et futurs

- **Tout projet a un domaine**, et un seul ; un sous-projet est dans le domaine de son parent.
- **Le domaine est le sommet de l'arbre :** un projet de premier niveau a pour parent le domaine lui-même.
- **Deux façons de poser un rôle sur un domaine :**

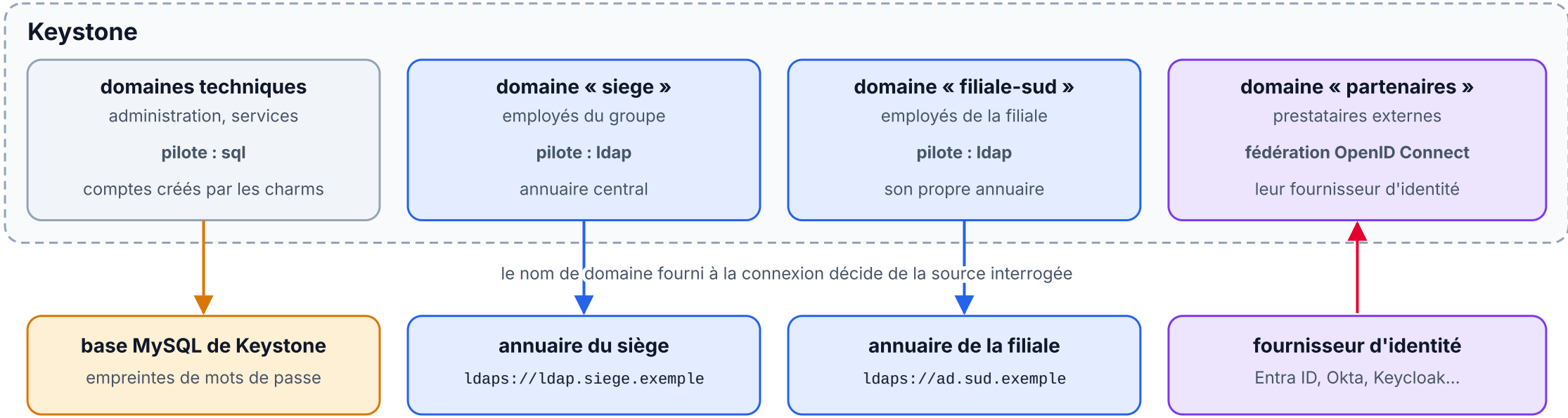
Affectation	Donne des droits sur
sur le domaine, directe	le domaine lui-même : gérer ses utilisateurs, groupes, projets
sur le domaine, héritée	tous les projets du domaine — mais **pas** le domaine lui-même

```
# Gérer le domaine
openstack role add --user bob --user-domain nord \
  --domain nord manager

# Lire tous les projets du domaine
openstack role add --group exploitants \
  --group-domain nord --domain nord --inherited reader
```

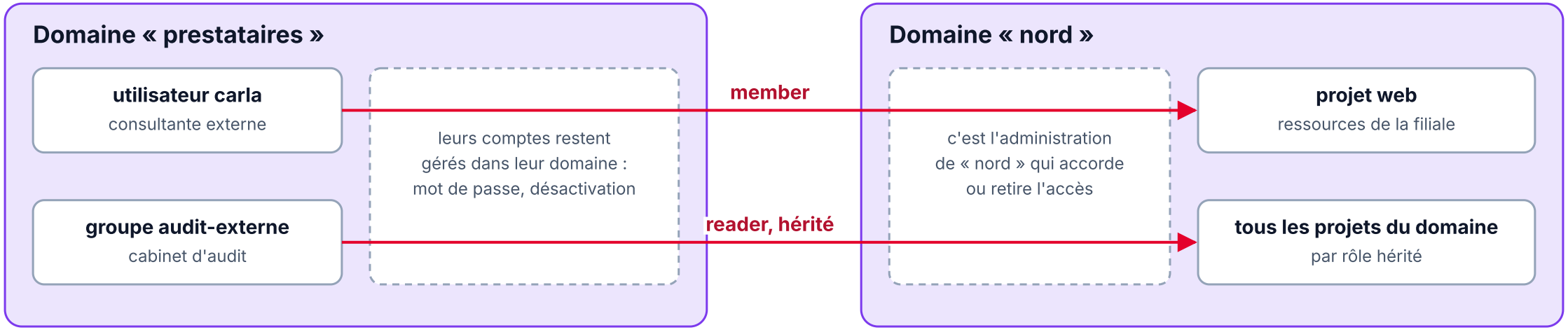
> Les deux se cumulent souvent : un responsable de domaine reçoit `manager` en direct, pour administrer, et `member` hérité, pour agir dans les projets.

Une source d'identité par domaine



- **L'aiguillage se fait sur le domaine :** « alice » du domaine « siege » est cherchée dans l'annuaire du siège, et nulle part ailleurs.
- **Les domaines ne se voient pas :** la panne de l'annuaire d'une filiale n'empêche pas les autres domaines de se connecter.
- **Une seule source par domaine.** Deux annuaires, deux domaines.
- **Projets et rôles restent dans Keystone**, quelle que soit la source : l'annuaire fournit des personnes et des groupes, jamais des droits.

Affectations entre domaines



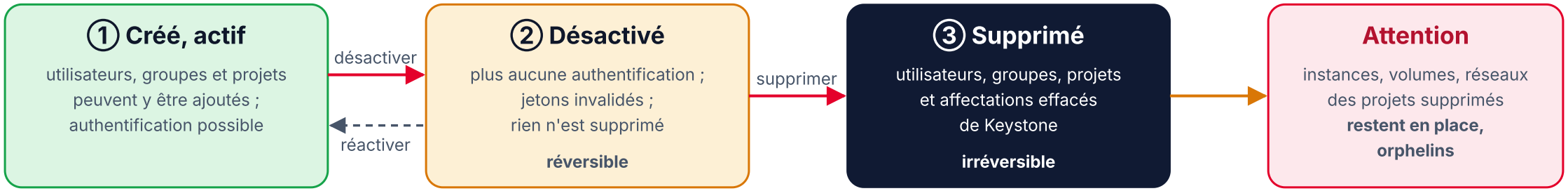

```
openstack role add \
  --user carla --user-domain prestataires \
  --project web --project-domain nord member

openstack role add \
  --group audit-externe --group-domain prestataires \
  --domain nord --inherited reader

# Carla s'authentifie avec deux domaines différents
export OS_USER_DOMAIN_NAME=prestataires
export OS_PROJECT_DOMAIN_NAME=nord
```

- **L'indépendance porte sur l'identité, pas sur l'accès :** rien n'interdit de donner à un acteur d'un domaine un rôle sur une cible d'un autre.
- **Usage type :** un domaine « prestataires » ou « exploitation », dont les membres interviennent chez plusieurs organisations.
- **Qui peut créer une telle affectation ?** Il faut des droits sur la **cible** ; un gestionnaire de domaine peut, selon la politique en vigueur, être limité aux acteurs de son propre domaine — à tester avant de s'y fier.
- **Revue :** ces affectations sont celles qui échappent le plus facilement aux contrôles ; les lister régulièrement.

Cycle de vie d'un domaine



La suppression n'est acceptée que sur un domaine désactivé : c'est un verrou contre l'erreur de manipulation.

```
# Créer
openstack domain create --description "Filiale Nord" nord

# Suspendre un locataire : effet immédiat
openstack domain set --disable nord
openstack domain set --enable nord

# Supprimer : nettoyer les ressources d'abord
openstack project list --domain nord -f value -c ID
openstack project purge --project <id>   # par projet
openstack domain set --disable nord
openstack domain delete nord
```

- **Keystone ne prévient pas les autres services :** il efface ses propres objets. Les ressources des projets disparus continuent de consommer capacité et adresses.
- **Ordre de retrait d'un locataire :** désactiver le domaine, inventorier ses projets, purger leurs ressources, puis seulement supprimer.
- `project purge` ne couvre pas tous les services : vérifier réseaux, répartiteurs, images et objets à la main.

> Le domaine par défaut et les domaines techniques ne se suppriment pas : le cloud en dépend.

M9 · ADMINISTRATION GLOBALE

Ce que la plateforme garde pour elle

Domaine d'action	Opérations réservées	Pourquoi ce n'est pas délégable
Les domaines	**créer, désactiver, supprimer un domaine ; y nommer les premiers responsables**	c'est l'acte qui fonde la délégation
Les sources d'identité	**raccorder un annuaire, déclarer un fournisseur d'identité, écrire les règles de correspondance**	touche à la configuration de Keystone et à la confiance accordée à un tiers
Les rôles	**créer des rôles, définir les rôles impliqués, attribuer** `admin`	un rôle a le même sens pour tous les domaines
Le catalogue	**services, points d'accès, régions**	commun à tout le cloud ; géré par les charms
Les politiques	**règles d'accès des services**	une règle s'applique à tous
Les plafonds	**quotas des projets, limites par défaut, limites d'un domaine**	arbitrage de la capacité entre organisations
Les ressources partagées	**gabarits, images publiques, réseaux externes et fournisseurs, types de volumes**	elles sont offertes à tous les domaines
L'infrastructure	**hyperviseurs, agrégats, zones, migration, évacuation, stockage**	mutualisée ; relève de l'exploitation
Le cloud lui-même	**fonctionnalités Sunbeam, mises à jour, TLS, sauvegardes**	hors d'OpenStack : Juju, Kubernetes, nœuds

Critère simple : si l'opération a un effet sur plus d'un domaine, ou sur du matériel, elle est globale.

Les plafonds sont le vrai levier de la plateforme : un gestionnaire de domaine crée autant de projets qu'il veut, mais ne peut pas s'accorder plus de capacité.

Portée système ou projet d'administration

	Jeton de portée système	Jeton du projet d'administration
Principe	**rôle `admin` sur « le système »**	rôle `admin` sur un projet particulier
Origine	**modèle récent, à portées explicites**	modèle historique
Contient un projet	**non**	oui
Dans Keystone	**toutes les opérations**	toutes les opérations également
Dans les autres services	**selon l'avancement de chaque service**	le plus largement accepté
Créer une instance	**impossible : pas de projet**	possible, dans ce projet

En pratique sur Sunbeam : le fichier produit par `sunbeam openrc` vise le projet d'administration. C'est ce jeton qui sert au quotidien ; la portée système s'obtient à la demande.

```
# Jeton du projet d'administration — ce que donne admin-openrc
source admin-openrc
ADM=$OS_USER_DOMAIN_NAME   # domaine d'administration
openstack token issue -c project_id -c user_id

# Jeton de portée système : retirer le projet, demander « all »
unset OS_PROJECT_NAME OS_PROJECT_DOMAIN_NAME
export OS_SYSTEM_SCOPE=all
openstack token issue -c system
openstack domain list

# Prérequis : détenir un rôle sur le système
openstack role assignment list --system all --names
openstack role add --user admin \
  --user-domain $ADM --system all admin
```

- **Les deux portées s'excluent :** un jeton vise le système **ou** un projet, jamais les deux.
- **Si une commande d'administration renvoie 403** avec l'un, essayer l'autre : c'est le signe d'une règle qui déclare ses portées.

Le piège du rôle admin

Ce que l'on croit faire

« je donne à bob le rôle admin sur le domaine nord, pour qu'il administre sa filiale »

→

Ce que dit la règle

`"admin_required": "role:admin or is_admin:1"`

le rôle admin, sans condition de portée

→

Ce que l'on obtient

là où la règle se limite à « admin_required », bob passe, quel que soit le domaine visé **au-delà de sa filiale**

La bonne délégation : le rôle manager sur le domaine

les règles qui l'acceptent vérifient toutes que la cible appartient au domaine du jeton — et il ne peut pas attribuer admin

- **Héritage historique :** pendant des années, « admin quelque part » a signifié « admin partout ». Les règles récentes corrigent cela opération par opération, mais pas encore partout ni dans tous les services.
- **Conséquence :** considérer toute attribution du rôle `admin` — sur un projet, sur un domaine — comme une attribution potentiellement **globale**.
- **Le rôle `manager`** a été introduit précisément pour offrir une délégation sans ce risque.

```
# Inventaire : qui détient admin, et sur quoi ?
openstack role assignment list --names --role admin

# Tester ce qu'un « admin de domaine » peut réellement faire
# (avec un jeton de bob, portée domaine nord)
openstack user list --domain sud      # doit échouer
openstack domain create essai         # doit échouer
```

Ne pas supposer, tester : le comportement dépend de la version de Keystone et des options de politique. Les deux commandes ci-dessus tranchent en une minute sur votre cloud.

Les profils de domaine

Profil	Affectation	Ce qu'il peut faire	À qui le donner
Gestionnaire de domaine	`manager` **sur le domaine**	gérer utilisateurs, groupes, projets et affectations du domaine ; attribuer `manager`, `member`, `reader` — pas `admin` ; ne modifie pas le domaine lui-même	**le profil de délégation par défaut**
Administrateur de domaine	`admin` **sur le domaine**	tout ce que fait le gestionnaire, plus la plupart des réglages du domaine ; risque de débordement (p. 209)	à éviter ; réserver à la plateforme
Lecteur de domaine	`reader` **ou** `member` **sur le domaine**	consulter utilisateurs, groupes, projets et affectations du domaine	audit interne, responsable sécurité de l'organisation
Intervenant sur tous les projets	`member` **ou** `reader` **hérité sur le domaine**	agir, ou lire, dans chaque projet du domaine ; aucun droit sur les identités	exploitants de l'organisation, supervision

```
# Gestionnaire
openstack role add --user bob --user-domain nord \
  --domain nord manager

# Lecteur de domaine
openstack role add --group audit --group-domain nord \
  --domain nord reader

# Exploitants : member sur tous les projets, présents et futurs
openstack role add --group exploitants --group-domain nord \
  --domain nord --inherited member
```

- `manager` **implique** `member`**, qui implique** `reader` **:** le gestionnaire est donc aussi lecteur de son domaine.
- **Gérer n'est pas utiliser :** le rôle `manager` sur le domaine ne donne **aucun** accès aux instances des projets. Pour cela, il lui faut aussi un rôle dans les projets — direct ou hérité.
- **Le rôle** `manager` **existe depuis la version 2023.2** ; son usage pour la gestion de domaine a été complété dans les versions suivantes.

Tableau des droits : module 8

Le gestionnaire de domaine en détail

Ce qu'il peut faire — dans son domaine uniquement

Objet	Opérations
Projets	créer, modifier, désactiver, supprimer ; créer des sous-projets ; poser des étiquettes
Utilisateurs	créer, modifier, désactiver, supprimer ; réinitialiser un mot de passe
Groupes	créer, supprimer ; ajouter et retirer des membres
Affectations	attribuer et retirer `manager`, `member`, `reader` sur le domaine et ses projets ; directes ou héritées
Consultation	lister tout ce qui précède ; voir les affectations effectives

Il peut nommer d'autres gestionnaires : la délégation se transmet, sans que la plateforme ait à intervenir.

Ce qu'il ne peut pas faire

Opération	Raison
attribuer ou retirer `admin`	**empêche l'escalade de privilèges**
modifier, désactiver, supprimer son domaine	**le domaine appartient à la plateforme**
voir un autre domaine	**cloison entre organisations**
créer un rôle	**les rôles sont globaux**
modifier les quotas de ses projets	**arbitrage de capacité : global**
raccorder un annuaire	**configuration de Keystone**
créer un gabarit, une image publique	**ressources partagées**
agir sur les instances de ses projets	**il faut un rôle dans le projet**

Liste établie d'après la description des profils par défaut de Keystone. Le détail, opération par opération, se lit dans les règles effectives et se vérifie par un test.

Ce qu'il voit, ce qu'il ne voit pas



Une cloison, pas un voile : ce n'est pas seulement que bob ne peut pas modifier le domaine « sud » — il ne peut pas en lister les utilisateurs ni même en connaître les projets.

Fuites possibles par les ressources partagées : le nom d'une image publique, d'un gabarit ou d'un réseau partagé est visible de tous les domaines. Ne pas y mettre le nom d'un client.

Les garde-fous de la délégation

Risque	Garde-fou intégré	Ce que la plateforme doit ajouter
Escalade : le gestionnaire se nomme administrateur	**la liste des rôles qu'un gestionnaire peut attribuer exclut `admin`**	ne jamais donner `admin` sur un domaine ou un projet à un délégué
Débordement : agir sur un autre domaine	**chaque règle compare le domaine du jeton à celui de la cible**	tester la cloison après chaque montée de version
Accaparement de capacité	**aucun : créer des projets est libre**	quotas par projet, plafond par domaine ; quota par défaut bas
Prolifération de comptes	**aucun dans la base locale**	source d'identité externe ; revue périodique
Comptes dormants	**désactivation après inactivité, si configurée**	politique de sécurité des mots de passe
Perte de traçabilité	**événements d'audit de Keystone**	les collecter ; comptes nominatifs pour les gestionnaires
Gestionnaire unique indisponible	**il peut nommer des adjoints**	exiger au moins deux gestionnaires par domaine
Départ du gestionnaire	**la plateforme garde tous les droits**	procédure de reprise : retirer son rôle, nommer le suivant

Le garde-fou qui manque le plus souvent : la capacité. Un gestionnaire ne peut pas augmenter un quota, mais il peut créer cent projets qui auront chacun le quota par défaut.

Rôles attribuables par un gestionnaire : `manager`, `member` et `reader` par défaut ; la liste est une règle de politique de Keystone, donc ajustable par la plateforme.

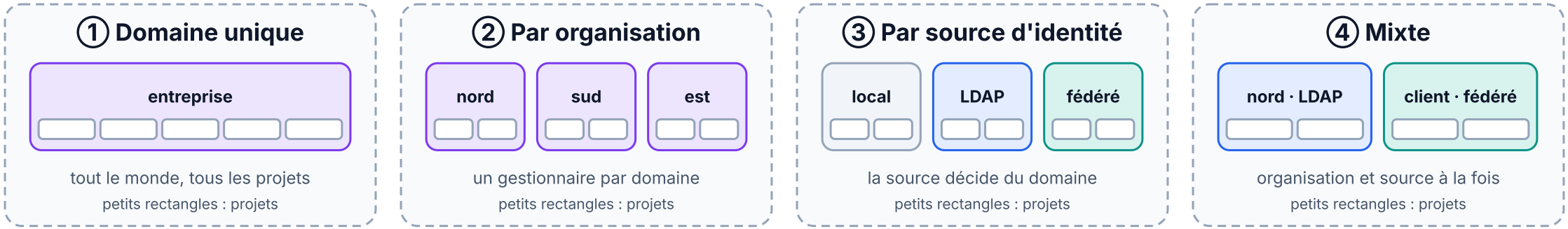
Vérifier la délégation : dix tests

#	Commande, avec le jeton de portée domaine de bob	Attendu	Ce que le test prouve
1	`openstack project create --domain nord essai`	**réussit**	la délégation fonctionne
2	`openstack user create --domain nord essai-u`	**réussit**	gestion des comptes du domaine
3	`openstack role add … --project essai member`	**réussit**	attribution des rôles courants
4	`openstack role add … --project essai admin`	**403**	pas d'escalade vers `admin`
5	`openstack user list --domain sud`	**403**	cloison en lecture
6	`openstack project create --domain sud intrus`	**403**	cloison en écriture
7	`openstack domain list`	son domaine seul, ou 403	pas de vue sur les autres domaines
8	`openstack domain set --disable nord`	**403**	le domaine reste à la plateforme
9	`openstack role create mon-role`	**403**	les rôles restent globaux
10	`openstack quota set --instances 999 <projet>`	**403**	la capacité reste à la plateforme

À rejouer après chaque montée de version et après toute modification des politiques : c'est la seule preuve que la cloison tient. Les dix commandes tiennent dans un script.

Un test qui « réussit » alors qu'il devrait échouer est une faille de délégation : chercher une attribution du rôle `admin`, ou une règle surchargée trop large.

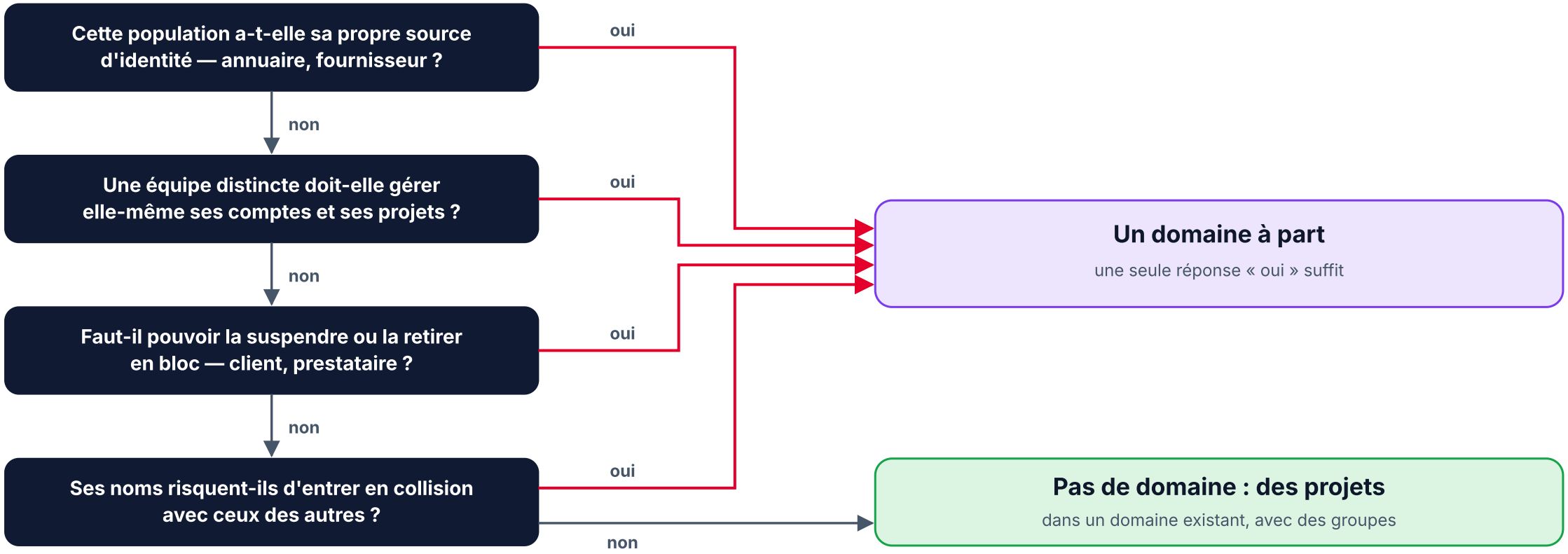
Quatre modèles de découpage



Modèle	Convient à	Avantage	Limite
① **Domaine unique**	**une équipe, un petit cloud**	rien à expliquer	pas de délégation ; noms à préfixer
② **Un domaine par organisation**	**filiales, clients, directions**	délégation complète, noms libres	un domaine à créer par arrivée
③ **Un domaine par source d'identité**	**un annuaire commun, plus des comptes externes**	suit l'existant de l'entreprise	la délégation se fait par projets, pas par domaine
④ **Mixte**	**grands groupes, hébergeurs avec clients fédérés**	chaque organisation a son domaine **et** sa source	plus de raccordements à maintenir

Sur Sunbeam, on part toujours de ③ au minimum : les domaines techniques existent déjà. La question est de savoir combien de domaines « métier » ajouter.

Combien de domaines ? Arbre de décision



Règle de sobriété : un domaine se justifie par une frontière d'**administration** ou de **source d'identité**. Pour séparer des ressources, des budgets ou des environnements, les projets suffisent.

Erreur fréquente : un domaine par environnement — production, recette, développement. Ce sont les mêmes personnes et le même annuaire : trois projets, ou trois arbres de projets, dans un seul domaine.

10

Projets, hiérarchie et quotas

Organiser un système d'information cloisonné sur Canonical OpenStack (Sunbeam) : le projet comme unité de propriété, l'arbre de projets, les plafonds de consommation

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Dire ce qu'un **projet** isole, service par service, et ce qu'il laisse partagé.
- Lire, fixer et dimensionner des **quotas** pour le calcul, le stockage et le réseau.
- Comparer plusieurs **modèles d'organisation** et en choisir un.
- Construire un **arbre de projets** et y poser des **rôles hérités**.
- Expliquer les **limites unifiées** et leurs deux modèles d'application.
- Compléter le cloisonnement par le réseau et le matériel.

Ce qu'est un projet

Projet « web » — identifiant 9a7e… — domaine « nord »

① Propriété
chaque instance, volume, réseau, image porte l'identifiant du projet
« à qui est-ce ? »

② Isolation
par défaut, un projet ne voit ni n'atteint les ressources d'un autre
« qui peut le voir ? »

③ Droits
les rôles s'attribuent sur le projet : c'est la portée la plus courante d'un jeton
« qui peut agir ? »

④ Plafonds
les quotas se fixent par projet et se mesurent sur ce qu'il possède
« jusqu'à combien ? »

instances
clés, groupes de serveurs

volumes
instantanés, sauvegardes

réseaux
sous-réseaux, routeurs, adresses flottantes

sécurité
groupes de sécurité et leurs règles

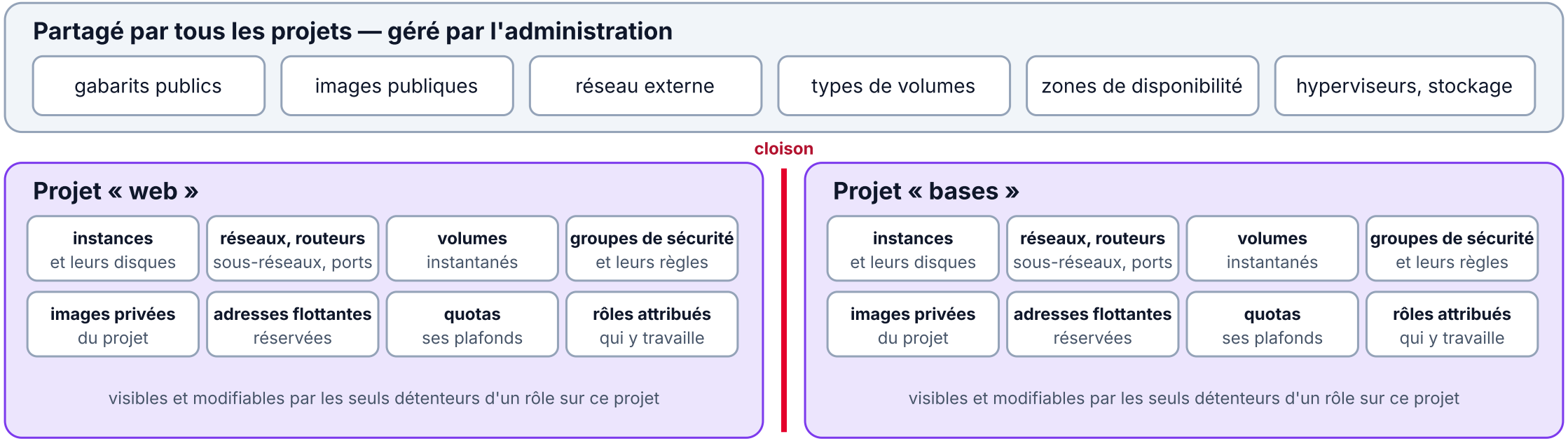
images
privées et partagées

et, selon les options
répartiteurs, piles, secrets, partages de fichiers, clusters

Vocabulaire : « projet » et « tenant » désignent la même chose ; le second est l'ancien nom, encore présent dans certaines commandes, options et messages d'erreur.

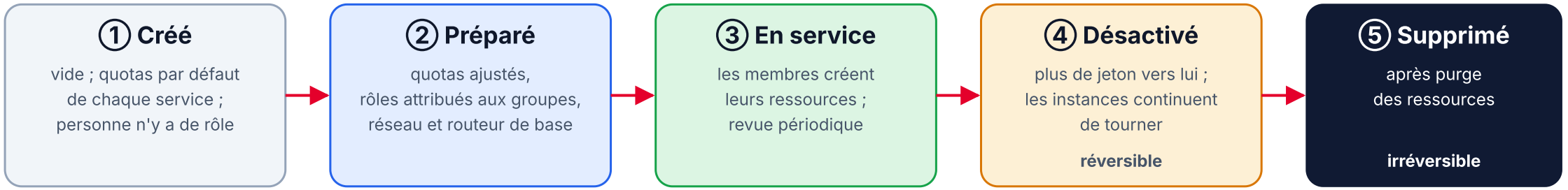
Un projet n'est pas un groupe de personnes. C'est un conteneur de ressources ; les personnes y accèdent par des rôles, et une même personne peut travailler dans plusieurs projets.

Ce qui est étanche, ce qui est partagé



- **Réseau :** deux réseaux de deux projets ne communiquent pas, sauf par un routeur et une adresse externe, comme deux réseaux d'entreprises distinctes.
- **API :** lister les instances ne renvoie que celles du projet du jeton ; demander celle d'un autre renvoie « introuvable ».
- **Matériel :** la cloison est logique. Les instances de deux projets partagent hyperviseurs et disques ; l'étanchéité repose sur la virtualisation.
- **Entre membres d'un même projet :** aucune cloison. Tout membre voit et peut modifier toutes les ressources du projet.

Cycle de vie d'un projet



Supprimer un projet dans Keystone ne supprime **pas** ses ressources dans les autres services : elles deviennent orphelines.

```
# Prérequis : jeton d'administrateur ou de gestionnaire de domaine
openstack project create --domain nord \
  --description "Site vitrine" vitrine
openstack project show vitrine --domain nord

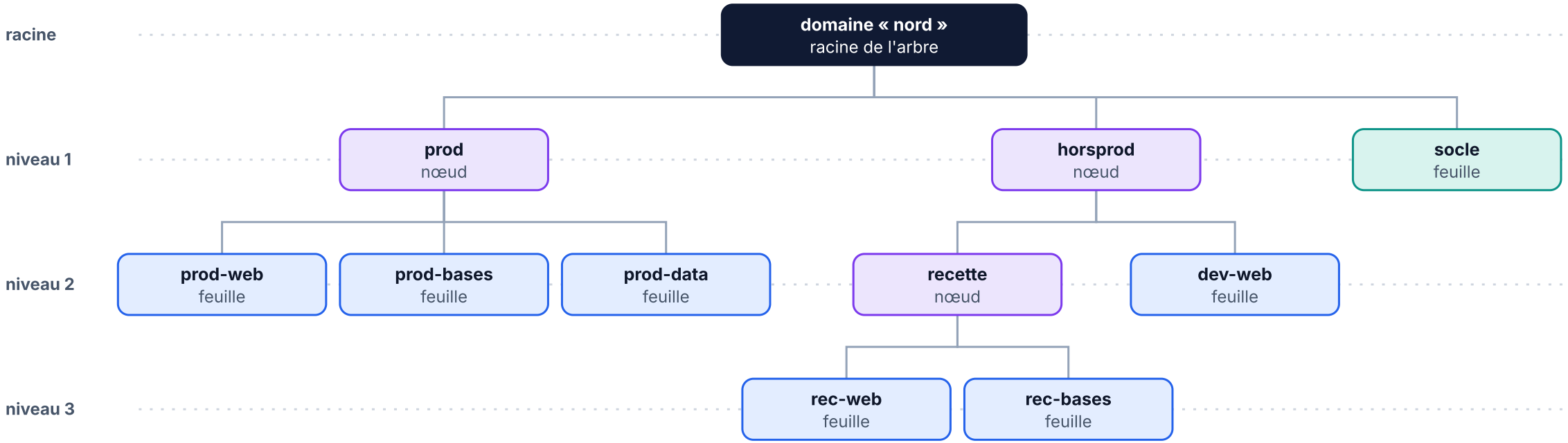
# Suspendre, reprendre
openstack project set --disable vitrine --domain nord
openstack project set --enable  vitrine --domain nord

# Retirer : purger d'abord, supprimer ensuite
openstack project purge --dry-run --project <id>
openstack project purge --project <id>
openstack project delete <id>
```

- **Désactiver n'arrête rien :** c'est l'accès par l'API qui est coupé. Pour libérer de la capacité, il faut arrêter ou supprimer les instances.
- `project purge` supprime instances, volumes, instantanés et images du projet ; l'option `--dry-run` liste sans agir. Réseaux, routeurs, adresses flottantes, répartiteurs et secrets sont à vérifier à part.
- **Ordre de nettoyage réseau :** ports et adresses flottantes, interfaces de routeur, routeurs, sous-réseaux, réseaux.

> Un projet supprimé sans purge laisse des instances qui tournent et consomment, mais que plus personne ne peut atteindre sans droits d'administration.

L'arbre de projets



Lecture : le domaine est la racine ; chaque projet a exactement un parent ; les projets sans enfant — les feuilles — sont en général ceux qui portent les ressources.

Rien n'oblige à mettre des ressources dans un nœud intermédiaire : « production » peut n'être qu'un dossier, qui sert à ranger et à porter des rôles hérités.

Les règles de l'arbre

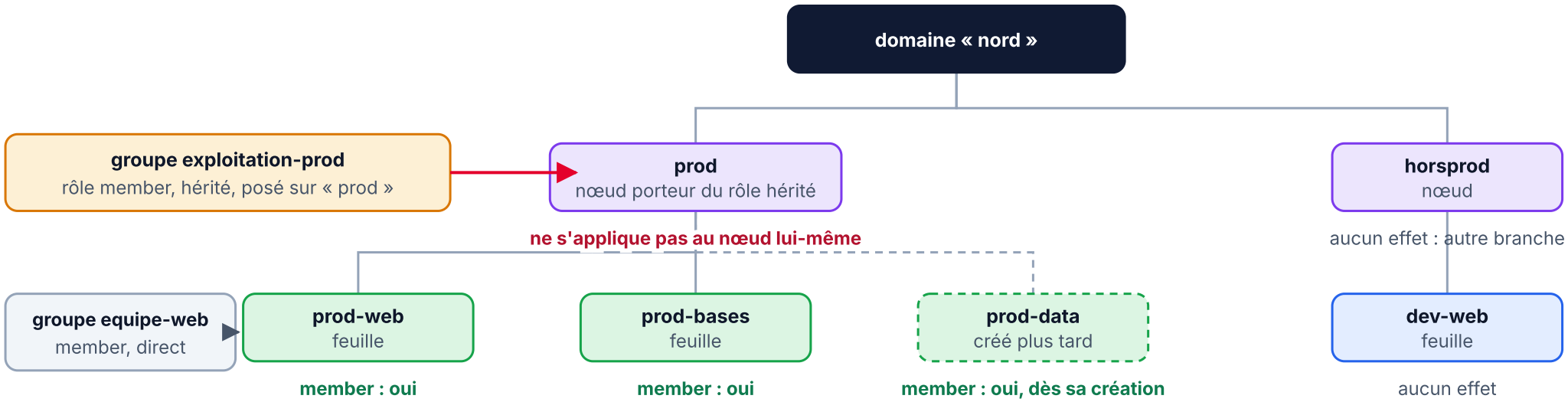
Règle	Détail	Conséquence pratique
Un seul parent	**fixé à la création du projet**	c'est un arbre, pas un graphe
Parent immuable	**on ne déplace pas un projet sous un autre parent**	concevoir l'arbre avant de créer ; sinon recréer et migrer
Même domaine	**un sous-projet est dans le domaine de son parent**	un arbre ne traverse pas les domaines
Noms uniques dans le domaine	**deux projets d'un même domaine ne peuvent porter le même nom, même sous des parents différents**	« prod/web » et « recette/web » : impossible ; préfixer ou suffixer
Profondeur limitée	**option `max_project_tree_depth` de Keystone, cinq niveaux par défaut**	deux ou trois niveaux suffisent presque toujours
Désactivation par les feuilles	**un projet désactivé ne peut pas avoir d'enfant actif**	pour suspendre une branche : désactiver les enfants, puis le parent
Suppression par les feuilles	**un projet qui a des enfants ne se supprime pas tel quel**	remonter l'arbre : feuilles d'abord
Isolation inchangée	**parent et enfant sont deux projets distincts pour tous les services**	le parent ne « voit » pas les instances de l'enfant

La règle qui surprend le plus : l'unicité des noms. L'arbre n'est pas un système de fichiers ; le nom complet d'un projet reste « nom + domaine », pas un chemin.

Règles décrites d'après le comportement connu de Keystone ; la profondeur maximale se vérifie sur votre version.

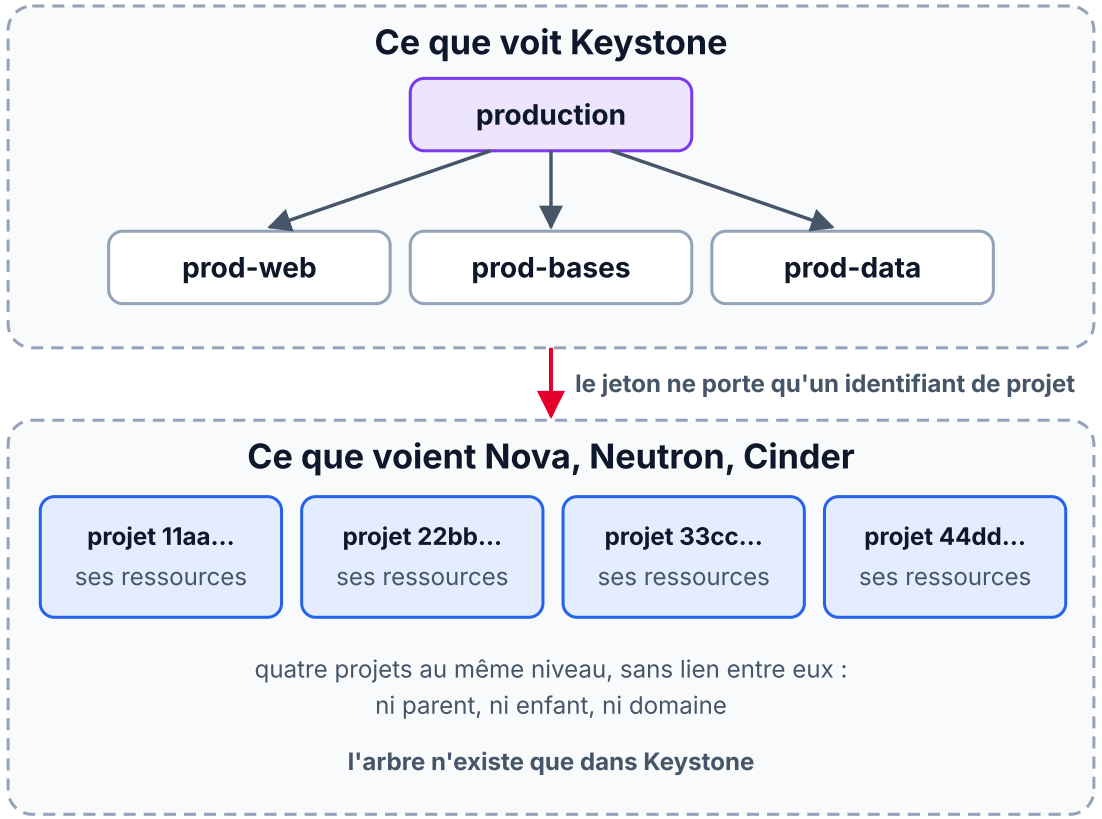
 Héritage depuis le domaine : module 9

Les rôles hérités



- **Un rôle hérité se pose sur un nœud et s'applique en dessous :** à tous ses descendants, présents et futurs — mais **pas** au nœud lui-même.
- **Pour avoir aussi le rôle sur le nœud,** on ajoute une affectation directe : les deux se cumulent.
- **Posé sur le domaine,** un rôle hérité couvre tous les projets du domaine.
- **Un nouveau sous-projet reçoit les droits sans rien faire :** c'est tout l'intérêt, et aussi le risque — il faut savoir ce qui descend avant de créer.

Ce que la hiérarchie ne fait pas



On pourrait croire que...	En réalité
un membre du parent voit les instances des enfants	non : il lui faut un rôle hérité, ou direct, sur chaque enfant
le quota du parent borne les enfants	non avec les quotas classiques ; oui seulement avec les limites unifiées en mode strict
les réseaux du parent sont accessibles aux enfants	non : il faut un partage explicite
supprimer le parent supprime la branche	non : feuilles d'abord, et les ressources à part
la facture se cumule vers le haut	à faire par l'outil de facturation, en lisant l'arbre

> **À retenir :** la hiérarchie est un outil d'**organisation des droits**. Pour tout le reste — réseau, capacité, facturation — elle n'est qu'une information que d'autres mécanismes peuvent exploiter.

À quoi servent les quotas

Protéger la capacité

Aucun projet ne peut, par erreur ou par excès, épuiser les hyperviseurs, le stockage ou les adresses.

Répartir équitablement

La capacité achetée ou allouée à chaque équipe est matérialisée et opposable.

Borner les erreurs

Une boucle dans un script d'automatisation s'arrête au quota, pas à la saturation du cloud.

Maîtriser les coûts

Le plafond de consommation est aussi un plafond de dépense, connu d'avance.

Un quota est un plafond, pas une réservation



La somme des quotas — 600 — dépasse la capacité — 400. C'est permis, et courant : tous les projets ne consomment pas leur maximum en même temps.
Mais rien ne garantit alors à un projet qu'il pourra atteindre son quota.

Deux réglages distincts : le quota dit ce qu'un projet a **le droit** de demander ; la capacité dit ce que le cloud **peut** fournir. Les deux refus n'ont pas le même message d'erreur.

Chiffres d'illustration. Le rapport entre somme des quotas et capacité est un choix d'exploitation, à suivre dans le temps.

Deux générations de mécanismes

Quotas « classiques » — par service

Nova
ses limites, dans sa base
instances, cœurs…

Cinder
ses limites, dans sa base
volumes, Go…

Neutron
ses limites, dans sa base
réseaux, ports…

chaque service stocke, vérifie et expose ses quotas ;
trois API, trois jeux de valeurs par défaut

le plus répandu ; projets tous au même niveau

commande : openstack quota …

Limites unifiées — centralisées dans Keystone

Keystone stocke toutes les limites
par défaut, par domaine, par projet

Nova
compte et vérifie

Glance
compte et vérifie

autres services
à mesure qu'ils l'adoptent

un seul endroit pour définir ; chaque service lit la limite
et connaît la hiérarchie des projets

commandes : openstack registered limit … , openstack limit …

	Quotas classiques	Limites unifiées
Où sont les valeurs	**dans chaque service**	dans Keystone
Hiérarchie de projets	**ignorée**	exploitable, selon le modèle choisi
Plafond par domaine	**non**	oui
Adoption	**tous les services**	Glance ; Nova sur option ; les autres progressivement

> **Sur un cloud donné, les deux coexistent :** il faut savoir, service par service, lequel est actif. Les slides suivantes décrivent d'abord les quotas classiques, puis les limites unifiées.

Quotas de calcul — Nova

Quota	Ce qu'il plafonne	Défaut amont
`instances`	**nombre d'instances du projet**	10
`cores`	**somme des cœurs virtuels des instances**	20
`ram`	**somme de la mémoire, en mégaoctets**	51 200
`key-pairs`	**paires de clés — par utilisateur**	100
`server-groups`	**groupes de serveurs (affinité)**	10
`server-group-members`	**instances par groupe**	10
`properties`	**métadonnées par instance**	128

- **Les trois premiers font tout le travail :** une instance est refusée dès que l'un d'eux serait dépassé.
- **Le gabarit décide de la consommation :** une instance « 4 cœurs, 8 Gio » retire 1, 4 et 8192 des trois compteurs.
- **Instance arrêtée :** elle compte toujours. Seule une instance « remisée et déchargée » ne consomme plus cœurs ni mémoire.

```
# Quotas de calcul d'un projet
openstack quota show --compute prod-web

# Fixer
openstack quota set prod-web \
  --instances 40 --cores 160 --ram 327680
```

Cohérence à vérifier : 40 instances, 160 cœurs, 320 Gio correspondent à un gabarit moyen de 4 cœurs et 8 Gio. Des quotas incohérents entre eux rendent l'un des trois inutile.

Ce qui n'est pas plafonné ici : le disque éphémère des instances — sauf avec les limites unifiées — et les cartes graphiques ou autres ressources spéciales, qui se bornent par gabarits privés.

Valeurs par défaut du code amont, données de mémoire : celles de votre cloud se lisent avec `openstack quota show --default .`

Quotas de stockage — Cinder et Glance

Cinder : volumes

Quota	Ce qu'il plafonne	Défaut amont
`volumes`	nombre de volumes	10
`gigabytes`	taille cumulée des volumes et des instantanés	1000
`snapshots`	nombre d'instantanés	10
`backups`	nombre de sauvegardes	10
`backup-gigabytes`	taille cumulée des sauvegardes	1000
`per-volume-gigabytes`	taille maximale d'un volume	illimitée

- **Par type de volume :** chaque type a en plus ses propres compteurs — utile pour rationner un stockage rapide et coûteux.
- **Les instantanés comptent dans `gigabytes` :** une équipe qui en abuse se bloque elle-même.

```
openstack quota show --volume prod-web
openstack quota set prod-web \
  --volumes 80 --gigabytes 4000 --snapshots 80

# Quota propre à un type de volume
openstack quota set prod-web \
  --volume-type rapide --gigabytes 500
```

Glance : images

Limite	Ce qu'elle plafonne
`image_size_total`	espace total des images du projet, en Mio
`image_stage_total`	espace des téléversements en cours
`image_count_total`	nombre d'images
`image_count_uploading`	téléversements simultanés

> Glance ne propose ses quotas par projet **que** par les limites unifiées : ils se règlent dans Keystone, pas avec `openstack quota set` .

> Sur un stockage Ceph, les volumes sont alloués à la demande : le quota borne la taille **promise**, pas l'espace réellement écrit.

Quotas réseau — Neutron

Quota	Ce qu'il plafonne	Défaut amont
`networks`	réseaux du projet	100
`subnets`	sous-réseaux	100
`ports`	ports : un par carte d'instance, plus ceux des routeurs, du DHCP, des répartiteurs	500
`routers`	routeurs	10
`floating-ips`	adresses flottantes — donc adresses externes	50
`secgroups`	groupes de sécurité	10
`secgroup-rules`	règles, tous groupes confondus	100
`subnetpools` , `rbac-policies`	pools de sous-réseaux, règles de partage	illimité, 10

- **Le quota qui compte vraiment : les adresses flottantes.** Elles puisent dans la plage du réseau externe, ressource rare et commune à tous.
- **Le quota qui bloque sans prévenir : les ports.** Une instance sans port disponible échoue à la création avec une erreur réseau, pas une erreur de calcul.

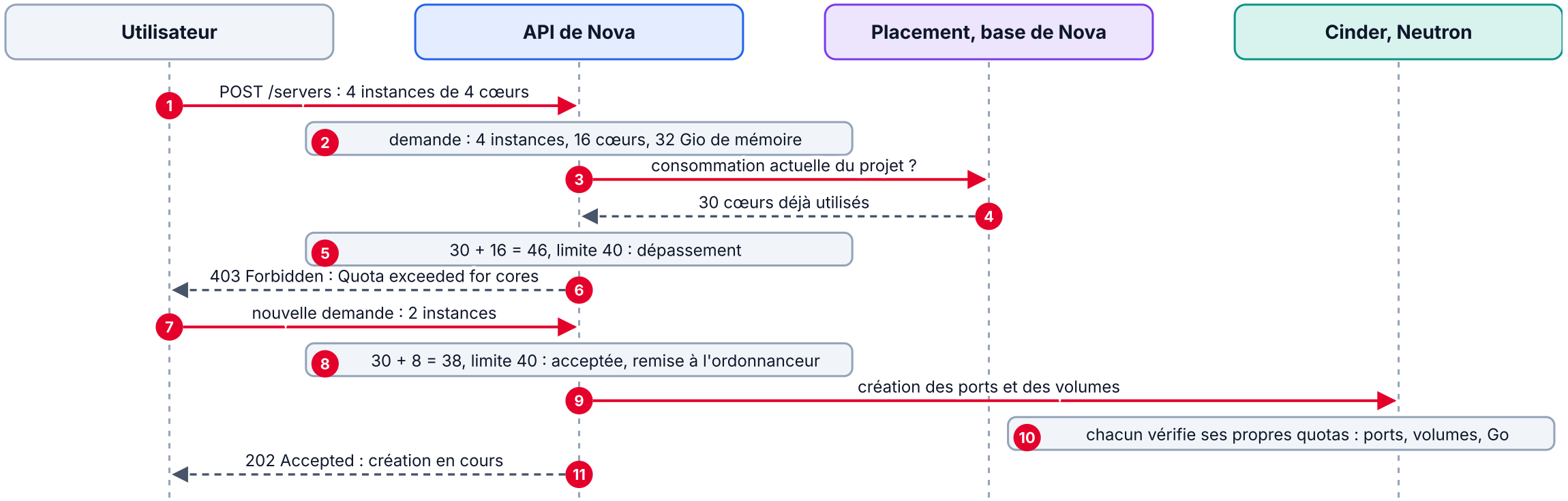
```
openstack quota show --network prod-web
openstack quota set prod-web \
  --networks 10 --subnets 20 --routers 3 \
  --ports 300 --floating-ips 8 \
  --secgroups 30 --secgroup-rules 300
```

Règle de cohérence : prévoir au moins un port par instance autorisée, plus une marge pour routeurs, DHCP et répartiteurs. Avec 40 instances, 300 ports sont confortables ; 40 seraient un piège.

Somme des adresses flottantes : la somme des quotas de tous les projets ne devrait pas dépasser de beaucoup la taille de la plage externe ; c'est le seul quota que l'on évite de surallouer.

L'équilibrage de charge a ses propres quotas, dans son propre service, lorsqu'il est activé.

Comment un quota est vérifié



- **La vérification se fait à l'entrée**, dans l'API du service, avant toute planification : un refus de quota ne laisse aucune trace d'instance.
- **Le service compte lui-même la consommation** à chaque demande — dans ses bases, ou auprès de Placement pour les cœurs et la mémoire.
- **Deux refus à distinguer :** « Quota exceeded », code 403, vient du quota ; « No valid host », instance en erreur, vient de la capacité.
- **Opération composée :** créer une instance avec un volume et une adresse flottante interroge trois quotas, dans trois services.

Les quotas par défaut : le réglage le plus important



- **Créer un projet est une opération déléguée ; fixer un quota ne l'est pas.** Entre les deux, le projet vit avec le défaut.
- **Un défaut généreux multiplie la capacité promise** par le nombre de projets, sans aucun contrôle.
- **Défaut des adresses flottantes :** cinquante par projet épuisent une petite plage externe en un projet.

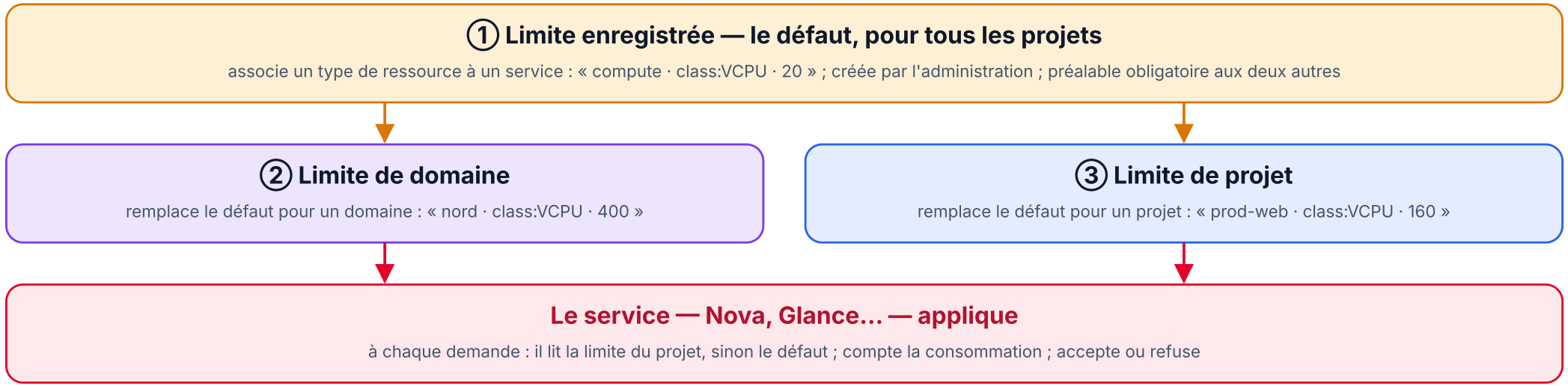
Stratégie	Défaut	Effet
Ouverte	**valeurs amont**	tout nouveau projet est utilisable ; capacité non maîtrisée
Bac à sable	**très bas : 2 instances, 4 cœurs, 0 adresse flottante**	on peut essayer ; tout usage réel passe par une demande
Fermée	**zéro partout**	rien ne fonctionne sans quota explicite ; contrôle total

```
# Lire les défauts en vigueur
openstack quota show --default

# Les changer : défauts de calcul et de volumes
openstack quota set --class default \
  --instances 2 --cores 4 --ram 8192 \
  --volumes 2 --gigabytes 40
```

Sur Sunbeam : certains défauts — ceux de Neutron notamment — sont des options de configuration du service, donc du charm. À vérifier avant de compter sur la commande ci-dessus.

Les limites unifiées : trois niveaux de valeurs



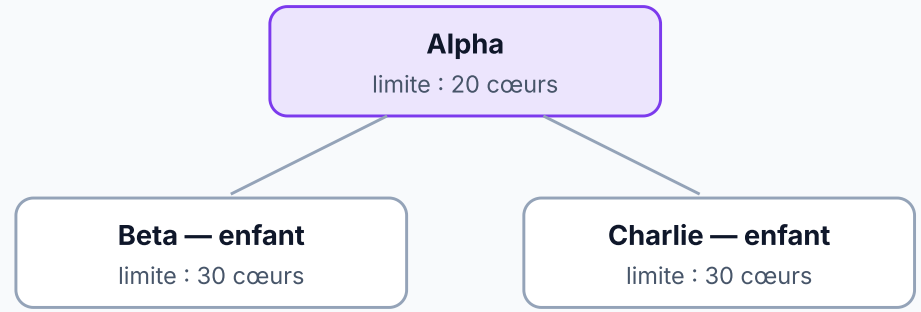
- **Keystone stocke et valide les limites ; il ne compte rien.** La consommation reste l'affaire de chaque service.
- **Valeurs :** de 0 à un très grand nombre ; `-1` pour illimité.
- **Abaisser sous la consommation** ne détruit rien : les nouvelles demandes sont refusées.

Ressource (Nova)	Équivalent classique
`servers`	instances
`class:VCPU`	cores
`class:MEMORY_MB`	ram
`class:DISK_GB`	— nouveau : disque des instances
`server_key_pairs`, `server_groups`…	key-pairs, server-groups…

> Toute classe de ressource de Placement peut être plafonnée ainsi, cartes graphiques comprises.

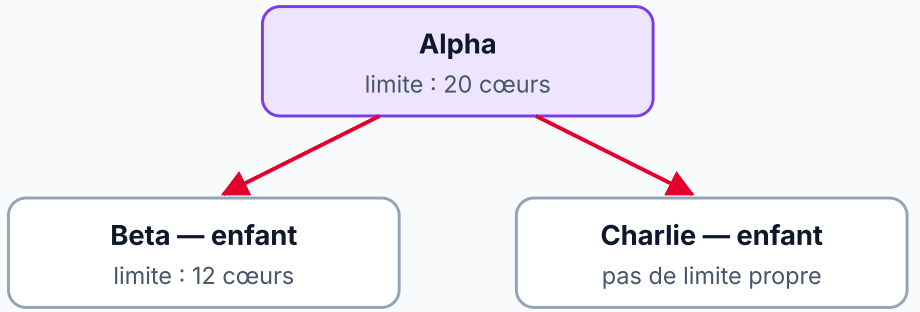
Limites unifiées : deux modèles d'application

Modèle « flat » — à plat



chaque projet est jugé seul : la limite du parent
ne borne pas les enfants, qui peuvent le dépasser
total possible : 20 + 30 + 30 = 80 cœurs

Modèle « strict_two_level » — deux niveaux stricts



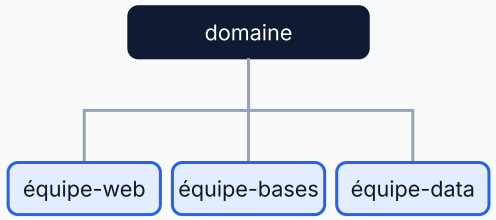
la consommation de tout l'arbre est plafonnée par le parent ;
aucun enfant ne peut dépasser la limite du parent
total possible : 20 cœurs — Charlie dispose de ce qui reste

	flat	strict_two_level
Rapport parent / enfant	**aucun**	le parent borne la somme de l'arbre
Profondeur de l'arbre	**quelconque**	**deux niveaux au plus** : créer un petit-enfant est refusé
Partage de capacité	**manuel, limite par limite**	automatique à l'intérieur de l'arbre
Validation des valeurs	**aucune**	à la création et à la modification
Usage type	**projets indépendants**	une enveloppe par équipe ou par domaine, répartie librement

Un seul modèle pour tout le cloud, choisi dans la configuration de Keystone ; en changer après coup peut invalider des limites existantes. Le modèle strict contraint aussi la forme des arbres de projets : à décider avant de les dessiner.

Quatre modèles d'organisation

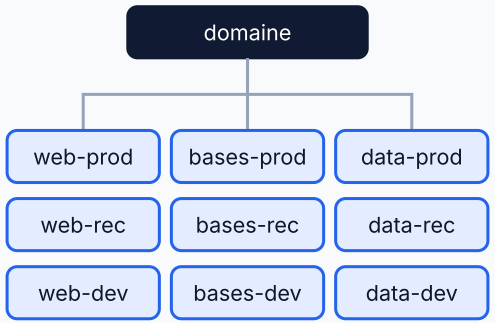
① Par équipe



production, recette et essais dans le même projet

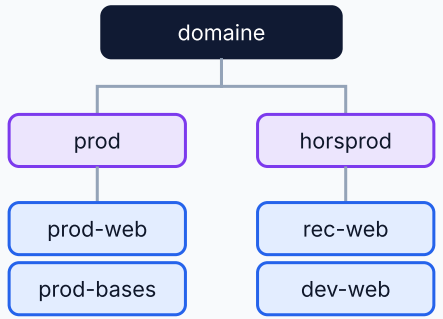
tout ce que fait l'équipe, au même endroit

② Application × environnement



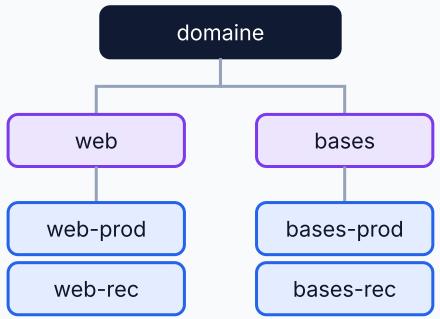
une case par couple, tout au même niveau

③ Arbre par environnement



environnement en nœud, application en feuille

④ Arbre par application



application en nœud, environnement en feuille

① Par équipe
Simple, vite en place. Convient à un cloud d'essai ; mélange les environnements.

② Application × environnement
Cloisonnement fin, à plat. Les droits se posent projet par projet.

③ Arbre par environnement
Un rôle hérité sur « production » couvre toute la production.

④ Arbre par application
Un rôle hérité sur « web » donne tous les environnements de l'application.

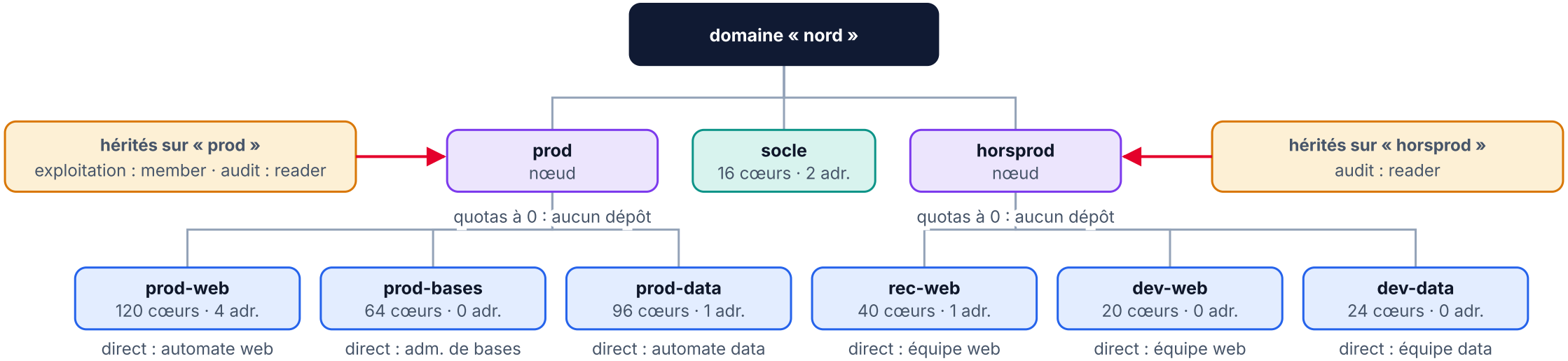
Comparer les quatre modèles

Critère	① Par équipe	② Application × environnement	③ Arbre par environnement	④ Arbre par application
Nombre de projets	**faible**	élevé	élevé, plus les nœuds	élevé, plus les nœuds
Production isolée des essais	**non**	**oui**	**oui**	**oui**
Incident limité à une application	**non**	**oui**	**oui**	**oui**
Affectations de rôles à gérer	**peu**	beaucoup : une par projet	peu : héritées par environnement	peu : héritées par application
Donner « toute la production » à l'exploitation	**impossible**	une affectation par projet	**une seule, héritée**	une par projet de production
Donner « toute l'application » à une équipe	**—**	une affectation par projet	une par projet de l'application	**une seule, héritée**
Plafond global par environnement	**non**	somme tenue à la main	**oui, avec** `strict_two_level`	non
Plafond global par application	**non**	somme tenue à la main	non	**oui, avec** `strict_two_level`
Risque principal	**un essai consomme le quota de la production**	dérive : droits oubliés, incohérents	un rôle hérité trop large en production	un développeur hérite de la production
Convient à	**laboratoire, équipe unique**	petit nombre d'applications	exploitation centralisée	équipes produit autonomes

Le choix entre ③ et ④ suit l'organisation : qui a le plus besoin d'un droit transversal — l'exploitation sur un environnement, ou une équipe sur son application ?

Le parent étant immuable, passer de ③ à ④ impose de recréer les projets. Le modèle se choisit avant la première création.

Un modèle de départ : l'arbre par environnement



- **Deux niveaux de projets, pas plus :** lisible, et compatible avec le modèle de limites `strict_two_level`.
- **Les nœuds ne portent aucune ressource :** quotas à zéro, ils rangent et transmettent.
- **Rôles hérités sur les nœuds** pour les populations transversales : exploitation, audit, sécurité.
- **Rôles directs sur les feuilles** pour les équipes applicatives.
- **Préfixe d'environnement dans le nom :** imposé par l'unicité des noms dans le domaine.
- **Une branche « socle »** pour ce qui sert à tous : images, réseau commun, supervision.

Cloisonner le matériel

Par défaut, tous les projets partagent tous les hyperviseurs. Trois outils, tous réservés à l'administration, permettent d'aller plus loin :

Outil	Effet
Agrégat d'hôtes	**groupe nommé d'hyperviseurs, porteur de propriétés ; invisible des utilisateurs**
Zone de disponibilité	**agrégat rendu visible : l'utilisateur la choisit à la création**
Agrégat réservé à des projets	**ces hôtes n'acceptent que les projets listés**
Gabarit privé	**gabarit visible des seuls projets autorisés ; peut cibler un agrégat par ses propriétés**
Type de volume privé	**même principe pour une classe de stockage**

Réserver un agrégat ne suffit pas : les hôtes refusent les autres projets, mais le projet réservataire peut encore être placé ailleurs. Pour une séparation dans les deux sens, lui donner uniquement des gabarits privés qui ciblent l'agrégat.

```
# Un agrégat pour la production sensible
openstack aggregate create prod-sensible
openstack aggregate add host prod-sensible noeud-4
openstack aggregate add host prod-sensible noeud-5

# Le réserver à un projet (identifiant)
openstack aggregate set prod-sensible \
  --property filter_tenant_id=$PID \
  --property classe=sensible

# Gabarit privé, qui cible l'agrégat
openstack flavor create s.4c8g --private \
  --vcpus 4 --ram 8192 --disk 40 \
  --property aggregate_instance_extra_specs:classe=sensible
openstack flavor set s.4c8g --project $PID
```

À vérifier sur votre déploiement : ces propriétés n'ont d'effet que si les filtres correspondants de l'ordonnanceur sont actifs. Sur Sunbeam, la configuration de Nova est écrite par les charms : contrôler par un essai avant de s'y fier.

11

Authentification externe et mots de passe

Fédération d'identité avec une autorité d'authentification externe, et politique de mots de passe selon le référentiel de l'ANSSI, sur Canonical OpenStack (Sunbeam)

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Expliquer pourquoi et comment déléguer l'authentification à une **autorité externe**.
- Écrire et tester des **règles de correspondance**.
- Traduire le guide de l'**ANSSI** en règles : longueur, complexité, expiration, verrouillage.
- Décrire les objets de la **fédération** dans Keystone et le déroulé d'une connexion.
- Raccorder un fournisseur d'identité à un cloud **Sunbeam**, pas à pas.
- Appliquer ces règles dans Keystone sans casser les **comptes de service**.

M11 · AUTORITÉ EXTERNE

Identité locale ou identité externe

Sujet	Comptes locaux dans Keystone	Autorité externe, par fédération
Où vit le mot de passe	**dans la base de Keystone, haché**	chez l'autorité ; Keystone ne le voit jamais
Politique de mot de passe	**options de Keystone, globales**	celle de l'autorité, déjà appliquée à toute l'entreprise
Multifacteur	**TOTP de Keystone, compte par compte**	celui de l'autorité : application, clé physique, carte
Départ d'un salarié	**à désactiver dans Keystone, en plus du reste**	désactivé une fois, chez l'autorité : plus de nouvelle connexion
Droits	**affectations aux comptes ou aux groupes**	groupes de l'autorité, traduits en groupes Keystone par des règles
Traçabilité	**journaux de Keystone**	journaux de l'autorité et de Keystone, avec le même identifiant
Dépendance	**aucune**	l'autorité doit être joignable pour se connecter

Principe : une seule source de vérité par personne. Chaque compte local supplémentaire est un mot de passe de plus à protéger, à faire expirer, à révoquer au départ.

Ce qui reste local : les comptes des services OpenStack, un ou deux comptes d'administration de secours, et les comptes techniques des automates. Ils sont l'objet de la dernière partie.

M11 · AUTORITÉ EXTERNE

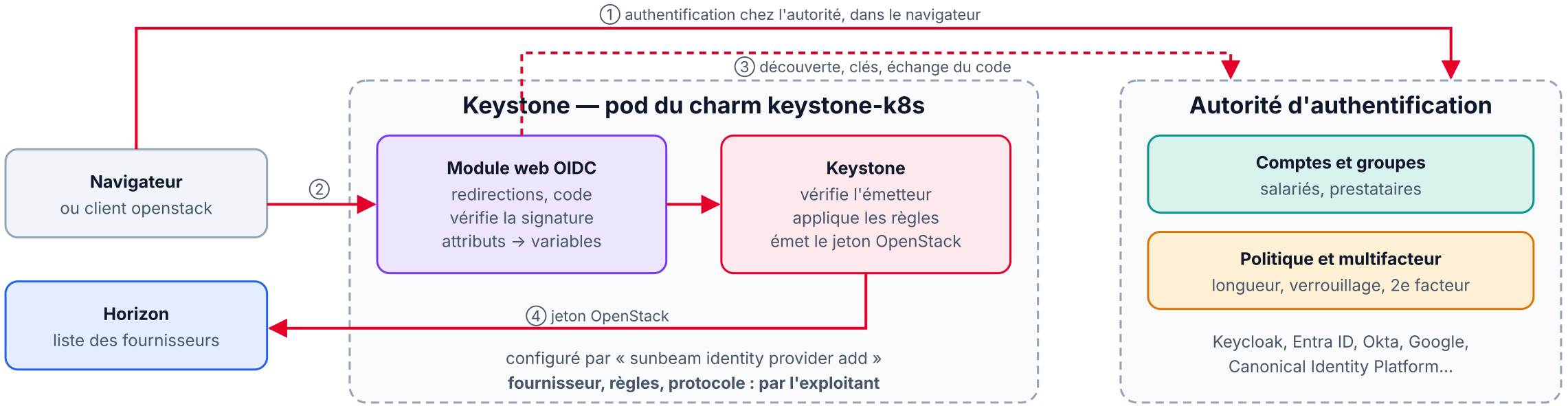
OpenID Connect ou SAML 2

Critère	OpenID Connect (OIDC)	SAML 2
Fondation	**OAuth 2.0, jetons JSON signés (JWT)**	assertions XML signées
Configuration	**adresse de découverte de l'émetteur, identifiant et secret de client**	échange de fichiers de métadonnées, certificats
Fournisseurs courants	**Keycloak, Entra ID, Okta, Google, Canonical Identity Platform**	ADFS, Shibboleth, Keycloak, Entra ID
Ligne de commande	**possible : mot de passe délégué, autorisation par appareil, jeton d'accès**	difficile : protocole pensé pour le navigateur
Dans Keystone	**protocole `openid`, module Apache `mod_auth_openidc`**	protocole `saml2`, module Apache `mod_auth_mellon` ou Shibboleth
Sur Sunbeam	**chemin documenté :** `sunbeam identity provider add … openid`	le charm prévoit une paire de clés SAML ; procédure à vérifier

Choix par défaut : OIDC. Plus simple à configurer, utilisable en ligne de commande, et c'est le chemin que Canonical documente pour Sunbeam.

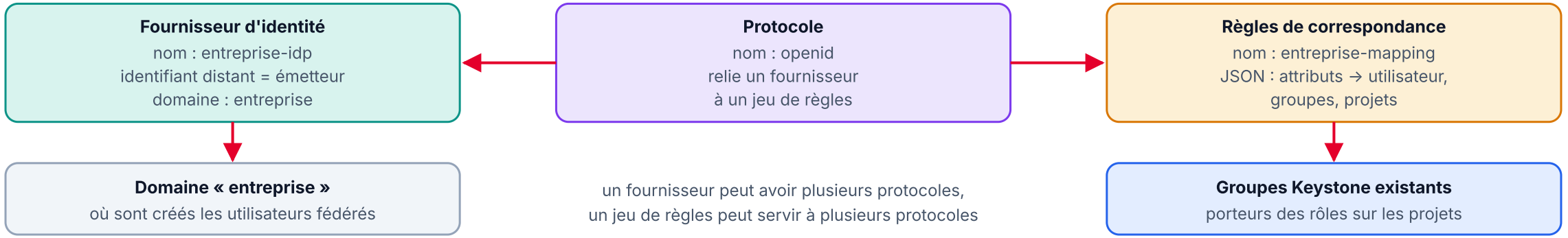
SAML 2 reste pertinent quand l'autorité de l'entreprise ne parle que lui — un ADFS ancien par exemple — ou pour une fédération entre établissements qui l'impose.

Architecture



- **Le module web fait le protocole** — redirections, échange du code, vérification de la signature — et transmet les attributs à Keystone sous forme de variables d'environnement.
- **Keystone fait l'autorisation :** il vérifie l'émetteur, applique les règles, crée ou retrouve l'utilisateur et émet un jeton.
- **Le préfixe des attributs** dépend de la configuration du module : `OIDC-email`, `HTTP_OIDC_EMAIL`… Les règles de correspondance doivent utiliser exactement ces noms.
- **Horizon** propose le fournisseur dans sa liste de connexion et reçoit le jeton en retour.

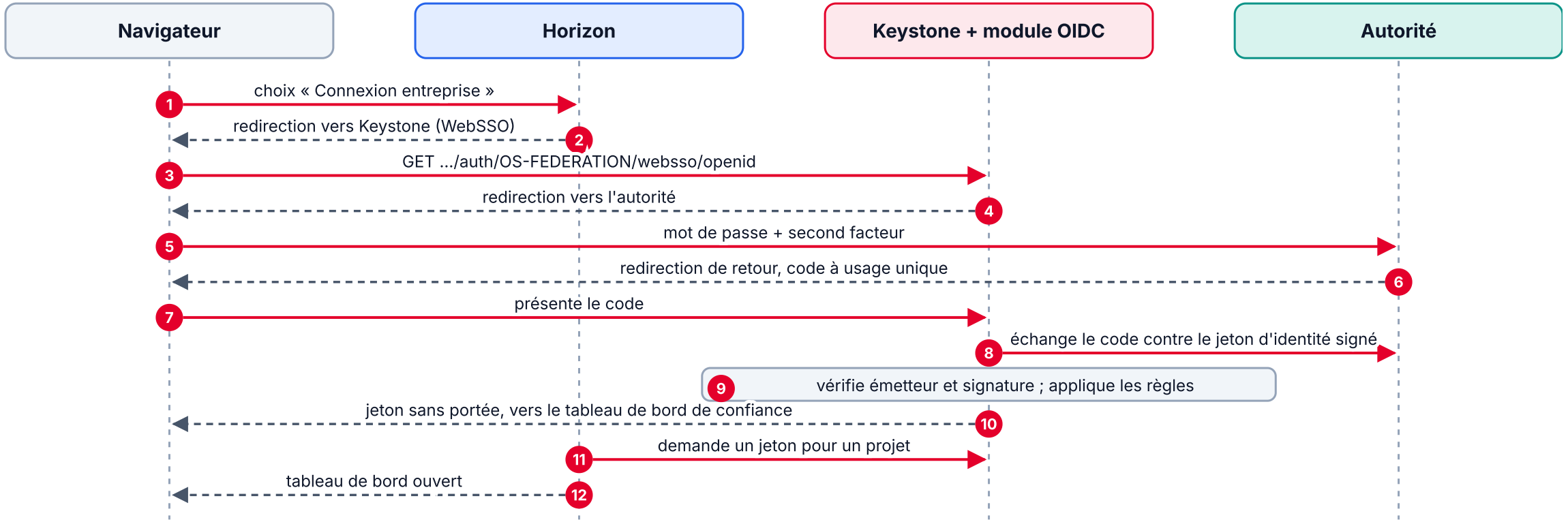
Les trois objets de Keystone




```
# Prérequis : identifiants d'administrateur
source admin-openrc
openstack identity provider create entreprise-idp \
  --remote-id https://sso.exemple.fr/realms/entreprise \
  --domain entreprise
openstack mapping create entreprise-mapping --rules rules.json
openstack federation protocol create openid \
  --identity-provider entreprise-idp --mapping entreprise-mapping
```

- **Identifiant distant :** doit être égal, caractère pour caractère, à l'émetteur présenté par l'attestation — barre oblique finale comprise.
- **Domaine du fournisseur :** les utilisateurs fédérés y sont créés ; un domaine par fournisseur.
- **Le nom du protocole** fait partie des adresses de connexion : `…/identity_providers/entreprise-idp/protocols/openid/auth`.

Connexion par le navigateur (WebSSO)



- **Le mot de passe et le second facteur** ne sont saisis que chez l'autorité (étape 4) : ni Horizon ni Keystone ne les voient.
- **Le premier jeton est sans portée :** il dit qui est l'utilisateur. Horizon en demande ensuite un pour un projet.
- **Tableau de bord de confiance :** Keystone ne renvoie le jeton qu'à une adresse d'Horizon déclarée d'avance — protection contre le vol de jeton par redirection.
- **Aucun rôle, aucun projet ?** La connexion réussit chez l'autorité, puis échoue dans Horizon : la règle n'a donné aucun groupe utile.

Les règles de correspondance : anatomie

```
[
  {
    "local": [
      { "user":  { "name": "{0}", "email": "{1}" } },
      { "group": { "name": "exploitation-prod",
                   "domain": { "name": "entreprise" } } }
    ],
    "remote": [
      { "type": "OIDC-preferred_username" },
      { "type": "OIDC-email" },
      { "type": "OIDC-groups",
        "any_one_of": [ "openstack-exploitation" ] }
    ]
  },
  {
    "local": [
      { "user":  { "name": "{0}", "email": "{1}" } },
      { "group": { "name": "developpeurs",
                   "domain": { "name": "entreprise" } } }
    ],
    "remote": [
      { "type": "OIDC-preferred_username" },
      { "type": "OIDC-email" },
      { "type": "OIDC-groups",
        "any_one_of": [ "openstack-dev" ] }
    ]
  }
]
```

Partie	Rôle
`remote`	ce que l'attestation doit contenir ; toutes les conditions doivent être vraies
`local`	ce que Keystone en tire : utilisateur, groupes, éventuellement projets
`{0}`, `{1}`	valeur du 1er, du 2e attribut de `remote`, dans l'ordre
Groupe local	doit exister dans Keystone et porter les rôles

- **Les droits ne sont pas dans la règle :** elle ne fait que placer la personne dans un groupe Keystone ; les rôles sont affectés au groupe, comme pour tout autre utilisateur.
- **Les règles se cumulent :** une personne présente dans les deux groupes de l'autorité reçoit les deux groupes Keystone.
- **Aucune règle ne correspond :** refus de la connexion (401). C'est le comportement par défaut voulu — n'entre que qui est prévu.

> Noms d'attributs d'exemple ; leur forme exacte dépend du module web et de l'autorité.

Utilisateurs et groupes fédérés

Type d'utilisateur	Comportement
éphémère (par défaut)	**créé à la première connexion dans le domaine du fournisseur ; attributs venus de l'autorité**
local	**doit exister déjà dans Keystone ; la fédération ne sert qu'à l'authentifier ; ses groupes et rôles locaux s'appliquent**

- **Appartenance aux groupes :** recalculée à chaque connexion à partir de l'attestation. Quitter le groupe chez l'autorité retire le droit à la connexion suivante.
- **Utilisateur « fantôme » :** Keystone garde une trace de chaque utilisateur fédéré — identifiant stable, dernière connexion — ce qui permet audit et affectations directes.
- **Appartenance persistante :** pour qu'un identifiant d'application ou une délégation d'un utilisateur fédéré reste valable entre deux connexions, Keystone peut conserver l'appartenance pendant une durée réglée sur le fournisseur (`authorization_ttl`) — à vérifier sur votre version.

Départ d'un salarié

Moment	Effet
Compte désactivé chez l'autorité	plus de nouvelle connexion
Jetons Keystone déjà émis	valides jusqu'à expiration, quelques heures
Identifiants d'application créés par lui	à supprimer : ils ne passent pas par l'autorité
Ressources qu'il a créées	restent dans les projets : elles appartiennent au projet

La fédération ne coupe pas tout : la procédure de départ doit inclure la suppression des identifiants d'application ; pour un départ sensible, tenir compte de la durée de vie des jetons déjà émis.

```
# Utilisateurs fédérés et leurs identifiants
openstack user list --domain entreprise
openstack application credential list --user <id>
```

Limites à connaître

Limite	Conséquence et contournement
Groupes recalculés à chaque connexion	**identifiants d'application et délégations d'un utilisateur fédéré peuvent perdre leurs droits ; durée d'appartenance persistante, ou compte technique local**
Pas de révocation à la source	**désactiver chez l'autorité n'invalide pas les jetons déjà émis ; durée de vie courte des jetons**
Ligne de commande	**selon les flux autorisés par l'autorité ; sinon, passer par Horizon et un identifiant d'application**
Un domaine par fournisseur	**un même salarié présent dans deux fournisseurs = deux utilisateurs**
Politique de Keystone	**les options de conformité de Keystone — verrouillage, expiration — ne s'appliquent pas aux utilisateurs fédérés : c'est l'autorité qui décide**

Ce que la fédération apporte

- une seule politique de mot de passe, celle de l'entreprise ;
- le multifacteur sans rien développer ;
- des départs traités une fois ;
- aucun mot de passe humain dans Keystone.

Ce qu'elle n'apporte pas

- les droits OpenStack : ils restent à concevoir dans Keystone ;
- l'authentification des services et des automates ;
- une révocation instantanée des sessions ouvertes.

> C'est la dernière ligne du tableau qui fait le lien avec la suite : une fois les personnes fédérées, la politique de mots de passe de Keystone ne concerne plus que les comptes locaux.

Ce que Sunbeam fait, ce qui reste à faire

Étape	Qui la fait	Commande ou action
Déclarer Keystone comme client chez l'autorité	**l'exploitant, chez l'autorité**	URL de redirection donnée par `sunbeam identity provider get-oidc-redirect-url`
Configurer le module OIDC de Keystone	**Sunbeam**	`sunbeam identity provider add`
Proposer le fournisseur dans Horizon	**Sunbeam**	automatique : le nom choisi sert d'étiquette
Créer le domaine du fournisseur	**l'exploitant**	`openstack domain create`
Enregistrer le fournisseur dans Keystone	**l'exploitant**	`openstack identity provider create`, même nom que dans Sunbeam
Groupes, projets, rôles	**l'exploitant**	module 10 (projets), module 12 (rôles, moindre privilège)
Règles de correspondance et protocole	**l'exploitant**	`openstack mapping create`, `openstack federation protocol create`
Retirer le fournisseur	**Sunbeam, puis l'exploitant**	`sunbeam identity provider remove` ne supprime pas les objets créés avec `openstack`

Types documentés par Canonical : `canonical` (Canonical Identity Platform), `google`, `okta`, `entra`, tous en OpenID Connect. Pour un autre fournisseur — Keycloak par exemple —, consulter `sunbeam identity provider add --help` sur votre version.

Pas à pas (1) : déclarer le fournisseur

```
# Exemple : Entra ID
cat > entra.yaml <<'EOF'
client-id: 1b2c…
client-secret: "••••••••"
microsoft-tenant: 7f3e…
label: Connexion entreprise
EOF
chmod 600 entra.yaml

sunbeam identity provider add entra openid \
  entreprise-idp --config entra.yaml

# Vérifier, et lire l'émetteur pour la suite
sunbeam identity provider list
sunbeam identity provider list --format=yaml

# Supprimer le fichier : le secret est désormais
# conservé par le déploiement
shred -u entra.yaml
```

Type	Champs du fichier
`google`	`client-id`, `client-secret`, `label`
`okta`	idem + `okta-org`
`entra`	idem + `microsoft-tenant`
`canonical`	offres Juju du modèle `iam` : `oauth_offer`, `cert_offer`

- **Le nom choisi** — `entreprise-idp` — sera repris tel quel dans Keystone ; il apparaît dans Horizon sous l'étiquette.
- **Le secret du client** ne doit pas traîner dans l'historique ni dans un dépôt : fichier en mode 600, supprimé après usage.

> La documentation écrit les clés tantôt avec tiret, tantôt avec soulignement : vérifier la forme attendue par votre version.

Pas à pas (2) : les objets de Keystone

```
# Prérequis : identifiants d'administrateur
source admin-openrc
ISSUER=$(sunbeam identity provider list --format=yaml \
  | yq -r '."entreprise-idp".issuer_url')
# (syntaxe de la documentation Canonical)

# 1 · Domaine dédié au fournisseur
openstack domain create entreprise \
  --description "Utilisateurs fédérés — Entra ID"

# 2 · Fournisseur : MÊME nom que dans Sunbeam
openstack identity provider create entreprise-idp \
  --remote-id "$ISSUER" --domain entreprise

# 3 · Groupes porteurs des rôles
openstack group create developpeurs --domain entreprise
openstack role add member \
  --group developpeurs --group-domain entreprise \
  --project rec-web --project-domain nord
```

```
# 4 · Règles de correspondance, testées hors ligne
openstack mapping create entreprise-mapping \
  --rules rules.json

# 5 · Protocole : relie fournisseur et règles
openstack federation protocol create openid \
  --identity-provider entreprise-idp \
  --mapping entreprise-mapping

# Relire l'ensemble
openstack identity provider show entreprise-idp
openstack federation protocol list \
  --identity-provider entreprise-idp
openstack mapping show entreprise-mapping
```

Groupes et projets peuvent être dans des domaines différents : le groupe vit dans le domaine du fournisseur ; il reçoit des rôles sur les projets du domaine « nord ».

Le nom du protocole, `openid`, est celui que le module configuré par Sunbeam attend.

Longueur et entropie (R21)

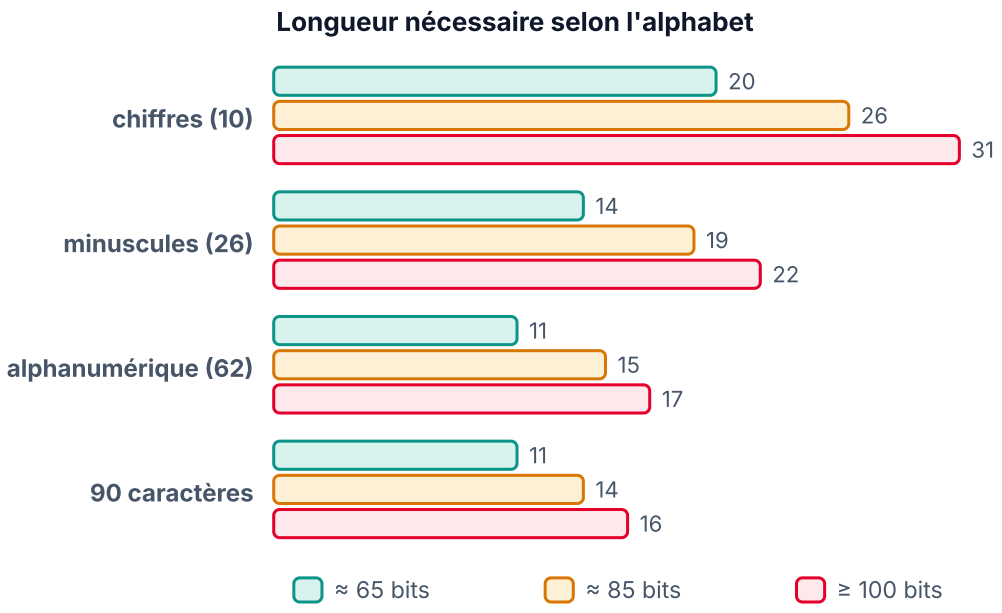


Tableau du guide — alphabet de 90 caractères

Sensibilité	Longueur minimale	Équivalent
Faible à moyenne	**9 à 11**	≈ 65 bits
Moyenne à forte	**12 à 14**	≈ 85 bits
Forte à très forte	**15 et plus**	≥ 100 bits

- **Forte à très forte sensibilité :** le guide recommande en plus le multifacteur.
- **Mots de passe non mémorisés** — ceux d'un gestionnaire, d'un compte technique — : au moins 20 caractères.
- **Code court acceptable** seulement avec des mesures fortes : blocage définitif après trois échecs, comme une carte à puce.

Pour un cloud : les comptes d'administration relèvent de la sensibilité forte ; un compte de travail ordinaire, au minimum de la moyenne à forte.

Calcul : longueur × $\log_2$(taille de l'alphabet). Valable seulement pour un tirage aléatoire uniforme — ce que n'est presque jamais un mot de passe choisi par une personne.

Expiration (R24, R25, R26)

R24 — Comptes ordinaires

Si la robustesse est imposée et que le système le permet : **pas d'expiration par défaut**. Le guide note qu'une expiration courte, de 3 à 6 mois, peut être contre-productive : les utilisateurs incrémentent un chiffre.

R25 — Comptes à privilèges

Imposer une expiration, par exemple **entre 1 et 3 ans**. Le guide préfère, pour ces comptes, une authentification à deux facteurs.

R26 — Compromission suspectée

Renouveler sans délai — dans la journée — tous les mots de passe concernés ; à défaut, désactiver les comptes et prévoir une procédure de réactivation.

Habitude ancienne	Problème	Pratique conforme au guide
Expiration à 90 jours pour tous	**mots de passe prévisibles : « Printemps2026! »**	pas d'expiration pour les comptes ordinaires robustes
Jamais d'expiration pour l'administration	**un mot de passe ancien a eu le temps de fuiter**	1 à 3 ans, plus multifacteur
Attendre l'expiration après une fuite	**fenêtre d'exploitation ouverte**	renouvellement immédiat, procédure prête

Dans Keystone, l'expiration est une option globale : elle ne distingue pas comptes ordinaires et privilégiés. On la module compte par compte avec une option d'exemption — p. 254.

Verrouillage et temporisation (R10)



- **R10 :** limiter le nombre de tentatives sur une période — blocage temporaire de quelques secondes ou minutes après plusieurs échecs, linéaire ou exponentiel.
- **Un blocage définitif** n'est envisagé par le guide que pour des codes courts protégés autrement — une carte à puce, par exemple.
- **Le revers du verrouillage :** quiconque connaît un identifiant peut le bloquer en tapant de faux mots de passe. Sur un compte de service, c'est une panne du cloud.
- **R14 :** en multifacteur, un échec ne doit pas révéler quel facteur a échoué.

De la règle à la valeur : une politique par population

Règle	Personnes, compte ordinaire	Personnes, à privilèges	Compte local de secours	Compte de service
Où il vit	**autorité externe**	autorité externe	Keystone, domaine local	Keystone, domaine des services
Longueur	**≥ 12**	≥ 15	≥ 20, tiré au sort	≥ 20, tiré au sort
Complexité	**largeur du jeu de caractères, liste de refus**	idem	sans objet : aléatoire	sans objet : aléatoire
Expiration	**aucune (R24)**	1 à 3 ans (R25)	1 an, et après chaque usage	rotation planifiée, hors expiration automatique
Historique	**5**	5	5	sans objet
Verrouillage	**temporisation après 5 à 10 échecs**	idem	temporisation, alerte	exempté — surveillance des échecs
Multifacteur	**recommandé**	obligatoire	obligatoire (TOTP)	impossible : secret protégé à la place
Inactivité	**désactivation après 90 jours**	idem	exempté — revue périodique	exempté
Premier usage	**selon l'autorité**	selon l'autorité	changement imposé	exempté

Valeurs proposées, à adapter : elles découlent du guide (R10, R21, R24, R25, R27) et d'une analyse de risque type ; elles ne remplacent pas la vôtre.

Lecture : deux colonnes se règlent chez l'autorité, deux dans Keystone. Les exemptions de la dernière colonne évitent de mettre le cloud en panne — p. 257.

Les options de conformité — `[security_compliance]`

Option	Effet	Guide	Exemption par compte
`lockout_failure_attempts`	**verrouille après N échecs consécutifs**	R10	`ignore_lockout_failure_attempts`
`lockout_duration`	**durée du verrouillage en secondes ; sans valeur, verrouillage jusqu'à réactivation**	R10	—
`password_expires_days`	**expiration des mots de passe changés après activation**	R24, R25	`ignore_password_expiry`
`unique_last_password_count`	**nombre de derniers mots de passe interdits, l'actuel compris**	R27	—
`minimum_password_age`	**délai minimal entre deux changements, en jours**	R27	—
`password_regex` , `password_regex_description`	**règle de composition, et son explication affichée**	R21, R23	—
`change_password_upon_first_use`	**changement imposé après création ou réinitialisation par l'administration**	R30	`ignore_change_password_upon_first_use`
`disable_user_account_days_inactive`	**désactive les comptes sans authentification depuis N jours**	R18	`ignore_user_inactivity`
— (options de compte)	`multi_factor_auth_enabled` , `multi_factor_auth_rules` , `lock_password`	R1	—

Toutes désactivées par défaut, et ne s'appliquent qu'au stockage SQL de Keystone : ni aux domaines LDAP, ni aux utilisateurs fédérés.

Globales : une seule valeur pour tout le cloud. La différenciation par population passe par les options d'exemption, posées compte par compte.

Une configuration alignée sur le guide

```
# keystone.conf — comptes locaux restants :
# administration de secours et comptes techniques
[security_compliance]
# R10 : temporisation, pas de blocage définitif
lockout_failure_attempts = 5
lockout_duration = 900
# R25 : comptes locaux = comptes à privilèges
password_expires_days = 365
# R27 : historique
unique_last_password_count = 5
minimum_password_age = 1
# R21 / R22 : longueur d'abord, pas de maximum utile
password_regex = ^.{15,256}$
password_regex_description = Au moins 15 caractères.
# R30 : mot de passe remis par l'administration
change_password_upon_first_use = true
# R18 : comptes oubliés
disable_user_account_days_inactive = 90

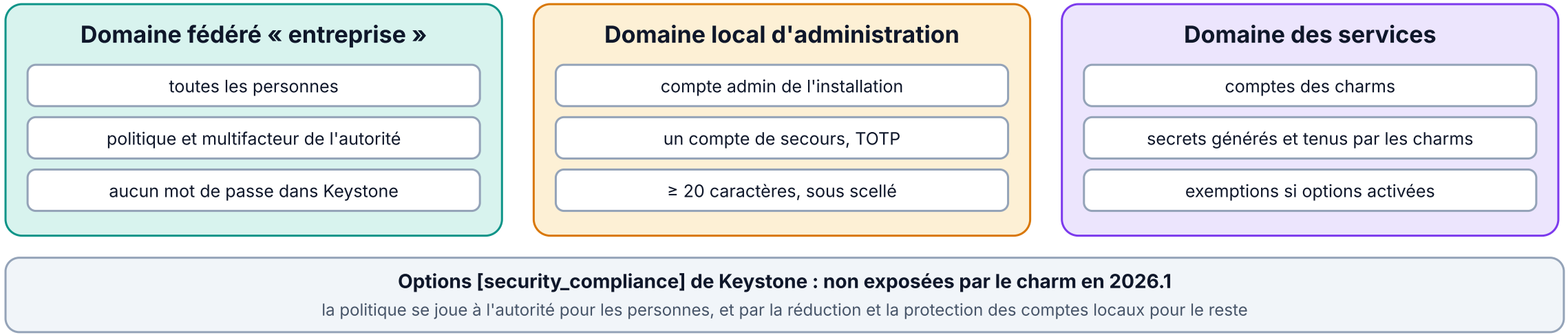
[identity]
# R29- : PBKDF2 ; R29 : scrypt
password_hash_algorithm = scrypt
```

- **Expiration à 365 jours pour tous les comptes locaux :** cohérent avec R25 dès lors que les personnes ordinaires sont fédérées ; il ne reste en local que des comptes à privilèges.
- **Expression régulière centrée sur la longueur :** 15 caractères minimum, sans classes imposées ; la borne de 256 protège seulement contre les abus.
- `minimum_password_age` **inférieur à** `password_expires_days`**,** sinon impossible de changer avant expiration.
- **Expiration des mots de passe existants :** l'option ne s'applique qu'aux mots de passe changés après son activation.

> **Avant toute activation :** poser les exemptions sur les comptes de service et le compte de secours — p. 258. Activer d'abord, exempter ensuite, c'est risquer de verrouiller le cloud.

> Valeurs proposées pour ce module, à valider par votre analyse de risque.

La stratégie sur Sunbeam



- **Le charm de Keystone n'expose aucune option de conformité** en 2026.1 : ses options portent sur le cache, la journalisation, la région, le stockage d'identité et la paire de clés SAML.
- **Modifier `keystone.conf` dans le pod** n'est pas durable : le charm réécrit sa configuration.
- **Donc :** la politique des personnes va à l'autorité ; Keystone garde un minimum de comptes locaux, protégés par la longueur, le multifacteur et les procédures.
- **À suivre :** si une version ultérieure du charm expose ces options, appliquer la configuration de la p. 255, exemptions d'abord.

Quatre risques

Verrouillage

`lockout_failure_attempts`

quelqu'un tape « nova » et
de faux mots de passe
→ Nova ne s'authentifie plus
→ le cloud ne crée plus rien

exemption
`ignore_lockout_failure_attempts`

Expiration

`password_expires_days`

le mot de passe du compte
expire à date fixe
→ tous les services liés
tombent le même jour

exemption
`ignore_password_expiry`

Premier usage

`change_password_upon_first_use`

après création ou
réinitialisation par script
→ changement exigé
qu'aucun service ne sait faire

exemption
`ignore_change_password_upon_first_use`

Inactivité

`disable_user_account_days_inactive`

un compte rarement utilisé —
le compte de secours —
→ désactivé sans bruit
→ indisponible le jour J

exemption
`ignore_user_inactivity`

Le plus grave est le verrouillage : il transforme une simple connaissance du nom d'un compte — `nova`, `neutron` — en moyen de mettre le cloud hors service, sans aucun mot de passe.

Ces risques concernent aussi les comptes techniques des automates qui s'authentifient par mot de passe. Les identifiants d'application, eux, ne passent pas par le mot de passe du compte.

Les exemptions

```
# Prérequis : identifiants d'administrateur
source admin-openrc

# Le domaine des services : son nom se lit, il ne se suppose pas
SVC=$(openstack domain list -f value -c Name | grep -i service)
openstack user list --domain "$SVC"

# Exempter chaque compte de service
for u in $(openstack user list --domain "$SVC" \
            -f value -c ID); do
  openstack user set "$u" \
    --ignore-lockout-failure-attempts \
    --ignore-password-expiry \
    --ignore-change-password-upon-first-use
done

# Inactivité, pour le compte de secours : par l'API
# (option sans équivalent dans le client selon les versions)
SID=$(openstack user show secours \
  --domain "$OS_USER_DOMAIN_NAME" -f value -c id)
TOKEN=$(openstack token issue -f value -c id)
curl -s -X PATCH "$OS_AUTH_URL/users/$SID" \
  -H "X-Auth-Token: $TOKEN" -H "Content-Type: application/json" \
  -d '{"user":{"options":{"ignore_user_inactivity":true}}}'

# Contrôle
openstack user show nova --domain "$SVC" -c options
```

Option de compte	À poser sur
`ignore_lockout_failure_attempts`	**comptes de service ; comptes techniques**
`ignore_password_expiry`	**comptes de service ; comptes techniques**
`ignore_change_password_upon_first_use`	**comptes de service ; comptes créés par script**
`ignore_user_inactivity`	**compte de secours ; comptes techniques peu utilisés**
`lock_password`	**comptes dont le secret est géré ailleurs**

- **Une exemption n'est pas une absence de contrôle :** elle déplace la protection vers la longueur du secret, son stockage et la surveillance des échecs.
- **Les exemptions sont visibles :** `openstack user show … -c options` ; les lister à chaque revue.

> Noms des options du client et de l'API à vérifier sur votre version. Sur Sunbeam, poser ces options sur les comptes gérés par les charms est sans risque pour eux, mais à refaire si un charm recrée son compte.

12

Rôles et moindre privilège

Rôles standards, contrôle d'accès par périmètre (Secure RBAC) et comptes de service sur Canonical OpenStack (Sunbeam)

À L'ISSUE DE CE MODULE, VOUS SAUREZ

- Énoncer le principe de **moindre privilège** et le traduire en affectations de rôles.
- Expliquer ce que **Secure RBAC** change : portées, nouvelles règles par défaut, fin de « admin partout ».
- Créer des **comptes de service** et des **identifiants d'application** restreints, et les faire tourner.
- Distinguer les cinq rôles standards : `reader`, `member`, `manager`, `admin`, `service`.
- Lire une règle de politique et diagnostiquer un refus 403.
- Bâtir et auditer une matrice de rôles pour une organisation.

Quatre leviers dans OpenStack

Affectation de rôle = acteur + rôle + cible

+ durée de la preuve

① Qui ? — l'acteur

- un groupe plutôt qu'une personne
- un compte nominatif, jamais partagé
- un identifiant par automate

levier : ne donner qu'aux bons

② Quoi ? — le rôle

- reader avant member
- member avant manager
- admin : jamais par défaut

levier : le plus petit rôle

③ Où ? — la cible

- un projet plutôt qu'un domaine
- héritage seulement s'il le faut
- portée système : exploitation

levier : le plus petit périmètre

④ Combien de temps ?

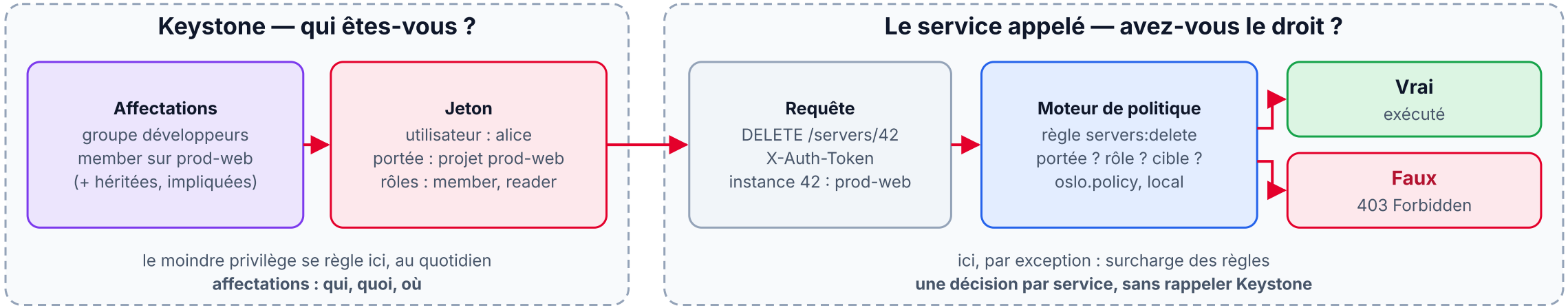
- jeton : quelques heures
- identifiant d'application : daté
- affectation : revue périodique

levier : une date de fin

Le seul objet qui donne des droits est l'affectation de rôle : acteur + rôle + cible. Le moindre privilège se joue donc sur chacun de ces trois termes, plus la durée de vie de la preuve d'identité.

Ce que les leviers ne font pas : ils n'isolent pas le matériel ni le réseau. Un droit bien taillé ne remplace ni les groupes de sécurité, ni les quotas.

La chaîne de décision



- **Keystone décide qui vous êtes et quels rôles vous avez, sur quelle portée.** Il met tout cela dans le jeton.
- **Chaque service décide seul** si ces rôles suffisent pour l'opération : Nova applique ses règles, Neutron les siennes.
- **Conséquence :** un rôle n'a aucun pouvoir propre. `member` ne signifie quelque chose que parce que les règles des services le citent.
- **Le moindre privilège se règle à deux endroits :** les affectations, côté Keystone — le quotidien ; les règles, côté services — l'exception.

Avant Secure RBAC : « admin quelque part, admin partout »

Le modèle historique ne connaissait que deux niveaux :

Profil	Ce qu'il permettait
Propriétaire	un rôle quelconque sur le projet : lire et modifier toutes ses ressources
Administrateur	`admin` sur un projet — n'importe lequel — et les services le traitaient en administrateur du cloud entier

- **Pas de lecture seule** utilisable : tout membre d'un projet pouvait tout modifier.
- **Pas de délégation propre :** pour gérer des comptes, il fallait `admin`, donc tout.
- **Des services s'appelant entre eux avec `admin`** : un compte de service compromis valait un administrateur.

> Ce comportement a fait l'objet d'un signalement de sécurité ancien et célèbre de Keystone : le rôle `admin` n'était pas « cantonné » à son projet.

Ce que Secure RBAC apporte

Besoin	Réponse
Lecture seule	`reader`, respecté par les règles par défaut
Travailler dans un projet	`member`
Déléguer la gestion	`manager` sur un domaine ou un projet
Appels entre services	`service`, hors hiérarchie
Cibler le bon périmètre	portées : projet, domaine, système
Activer sans tout casser	deux options par service : nouvelles règles, contrôle de portée

> **Ce qui n'a pas changé :** le rôle `admin` reste global dans la plupart des services — un `admin` sur un projet administre encore le cloud. C'est le point le plus important de ce module.

Cinq rôles standards

reader

Lire. Lister les instances, voir les volumes, consulter les quotas. Aucune modification.

Pour : audit, support, supervision, développeurs en production.

member

Agir sur les ressources de la cible : créer, modifier, supprimer.

Pour : les équipes qui travaillent dans un projet, les automates de déploiement.

manager

Gérer : comptes, groupes, projets et affectations dans un domaine ; certaines opérations élevées dans un projet.

Pour : responsables délégués.

admin

Tout, y compris l'infrastructure : hyperviseurs, réseaux externes, quotas, domaines.

Pour : la poignée de personnes qui exploite la plateforme.

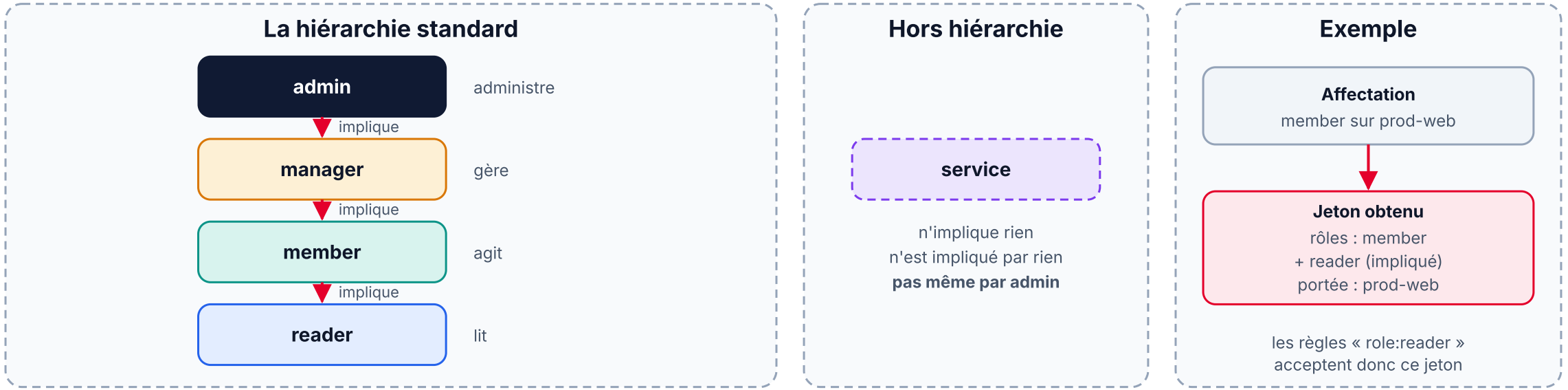
service

Appels d'un service OpenStack à un autre : notifications d'événements, opérations internes.

Pour : les comptes des services, jamais des humains.

Rôle	Créé par	Implique	Remarque
`admin`	**initialisation de Keystone**	`manager`, donc tous les autres	global : voir p. 266
`manager`	**initialisation (depuis 2023.2)**	`member`, `reader`	ne peut pas attribuer `admin`
`member`	**initialisation**	`reader`	le rôle de travail par défaut
`reader`	**initialisation**	—	base de la hiérarchie
`service`	**initialisation (depuis 2023.2) ; attribué par le charm sur Sunbeam**	— : hors hiérarchie	`admin` ne l'implique pas

La hiérarchie d'implication



- **Un rôle impliqué est ajouté au jeton automatiquement :** qui a `member` reçoit aussi `reader` dans son jeton, sans affectation supplémentaire.
- **Une seule affectation par personne et par cible suffit :** le rôle le plus haut nécessaire.
- `service` **est volontairement isolé :** un administrateur humain ne peut pas se faire passer pour un service, et un compte de service n'hérite pas de droits humains.
- **Les règles peuvent donc s'écrire simplement :** « `role:reader` » suffit pour une lecture — tous les rôles supérieurs y satisfont.

 Délégation détaillée : module 9

manager : déléguer la gestion

Sur un domaine — avec un jeton de portée domaine

Opération	Domain manager
Créer, désactiver des utilisateurs du domaine	**oui**
Gérer groupes et appartenances	**oui**
Créer, désactiver des projets du domaine	**oui**
Attribuer `reader` , `member` , `manager`	**oui, dans le domaine**
Attribuer `admin`	**non : pas d'élévation**
Modifier le domaine lui-même	**non**
Voir les autres domaines	**non**
Changer les quotas	**non**

> La liste des rôles qu'un `manager` peut attribuer est fixée par la politique de Keystone ; elle exclut `admin` et peut être étendue à un rôle maison.

Sur un projet — avec un jeton de portée projet

- **Dans Nova, depuis 2025.2 :** des opérations jusque-là réservées à `admin` deviennent accessibles au `manager` du projet — par exemple migrer à froid une instance sans choisir l'hôte.
- **Dans les autres services :** l'adoption du persona est progressive. À vérifier service par service avant de s'en servir.
- **Jamais de choix d'hôte, jamais d'infrastructure :** le `manager` reste dans son projet.

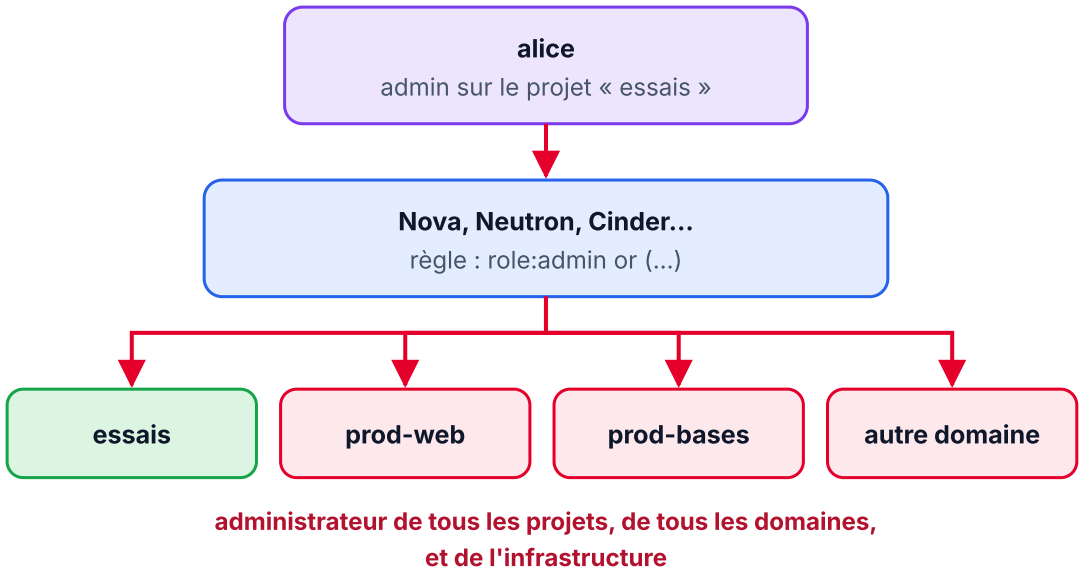
```
# Responsable du domaine « nord », délégué
openstack role add manager \
  --group gestionnaires-nord --group-domain nord \
  --domain nord

# Il travaille avec un jeton de portée domaine
export OS_DOMAIN_NAME=nord
unset OS_PROJECT_NAME OS_PROJECT_DOMAIN_NAME
openstack user create --domain nord alice
```

> Le module 9 détaille l'administration déléguée : ce que voit et ne voit pas un gestionnaire.

 Le piège admin : module 9

admin : le rôle qui déborde de sa cible



- **Dans Nova, Neutron, Cinder...** `admin` sur n'importe quel projet donne les droits d'administration sur tout le cloud. C'est le « legacy admin », conservé volontairement pour ne pas casser les déploiements existants.
- **Dans Keystone aussi :** les règles commencent par `admin_required` (`role:admin or is_admin:1`), sans condition de domaine. Un `admin` de domaine peut donc agir hors de son domaine — à tester (p. 209).
- **Conséquence pratique :** `admin` ne se délègue jamais. Pour déléguer, `manager` .

Bonne pratique	Pourquoi
Deux à cinq personnes nommées	surface minimale, imputabilité
Comptes d'administration distincts des comptes de travail	on ne lit pas ses courriels en `admin`
Affectation sur le seul projet d'administration	une seule affectation à surveiller
Revue mensuelle de la liste	retrait des départs

Règle typique de Nova :
`role:admin or (role:member and project_id:%(project_id)s)` . La partie `admin` ne vérifie pas le projet : elle vaut pour tous.

service : le rôle des machines

Certaines API ne sont faites que pour les services entre eux. Exemple : Neutron prévient Nova qu'un port est prêt ; Cinder prévient Nova qu'un volume a changé de taille.

Avant 2023.2	Avec le rôle `service`
le compte de Neutron avait `admin` pour appeler Nova	il a `service` : la règle exige ce rôle précis
un humain `admin` pouvait appeler ces API internes	un humain `admin` n'a pas `service` : refus
compromettre un compte de service = administrateur	objectif : compte de service limité aux API internes

Règle de la documentation de Keystone : `"identity:create_foo": "role:service"`, ou `"role:service or role:admin"` pendant la transition.

- **Hors hiérarchie :** ni `admin` ne l'implique, ni il n'implique `reader`.
- **Ne jamais l'attribuer à une personne** : il donne accès à des API dont l'appel à tort peut détruire des ressources.
- **Sur Sunbeam,** le charm de Keystone crée les comptes des services avec `service` **et** `admin` : le second est gardé tant que tous les services ne savent pas se contenter du premier.

```
# Qui détient le rôle service ?
openstack role assignment list --role service --names

# Attendu : uniquement des comptes du domaine
# des services, jamais une personne
```

Le jour où les comptes de service n'ont plus que `service`, un vol de leurs identifiants ne donnera plus l'administration du cloud — c'est la cible de la phase 2 de Secure RBAC.

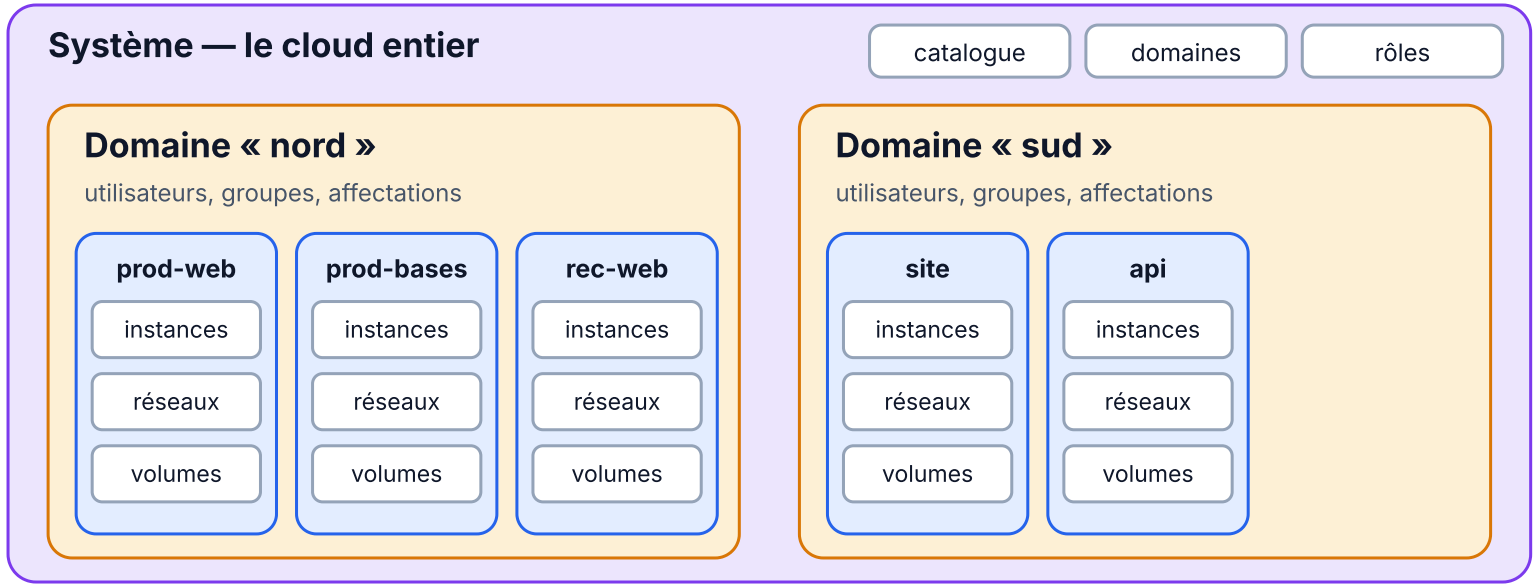
Matrice rôle × opération

Opération (jeton de portée projet, sauf mention)	reader	member	manager	admin	service
Lister les instances du projet	**oui**	**oui**	**oui**	**oui, tous projets**	—
Créer, supprimer une instance	**non**	**oui**	**oui**	**oui**	—
Migrer à froid sans choisir l'hôte (Nova 2025.2+)	**non**	**non**	**oui**	**oui**	—
Migrer à chaud, choisir l'hôte	**non**	**non**	**non**	**oui**	—
Lister les hyperviseurs	**non**	**non**	**non**	**oui**	—
Créer un réseau, un routeur du projet	**non**	**oui**	**oui**	**oui**	—
Créer un réseau externe ou partagé	**non**	**non**	**non**	**oui**	—
Modifier les quotas	**non**	**non**	**non**	**oui**	—
Créer un utilisateur (jeton de portée domaine)	**non**	**non**	**oui, son domaine**	**oui**	—
Attribuer `admin`	**non**	**non**	**non**	**oui**	—
Événements externes de Nova (appel de Neutron)	**non**	**non**	**non**	selon la version	**oui**

Lecture : d'après les règles par défaut de Nova, Neutron et Keystone. Une surcharge de politique sur votre cloud peut changer n'importe quelle case.

Vérifier une case : lire la règle effective du service (p. 271) plutôt que de faire confiance à un tableau — celui-ci compris.

Trois portées



Jeton et ce qu'il ouvre

jeton système
`OS_SYSTEM_SCOPE=all`
identité de tout le cloud

jeton de domaine
`OS_DOMAIN_NAME=nord`
identité du domaine nord

jeton de projet
`OS_PROJECT_NAME=prod-web`
ressources de prod-web

Système
Ce qui n'appartient à personne : catalogue, domaines, rôles, et dans certains services l'infrastructure. Réservée à l'exploitation.

Domaine
L'identité d'une organisation : ses utilisateurs, groupes, projets, affectations. Portée de la délégation.

Projet
Les ressources : instances, réseaux, volumes, images. La portée de presque tout le travail quotidien.

Personas : un rôle sur une portée

	Portée système	Portée domaine	Portée projet
`admin`	**administre Keystone : catalogue, domaines, fédération**	administre l'identité — et déborde du domaine (`admin_required`)	« legacy admin » : administre tous les services, tous projets
`manager`	—	gère utilisateurs, groupes, projets, affectations du domaine	opérations élevées dans le projet, là où le service le prévoit
`member`	**lit l'identité de tout le cloud**	lit le domaine	crée et gère les ressources du projet
`reader`	**lit l'identité de tout le cloud — audit**	lit le domaine	lit les ressources du projet

- **Un persona = un couple.** « Lecteur système » et « lecteur de projet » sont deux profils différents, obtenus par deux affectations différentes.
- **Les cases de la colonne système** concernent Keystone. Les services de ressources — Nova en tête — n'acceptent plus de jeton système : leurs règles sont de portée projet.
- **Les droits ne se transportent pas d'une portée à l'autre :** un `reader` système ne lit pas les instances ; il lui faut un jeton de projet.
- **Exception :** `admin`, global (p. 266).

Anatomie d'une règle

```
# Nova — règles par défaut (extraits)
"project_reader":  "role:reader and project_id:%(project_id)s"
"project_member":  "role:member and project_id:%(project_id)s"
"project_manager_api":
                   "role:manager and project_id:%(project_id)s"
"admin_api":       "role:admin"

"os_compute_api:servers:show":
  "role:admin or (role:reader and project_id:%(project_id)s)"
"os_compute_api:servers:create":
  "role:admin or (role:member and project_id:%(project_id)s)"
"os_compute_api:os-hypervisors:list":
  "role:admin"

# Keystone — lire les utilisateurs
"identity:list_users":
  "(rule:admin_required)
   or (role:reader and system_scope:all)
   or (role:reader and domain_id:%(target.domain_id)s)"
```

Élément	Ce qu'il vérifie
`role:member`	le jeton contient ce rôle, directement ou par implication
`project_id:%(project_id)s`	la ressource visée appartient au projet du jeton
`domain_id:%(target.domain_id)s`	la cible est dans le domaine du jeton
`system_scope:all`	le jeton est de portée système
`rule:…`	renvoie à une autre règle nommée
Portées acceptées	attribut de chaque règle dans le code : projet, domaine, système ; non surchargeable par fichier dans Nova

Trois contrôles successifs : la portée du jeton est-elle acceptée par cette opération ? le rôle est-il présent ? la cible appartient-elle à la portée ? Il faut les trois.

`role:admin` **seul, sans condition de projet :** c'est là que se loge le « legacy admin ».

Les deux options d'activation

`[oslo_policy] enforce_new_defaults`

Vrai : seules les nouvelles règles par défaut — avec `reader`, `member`, `manager` — s'appliquent. Faux : les anciennes règles restent acceptées en alternative, et tout rôle sur un projet suffit souvent à y écrire.

`[oslo_policy] enforce_scope`

Vrai : un jeton dont la portée ne figure pas parmi celles de la règle est refusé (403). Faux : la portée n'est que signalée dans les journaux.

Combinaison	Comportement	Quand
faux · faux	**comportement historique : `reader` peut écrire, jeton système accepté partout**	clouds anciens, à faire évoluer
vrai · faux	**rôles respectés, portées seulement journalisées**	étape de migration : observer les refus de portée à venir
faux · vrai	**combinaison incohérente ; à éviter**	—
vrai · vrai	**Secure RBAC complet**	cible ; valeur par défaut depuis oslo.policy 4.4 (2024.2)

Une option par service : chaque service lit son propre fichier de configuration. Un cloud peut être en Secure RBAC complet dans Nova et en mode historique dans un autre service.

Nova recommande d'activer les deux ensemble, et rend le contrôle de portée obligatoire en 2026.2 : l'option n'y aura plus d'effet.

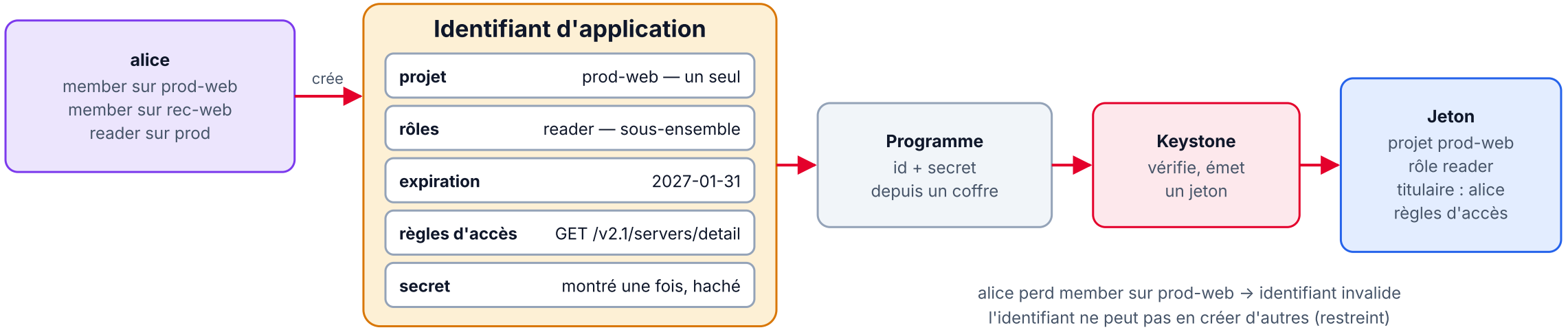
Comptes d'automates : quatre options

Critère	Compte personnel	Utilisateur technique dédié	Identifiant d'application	Délégation (trust)
Secret	**mot de passe d'une personne**	mot de passe propre	secret propre, haché par Keystone, montré une fois	celui du service délégataire
Portée	**tous les projets de la personne**	ses affectations	**un seul projet**	un projet, rôles choisis
Rôles	**tous ceux de la personne**	ceux qu'on lui donne	**sous-ensemble de ceux du créateur**	**sous-ensemble de ceux du délégant**
Restriction par API	non	non	**oui : règles d'accès**	non
Expiration	**celle du mot de passe**	celle du mot de passe	**date d'expiration**	**date d'expiration**
Départ du créateur	**le pipeline s'arrête**	sans effet	invalide : à faire tourner avant	invalide
Imputabilité	**confondue avec la personne**	**compte nommé**	rattaché au créateur, nommé	délégant et délégataire
Usage recommandé	jamais	automate de plateforme sans titulaire humain	**cas général des automates**	services qui agissent plus tard pour un utilisateur : Heat, Magnum

Compte personnel dans un pipeline : l'automate a tous les droits de la personne, sur tous ses projets, et s'arrête le jour de son départ. C'est l'anti-modèle.

Utilisateur technique ou identifiant d'application ? L'identifiant d'application est plus étroit ; l'utilisateur technique survit au départ de quiconque. On combine souvent : un utilisateur technique, qui crée ses identifiants d'application.

Identifiants d'application : principe



- **Créés par l'utilisateur lui-même,** sans intervention de l'administration : la délégation est en libre-service.
- **Le secret n'est montré qu'une fois :** Keystone le hache avant de le stocker. Perdu, il ne se récupère pas — on en crée un autre.
- **Ne peuvent jamais dépasser leur créateur :** on ne donne pas un rôle qu'on n'a pas. Retirer un rôle au créateur invalide ses identifiants sur ce projet.
- **Restreints par défaut :** ils ne peuvent pas créer ni supprimer d'autres identifiants d'application — sauf option `--unrestricted`, à éviter.

Créer un identifiant d'application restreint

```
# Prérequis : être connecté avec un jeton de portée projet,
# sur le projet visé, avec le compte qui en sera titulaire
openstack --os-cloud ci-web token issue -c project_id

# Identifiant d'un pipeline de déploiement : member,
# sur prod-web seulement, valable jusqu'à fin janvier
openstack --os-cloud ci-web application credential create \
  deploy-web-2026q4 \
  --role member \
  --expiration 2027-01-31T00:00:00 \
  --description "Pipeline GitLab web — titulaire : equipe-web"

# Identifiant d'un outil d'inventaire : lecture seule
openstack --os-cloud ci-web application credential create \
  inventaire-web --role reader \
  --expiration 2027-03-31T00:00:00

# Lister, afficher, supprimer
openstack application credential list
openstack application credential show deploy-web-2026q4
openstack application credential delete inventaire-web
```

Option	Effet
`--role`	sous-ensemble des rôles du créateur ; par défaut, tous
`--expiration`	date de fin ; sans elle, aucune expiration
`--access-rules`	liste blanche d'appels d'API autorisés
`--secret`	fournir le secret au lieu de le laisser générer — à éviter
`--unrestricted`	autorise à créer d'autres identifiants — à éviter

La réponse contient `id` et `secret` : les copier immédiatement dans le coffre de secrets du pipeline, jamais dans un dépôt, un ticket ou une messagerie.

Nom porteur de sens : usage + périmètre + période. À la revue, on sait ce qu'il fait et quand il doit tourner.

Règles d'accès : restreindre aux appels utiles

```
# Une sauvegarde qui ne fait que lister les volumes
# et créer des instantanés
openstack application credential create sauvegarde-web \
  --role member --expiration 2027-01-31T00:00:00 \
  --access-rules '[
    {"service": "volumev3", "method": "GET",
     "path": "/v3/*/volumes/detail"},
    {"service": "volumev3", "method": "POST",
     "path": "/v3/*/snapshots"},
    {"service": "volumev3", "method": "GET",
     "path": "/v3/*/snapshots/**"}
  ]'

# Règles déjà connues, réutilisables par identifiant
openstack access rule list
```

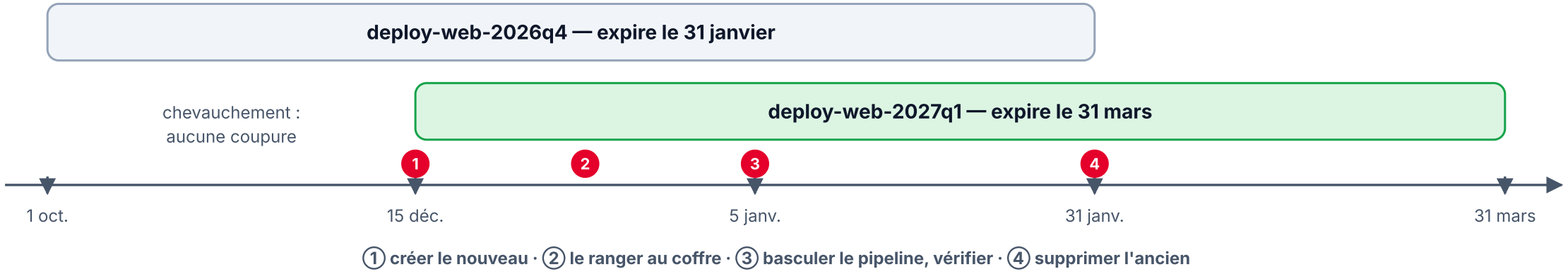
Chemins : `*` remplace un segment, `**` un nombre quelconque de segments. Le chemin part de la racine de l'adresse du service, version comprise.

Usage	Règle(s) — service, méthode, chemin
Créer des instances	`compute POST /v2.1/servers`
Lister les instances	`compute GET /v2.1/servers/detail`
Téléverser des images	`image POST /v2/images`, `image PUT /v2/images/*/file`
Lire les ports	`network GET /v2.0/ports`

- **Le rôle reste nécessaire :** la règle d'accès filtre en plus. Il faut que la politique du service **et** la règle d'accès disent oui.
- **Le service est désigné par son type** dans le catalogue — `openstack catalog list` le donne.
- **Mettre au point par essais :** lancer l'automate, relever les 401 et les chemins refusés, compléter. Les clients appellent parfois des API inattendues — découverte de versions, extensions.

Les services doivent déclarer leur type à keystonemiddleware pour que les règles s'appliquent ; à confirmer sur Sunbeam par un essai avant de s'y fier.

Rotation et révocation



- **Rotation sans coupure :** le nouvel identifiant existe avant que l'ancien disparaisse ; deux noms différents.
- **Fréquence :** à chaque trimestre, ou plus court pour la production.

- **Révocation d'urgence :** `application credential delete` — effet immédiat pour les nouveaux jetons ; les jetons déjà émis vivent jusqu'à leur expiration.
- **Pour tout couper :** retirer le rôle du titulaire sur le projet, ce qui invalide ses identifiants.

- **Départ du titulaire :** recréer les identifiants sous un autre titulaire **avant** de désactiver le compte.
- **Fuite suspectée :** révoquer d'abord, enquêter ensuite.

Une matrice de rôles type

Population	Rôle	Cible	Hérité	Commentaire
Équipe plateforme (2 à 5 personnes)	`admin`	projet d'administration	non	comptes d'administration distincts des comptes de travail
Équipe plateforme — lecture	`reader`	système	—	voir toute l'identité sans passer par `admin`
Gestionnaires du domaine nord	`manager`	domaine nord	non	comptes, groupes, projets, affectations
Exploitation de la production	`member`	projet `prod` (nœud)	oui	tous les projets de production, présents et futurs
Équipe web (`equipe-web`)	`member`	projets de recette et de développement	non	affectation directe, projet par projet
Développeurs — production	`reader`	projet `prod` (nœud)	oui	diagnostiquer sans corriger
Audit	`reader`	domaine nord	oui	et `reader` sur le domaine, pour lire les comptes
Pipeline de déploiement web	`member`	projet `prod-web`	non	identifiant d'application, expiration trimestrielle
Supervision	`reader`	chaque projet surveillé	non	identifiant d'application par projet
Services OpenStack	`admin` + `service`	projet des services	non	géré par les charms : ne pas toucher

Aucun rôle donné à une personne : toutes les affectations humaines passent par des groupes. Ajouter ou retirer quelqu'un = modifier une appartenance.

Cette matrice suppose l'arbre de projets « par environnement » du module 10 : nœuds `prod` et `horsprod`, feuilles par application.

Pas à pas (1) : groupes et affectations

```
# Étape 0 — prérequis : cloud Sunbeam déployé, client
# openstack installé, domaine nord et projets créés
sunbeam openrc > admin-openrc
source admin-openrc
openstack role list      # reader member manager admin service

# Étape 1 — les groupes
for g in gestionnaires-nord exploitation-prod \
         developpeurs equipe-web audit; do
  openstack group create $g --domain nord
done

# Étape 2 — délégation du domaine
openstack role add manager --domain nord \
  --group gestionnaires-nord --group-domain nord

# Étape 3 — audit : lecture du domaine et des projets
openstack role add reader --domain nord \
  --group audit --group-domain nord
openstack role add reader --domain nord --inherited \
  --group audit --group-domain nord
```

```
# Étape 4 — production : exploitation et développeurs
openstack role add member --inherited \
  --project prod --project-domain nord \
  --group exploitation-prod --group-domain nord
openstack role add reader --inherited \
  --project prod --project-domain nord \
  --group developpeurs --group-domain nord

# Étape 5 — hors production : affectations directes
for p in rec-web dev-web; do
  openstack role add member \
    --project $p --project-domain nord \
    --group equipe-web --group-domain nord
done

# Étape 6 — les personnes, dans les groupes
for g in developpeurs equipe-web; do
  openstack group add user --group-domain nord \
    --user-domain nord $g alice
done

# Contrôle
openstack role assignment list --names \
  --effective --user alice --user-domain nord
```

> Domaine raccordé à un annuaire (`sunbeam ldap add-domain`) : les groupes viennent de l'annuaire ; on saute les étapes 1 et 6.

13

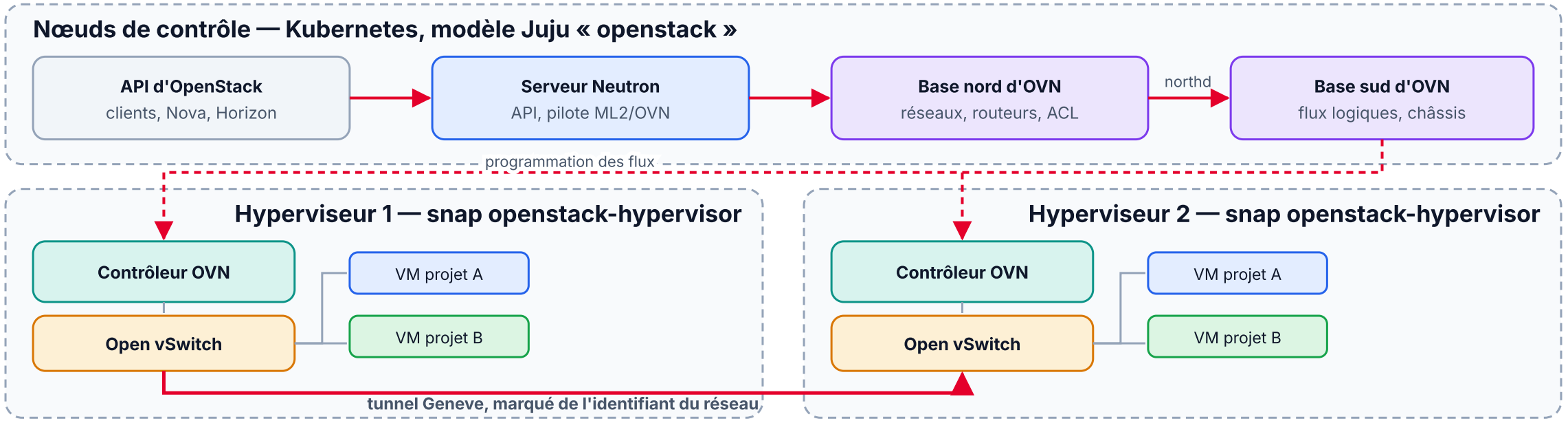
Micro-segmentation, audit et traçabilité

Réseaux partagés et réseaux privés, isolation entre locataires, groupes de sécurité ; journalisation des services, corrélation et centralisation des journaux, sur Canonical OpenStack (Sunbeam)

À L'ISSUE DE CE MODULE, VOUS SAUREZ

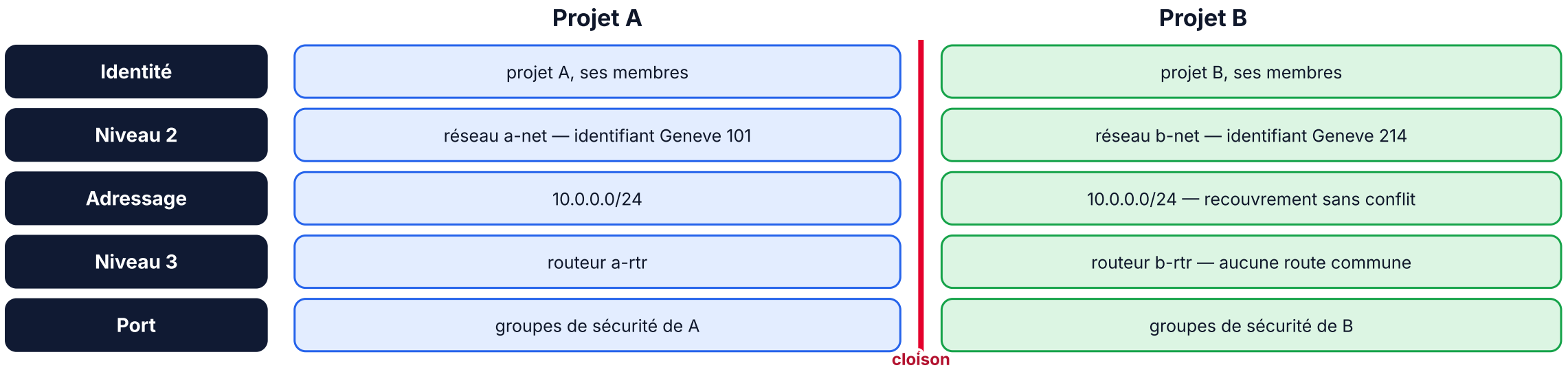
- Expliquer ce qui isole deux projets dans **Neutron et OVN**.
- Concevoir une **micro-segmentation** par groupes de sécurité et la tester.
- Centraliser les journaux d'un cloud Sunbeam et écrire des **règles d'alerte** utiles.
- Choisir entre **réseau partagé**, **réseau privé** et partage ciblé par RBAC.
- Retrouver une action dans les journaux grâce aux **identifiants de requête** et aux événements d'audit.

Neutron et OVN sur Sunbeam



- **Neutron décrit,** OVN réalise : chaque réseau devient un commutateur logique, chaque routeur un routeur logique, chaque groupe de sécurité un ensemble de règles de filtrage (ACL).
- **Le plan de contrôle** — serveur Neutron, bases nord et sud d'OVN — tourne dans Kubernetes sur les nœuds de contrôle.
- **Le plan de données est distribué :** le contrôleur OVN de chaque hyperviseur programme Open vSwitch localement ; le filtrage se fait au plus près de l'instance.
- **Entre hyperviseurs,** le trafic des réseaux de projet circule dans des tunnels Geneve, marqué de l'identifiant de son réseau.

Ce qui isole deux projets



Par défaut, deux projets ne se voient pas : réseaux distincts, adresses indépendantes, aucun routeur commun. Pour communiquer, ils doivent passer par le réseau externe et des adresses flottantes — comme deux entreprises sur Internet.

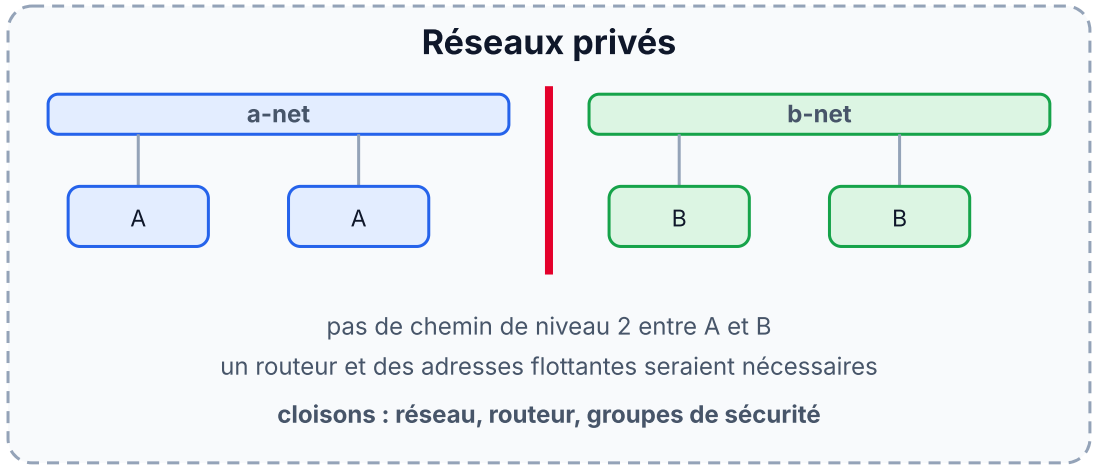
Chaque couche se défait par une décision explicite : un réseau partagé, un routeur commun, un groupe de sécurité trop ouvert, une sécurité de port désactivée. La suite du module passe ces décisions en revue.

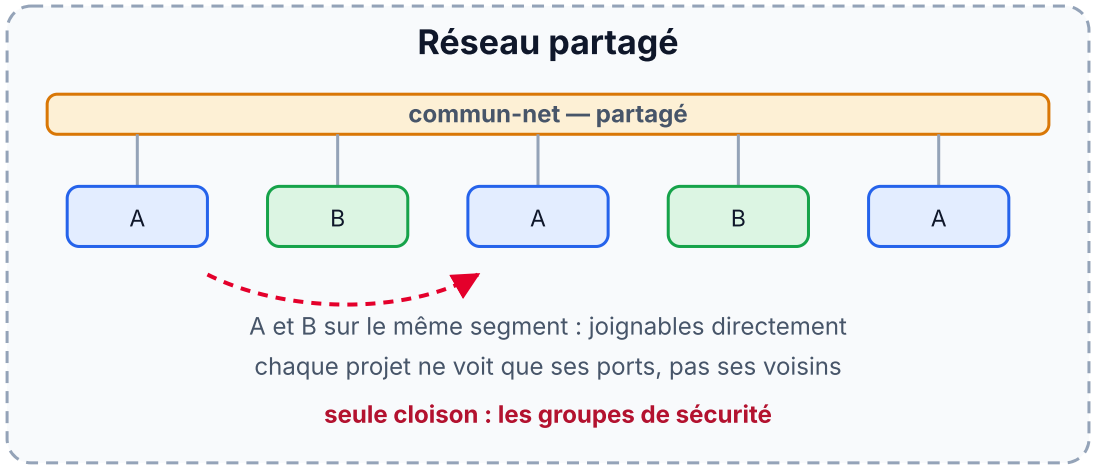
Le chemin d'un paquet entre deux projets



- **Quatre contrôles successifs :** groupe de sécurité de sortie de A, traduction d'adresse par le routeur de A, routeur de B qui traduit l'adresse flottante, groupe de sécurité d'entrée de B.
- **Le groupe de sécurité de B** voit l'adresse source traduite de A — l'adresse de passerelle de son routeur —, pas son adresse privée.
- **Avec OVN,** routage et traduction d'adresses sont distribués : le paquet peut ne jamais quitter l'hyperviseur, et pourtant subir les mêmes contrôles.
- **Sans adresse flottante sur B,** aucun chemin : B est injoignable depuis A.

Ce que partager un réseau veut dire





- **Sur un réseau partagé,** les instances de plusieurs projets sont sur le même segment de niveau 2. Elles se joignent directement, sans routeur — **seuls les groupes de sécurité les séparent.**
- **Chaque projet ne voit que ses propres ports,** mais il peut découvrir les autres adresses du segment par balayage.
- **Le groupe `default`** d'un projet n'autorise l'entrée que depuis ce même groupe — donc pas depuis les autres projets. C'est ce qui protège le cas général ; une seule règle trop large l'annule.
- **Usage légitime :** réseau d'administration ou de supervision, avec des règles d'entrée strictes du côté des instances.

 Vu au module 10

Le partage ciblé : RBAC de Neutron

Type d'objet	Ce que le partage permet
`network`	**créer des ports sur le réseau ; ou s'en servir comme réseau externe**
`security_group`	**appliquer le groupe à ses ports**
`address_group`	**utiliser le groupe d'adresses dans ses règles**
`qos_policy`	**appliquer la politique de qualité de service**
`subnetpool`, `address_scope`	**tirer des sous-réseaux d'un réservoir commun**

Action	Sens
`access_as_shared`	utilisation partagée
`access_as_external`	usage comme réseau externe : passerelle de routeur, adresses flottantes

```
# Partager le réseau d'administration avec UN projet
openstack network rbac create \
  --type network --action access_as_shared \
  --target-project prod-web --target-project-domain nord \
  admin-net

# Équivalent de --share : cible « * » (tous)
# — à éviter, et souvent réservé à l'administration
openstack network rbac create --type network \
  --action access_as_shared --target-all-projects admin-net

# Revue
openstack network rbac list --long
openstack network rbac delete <id>
```

- `--share` **à la création = cible « * » :** tous les projets, présents et futurs.
- **Retrait bloqué** tant que le projet cible a des ports sur le réseau : nettoyer d'abord.
- **Qui peut partager :** le propriétaire, avec un projet nommé ; la cible « * » est en principe réservée à l'administration — règle `create_rbac_policy` à resserrer si besoin.

Le groupe de sécurité

Propriété	Conséquence
Liste d'autorisations	**tout ce qui n'est pas autorisé est refusé ; aucune règle de refus**
Avec état	**la réponse à un flux autorisé est autorisée d'office**
Appliqué au port	**filtre au plus près de l'instance, même entre deux instances du même réseau**
Cumulatif	**plusieurs groupes sur un port = union des autorisations**
Dynamique	**une règle modifiée s'applique aux instances en cours**
Par projet	**chaque projet gère ses groupes ; partageables par RBAC**

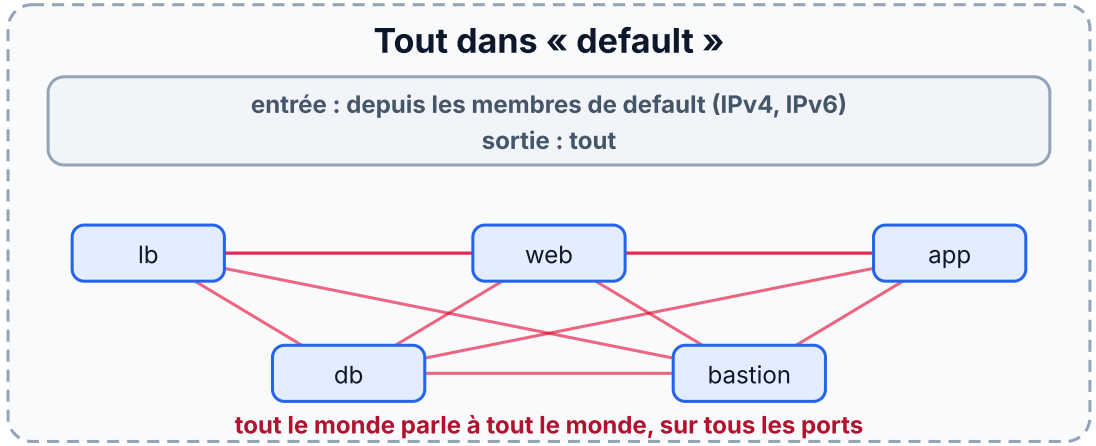
Sans état (`--stateless`) : les réponses doivent être autorisées explicitement. Réservé à des cas de performance particuliers.

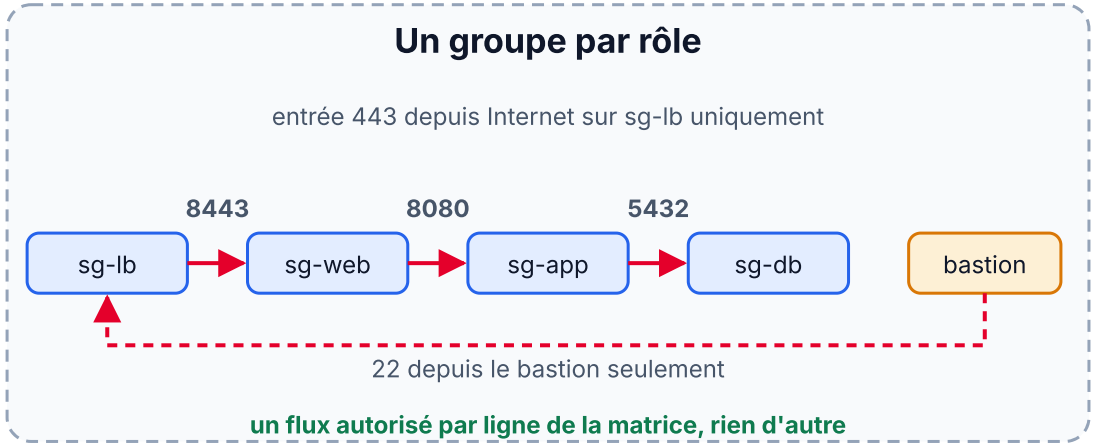
Une règle

Champ	Valeurs
Direction	entrée (`--ingress`) ou sortie (`--egress`)
Famille	IPv4 ou IPv6
Protocole	tcp, udp, icmp, numéro, ou tous
Ports	un port ou une plage
Origine ou destination	un bloc d'adresses, **ou** un groupe de sécurité, **ou** un groupe d'adresses

Pas de règle de refus : on ne peut pas écrire « tout sauf ce port ». Pour interdire, on retire l'autorisation. Les refus explicites relèvent du pare-feu de réseau (FWaaS) ou du pare-feu périmétrique.

Le groupe « default »





- **Créé automatiquement** dans chaque projet, et appliqué à toute instance créée sans groupe explicite.
- **Entrée :** uniquement depuis les membres du même groupe. **Sortie :** tout.
- **Conséquence :** toutes les instances d'un projet qui gardent `default` se parlent librement, sur tous les ports — l'inverse de la micro-segmentation.
- **Bonne pratique :** ne pas y ajouter de règles ; créer des groupes par rôle et les appliquer explicitement.
- **Sur le projet de démonstration Sunbeam,** il a été ouvert en SSH et ping depuis partout : ne pas s'en inspirer.
- **Il ne se supprime pas** tant que le projet existe ; on peut en revanche en retirer les règles.

Une application à trois niveaux : la matrice de flux

Source	Destination	Flux	Règle (sur le groupe destination)	Justification
Internet	`sg-lb`	TCP 443	entrée 443 depuis `0.0.0.0/0`	seul point exposé
`sg-lb`	`sg-web`	TCP 8443	entrée 8443 depuis groupe `sg-lb`	le répartiteur seul parle aux serveurs web
`sg-web`	`sg-app`	TCP 8080	entrée 8080 depuis groupe `sg-web`	appel de l'API métier
`sg-app`	`sg-db`	TCP 5432	entrée 5432 depuis groupe `sg-app`	seule l'application touche aux données
`sg-bastion`	**tous**	TCP 22	entrée 22 depuis groupe `sg-bastion`, sur chaque groupe	administration par rebond
supervision	**tous**	TCP 9100	entrée 9100 depuis l'adresse de la sonde	métriques
`sg-db`	**extérieur**	aucun	sortie limitée au réseau du projet	pas d'exfiltration directe

Méthode

Lister les flux nécessaires avant d'écrire une règle ; chaque ligne de la matrice devient une règle, et réciproquement.

Ce qui n'y figure pas

Serveur web vers base de données, Internet vers l'application, bastion vers Internet : ces flux sont refusés sans rien écrire.

Un seul réseau suffit

Les quatre niveaux peuvent partager un réseau : les groupes séparent. Des réseaux distincts ajoutent une seconde barrière.

Pas à pas : les groupes de l'application

```
# Prérequis : projet prod-web, rôle member
for g in sg-lb sg-web sg-app sg-db sg-bastion; do
  openstack security group create $g
done

R="openstack security group rule create"
$R sg-lb  --ingress --protocol tcp --dst-port 443 \
   --remote-ip 0.0.0.0/0 --description "HTTPS public"
$R sg-web --ingress --protocol tcp --dst-port 8443 \
   --remote-group sg-lb  --description "LB vers web"
$R sg-app --ingress --protocol tcp --dst-port 8080 \
   --remote-group sg-web --description "web vers app"
$R sg-db  --ingress --protocol tcp --dst-port 5432 \
   --remote-group sg-app --description "app vers base"
$R sg-bastion --ingress --protocol tcp --dst-port 22 \
   --remote-ip 192.168.50.0/24 --description "admin"
for g in sg-web sg-app sg-db; do
  $R $g --ingress --protocol tcp --dst-port 22 \
     --remote-group sg-bastion --description "rebond SSH"
done
# sg-db : sortie limitée au réseau du projet (matrice)
for r in $(openstack security group rule list sg-db \
           --egress -f value -c ID); do
  openstack security group rule delete $r
done
$R sg-db --egress --remote-ip 10.10.0.0/16 --description "sortie interne"
```

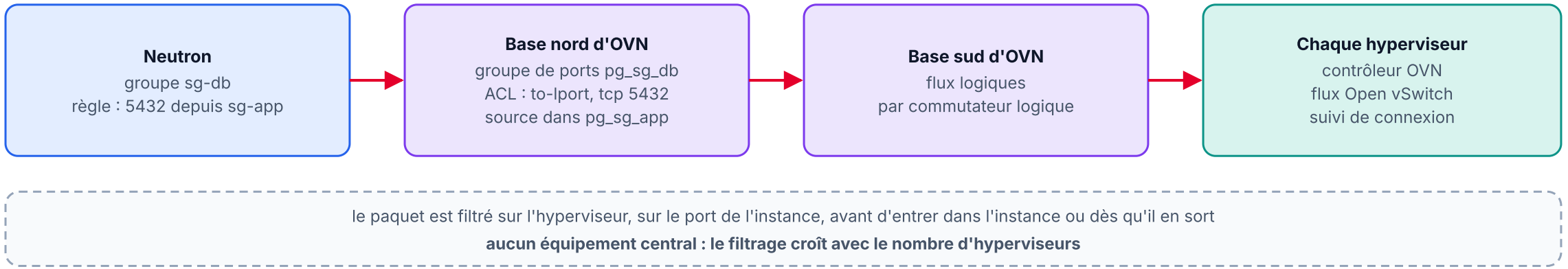
```
# Créer les instances avec leur seul groupe
openstack server create db-01 --flavor m1.medium \
  --image ubuntu-24.04 --network prod-web-app \
  --security-group sg-db --key-name exploitation

# Vérifier les groupes effectifs d'un port
openstack port list --server db-01 -c ID -c "Fixed IP Addresses"
openstack port show <port> -c security_group_ids
```

Test de cloison	Attendu
web-01 → db-01:5432	refusé
app-01 → db-01:5432	accepté
app-01 → db-01:22	refusé
bastion → db-01:22	accepté
Internet → web-01:8443	refusé

> Tester avec `nc -zv -w 3 <ip> <port>` depuis chaque instance : un refus par groupe de sécurité se traduit par un délai dépassé, pas par un refus de connexion.

Mise en œuvre par OVN



- **Un groupe de sécurité devient un groupe de ports OVN** et ses règles des ACL, inscrites dans la base nord puis traduites pour chaque hyperviseur.
- **Le filtrage se fait dans Open vSwitch** sur l'hyperviseur de l'instance, avec suivi de connexion : il n'y a pas de pare-feu central à saturer.
- **Une instance compromise ne peut pas contourner ses règles :** elles s'appliquent hors de l'instance, sur le port.
- **Lire ce qu'OVN a reçu :** `ovn-nbctl acl-list` dans le conteneur de la base nord — réservé au diagnostic, jamais pour modifier.

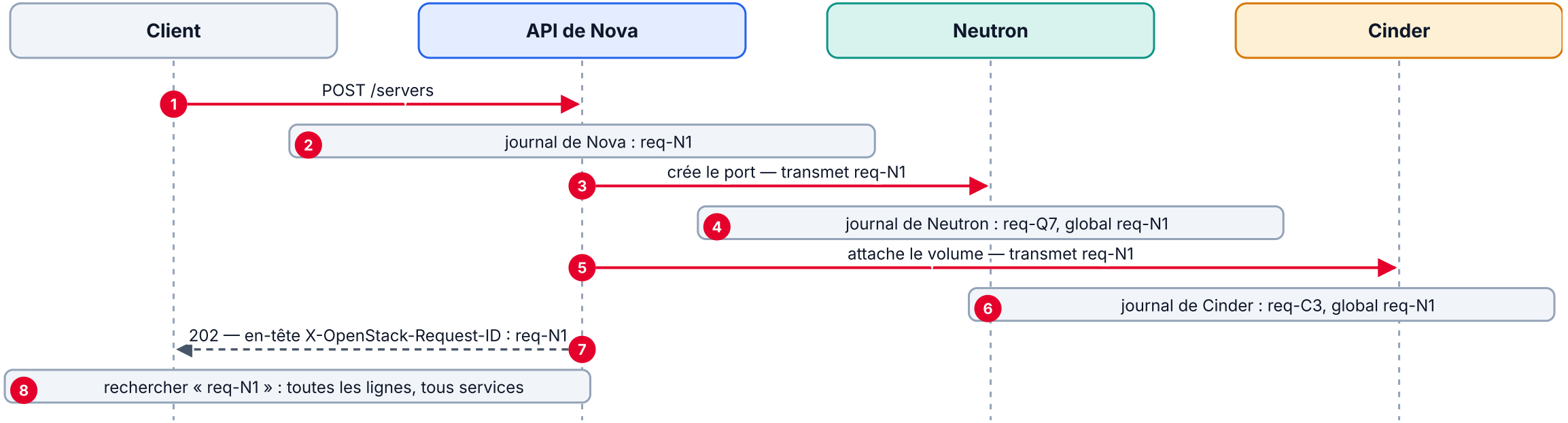
Cloisonnement inter-locataires : synthèse et contrôles

Couche	Ce qui cloisonne	Ce qui le défait	Contrôle périodique
Identité	**projets, rôles, domaines**	rôle sur plusieurs projets, `admin`	`role assignment list --effective`
Niveau 2	**réseau privé par projet, Geneve**	réseau partagé	`network rbac list` ; réseaux `Shared`
Niveau 3	**routeur propre, pas de route commune**	routeur relié aux sous-réseaux de deux projets	`router list --long` , interfaces
Exposition	**pas d'adresse flottante**	adresses flottantes inutiles	`floating ip list --long`
Port	**groupes de sécurité par rôle**	`0.0.0.0/0` en entrée, `default` élargi	règles ouvertes sur 0.0.0.0/0
Anti-usurpation	**sécurité de port**	sécurité de port désactivée	ports sans sécurité de port
Ressources	**quotas**	quotas illimités	`quota show --usage`

Test de cloison entre locataires, après chaque changement : depuis une instance du projet A, tenter de joindre l'adresse fixe et l'adresse flottante d'une instance du projet B, sur un port non autorisé. Attendu : échec dans les deux cas.

Ces contrôles s'automatisent en un script lancé chaque semaine, dont le résultat part dans les journaux centralisés — ce qui fait la transition avec la suite.

Les identifiants de requête



- **Chaque service** attribue un identifiant `req-…` à chaque requête reçue et le renvoie dans l'en-tête de réponse `X-OpenStack-Request-ID`.
- **Lorsqu'un service en appelle un autre,** il transmet l'identifiant d'origine : le service appelé l'inscrit comme identifiant global à côté du sien.
- **Le client peut fournir son propre identifiant** global ; un outil d'automatisation l'utilise pour relier ses actions aux journaux du cloud.
- **En pratique :** on part de l'identifiant renvoyé à l'utilisateur ou affiché par `openstack --debug`, puis on cherche cette chaîne dans tous les journaux centralisés.

Les événements d'instance

```
# Historique des actions sur une instance
openstack server event list db-01

+------------------+--------------+--------+---------------------+
| Request ID       | Server ID    | Action | Start Time          |
+------------------+--------------+--------+---------------------+
| req-7c1e…        | 5e3b…        | delete | 2026-10-13T09:42:17 |
| req-31aa…        | 5e3b…        | reboot | 2026-10-12T22:05:03 |
| req-f02c…        | 5e3b…        | create | 2026-10-01T08:11:46 |
+------------------+--------------+--------+---------------------+

# Détail : qui, depuis quel projet, résultat
openstack server event show db-01 req-7c1e…

# Instance déjà supprimée : par identifiant,
# avec les droits d'administration
openstack server event list 5e3b…
```

> Sortie reconstituée pour l'exemple ; les colonnes dépendent de la microversion de l'API de Nova.

Champ	Usage
Action	**création, arrêt, redémarrage, migration, suppression…**
Identifiant de requête	**pivot vers les journaux de tous les services**
Utilisateur, projet	**auteur de l'action**
Étapes, résultat	**succès ou erreur de chaque étape interne**

- **Première source à consulter** pour toute question sur une instance : elle est dans la base de Nova, donc survit aux redémarrages des pods.
- **Limites :** elle ne couvre que les instances. Pour les réseaux, les volumes, les rôles, il faut les journaux des autres services.
- **Les membres du projet** voient les événements de leurs instances ; utile au support de premier niveau.

Les notifications CADF de Keystone

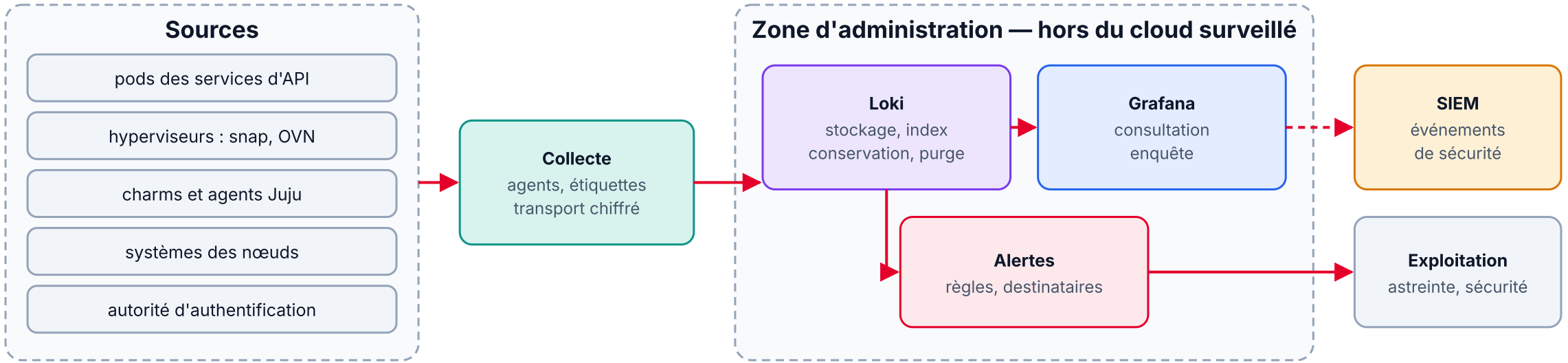
```
{
  "event_type": "identity.role_assignment.created",
  "priority": "INFO",
  "publisher_id": "identity.keystone-0",
  "payload": {
    "typeURI": "http://schemas.dmtf.org/cloud/audit/1.0/event",
    "eventType": "activity",
    "eventTime": "2026-10-13T09:58:02.114+0000",
    "action": "created.role_assignment",
    "outcome": "success",
    "initiator": {
      "id": "4f2a…",
      "host": { "address": "10.20.0.15",
                "agent": "openstacksdk/4.x" }
    },
    "target": { "typeURI": "…/role_assignment", "id": "…" },
    "observer": { "typeURI": "service/security", "id": "…" },
    "role": "…", "user": "…", "project": "…"
  }
}
```

Réglage de Keystone	Effet
`notification_format = cadf`	**événements au format d'audit CADF, plutôt que le format simple**
`notification_opt_out`	**liste des événements à ne pas émettre ; par défaut, les authentifications réussies ou échouées sont supprimées**
Pilote de notifications	**messagerie (vers un consommateur) ou journal**

- **Événements émis :** création, modification, suppression des utilisateurs, groupes, projets, domaines, rôles, affectations, points d'accès ; authentifications si on les réactive.
- **Pour l'audit,** réactiver les échecs d'authentification : c'est la trace d'une attaque par essais.

Sur Sunbeam : le charm de Keystone propose l'option `enable-telemetry-notifications`, liée à la fonctionnalité de télémétrie. Format et destination des notifications sont à vérifier sur votre déploiement avant d'en faire une source d'audit.

Architecture de centralisation



- **Collecte au plus près :** un agent par nœud ou par modèle lit les journaux et les envoie, étiquetés par origine.
- **Stockage hors des systèmes surveillés :** les journaux échappent à un administrateur ou un attaquant présent sur un nœud du cloud.
- **Exploitation à deux usages :** consultation et enquête d'un côté, alertes de l'autre.
- **Le SIEM de l'entreprise,** s'il existe, reçoit une copie des événements de sécurité pour les corréler avec le reste du système d'information.

L'observabilité de Sunbeam

```
# Option 1 — pile d'observabilité intégrée au cloud
sunbeam enable observability embedded

# Option 2 — pile existante, sur un autre contrôleur Juju
# (contrôleur enregistré, droits de consommation des offres)
sunbeam enable observability external \
  cos-controller \
  <offre-grafana-dashboard> \
  <offre-prometheus-receive-remote-write> \
  <offre-loki-logging>

# Adresse de Grafana et mot de passe initial
sunbeam observability dashboard-url
juju run --model observability grafana/leader \
  get-admin-password
```

Taille du stockage de la pile intégrée : fixée dans le fichier manifeste au déploiement ; la modifier ensuite n'est pas pris en charge. À dimensionner dès le départ.

Composant	Rôle
Loki	**stockage et requête des journaux**
Prometheus	**métriques : services, hyperviseurs, capacité**
Grafana	**tableaux de bord, consultation, alertes**

Tableau de bord fourni	Usage
OpenStack Logging	journaux des services, codes HTTP des API
OpenStack Service Overview	état des services
OpenStack Project Overview	consommation par projet
Capacity	projection de la capacité

Pile externe recommandée pour l'audit : les journaux sortent du cloud qu'ils décrivent, comme le demande l'ANSSI.

Interroger les journaux

```
# Requêtes LogQL, dans Grafana > Explore > Loki
# (étiquettes Juju indicatives : juju_model,
#  juju_application, juju_unit)

# Toutes les lignes d'une requête, tous services
{juju_model="openstack"} |= "req-7c1e"

# Suppressions d'instances
{juju_application="nova"} |= "DELETE /v2.1/servers"

# Échecs d'authentification dans Keystone
{juju_application="keystone"} |= "Authorization failed"

# Ajouts de règles de groupe de sécurité
{juju_application="neutron"} |= "POST /v2.0/security-group-rules"

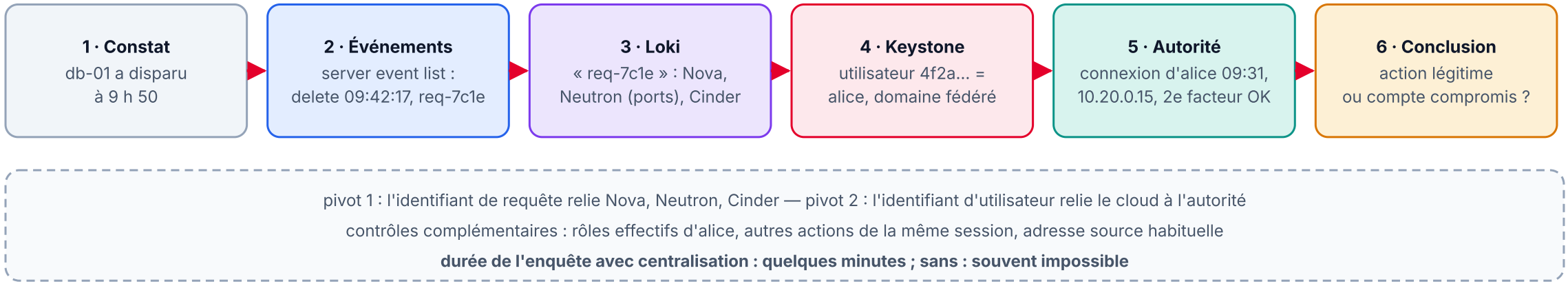
# Nombre de refus 403 par service, sur 5 minutes
sum by (juju_application) (
  count_over_time({juju_model="openstack"} |= "status: 403" [5m]))
```

Élément LogQL	Rôle
`{…}`	**sélection des flux par étiquettes : modèle, application, unité**
`|=` , `!=`	**contient, ne contient pas**
`|~`	**expression régulière**
`count_over_time`	**compter sur une fenêtre : base des alertes**

- **Commencer large, filtrer ensuite :** un identifiant de requête suffit à retrouver toute une action.
- **Les étiquettes** disponibles se lisent dans Grafana : explorer le sélecteur avant d'écrire une requête.
- **Chaînes de recherche** à adapter au format réel de vos journaux — vérifier sur une ligne connue.

> Ces requêtes, enregistrées comme tableaux de bord, deviennent le poste de travail de l'enquête.

Une enquête pas à pas : qui a supprimé la base ?



La corrélation tient en deux pivots : l'identifiant de requête relie les services entre eux ; l'identifiant d'utilisateur relie le cloud à l'autorité d'authentification.

Ce qui la rend possible : horloges synchronisées, niveau INFO sur les API, journaux centralisés et conservés, événements d'authentification réactivés.

Règles d'alerte utiles

Événement	Pourquoi	Source	Seuil indicatif
Échecs d'authentification répétés sur un compte	**attaque par essais, compte verrouillé**	Keystone	> 10 en 5 minutes
Échec d'authentification d'un compte de service	**secret changé, attaque, panne imminente**	Keystone	dès le premier
Affectation du rôle `admin`	**élévation de privilège**	Keystone	chaque occurrence
Règle d'entrée depuis `0.0.0.0/0` hors port 443	**exposition non voulue**	Neutron	chaque occurrence
Création d'une politique RBAC réseau	**ouverture entre locataires**	Neutron	chaque occurrence
Association d'une adresse flottante	**nouvelle exposition**	Neutron	revue quotidienne
Sécurité de port désactivée	**perte d'anti-usurpation**	Neutron	chaque occurrence
Suppressions en masse	**erreur ou sabotage**	Nova, Cinder	> 20 en 10 minutes
Connexion du compte de secours	**usage exceptionnel**	Keystone	chaque occurrence
Source de journaux muette	**chaîne de collecte cassée**	Loki	aucune ligne depuis 15 minutes

Une alerte doit appeler une action : chacune a un destinataire et une procédure. Une alerte que personne ne traite sera ignorée avec les autres.

La dernière ligne est la plus importante : sans elle, une chaîne de collecte cassée passe inaperçue jusqu'au jour de l'enquête.

SYNTHÈSE

Ce qu'il faut retenir, module par module

Module	L'idée à garder
01 · Panorama	OpenStack est un IaaS libre : une API par service, Keystone au centre, un plan de contrôle qui pilote calcul, réseau et stockage.
02 · Architecture interne	REST entre services, RPC par le bus à l'intérieur d'un service, OVSDB pour OVN ; trois contrôleurs pour le quorum ; réseaux séparés par domaine de confiance.
03 · Composants système	Bus, base et serveur d'API portent tous les services : savoir lire leur état, c'est diagnostiquer la moitié des incidents.
04 · Principaux services	Nova orchestre Glance, Placement, Neutron et Cinder ; l'étape où une opération se bloque désigne le service à examiner.
05 · Méthodes de déploiement	Choisir d'abord qui exploite, puis l'outil ; trancher par une maquette qui inclut une montée de version.
06 · La stack Canonical	MAAS provisionne, Juju orchestre par charms, Sunbeam assemble ; fournisseur de machines et version LTS de départ sont des choix définitifs.
07 · MicroCloud	LXD, MicroCeph et MicroOVN sans plan de contrôle séparé : un cluster de virtualisation, pas un cloud en libre-service.
08 · Keystone	Acteur + rôle + cible : la seule source de droits ; jetons Fernet non stockés ; 401 = identité, 403 = autorisation.
09 · Domaines	Le domaine isole les identités et leur administration, pas les ressources ; déléguer par manager, jamais par admin.
10 · Projets et quotas	Toute ressource appartient à un projet ; l'arbre fait descendre les rôles ; les quotas par défaut sont le réglage le plus important.
11 · Authentification	Les personnes s'authentifient chez l'autorité fédérée ; Keystone garde peu de comptes locaux, exemptés avec soin pour les services.
12 · Moindre privilège	Rôles standards sur la cible la plus étroite, portés par des groupes ; identifiants d'application expirants pour les automates.
13 · Segmentation et audit	Réseau privé par projet, groupes de sécurité par rôle ; identifiant de requête et journaux centralisés hors du cloud pour enquêter.